



Universidad de Oviedo

UNIVERSIDAD DE OVIEDO
ESCUELA DE INGENIERÍA INFORMÁTICA

Trabajo Fin de Grado

Diseño y Desarrollo de un Sistema de Verificación Automática de Entregas

Grado en Ingeniería Informática del Software

Autor: Adrián Martínez Fuentes (UO295454)

Tutora universitaria: María del Carmen Suárez Torrente

Curso 2025/2026

DECLARACIÓN DE ORIGINALIDAD

De acuerdo con lo expresado en el artículo 8.3 del Reglamento sobre la asignatura Trabajo Fin de Grado en la Universidad de Oviedo, aprobado por su Consejo de Gobierno el 5 de marzo de 2020 (BOPA de 30 de marzo de 2020), declaro lo siguiente:

Yo, Adrián Martínez Fuentes, con NIE 71743428B, en relación con la memoria que presento ante el Tribunal para su evaluación como Trabajo Fin de Grado, declaro que soy el autor de la misma, habiendo citado debidamente todas las fuentes utilizadas en su elaboración.

Para que conste, firmo el presente documento.

En Oviedo, a 2 de julio de 2026

Firmado por ADRIAN
MARTINEZ FUENTES
el día 02/07/2026

Fdo.: _____

A mi familia, por su apoyo incondicional y su confianza constante.

A mi tutora, por su guía y orientación.

A todos los que han hecho posible este camino.

AGRADECIMIENTOS

En primer lugar, quisiera expresar mi más sincero agradecimiento a mi tutora, María del Carmen Suárez Torrente, por su orientación, disponibilidad y apoyo constante a lo largo de todo el desarrollo de este Trabajo Fin de Grado. Sus comentarios y guía han sido fundamentales para dar forma a este proyecto.

Quiero agradecer también a la Escuela de Ingeniería Informática de la Universidad de Oviedo por la formación recibida durante estos años, que ha proporcionado las bases necesarias para abordar un proyecto de esta envergadura. En particular, agradezco a todos los profesores que han contribuido a mi aprendizaje en áreas como la ingeniería del software, las bases de datos y los sistemas distribuidos.

A mi familia, por su apoyo incondicional y su confianza sostenida a lo largo de esta etapa. Sin su respaldo, nada de esto habría sido posible.

Por último, quiero agradecer a todas las personas que, de forma directa o indirecta, han formado parte de este camino universitario, por los momentos compartidos y el apoyo recibido.

RESUMEN

El presente Trabajo Fin de Grado aborda el diseño y desarrollo de un sistema centralizado para la verificación automática de entregas de software, denominado SVAES (Sistema de Verificación Automática de Entregas de Software). El proyecto nace de la necesidad de resolver un problema habitual en entornos de desarrollo modernos: la información necesaria para validar una entrega se encuentra distribuida entre múltiples fuentes heterogéneas, como sistemas de gestión de tareas, repositorios de documentación, herramientas de planificación y sistemas de control de versiones.

En estos contextos, la validación manual de entregas resulta un proceso costoso, repetitivo y propenso a errores humanos. Determinar si una entrega cumple los requisitos de coherencia e integridad requiere consultar cada fuente, contrastar la información entre ellas y verificar que las relaciones entre los distintos artefactos se mantienen correctamente.

La solución propuesta consiste en un sistema modular capaz de obtener información desde distintas fuentes mediante interfaces programáticas (APIs REST), unificarla en un modelo interno común y aplicar sobre ella un conjunto de reglas de verificación automática. El sistema permite evaluar la integridad de cada entrega, detectar inconsistencias entre artefactos y generar resultados reproducibles, auditables y trazables. El motor de verificación está implementado en Rust para garantizar rendimiento y seguridad en memoria, mientras que el backend utiliza FastAPI con Python y el frontend está desarrollado con Angular. La persistencia se gestiona mediante PostgreSQL y la ejecución asíncrona mediante Celery con Redis.

Además, la arquitectura del sistema se ha diseñado con un enfoque modular y extensible, lo que facilita su adaptación a distintos escenarios de desarrollo software y la incorporación de nuevas fuentes de información y reglas de verificación.

El presente proyecto tiene su origen en una propuesta formulada en el contexto de las prácticas en empresa realizadas en Indra Sistemas, S.A., donde se identificó una necesidad organizativa real relacionada con la validación de entregas de software en entornos de desarrollo de cierta complejidad. Tras la desvinculación formal del convenio de prácticas y del Trabajo Fin de Grado inicialmente asociado a dicha empresa, el trabajo ha sido desarrollado de forma individual e independiente, con orientación *open source* y empleando un entorno de datos simulado construido íntegramente por el autor para la validación del sistema.

Palabras clave: verificación automática, entregas software, integración de sistemas, APIs

REST, ciclo de vida del software, automatización, trazabilidad, coherencia de datos, FastAPI, Angular, Rust, PostgreSQL, Docker Compose.

ABSTRACT

This Bachelor's Thesis presents the design and development of a centralized system for the automatic verification of software deliveries, named SVAES (Automated Software Delivery Verification System). The project addresses a common challenge in modern software development environments: the information required to validate a delivery is distributed across multiple heterogeneous sources, including task management systems, documentation repositories, planning tools, and version control systems.

In such contexts, manual delivery validation becomes a costly, repetitive process prone to human error. Determining whether a delivery meets the required consistency and integrity criteria demands consulting each source, cross-referencing information, and verifying that relationships between different artifacts are maintained correctly.

The proposed solution is a modular system capable of retrieving information from different sources through programmatic interfaces (REST APIs), unifying it into a common internal model, and applying a set of automatic verification rules. The system evaluates each delivery's integrity, detects inconsistencies between artifacts, and produces reproducible, auditable, and traceable results. The verification engine is implemented in Rust for performance and memory safety, while the backend uses FastAPI with Python and the frontend is developed with Angular. Persistence is managed through PostgreSQL and asynchronous execution through Celery with Redis.

Furthermore, the system architecture follows a modular and extensible design, facilitating its adaptation to different software development scenarios and the incorporation of new information sources and verification rules.

This project originates from a proposal formulated during an internship at Indra Sistemas, S.A., where a real industrial need related to the automated validation of software deliveries in complex development environments was identified. Following the formal disengagement from both the company-sponsored internship and the associated thesis arrangement, the work has been developed independently and with an open-source orientation, using a simulated data environment constructed entirely by the author for system validation, with no access to or involvement from the company during execution.

Keywords: automatic verification, software deliveries, system integration, REST APIs, software lifecycle, automation, traceability, data consistency, FastAPI, Angular, Rust, PostgreSQL, Docker Compose.

Índice general

Declaración de originalidad	i
1 Memoria del Proyecto	2
1.1 Resumen de la Motivación, Objetivos y Alcance del Proyecto	2
1.2 Estructura de la Documentación	3
2 Introducción	5
2.1 Justificación del Proyecto	5
2.2 Objetivos del Proyecto	6
2.3 Planteamiento del Problema	7
2.4 Escenario de Referencia	7
2.5 Proceso Manual de Referencia	8
2.6 Fuentes de Información Implicadas	8
2.7 Limitaciones del Enfoque Manual	9
2.8 Ámbito y Alcance del Sistema	10
2.9 Herramientas y Soluciones Similares	11
2.10 Análisis de Alternativas Tecnológicas	12
3 Marco Teórico y Estado del Arte	15
3.1 Ciclo de Vida del Software y Gestión de Entregas	15
3.2 Trazabilidad de Artefactos Software	16
3.3 Integración de Sistemas y Fuentes de Información	16
3.4 Application Programming Interfaces (APIs) Representational State Transfer (REST) y Acceso a Información	17
3.5 Verificación Automática Basada en Reglas	17
3.6 Estado del Arte en Automatización del Desarrollo Software	18
3.6.1 Integración Continua y Entrega Continua	18
3.6.2 Quality Gates	18
3.6.3 Plataformas Application Lifecycle Management (ALM)	19
3.6.4 Minería de Repositorios Software	19
3.6.5 Automatización de Procesos	19
3.6.6 Conclusiones del Estado del Arte	19

4	Especificación de Requisitos del Sistema	20
4.1	Propósito	20
4.2	Ámbito del sistema	20
4.3	Definiciones, acrónimos y abreviaturas	20
4.4	Visión general	21
4.5	Stakeholders	21
4.5.1	Stakeholders internos	21
4.5.2	Stakeholders externos	23
4.5.3	Usuarios del sistema	23
4.6	Fuentes de información	24
4.7	Ámbito y alcance del sistema	25
4.8	Normativa aplicable	25
4.8.1	Estándares de ingeniería del software	25
4.8.2	Normativa de seguridad y protección de datos	25
4.8.3	Estándares técnicos	26
4.9	Funcionalidades de alto nivel	26
4.9.1	Épica 1: Arquitectura Multi-inquilino y Seguridad	26
4.9.2	Épica 2: Integración y Estandarización de Políticas	26
4.9.3	Épica 3: Núcleo Operativo (Motor de Verificación)	27
4.9.4	Épica 4: Observabilidad y Comunicación	27
4.9.5	Épica 5: Extensibilidad	28
4.10	Necesidades no funcionales	29
4.10.1	Rendimiento	29
4.10.2	Disponibilidad	29
4.10.3	Fiabilidad	29
4.10.4	Seguridad	30
4.10.5	Usabilidad	31
4.10.6	Mantenibilidad	31
4.10.7	Portabilidad	32
4.10.8	Extensibilidad	32
4.10.9	Trazabilidad	32
4.10.10	Requisitos organizacionales y externos	33
4.11	Catálogo de requisitos funcionales	33
4.11.1	Épica 1 — Arquitectura Multi-inquilino y Seguridad	34
4.11.2	Épica 2 — Integración y Estandarización de Políticas	42
4.11.3	Épica 3 — Núcleo Operativo (Motor de Verificación)	46
4.11.4	Épica 4 — Observabilidad y Comunicación	54
4.11.5	Épica 5 — Extensibilidad	60
4.12	Casos de uso	62

4.12.1	CU-01: Verificar Release	63
4.12.2	CU-02: Configurar Perfil de Verificación	67
4.13	Diagramas del sistema	70
4.13.1	DF-01 — Autenticación	70
4.13.2	DF-02 — Ciclo de vida de release	71
5	Diseño del Sistema	73
5.1	Visión Arquitectónica y Decisiones de Diseño	73
5.1.1	Tabla de Decisiones Arquitectónicas	74
5.1.2	Stack Tecnológico	75
5.2	Arquitectura del Sistema	76
5.2.1	Diagrama de Contexto (C4 Nivel 1)	76
5.2.2	Diagrama de Contenedores (C4 Nivel 2)	77
5.2.3	Diagrama de Componentes: Backend (C4 Nivel 3)	78
5.2.4	Diagrama de Despliegue	79
5.2.5	Soporte Multi-organización	80
5.3	Diseño de la Capa de Dominio y Aplicación	81
5.3.1	Organización en Capas	81
5.3.2	Diagrama de Clases del Dominio	82
5.3.3	Diagramas de Secuencia	83
5.4	Diseño del Motor de Verificación (Rust)	85
5.4.1	Interfaz de Comunicación con el Backend	85
5.4.2	Modelo Interno del Motor	85
5.4.3	Catálogo de Reglas de Verificación (RV-01 a RV-10)	86
5.5	Diseño de la Integración con Herramientas Externas	90
5.5.1	Puerto IConnector	90
5.5.2	Conectores de Referencia	91
5.6	Diseño de la Base de Datos	92
5.6.1	Sistema Gestor de Base de Datos	92
5.6.2	Diagrama Entidad-Relación	92
5.6.3	Descripción de Tablas del Esquema	93
5.6.4	Consideraciones de Diseño del Esquema	98
5.7	Diseño de la Interfaz de Usuario	99
5.7.1	Mapa de Navegación	99
5.7.2	Wireframes de las Vistas Principales	100
5.7.3	Diseño de la API REST Pública	102
5.8	Seguridad en el Diseño	105
5.9	Trazabilidad Diseño–Requisitos	106
5.10	Resumen del Capítulo	107

6	Implementación	109
6.1	Estándares y Normas Seguidas	109
6.2	Lenguajes de Programación	109
6.3	Tecnologías y Herramientas Utilizadas	110
6.4	Implementación del Backend (FastAPI)	110
6.4.1	Organización del Código Fuente	110
6.4.2	Capa de Dominio	111
6.4.3	Capa de Aplicación: Puertos y Casos de Uso	112
6.4.4	Adaptadores Primarios: API REST y Middleware	113
6.4.5	Adaptadores Secundarios: Persistencia y Migraciones	114
6.4.6	Seguridad Transversal	115
6.5	Motor de Verificación en Rust	115
6.5.1	Estructura del Proyecto y Servidor Hypertext Transfer Protocol (HTTP)	115
6.5.2	Pipeline de Verificación y Ejecución Paralela	116
6.5.3	Implementación de las Reglas RV-01 a RV-10	117
6.6	Sistema de Conectores	118
6.6.1	Puerto IConnector	118
6.6.2	Catálogo de Implementaciones	119
6.6.3	ConnectorRegistry y Cifrado de Credenciales	120
6.7	Cola de Tareas Asíncrona (Celery + Redis)	120
6.7.1	Servicios de Aplicación: Gestión Multi-organización	121
6.8	Frontend Angular	122
6.8.1	Estructura de la Aplicación	122
6.8.2	Autenticación, 2FA y RBAC en Cliente	123
6.8.3	Módulos de Gestión de Releases y Conectores	124
6.8.4	Internacionalización	125
6.9	Infraestructura de Despliegue	125
6.9.1	Configuración Docker Compose	125
6.9.2	Análisis Estático y Calidad Continua del Código	126
6.10	Problemas Encontrados y Soluciones Adoptadas	126
6.11	Descripción de la Estructura del Código Fuente	127
6.12	Implementación de la Suite de Pruebas	128
6.12.1	Estructura de la Suite	128
6.12.2	Configuración de Cobertura	130
6.12.3	Resultados de Cobertura	132
6.12.4	Referencias Cruzadas con el Plan de Pruebas	132
6.13	Resumen del Capítulo	133

7	Plan y ejecución de pruebas del sistema	134
7.1	Técnicas de diseño de pruebas	134
7.1.1	Clases de equivalencia y valores límite	135
7.1.2	Each Choice	135
7.1.3	Base Choice	135
7.1.4	Transición de estados	136
7.2	Pruebas unitarias	136
7.2.1	Aggregator del motor de verificación	137
7.2.2	Catálogo de reglas RV-01 a RV-10	138
7.2.3	Endpoints del backend FastAPI	141
7.2.4	Implementaciones de IConnector	142
7.2.5	Backend Python: capas de dominio, servicios y repositorios . . .	143
7.2.6	Frontend Angular: guards, interceptors, servicios y componentes	146
7.3	Pruebas de integración	152
7.3.1	Ciclo de vida de la release	152
7.3.2	Flujo completo, rate limiting, resiliencia y migración	153
7.3.3	Autenticación mediante API Key	153
7.4	Pruebas de aceptación y usabilidad	154
7.5	Pruebas de rendimiento	158
7.6	Pruebas de seguridad	159
7.7	Pruebas de usabilidad con usuarios	159
7.7.1	Objetivos	160
7.7.2	Perfil de los participantes	160
7.7.3	Diseño del experimento	161
7.7.4	Métricas e instrumentos	162
7.7.5	Resultados por participante	162
7.7.6	Resultados agregados y comparación entre perfiles	164
7.7.7	Hallazgos cualitativos	164
7.7.8	Conclusiones de la prueba de usabilidad	166
7.8	Ejecución y resultados	168
7.8.1	Inventario consolidado	168
7.8.2	Resultados por nivel	169
7.8.3	Cobertura medida y Quality Gate	169
7.8.4	Tests faltantes y deuda asumida	170
7.9	Matriz de trazabilidad y resumen	170
7.9.1	Matriz de trazabilidad requisitos – casos de prueba	171
7.9.2	Herramientas y entorno de ejecución	172
8	Manuales del Sistema	174

8.1	Manual de Instalación y Despliegue	174
8.1.1	Requisitos Previos	174
8.1.2	Obtención del Código Fuente	175
8.1.3	Configuración del Entorno	175
8.1.4	Despliegue con Docker Compose	178
8.1.5	Verificación del Despliegue	179
8.1.6	Despliegue en Producción	179
8.1.7	Actualización del Sistema	180
8.1.8	Resolución de Problemas Comunes	181
8.2	Manual de Usuario	182
8.2.1	Acceso al Sistema	182
8.2.2	Solicitud de Acceso al Sistema	182
8.2.3	Recuperación de Contraseña	183
8.2.4	Autenticación de Doble Factor	183
8.2.5	Selección de Organización y Proyecto	183
8.2.6	Panel Principal (Dashboard)	184
8.2.7	Gestión de Releases	184
8.2.8	Lanzamiento de una Verificación	186
8.2.9	Consulta de Resultados	187
8.2.10	Gestión del Perfil de Usuario	188
8.3	Manual de Administración	189
8.3.1	Gestión de la Organización	189
8.3.2	Gestión de Usuarios y Roles	190
8.3.3	Gestión de Conectores	190
8.3.4	Gestión de Perfiles de Verificación	192
8.3.5	Gestión de Notificaciones	192
8.3.6	Administración Global (Rol Administrador global)	193
9	Planificación y Presupuesto	195
9.1	Introducción	195
9.2	Planificación temporal	195
9.2.1	Marco metodológico	195
9.2.2	Estructura de descomposición del trabajo (EDT)	196
9.2.3	Hitos del proyecto	196
9.2.4	Planificación inicial (ideal)	197
9.2.5	Planificación real (ejecutada)	198
9.2.6	Comparativa Ideal vs. Real y desviaciones	198
9.3	Estudio de riesgos	199
9.3.1	Metodología y modelo de valoración	199

9.3.2	Distribución y priorización	200
9.3.3	Registro de riesgos	201
9.3.4	Plan de respuesta a los riesgos priorizados	202
9.3.5	Valor monetario esperado y reserva de contingencia	203
9.4	Presupuesto	204
9.4.1	Metodología	204
9.4.2	Definición de empresa	204
9.4.3	Presupuesto interno (de costes)	205
9.4.4	Presupuesto del cliente	208
9.4.5	Verificación contable	209
9.5	Conclusiones	210
10	Conclusiones y Trabajo Futuro	211
10.1	Conclusiones	211
10.1.1	Valoración global	211
10.1.2	Grado de cumplimiento de los objetivos	212
10.1.3	Del proceso manual al proceso automatizado	213
10.1.4	Cobertura de requisitos y arquitectura	215
10.1.5	Planificación, esfuerzo y desviaciones	215
10.1.6	Aprendizajes técnicos	216
10.2	Trabajo Futuro	217
10.2.1	Extensión funcional	218
10.2.2	Integración con pipelines CI/CD	218
10.2.3	Arquitectura, despliegue y rendimiento	218
10.2.4	Observabilidad y métricas	219
10.2.5	Análisis predictivo y aprendizaje automático	219
10.2.6	Seguridad, cumplimiento y comunidad	219
10.2.7	Resumen y priorización	220
10.3	Reflexión final	220
11	Referencias Bibliográficas	223
12	Apéndices	223
12.1	Contenido Entregado en Archivo Adjunto	224
12.2	Estructura general del entregable	224
12.3	Estructura del código fuente	224
12.4	Instrucciones de despliegue	225
12.5	Repositorio de código y trazabilidad	226
12.6	Estudio Detallado de Riesgos	226
12.7	Metodología	226

12.8	Registro de riesgos	227
12.8.1	R01 — Plazo cerrado del TFG sin posibilidad de extensión . . .	227
12.8.2	R02 — Subestimación de la complejidad del motor	228
12.8.3	R03 — Imposibilidad de alcanzar el <i>Quality Gate</i> del 80 % de cobertura	228
12.8.4	R04 — Pérdida de trabajo por fallo técnico	228
12.8.5	R05 — Disponibilidad limitada de la tutora	229
12.8.6	R06 — Falta de experiencia previa con Rust en proyectos de producción	229
12.8.7	R07 — Cambios significativos en requisitos durante el desarrollo	230
12.8.8	R08 — Dificultades de despliegue sobre arquitectura AArch64	230
12.8.9	R09 — Vulnerabilidades de seguridad detectadas tardíamente . .	230
12.8.10	R10 — Caída del servicio de Overleaf durante la redacción . . .	231
12.8.11	OP01 — Reutilización en futuros TFG o trabajos académicos . .	231
12.8.12	OP02 — Publicación del repositorio como referencia profesional .	232
12.9	Resumen y priorización	232
12.9.1	Distribución por bandas de exposición	232
12.9.2	Tabla resumen del registro de riesgos	233
12.9.3	Conclusiones del análisis de riesgos	234
12.10	Glosario y Acrónimos	235
12.11	Glosario de términos de dominio	235
12.12	Reglas de verificación	236
12.13	Acrónimos técnicos	237
12.14	Tecnologías y herramientas	237

Índice de figuras

1	Diagrama de casos de uso CU-01 — Verificar Release.	64
2	Diagrama de casos de uso CU-02 — Configurar Perfil de Verificación. . . .	67
3	DF-01. Flujo de autenticación.	71
4	DF-02. Ciclo de vida de una <i>release</i>	72
5	Diagrama de contexto del Sistema de Verificación Automática de Entregas de Software (SVAES) (C4 Nivel 1).	76
6	Diagrama de contenedores del SVAES (C4 Nivel 2).	77
7	Diagrama de componentes del SVAES (C4 Nivel 3).	78

8	Diagrama de despliegue del SVAES mediante Docker Compose.	79
9	Diagrama de clases del subsistema de conectores del SVAES.	82
10	Diagrama de secuencia DS-01: lanzamiento de una verificación.	83
11	Diagrama de secuencia DS-02: registro y activación de un conector.	84
12	Estructura interna del motor de verificación del SVAES.	86
13	Diagrama entidad-relación del SVAES (notación Crow's Foot). Se muestran las entidades del núcleo operativo del modelo; por claridad se omiten <code>user</code> y <code>user_membership</code> , que forman parte del núcleo pero cuyas relaciones con el resto del esquema son transversales. Las tablas <code>user</code> , <code>user_membership</code> , <code>api_key</code> , <code>custom_role</code> , <code>access_request</code> , <code>audit_log</code> , <code>notification_channel</code> , <code>notification_subscription</code> y <code>template</code> también forman parte del esquema (descritas en la Tabla 26) pero se omiten del diagrama por claridad.	93
14	Mapa de navegación del frontend del SVAES.	99
15	Wireframe de la vista de detalle de release.	100
16	Wireframe de la vista de gestión de conectores.	100
17	Wireframe de la vista de edición de perfil de verificación.	101
18	Wireframe de la vista Dashboard.	101
19	Diagrama de Gantt de la planificación inicial (ideal).	197
20	Diagrama de Gantt de la planificación real (ejecutada).	198
21	Matriz Probabilidad $\times$ Impacto con los riesgos y oportunidades identificados.	200
22	Distribución de riesgos y oportunidades por categoría.	200
23	Priorización por valor de riesgo ($P \cdot I$). Línea discontinua: umbral 0,15.	201
24	Presupuesto interno: barra apilada por partida.	205
25	Distribución relativa del coste interno por partida.	205
26	Partida 2 — Reparto entre predesarrollo, desarrollo y postdesarrollo.	207
27	Comparativa visual del presupuesto interno frente al del cliente.	209
28	Grado de cumplimiento de los objetivos específicos del proyecto.	212
29	Comparativa de Key Performance Indicators (KPIs) entre el proceso manual y el proceso automatizado con SVAES.	214
30	Cobertura de requisitos por categoría MoSCoW.	215
31	Distribución del esfuerzo por fase: planificado frente a real.	216
32	Hoja de ruta orientativa para la evolución de SVAES.	217

Índice de tablas

1	Stakeholders internos del SVAES.	22
2	Stakeholders externos del SVAES.	23
3	Usuarios del sistema SVAES.	24
4	Fuentes de información del SRS.	25
5	Requisitos no funcionales de rendimiento.	29
6	Requisitos no funcionales de disponibilidad.	29
7	Requisitos no funcionales de fiabilidad.	30
8	Requisitos no funcionales de seguridad.	31
9	Requisitos no funcionales de usabilidad.	31
10	Requisitos no funcionales de mantenibilidad.	32
11	Requisitos no funcionales de portabilidad.	32
12	Requisitos no funcionales de extensibilidad.	32
13	Requisitos no funcionales de trazabilidad.	33
14	Requisitos no funcionales organizacionales y externos.	33
15	Identificación del caso de uso CU-01.	64
16	Flujos alternativos del caso de uso CU-01.	66
17	Postcondición y trazabilidad del caso de uso CU-01.	66
18	Identificación del caso de uso CU-02.	68
19	Flujos alternativos del caso de uso CU-02.	69
20	Postcondición y trazabilidad del caso de uso CU-02.	69
21	Decisiones arquitectónicas del SVAES.	74
22	Stack tecnológico del SVAES.	75
23	Organización en capas de la arquitectura del SVAES.	82
24	Catálogo de reglas de verificación del motor SVAES.	89
25	Métodos del puerto IConnector.	91
26	Descripción de las diecisiete tablas del esquema relacional del SVAES.	98
27	Endpoints de la API REST pública del SVAES.	105
28	Matriz de trazabilidad diseño-requisitos.	107
29	Lenguajes de programación del SVAES.	110
30	Implementación de las reglas de verificación en Rust.	117

31	Catálogo de implementaciones de conectores del SVAES.	119
32	Servicios Docker Compose del SVAES.	125
33	Agrupación lógica de pruebas del backend Python por técnica de diseño. . .	129
34	Derivación <i>Base Choice</i> para <code>POST /releases</code>	136
35	Distribución de tests unitarios del motor Rust por regla.	139
36	Pruebas unitarias del <i>backend</i> Python por capa.	145
37	Distribución de archivos <code>spec.ts</code> del <i>frontend</i> Angular.	147
38	Suites de pruebas de aceptación.	155
39	Participantes en las pruebas de usabilidad.	161
40	Tareas planteadas en las pruebas de usabilidad.	161
41	Resultados por participante y tarea. Leyenda: ✓ éxito sin asistencia; A éxito con asistencia; ✗ no completada.	163
42	Puntuación System Usability Scale (SUS) por participante. Referencia: ≥ 68 se considera por encima de la media de la industria.	163
43	Métricas agregadas por perfil de participante.	164
44	Hallazgos cualitativos por participante.	166
45	Inventario consolidado de pruebas ejecutadas.	169
46	Resultados de ejecución agrupados por nivel.	169
47	Resumen del plan de pruebas por nivel, técnica y herramienta.	171
48	Matriz de trazabilidad requisitos – casos de prueba.	172
49	Requisitos previos del sistema anfitrión.	175
50	Variables de entorno del SVAES (las marcadas con * solo son obligatorias si se desea habilitar el envío de notificaciones por correo).	177
51	Servicios desplegados por Docker Compose ([†] el puerto de PostgreSQL solo se publica al host en desarrollo).	178
52	Resolución de problemas comunes durante el despliegue.	181
53	Estados del ciclo de vida de una release.	185
54	Reglas de verificación disponibles en el SVAES (RV-01 a RV-10), su severidad por defecto y nivel de aplicación.	187
55	Roles del sistema SVAES y sus capacidades principales.	189
56	Catálogo de conectores disponibles en el SVAES por categoría funcional. .	191
57	Estructura de descomposición del trabajo (resumen, nivel 3).	196
58	Hitos del proyecto (rama 1.1 de la EDT).	197
59	Desviaciones de la planificación real respecto a la línea base.	199
60	Registro de riesgos y oportunidades.	202
61	Plan de respuesta a los riesgos priorizados.	203
62	Cálculo de EMV para los riesgos priorizados con mayor exposición.	203

63	Parámetros de la definición de empresa.	204
64	Presupuesto interno por partida.	205
65	Partida 1 — Hardware y licencias.	206
66	Partida 2 — Detalle por sub-bloque.	207
67	Partida 5 — Composición de la reserva de contingencia.	208
68	Presupuesto del cliente (sin IVA).	209
69	Evidencia del cumplimiento de los objetivos específicos.	213
70	Comparativa cuantitativa entre verificación manual y SVAES.	214
71	Priorización de las líneas de trabajo futuro.	220
72	Estructura del material entregado.	224
73	Estructura del directorio de código fuente.	225
74	Matriz probabilidad-impacto (valores de exposición).	227
75	Resumen del registro de riesgos ordenado por exposición decreciente. . . .	234

LISTINGS

1	Estructura de directorios del backend SVAES.	110
2	Estructura del caso de uso <code>LaunchVerification</code>	112
3	<i>Endpoint</i> del motor de verificación.	115
4	Rasgo <code>Rule</code> del motor de verificación.	116
5	Puerto <code>IConnector</code>	118
6	Configuración de cobertura en <code>pyproject.toml</code> del backend Python. . . .	130
7	Exclusiones de cobertura en <code>sonar-project.properties</code>	131
8	Comando de cobertura del motor Rust con cargo <code>llvm-cov</code>	131

ACRÓNIMOS

ALM	Application Lifecycle Management. vii , 19 , 196
API	Application Programming Interface. vii , ix , x , xvi , 3 , 4 , 10 , 12 , 13 , 17 , 25 , 26 , 28–30 , 33 , 44 , 60 , 74 , 75 , 91 , 97 , 102 , 105 , 109 , 110 , 113 , 123 , 125 , 130 , 142 , 144 , 153 , 154 , 158 , 171 , 174 , 178 , 179 , 186 , 188 , 194 , 218 , 225
CI	Continuous Integration. 165
CORS	Cross-Origin Resource Sharing. 176 , 177
CSP	Content Security Policy. 180
DOM	Document Object Model. 150 , 151
HTTP	Hypertext Transfer Protocol. x , 17 , 29 , 113–116 , 124 , 133 , 136 , 141–143 , 146 , 148 , 150–153 , 158 , 159 , 173 , 175 , 179 , 180 , 186
HTTPS	Hypertext Transfer Protocol Secure. 175 , 180 , 191
JSON	JavaScript Object Notation. 17 , 116 , 148
JWT	JSON Web Token. 75 , 77 , 80 , 81 , 102 , 105 , 107 , 113 , 115 , 123 , 124 , 130 , 133 , 159 , 172 , 176–178 , 194 , 230
KPI	Key Performance Indicator. xv , 3 , 101 , 213 , 214 , 219

RBAC	Role-Based Access Control. 26, 39, 75, 77, 81, 107, 123, 133, 199, 211
REST	Representational State Transfer. vii, ix, x, xvi, 3, 4, 12, 17, 25, 26, 28–30, 33, 44, 60, 74, 91, 102, 105, 109, 110, 113, 125, 158, 174, 178, 179, 188, 194, 218, 225
SRS	Software Requirements Specification. 20, 134, 137, 170
SUS	System Usability Scale. xvii, 162, 163, 167, 168
SVAES	Sistema de Verificación Automática de Entregas de Software. xiv–xix, 10, 12, 20–26, 28, 31, 43, 44, 46, 60, 62, 63, 70, 73–82, 85, 86, 89–93, 98, 99, 102, 105, 107, 109–111, 119, 125, 134–136, 143, 167, 168, 174, 175, 177, 179–182, 187, 189–192, 194, 195, 210, 211, 214, 216–218, 224, 235
TFG	Trabajo Fin de Grado. 135

1 Memoria del Proyecto

1.1. Resumen de la Motivación, Objetivos y Alcance del Proyecto

Este proyecto surge de una problemática presente en organizaciones que desarrollan software en entornos de cierta complejidad: la ausencia de mecanismos automáticos que permitan verificar, de forma integrada y auditable, que una entrega de software cumple los requisitos definidos antes de avanzar en el flujo de desarrollo. Dicha necesidad fue identificada por primera vez durante las prácticas en empresa realizadas en Indra Sistemas, S.A., constituyendo el punto de partida del presente trabajo.

En estos entornos, los componentes software se desarrollan y entregan en el contexto de proyectos distintos, cada uno con sus propias herramientas, repositorios y procedimientos. Esta heterogeneidad genera ineficiencias operativas y riesgos de inconsistencia entre los datos registrados en los diferentes sistemas implicados en el ciclo de vida del software, especialmente cuando la información relativa a una misma entrega se encuentra distribuida entre herramientas de gestión de cambios, seguimiento de tareas, planificación de versiones y gestión documental.

El proceso de validación habitual en este tipo de entornos se realiza de forma manual y sigue una secuencia de comprobaciones entre distintas herramientas. En primer lugar, el responsable técnico revisa la petición de cambio o incidencia registrada en el sistema de gestión correspondiente, verificando su estado, alcance y correspondencia con la entrega. A continuación, accede a la herramienta de seguimiento de tareas, donde valida la existencia de los elementos de trabajo asociados, su correcta vinculación con la petición de cambio y su grado de completitud, paso clave para asegurar la trazabilidad entre requisitos, desarrollo y entrega. Posteriormente, se consulta el plan de construcción de la versión para comprobar que los componentes incluidos en la entrega se corresponden con la planificación prevista. Finalmente, se revisa la documentación asociada en el repositorio documental, verificando que los documentos requeridos están presentes, actualizados y aprobados conforme al proceso definido.

Durante todo este proceso, el responsable técnico debe contrastar manualmente la coherencia de la información entre las distintas herramientas, asegurando que existe alineación entre la petición de cambio, las tareas de desarrollo, la planificación de la *release* y la documentación asociada. Este procedimiento consume un tiempo considerable en cada ciclo de entrega y expone el proceso a errores humanos que pueden comprometer la trazabilidad

y la calidad de las entregas. Asimismo, la ausencia de un registro sistemático de las verificaciones realizadas dificulta la obtención de métricas fiables del proceso y limita la capacidad de mejora continua.

El **objetivo principal** de este Trabajo Fin de Grado es diseñar e implementar, desde cero, un sistema centralizado de verificación automática de entregas de software. El sistema se integrará con herramientas externas de gestión de proyectos y documentación mediante sus [API REST](#), y permitirá comprobar de forma automática y auditable que cada entrega cumple los criterios de coherencia e integridad definidos antes de avanzar en el flujo del proyecto. Con ello se pretende reducir el tiempo dedicado a la validación manual, minimizar los errores derivados de la intervención humana, mejorar la trazabilidad entre los distintos sistemas del ciclo de desarrollo y establecer una base de registro histórico que permita la medición y mejora continua del proceso.

El **alcance** del proyecto comprende el análisis del proceso de validación manual existente, la identificación de sus limitaciones, el diseño de la arquitectura del nuevo sistema, la implementación de los módulos de integración con las herramientas externas mediante [API REST](#), el desarrollo de la lógica de verificación automática, la definición de indicadores de rendimiento (**KPIs**) del proceso y la validación del sistema mediante pruebas funcionales sobre entornos controlados. Queda fuera del alcance las modificaciones sobre los sistemas externos con los que el sistema se integra. La delimitación detallada de la solución, incluyendo las funcionalidades incluidas y excluidas, se desarrolla en la sección [2.8](#) del capítulo [2](#).

1.2. Estructura de la Documentación

A continuación se describe brevemente el contenido de cada capítulo de esta memoria, con el fin de facilitar la navegación por el documento y proporcionar al lector una visión global del trabajo realizado.

Capítulo 1 — Memoria del Proyecto. El presente capítulo recoge el resumen ejecutivo del proyecto: motivación, objetivo principal, alcance y estructura de la documentación.

Capítulo 2 — Introducción. Presenta en detalle la justificación del proyecto y los objetivos específicos que se persiguen. Incluye el estudio de la situación actual, con la descripción del proceso de validación manual, las herramientas y roles involucrados, el diagrama de flujo del proceso, los indicadores [KPI](#) identificados y las limitaciones detectadas que motivan el desarrollo del sistema.

Capítulo 3 — Marco Teórico y Estado del Arte. Expone los fundamentos conceptuales y tecnológicos sobre los que se sustenta el proyecto: integración de herramientas mediante [API REST](#), patrones de verificación automática en el ciclo de vida del software y revisión del estado del arte en sistemas de verificación de entregas en el ámbito académico e industrial.

Capítulo 4 — Especificación de Requisitos del Sistema. Contiene el catálogo completo de requisitos funcionales y no funcionales del sistema, incluyendo casos de uso y criterios de aceptación.

Capítulo 5 — Diseño del Sistema. Describe la arquitectura del sistema, los componentes principales, los diagramas de clases y de secuencia, el diseño de la base de datos y la interfaz de usuario, así como las decisiones de diseño adoptadas.

Capítulo 6 — Implementación. Explica los lenguajes, herramientas y tecnologías utilizadas, la estructura del código desarrollado y los aspectos clave de la integración con las herramientas externas mediante sus [API REST](#).

Capítulo 7 — Pruebas. Recoge el plan de pruebas, los tipos de prueba realizados y los resultados obtenidos en relación con los requisitos definidos.

Capítulo 8 — Manuales. Incluye los manuales de instalación, configuración y uso del sistema.

Capítulo 9 — Planificación y Presupuesto. Presenta la planificación temporal, el análisis de riesgos y la estimación de costes del proyecto.

Capítulo 10 — Conclusiones y Trabajo Futuro. Resume los resultados obtenidos, las conclusiones del trabajo y posibles líneas de evolución futura.

Referencias bibliográficas. Lista las fuentes utilizadas a lo largo del documento.

Anexos. Incluyen documentación complementaria de apoyo al proyecto.

2 Introducción

2.1. Justificación del Proyecto

El desarrollo de software moderno se apoya habitualmente en un conjunto de herramientas especializadas que cubren distintas necesidades del ciclo de vida del sistema, como la gestión de tareas, el control de versiones, la documentación técnica, la planificación de entregas o la ejecución de pruebas. Esta especialización aporta flexibilidad y capacidad de adaptación, pero también introduce un problema recurrente: la información necesaria para Verificar una entrega de software queda distribuida entre varias fuentes heterogéneas.

En este contexto, determinar si una entrega puede considerarse correcta no depende únicamente del estado del código fuente o del resultado de una compilación, sino también de la coherencia entre distintos artefactos asociados al proceso de desarrollo. Una entrega válida debe poder justificarse a partir de la correspondencia entre las tareas planificadas, los cambios implementados, la documentación disponible y los elementos registrados en los sistemas que intervienen en el flujo de trabajo.

Cuando esta comprobación se realiza de forma manual, el proceso resulta costoso, repetitivo y propenso a errores. La persona encargada de la validación debe consultar múltiples fuentes, contrastar la información entre ellas y decidir si existen inconsistencias que impidan aceptar la entrega. A medida que crece la complejidad del sistema o aumenta la frecuencia de las entregas, esta forma de trabajo se vuelve poco eficiente y difícilmente escalable.

Ante esta situación, la automatización de la verificación de entregas constituye una línea de mejora clara desde el punto de vista de la ingeniería del software [10]. Disponer de un sistema capaz de recopilar información desde varias fuentes, aplicar reglas de coherencia de forma sistemática y generar un resultado verificable permitiría reducir la dependencia del trabajo manual, homogeneizar el proceso de revisión y mejorar la fiabilidad del resultado final.

Este Trabajo Fin de Grado se justifica, por tanto, por la necesidad de diseñar e implementar una solución software que aborde este problema desde un enfoque general y reutilizable, sin depender de un entorno corporativo concreto. El valor del proyecto no reside únicamente en automatizar una secuencia de comprobaciones, sino en proponer una arquitectura extensible que permita modelar el problema de la validación de entregas como un proceso formal, reproducible y soportado por software.

Cabe señalar que la propuesta de este trabajo tuvo su origen en el contexto de las prácticas en empresa realizadas en Indra Sistemas, S.A., donde se detectó la existencia de una

necesidad organizativa real relacionada con la validación sistemática de entregas en entornos de desarrollo de cierta complejidad. Debido a dificultades operativas surgidas durante el transcurso de las prácticas, se procedió a la desvinculación formal tanto del convenio de prácticas como del Trabajo Fin de Grado inicialmente asociado a la empresa. El trabajo ha sido continuado, a partir de ese momento, de forma individual e independiente y con orientación *open source*, sin participación de la empresa en ninguna fase de su ejecución. La validación del sistema se ha realizado íntegramente sobre un entorno simulado construido por el autor, sin acceso a sistemas ni infraestructuras corporativas. El proyecto conserva así el valor de abordar una necesidad industrial real, al tiempo que ofrece una solución genérica, reutilizable y tecnológicamente neutral.

2.2. Objetivos del Proyecto

El objetivo principal del proyecto es diseñar e implementar un sistema centralizado de verificación automática de entregas de software, capaz de obtener información desde múltiples fuentes, evaluar su coherencia mediante reglas definidas y emitir un resultado de validación de forma automática y reproducible.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Definir un modelo de referencia para el proceso de validación de entregas de software, identificando las fuentes de información, los artefactos implicados y las comprobaciones necesarias para determinar la validez de una entrega.
- Analizar las limitaciones de un proceso manual de validación cuando la información se encuentra distribuida entre varias herramientas o sistemas.
- Diseñar la arquitectura del sistema propuesto, incluyendo sus módulos de integración, el núcleo de verificación y el mecanismo de persistencia de resultados.
- Implementar conectores o mecanismos de acceso a las fuentes de información que intervienen en la validación, utilizando interfaces programáticas adecuadas para su consulta automática.
- Desarrollar un conjunto de reglas de verificación que permitan comprobar la integridad y coherencia de una entrega de software.
- Proporcionar una interfaz o mecanismo de consulta que permita visualizar el resultado de las validaciones realizadas y el detalle de las incidencias detectadas.
- Validar el funcionamiento del sistema en un entorno controlado, comprobando que el comportamiento obtenido se ajusta a los requisitos definidos.

2.3. Planteamiento del Problema

El problema abordado en este trabajo puede formularse de la siguiente manera: dado un conjunto de artefactos relacionados con una entrega de software y distribuidos entre distintas fuentes de información, determinar automáticamente si dicha entrega cumple las condiciones de coherencia e integridad necesarias para considerarse válida.

Este problema presenta varias dificultades. En primer lugar, la información relevante no se encuentra centralizada en un único sistema. En segundo lugar, los artefactos que deben compararse no son homogéneos, ya que pueden pertenecer a categorías distintas, como tareas, documentos o registros de entrega. En tercer lugar, la validación no consiste simplemente en comprobar la existencia de estos elementos, sino en verificar las relaciones que deben mantenerse entre ellos.

Por ello, el problema no puede resolverse únicamente mediante automatismos aislados o consultas puntuales, sino que requiere una solución estructurada capaz de:

1. Recopilar información desde varias fuentes.
2. Transformarla a un modelo interno común.
3. Aplicar reglas de validación sobre ese modelo.
4. Registrar y presentar el resultado obtenido.

Este planteamiento sitúa el proyecto dentro del ámbito de la integración de sistemas, la automatización de procesos y la verificación de artefactos software.

2.4. Escenario de Referencia

Para abordar el problema de forma sistemática, el proyecto parte de un escenario de referencia que representa una situación habitual en el desarrollo software: una entrega debe ser validada a partir de la información disponible en varias fuentes, sin asumir que todas ellas pertenecen al mismo ecosistema ni que comparten un modelo de datos común.

En este escenario, una entrega puede estar asociada, de manera general, a los siguientes elementos:

- Un identificador o referencia de entrega.
- Uno o varios elementos de trabajo asociados al cambio desarrollado.
- Documentación técnica o funcional relacionada.

- Información de versión o planificación.
- Un conjunto de estados o condiciones que deben cumplirse antes de aceptar la entrega.

La validación manual de este tipo de entregas exige consultar cada fuente de información, comprobar la existencia de los artefactos necesarios y contrastar si los datos son coherentes entre sí. El sistema desarrollado en este TFG toma este escenario como base para definir un modelo automatizable del proceso de validación.

2.5. Proceso Manual de Referencia

Antes de automatizar la validación, resulta útil describir cómo se realizaría este proceso de forma manual en un escenario genérico como el definido anteriormente.

En primer lugar, la persona encargada de revisar la entrega identifica la referencia principal sobre la que debe trabajar, por ejemplo un identificador de versión, de cambio o de entrega. A partir de esa referencia, consulta las distintas fuentes de información disponibles para localizar los artefactos asociados.

Una vez localizados, se comprueba que los elementos requeridos existen y que la información registrada mantiene una relación coherente. Entre otras verificaciones, este proceso puede incluir la comprobación de que una tarea asociada está finalizada, de que la documentación correspondiente existe, o de que la versión indicada en una fuente coincide con la registrada en otra.

Finalmente, la persona responsable determina si la entrega puede darse por válida o si, por el contrario, existen inconsistencias que obligan a revisarla o corregirla antes de continuar.

Aunque este procedimiento resulta conceptualmente sencillo, su ejecución manual presenta varias limitaciones: requiere tiempo, depende de la atención del revisor, no siempre se realiza de forma uniforme y dificulta la repetibilidad del proceso.

2.6. Fuentes de Información Implicadas

En un proceso de validación de entregas, la información necesaria puede proceder de varios tipos de fuentes. Sin vincular el proyecto a herramientas concretas, las categorías más relevantes son las siguientes.

Fuentes de seguimiento de trabajo. Incluyen sistemas donde se registran elementos como incidencias, tareas, historias de usuario o actividades asociadas al desarrollo. Estas fuentes permiten comprobar si los cambios vinculados a la entrega han sido correctamente registrados y cuál es su estado.

Fuentes documentales. Incluyen repositorios donde se almacena documentación técnica, funcional o de pruebas. Su consulta permite verificar si la documentación asociada a la entrega existe y se encuentra alineada con el cambio realizado.

Fuentes de planificación o versionado. Permiten conocer a qué versión, iteración o ciclo pertenece una entrega y si dicha asignación es coherente con el resto de artefactos relacionados.

Fuentes de persistencia interna del sistema. Además de las fuentes externas, el sistema desarrollado requiere una fuente de persistencia propia donde almacenar el historial de validaciones realizadas, los resultados obtenidos y el detalle de las incidencias detectadas.

2.7. Limitaciones del Enfoque Manual

El análisis del proceso manual de referencia permite identificar varias limitaciones que justifican la construcción del sistema propuesto.

En primer lugar, la validación manual implica un esfuerzo repetitivo que aumenta con el número de entregas y con la cantidad de fuentes consultadas. Cada nueva validación exige repetir comprobaciones similares, lo que introduce un coste operativo que no aporta valor directo al producto software.

En segundo lugar, la revisión manual depende del criterio y de la atención de la persona que la realiza. Esto puede provocar diferencias entre validaciones equivalentes, omisiones involuntarias o interpretaciones distintas de una misma situación.

En tercer lugar, la falta de automatización dificulta la trazabilidad del propio proceso de validación. Si no existe un sistema que registre de forma estructurada qué se ha comprobado, cuándo y con qué resultado, resulta más complejo auditar el proceso o analizar incidencias posteriores.

Por último, el enfoque manual presenta problemas de escalabilidad. Un procedimiento que puede resultar asumible en un escenario reducido deja de serlo cuando el número de entregas aumenta o cuando las fuentes de información se multiplican.

Estas limitaciones hacen razonable plantear una solución software que formalice el proceso, reduzca la intervención manual y permita aplicar las mismas reglas de forma consistente en todas las validaciones.

2.8. Ámbito y Alcance del Sistema

El [SVAES](#) es un sistema centralizado de verificación automática de entregas de software que actúa como mecanismo de control de calidad (*quality gate*), validando la integridad, coherencia y completitud de los artefactos asociados a una *release* mediante la consulta de múltiples fuentes externas. El sistema no pretende sustituir a las herramientas externas utilizadas en el proceso de desarrollo, sino actuar como una capa intermedia de consulta, validación y registro, reduciendo la intervención manual, minimizando errores y mejorando la trazabilidad entre los distintos elementos del ciclo de vida del software.

Funcionalidades principales del sistema. El sistema [SVAES](#) incluye las siguientes funcionalidades principales:

- Gestión de organizaciones y aislamiento de datos entre ellas.
- Gestión de proyectos y *releases* de software.
- Integración con sistemas externos mediante conectores.
- Ejecución automática de verificaciones sobre los artefactos de una *release*.
- Definición y aplicación de reglas de verificación configurables.
- Gestión de perfiles de verificación y plantillas de *releases*.
- Visualización de resultados de verificación e historial asociado.
- Generación de indicadores y estadísticas del proceso de validación.
- Sistema de notificaciones sobre eventos relevantes del sistema.
- Exposición de una [API](#) para consulta de resultados.

Dentro del alcance. Dentro del alcance del proyecto se incluyen:

- La definición de un modelo interno de entrega y de los artefactos asociados.
- El acceso automatizado a las fuentes de información seleccionadas.

- La aplicación de reglas de validación sobre la información recopilada.
- El almacenamiento de resultados e incidencias.
- La consulta de las validaciones realizadas.

Fuera del alcance. Quedan fuera del alcance del proyecto:

- La modificación del comportamiento interno de herramientas externas.
- La automatización del desarrollo, compilación o despliegue del software validado.
- La resolución automática de inconsistencias detectadas.
- La integración con un número ilimitado de fuentes, más allá de las contempladas en la implementación realizada.

La solución propuesta no está vinculada a ninguna herramienta, plataforma o proveedor concreto: las fuentes externas con las que el sistema se integra se abstraen mediante conectores configurables, lo que permite adaptar la solución a distintos ecosistemas de desarrollo software.

2.9. Herramientas y Soluciones Similares

Antes de definir la solución propuesta, resulta conveniente revisar qué tipos de herramientas abordan parcialmente problemas relacionados con la automatización del ciclo de vida del software.

Las plataformas de integración y entrega continua [3, 5] permiten automatizar tareas como la compilación, la ejecución de pruebas o el despliegue. Su principal fortaleza reside en el control técnico del pipeline de desarrollo. Sin embargo, su enfoque se sitúa principalmente en el código y en los artefactos de construcción, no en la coherencia global entre múltiples fuentes de información heterogéneas.

Por su parte, las plataformas de gestión del ciclo de vida del software ofrecen funcionalidades de trazabilidad y organización del trabajo dentro de un ecosistema determinado. No obstante, cuando la información se encuentra distribuida entre varios sistemas, su capacidad para verificar la coherencia entre fuentes externas resulta más limitada.

También existen soluciones orientadas a la automatización general de procesos. Estas herramientas permiten orquestar acciones entre aplicaciones, pero no suelen incorporar

un modelo específico para la validación de entregas software ni una lógica centrada en la coherencia entre artefactos del proceso de desarrollo.

A partir de esta revisión se observa que existe espacio para una solución específica centrada en la verificación automática de entregas a partir de múltiples fuentes de información, con una lógica de validación propia y desacoplada de un ecosistema único.

2.10. Análisis de Alternativas Tecnológicas

El diseño del [SVAES](#) requiere tomar decisiones tecnológicas en seis dimensiones independientes: el framework de backend, el lenguaje del motor de verificación, el framework de frontend, el mecanismo de ejecución asíncrona, el sistema gestor de base de datos y la estrategia de contenerización. Esta sección analiza las alternativas más relevantes en cada dimensión y justifica la opción adoptada.

Framework de backend: Flask frente a FastAPI. Flask es un microframework minimalista ampliamente utilizado en proyectos Python de alcance acotado. Su principal ventaja es la simplicidad inicial: no impone estructura y permite arrancar con pocas dependencias. Sin embargo, esta misma característica supone una limitación cuando el sistema expone una [API REST](#) documentada, con múltiples endpoints tipados y validación de esquema en entrada y salida. En ese contexto, Flask obliga a incorporar bibliotecas adicionales (Marshmallow, Flask-RESTX o similares) para cubrir funcionalidades que FastAPI proporciona de forma nativa: validación automática mediante Pydantic, generación del contrato OpenAPI sin configuración manual y soporte asíncrono integrado en el framework. FastAPI genera además la documentación interactiva (Swagger UI y Redoc) a partir del código, garantizando que el contrato [API](#) refleja siempre la implementación real. Para un sistema cuyo frontend consume la [API](#) mediante un cliente tipado generado desde la especificación OpenAPI, esta propiedad elimina una clase entera de errores de integración. Por estas razones se descarta Flask en favor de FastAPI como framework de backend.

Motor de verificación: Python frente a Rust. La alternativa más inmediata para el motor de verificación es implementarlo directamente en Python, integrado en el mismo proceso que el backend FastAPI. Este enfoque simplifica el despliegue y elimina la necesidad de un proceso independiente. No obstante, el motor de verificación es el componente más crítico del sistema en cuanto a corrección y predecibilidad de rendimiento: debe evaluar en paralelo múltiples reglas sobre los artefactos de una release, garantizando que el tiempo de verificación no supera el umbral establecido en los requisitos no funcionales independientemente de la carga del servidor. Python, al estar sujeto al Global Interpreter Lock

(GIL), no puede ejecutar código CPU-intensivo en paralelo real dentro del mismo proceso; el paralelismo real requiere multiproceso, lo que introduce complejidad de comunicación y serialización. Rust, en cambio, ofrece paralelismo de datos seguro sin GIL mediante el crate Rayon, garantías de seguridad en memoria verificadas en tiempo de compilación y latencias predecibles al no disponer de recolector de basura. Dado que el motor opera exclusivamente sobre datos en memoria (los conectores en Python recuperan los datos de los artefactos antes de invocar al motor), no es necesario que Rust acceda a la red ni a la base de datos, lo que simplifica su modelo de prueba. El motor se despliega como proceso independiente e interactúa con el backend mediante una interfaz bien definida. Se adopta Rust para el motor de verificación por sus propiedades de paralelismo, seguridad en memoria y rendimiento predecible.

Framework de frontend: Vue frente a Angular frente a React. El sistema requiere una interfaz de administración con múltiples roles de usuario, formularios complejos de configuración de conectores y reglas, y visualización de resultados de verificación con detalle por regla. React es una biblioteca de composición de componentes que requiere decisiones adicionales sobre enrutado, gestión de estado y validación de formularios. Vue ofrece un marco más integrado, pero su ecosistema TypeScript es menos maduro que el de Angular. Angular proporciona un framework completo con TypeScript nativo, inyección de dependencias integrada, módulo de formularios reactivos y soporte de generación automática de clientes REST tipados desde especificaciones OpenAPI mediante OpenAPI Generator. Para una aplicación de gestión con múltiples roles y un contrato API bien definido, la generación automática del cliente elimina la duplicación de tipos entre frontend y backend y reduce el riesgo de desincronización. Se adopta Angular como framework de frontend.

Ejecución asíncrona: ejecución síncrona en petición HTTP frente a cola de tareas. Las verificaciones implican llamadas a [APIs](#) externas con latencias variables e impredecibles, condicionadas por la red y el estado de los sistemas externos. Ejecutar estas llamadas de forma síncrona dentro de la petición HTTP produciría timeouts en el cliente y bloquearía el proceso del servidor durante el tiempo de espera. La alternativa es desacoplar la ejecución mediante una cola de tareas: el backend recibe la petición, encola el trabajo y responde de forma inmediata con un identificador de tarea; el worker procesa la verificación en background y el frontend consulta el estado mediante polling. Se evalúan Celery con broker Redis y RQ (Redis Queue). Celery es la solución más madura del ecosistema Python, con soporte de reintentos, prioridades, tareas periódicas y monitorización mediante Flower. RQ es más simple pero carece de varias de estas funcionalidades. Se adopta Celery con Redis como broker y almacén de estado de tareas.

Sistema gestor de base de datos: SQLite frente a PostgreSQL. SQLite es adecuado para prototipos o aplicaciones de un único usuario, pero presenta limitaciones en escritura concurrente que lo hacen inadecuado para un sistema con workers Celery ejecutando verificaciones en paralelo y un backend atendiendo peticiones simultáneas. PostgreSQL ofrece transacciones ACID, soporte del tipo de columna JSONB para almacenar configuraciones de conectores con esquema variable y resultados de verificación sin necesidad de tablas auxiliares, tipos enumerados nativos para los estados del dominio, y soporte de índices GIN sobre columnas JSONB para búsquedas eficientes sobre el contenido de los resultados históricos. Se adopta PostgreSQL 17 como sistema gestor de base de datos.

Contenerización y despliegue: instalación directa frente a Docker Compose. El sistema integra cinco servicios que deben ejecutarse de forma coordinada: el backend FastAPI, el frontend Angular servido por nginx, el motor Rust, el worker Celery y la base de datos PostgreSQL, con Redis como broker. Gestionar estas dependencias mediante instalación directa en el sistema operativo del host introduce complejidad de configuración y dificulta la reproducibilidad del entorno. Docker Compose permite declarar todos los servicios, sus dependencias de arranque, los volúmenes persistentes y la red interna en un único fichero versionado. El entorno de desarrollo es así reproducible con un único comando, y la incorporación de nuevos desarrolladores no requiere configuración manual de dependencias. Se adopta Docker Compose como mecanismo de contenerización y orquestación del entorno local.

En conjunto, las decisiones adoptadas conducen a un stack basado en FastAPI y Python para el backend, Rust para el motor de verificación, Angular para el frontend, Celery con Redis para la ejecución asíncrona, PostgreSQL para la persistencia y Docker Compose para el despliegue. Todas las tecnologías seleccionadas son software libre, con licencias compatibles entre sí y con el carácter abierto del proyecto.

3 Marco Teórico y Estado del Arte

El presente capítulo establece los fundamentos conceptuales y tecnológicos sobre los que se sustenta el sistema desarrollado en este Trabajo Fin de Grado. A diferencia de enfoques basados en entornos corporativos concretos, el sistema propuesto se define de forma independiente, partiendo de principios generales de ingeniería del software y de la necesidad de automatizar la verificación de entregas en contextos heterogéneos.

El capítulo se divide en dos bloques principales. En primer lugar, se presenta el marco teórico, donde se describen los conceptos fundamentales relacionados con el ciclo de vida del software, la trazabilidad de artefactos, la integración de sistemas y la verificación automática. En segundo lugar, se expone el estado del arte, situando el trabajo dentro del contexto académico e industrial de la automatización del desarrollo software.

El objetivo de este capítulo es proporcionar el contexto necesario para comprender el problema abordado y justificar las decisiones de diseño adoptadas en el sistema propuesto.

3.1. Ciclo de Vida del Software y Gestión de Entregas

El desarrollo de software se organiza habitualmente en torno a un ciclo de vida que abarca desde la definición de requisitos hasta la entrega y mantenimiento del sistema [12]. Dentro de este ciclo, la fase de entrega constituye un punto crítico, ya que implica la validación de que el software cumple con los criterios establecidos antes de su despliegue o liberación.

En entornos reales, una entrega software no se limita únicamente al código fuente, sino que incluye múltiples artefactos asociados, como:

- Requisitos funcionales y no funcionales.
- Tareas de desarrollo y seguimiento.
- Documentación técnica.
- Resultados de pruebas.
- Información de planificación.

La validación de una entrega implica comprobar la coherencia entre todos estos elementos, asegurando que existe alineación entre lo que se ha definido, lo que se ha implementado y lo que se entrega.

Sin embargo, esta validación suele realizarse de forma manual, lo que introduce problemas de eficiencia, errores humanos y falta de trazabilidad.

3.2. Trazabilidad de Artefactos Software

La trazabilidad es un concepto fundamental en ingeniería del software que hace referencia a la capacidad de relacionar distintos artefactos a lo largo del ciclo de vida del sistema [12].

La trazabilidad permite establecer vínculos entre:

- Requisitos y tareas de desarrollo.
- Tareas y código fuente.
- Código y pruebas.
- Entregas y documentación asociada.

El objetivo de la trazabilidad es garantizar la consistencia del sistema y facilitar tareas como la validación, el mantenimiento o la auditoría.

No obstante, en muchos entornos la información se encuentra distribuida en múltiples fuentes o sistemas, lo que dificulta la verificación global de la coherencia de los datos.

3.3. Integración de Sistemas y Fuentes de Información

En la actualidad, el desarrollo de software se apoya en múltiples herramientas especializadas, cada una orientada a un aspecto concreto del proceso, como gestión de código, seguimiento de tareas o documentación.

Esto genera entornos heterogéneos en los que la información relevante para una entrega está distribuida en diferentes sistemas. Para poder trabajar con esta información de forma conjunta es necesario aplicar mecanismos de integración.

La integración de sistemas permite:

- Acceder a datos desde múltiples fuentes.
- Unificar la información en un modelo común.
- Automatizar procesos de validación.

Este enfoque es clave en el sistema propuesto, que actúa como una capa intermedia encargada de recopilar y analizar información procedente de distintas fuentes.

3.4. APIs REST y Acceso a Información

Uno de los mecanismos más utilizados para la integración de sistemas es el uso de [APIs REST](#). Este tipo de interfaces permite acceder a recursos de forma programática mediante el protocolo [HTTP](#).

Las operaciones principales en una [API REST](#) son:

- GET: obtención de información.
- POST: creación de recursos.
- PUT: actualización de recursos.
- DELETE: eliminación de recursos.

Las [APIs REST](#) utilizan habitualmente formatos como [JavaScript Object Notation \(JSON\)](#) para el intercambio de datos, lo que facilita la interoperabilidad entre sistemas.

El uso de [APIs](#) permite desacoplar los sistemas, evitando dependencias directas con bases de datos o implementaciones internas, y facilitando la automatización de procesos.

3.5. Verificación Automática Basada en Reglas

La verificación automática consiste en evaluar si un conjunto de datos cumple una serie de condiciones predefinidas sin intervención manual.

En el contexto de este trabajo, la verificación se basa en la definición de reglas que comprueban la coherencia entre los distintos elementos de una entrega software.

Estas reglas pueden incluir, por ejemplo:

- Existencia de artefactos obligatorios.
- Correspondencia entre elementos relacionados.
- Validación de estados o condiciones.

Este enfoque permite transformar un proceso manual en un proceso automático, reproducible y auditable.

3.6. Estado del Arte en Automatización del Desarrollo Software

La automatización del ciclo de vida del software ha sido uno de los principales focos de evolución en la industria en los últimos años. Este proceso se ha materializado principalmente a través de prácticas como la integración continua y la entrega continua.

3.6.1. Integración Continua y Entrega Continua

El paradigma de Continuous Integration y Continuous Delivery propone la automatización de las fases de construcción, prueba y despliegue del software mediante pipelines [3, 5, 11].

Estos pipelines permiten:

- Integrar cambios de forma frecuente.
- Ejecutar pruebas automáticamente.
- Detectar errores de forma temprana.

No obstante, estos enfoques se centran principalmente en el código y en los aspectos técnicos del sistema, dejando en segundo plano la verificación de la coherencia entre artefactos de gestión.

3.6.2. Quality Gates

Los quality gates son puntos de control dentro del proceso de desarrollo que verifican el cumplimiento de ciertos criterios antes de permitir avanzar a la siguiente fase [3, 7].

Estos criterios suelen estar relacionados con:

- Calidad del código.
- Cobertura de pruebas.
- Ausencia de errores críticos.

El concepto de quality gate es relevante para este trabajo, ya que el sistema propuesto puede interpretarse como un mecanismo de control aplicado a la coherencia de la entrega.

3.6.3. Plataformas ALM

Las plataformas de gestión del ciclo de vida del software (ALM) integran diferentes funcionalidades en un único entorno, facilitando la trazabilidad entre artefactos.

Sin embargo, en muchos casos la información no está completamente centralizada, sino distribuida entre múltiples herramientas, lo que requiere mecanismos adicionales de verificación.

3.6.4. Minería de Repositorios Software

La minería de repositorios software analiza los datos generados durante el desarrollo con el objetivo de extraer conocimiento útil.

Estos enfoques demuestran que la información del ciclo de vida del software puede ser procesada automáticamente, lo que refuerza la viabilidad de sistemas de verificación automatizada.

3.6.5. Automatización de Procesos

Existen diversas tecnologías orientadas a la automatización de procesos, como herramientas de integración o sistemas de workflow.

Sin embargo, estas soluciones no están específicamente orientadas a verificar la coherencia de entregas software desde una perspectiva global.

3.6.6. Conclusiones del Estado del Arte

Del análisis realizado se concluye que:

- La automatización del ciclo de vida del software está ampliamente extendida.
- Los mecanismos existentes se centran en aspectos técnicos del desarrollo.
- Existe una carencia de soluciones orientadas a verificar la coherencia global de una entrega.

En este contexto, el sistema propuesto en este Trabajo Fin de Grado aborda este vacío, proporcionando un enfoque centrado en la verificación automática de entregas software a partir de múltiples fuentes de información.

4 Especificación de Requisitos del Sistema

El presente capítulo constituye la Especificación de Requisitos del Software ([Software Requirements Specification \(SRS\)](#)) del sistema **SVAES** (Sistema de Verificación Automática de Entregas de Software), elaborada siguiendo las recomendaciones del estándar *IEEE 830:1998* y las directrices de la norma *UNE 157801:2007*. Su objetivo es definir de forma precisa, completa y verificable qué debe hacer el sistema, sin entrar todavía en cómo se implementará, estableciendo así la base contractual para el diseño de la arquitectura, la implementación y la validación en los capítulos 5 y 6.

4.1. Propósito

Este documento tiene como propósito establecer la especificación completa e inequívoca de los requisitos del sistema **SVAES**. Sirve como referencia única para la comprensión común entre los distintos actores involucrados en el proyecto: el equipo de desarrollo, los *stakeholders* del sistema y los evaluadores del TFG. La especificación contenida en este capítulo será la base sobre la que se validará que la implementación final cumple con las expectativas definidas, y será el punto de referencia para la trazabilidad entre las decisiones de diseño y los requisitos originales.

4.2. Ámbito del sistema

El ámbito y alcance del sistema **SVAES** se define de forma unificada en la sección 2.8 del capítulo 2. Esta sección se mantiene en el documento de requisitos por consistencia estructural y remite al lector a dicha definición.

4.3. Definiciones, acrónimos y abreviaturas

A lo largo de este documento se utilizan los siguientes términos y abreviaturas:

- **API:** Application Programming Interface (Interfaz de Programación de Aplicaciones).
- **RBAC:** Role-Based Access Control (Control de Acceso Basado en Roles).

- **REST:** Representational State Transfer (Transferencia de Estado Representacional).
- *Release:* versión concreta de un producto de software que se entrega como una unidad coherente.
- *Quality gate:* punto de control en el proceso de entrega donde se verifica que se cumplen unos criterios de calidad antes de avanzar.
- *Connector:* módulo de integración con un sistema externo específico que permite consultar datos de forma estandarizada.
- *Tenant:* instancia aislada de una organización dentro de un sistema multi-inquilino.
- **UUID:** Universal Unique Identifier (Identificador Universal Único).
- **HTTPS:** Hypertext Transfer Protocol Secure (Protocolo de Transferencia de Hipertexto Seguro).
- **SaaS:** Software as a Service (Software como Servicio).

4.4. Visión general

El presente capítulo se organiza en las siguientes secciones: la sección 4.5 identifica todos los actores involucrados en el sistema, incluyendo los stakeholders internos detallados en la Tabla 1 y los stakeholders externos en la Tabla 2; la sección 4.6 enumera las fuentes de información utilizadas como referencia, detalladas en la Tabla 4; la sección 2.8 del capítulo 2 define el ámbito y alcance del sistema; la sección 4.8 presenta la normativa aplicable; la sección 4.9 describe las funcionalidades de alto nivel organizadas en épicas; la sección 4.10 establece las necesidades no funcionales del sistema clasificadas según el modelo de calidad ISO/IEC 25010 (Tablas 5 a 14); y la sección 4.11 presenta el catálogo completo de requisitos funcionales organizados por épica.

4.5. Stakeholders

4.5.1. Stakeholders internos

Los *stakeholders* internos se dividen en dos grupos: el contexto académico del TFG, formado por las partes directamente implicadas en la evaluación y supervisión del trabajo, y el contexto operativo del SVAES, que recoge los roles típicos de una organización software que haría uso del sistema.

ID	Rol	Descripción	Intereses
S1	Gestor de proyecto	Responsable de la planificación y seguimiento de las entregas del proyecto	Visibilidad del estado de las releases, reducción de tiempos de validación, KPIs claros
S2	Responsable de configuración	Encargado de la trazabilidad entre los artefactos de la release	Verificación automática de trazabilidad, detección de inconsistencias
S3	Desarrollador	Genera el código y las tareas asociadas a las releases	Feedback rápido sobre errores, reducción de re-trabajo
S4	Responsable de pruebas	Define y valida la documentación y resultados de pruebas	Validación automática de documentación y cobertura de pruebas
S5	Responsable técnico de entrega	Valida y autoriza las releases antes de su paso a producción	Validación completa automatizada, reducción de errores humanos, confianza en resultados
S6	Administrador del sistema	Gestiona la configuración global del sistema	Control del sistema, estabilidad, gestión de usuarios y conectores
S7	Manager de organización	Gestiona recursos y proyectos dentro de una organización	Autonomía organizativa, control de proyectos y configuraciones
S8	Desarrollador de conectores	Implementa integraciones con sistemas externos	Extensibilidad del sistema, facilidad de integración, estabilidad de conectores

Tabla 1: Stakeholders internos del SVAES.

4.5.2. Stakeholders externos

ID	Nombre / entidad	Tipo	Relación con el sistema
SE1	Sistema de gestión de tareas	Sistema externo	Fuente de datos de tareas, épicas y subtareas para la verificación
SE2	Repositorio de código	Sistema externo	Fuente de ramas, commits y etiquetas asociadas a la release
SE3	Sistema documental	Sistema externo	Fuente de documentación técnica
SE4	Herramienta de planificación	Sistema externo	Fuente del plan de versiones y releases
SE5	Sistema de gestión de cambios	Sistema externo	Fuente de peticiones de cambio e incidencias
SE6	Canal de notificaciones	Sistema externo	Destino de alertas y notificaciones
SE7	Sistema CI/CD consumidor	Sistema externo	Consume la API del sistema para automatizar decisiones en pipelines

Tabla 2: Stakeholders externos del [SVAES](#).

4.5.3. Usuarios del sistema

ID	Usuario	Descripción	Permisos principales	Stakeholders
U1	Usuario no autenticado	Usuario sin acceso al sistema autenticado	Acceso a login y documentación pública	—
U2	Usuario estándar	Usuario autenticado que trabaja con releases y verificaciones	Crear releases, ejecutar verificaciones, consultar resultados e historial	S1, S2, S3, S4, S5
U3	Administrador global	Usuario con control total del sistema	Gestión global de organizaciones, usuarios, conectores y configuración	S6

ID	Usuario	Descripción	Permisos principales	Stakeholders
U4	Manager de organización	Usuario con control sobre una organización concreta	Gestión de proyectos, miembros, conectores locales y perfiles de verificación	S7

Tabla 3: Usuarios del sistema [SVAES](#).

4.6. Fuentes de información

La especificación se apoya en las siguientes fuentes de referencia, agrupadas en estándares y normas, bibliografía técnica especializada en integración continua y calidad del software, y documentación técnica de las APIs de los sistemas externos considerados para los conectores de referencia.

ID	Fuente	Tipo	Uso en el sistema
F1	IEEE 830:1998 — Recommended Practice for SRS	Estándar	Base para la estructura y redacción de la especificación de requisitos
F2	UNE 157801:2007	Norma	Guía para la elaboración formal del proyecto de sistema de información
F3	ISO/IEC 25010:2011	Estándar	Referencia para la definición de requisitos de calidad (no funcionales)
F4	Humble & Farley (2010) — Continuous Delivery	Bibliografía	Fundamento del concepto de automatización y verificación en pipelines de software
F5	Shahin et al. (2017) — CI/CD Systematic Review	Bibliografía	Apoyo teórico sobre prácticas y beneficios de CI/CD
F6	Montgomery et al. (2022) — Automated Quality Gates	Bibliografía	Referencia para el diseño del motor de verificación como quality gate
F7	Jira REST API v2	Técnica	Fuente de datos de tareas, épicas y estados para la verificación
F8	GitLab REST API v4	Técnica	Fuente de datos de repositorio (commits, ramas, tags)
F9	Confluence REST API	Técnica	Fuente de documentación asociada a las releases

ID	Fuente	Tipo	Uso en el sistema
F10	OWASP Top 10 (2021)	Seguridad	Base para la definición de requisitos de seguridad
F11	OpenAPI Specification 3.x	Estándar	Definición del contrato de la API REST del sistema

Tabla 4: Fuentes de información del SRS.

4.7. Ámbito y alcance del sistema

El ámbito y alcance del sistema [SVAES](#) se define de forma unificada en la sección 2.8 del capítulo 2. Esta sección se mantiene en el documento de requisitos por consistencia estructural y remite al lector a dicha definición.

4.8. Normativa aplicable

El sistema [SVAES](#) se diseña y especifica de acuerdo con los siguientes estándares, normas y referencias técnicas, que condicionan tanto la estructura del presente documento como los requisitos funcionales y no funcionales del sistema.

4.8.1. Estándares de ingeniería del software

- **IEEE 830:1998:** define la estructura y buenas prácticas para la Especificación de Requisitos del Software.
- **UNE 157801:2007:** establece los criterios generales para la elaboración de proyectos de sistemas de información.
- **ISO/IEC 25010:2011:** proporciona el modelo de calidad del software utilizado como base para la definición de los requisitos no funcionales.

4.8.2. Normativa de seguridad y protección de datos

- **OWASP Top 10 (2021):** se utiliza como referencia para la identificación y mitigación de riesgos de seguridad en aplicaciones web.
- **Reglamento (UE) 2016/679 (RGPD):** regula el tratamiento de datos personales de los usuarios del sistema, condicionando los requisitos de seguridad, almacenamiento y acceso a la información.

4.8.3. Estándares técnicos

- **OpenAPI Specification 3.x:** define el formato del contrato de la [API REST](#) del sistema, garantizando su interoperabilidad con sistemas externos.

4.9. Funcionalidades de alto nivel

El sistema [SVAES](#) se articula en torno a módulos funcionales (*Épicas*) que aportan valor directo a los distintos *stakeholders* de la organización, permitiendo estandarizar, automatizar y auditar el proceso de entrega de software.

4.9.1. Épica 1: Arquitectura Multi-inquilino y Seguridad

FEAT-01: Estructuración y Aislamiento de Organizaciones.

- **Actor principal:** Administrador Global (U3).
- **Descripción:** el sistema proporciona la capacidad de particionar lógicamente la plataforma en múltiples organizaciones aisladas. Cada organización opera como un *tenant* independiente con sus propios proyectos, usuarios y configuraciones.
- **Valor de negocio:** garantiza la privacidad y el aislamiento de datos entre distintos clientes o departamentos, permitiendo un despliegue centralizado (SaaS o interno) seguro.

FEAT-02: Control de Acceso y Gestión de Identidad ([Role-Based Access Control \(RBAC\)](#)).

- **Actor principal:** Administrador Global (U3) y Manager de Organización (U4).
- **Descripción:** el sistema implementa un modelo de control de acceso basado en roles que permite delegar la administración de usuarios a nivel de organización, restringiendo las acciones sobre las *releases* y configuraciones estrictamente a los usuarios autorizados.
- **Valor de negocio:** protege la integridad del proceso de validación evitando que usuarios no autorizados alteren las reglas de calidad o manipulen entregables.

4.9.2. Épica 2: Integración y Estandarización de Políticas

FEAT-03: Integración y Sincronización con Herramientas de Terceros.

- **Actor principal:** Manager de Organización (U4).
- **Descripción:** el sistema proporciona mecanismos para configurar, autenticar y auditar la conexión bidireccional (vía conectores) con las fuentes externas.
- **Valor de negocio:** convierte al [SVAES](#) en una herramienta autónoma capaz de extraer datos en tiempo real sin requerir la entrada manual de datos por parte de

los desarrolladores.

FEAT-04: Definición de Políticas de Calidad (Perfiles y Plantillas).

- **Actor principal:** Manager de Organización (U4).
- **Descripción:** el sistema permite diseñar conjuntos de reglas de verificación reutilizables (Perfiles) y estructuras predefinidas de entregables (Plantillas) que actúan como «contratos» de obligado cumplimiento para los proyectos.
- **Valor de negocio:** estandariza los criterios de calidad en la organización, eliminando la ambigüedad sobre qué elementos deben estar presentes y validados.

4.9.3. Épica 3: Núcleo Operativo (Motor de Verificación)

FEAT-05: Orquestación del Ciclo de Vida de Releases.

- **Actor principal:** Usuario Estándar (U2) y Manager de Organización (U4).
- **Descripción:** el sistema facilita la creación de *releases* y la vinculación de sus artefactos asociados (código, tareas, documentación), orquestando su transición por una máquina de estados estricta (Borrador → Pendiente → En Verificación → Válida / No Válida).
- **Valor de negocio:** proporciona un inventario centralizado y estructurado de todas las entregas de software en curso y pasadas.

FEAT-06: Ejecución Automatizada de Verificaciones (Quality Gate).

- **Actor principal:** Usuario Estándar (U2).
- **Descripción:** el sistema ejecuta el motor de verificación asíncrono que consulta los artefactos en las fuentes externas y los evalúa contra el perfil de calidad asignado, calculando el resultado global del *quality gate*.
- **Valor de negocio:** elimina la validación manual (ahorro de tiempo y coste) y garantiza la objetividad en la aprobación/rechazo de una entrega.

4.9.4. Épica 4: Observabilidad y Comunicación

FEAT-07: Trazabilidad y Auditoría de Resultados.

- **Actor principal:** Responsable Técnico de Entrega (S5) y Auditores.
- **Descripción:** el sistema mantiene un registro histórico e inmutable de todas las verificaciones realizadas, incluyendo el perfil de calidad aplicado (*snapshot*) y los resultados detallados por regla.
- **Valor de negocio:** garantiza el cumplimiento normativo e histórico, permitiendo auditar por qué una *release* concreta fue aprobada o rechazada en el pasado, incluso si las reglas de la organización cambian posteriormente.

FEAT-08: Inteligencia de Negocio y Métricas de Calidad (Dashboard).

- **Actor principal:** Manager de Organización (U4) y Gestor de Proyecto (S1).
- **Descripción:** el sistema consolida los datos históricos de validación para exponer indicadores de rendimiento, cuellos de botella y tasas de éxito mediante paneles visuales filtrables.
- **Valor de negocio:** habilita la toma de decisiones basada en datos para aplicar mejoras continuas en los procesos de desarrollo de los equipos.

FEAT-09: Sistema de Alertas y Notificaciones Proactivas.

- **Actor principal:** todos los usuarios autenticados.
- **Descripción:** el sistema empuja la información relevante a los usuarios afectados a través de canales integrados (Email, Slack, MS Teams) según sus preferencias.
- **Valor de negocio:** minimiza los tiempos muertos de espera, permitiendo a los equipos reaccionar de inmediato ante un bloqueo o fallo en su entrega.

4.9.5. Épica 5: Extensibilidad

FEAT-10: Interoperabilidad Programática Automática.

- **Actor principal:** Sistema CI/CD Consumidor (SE7).
- **Descripción:** el sistema expone el subconjunto de operaciones operativas a través de una [API REST](#) securizada, permitiendo a sistemas externos hacer verificaciones o consultar su estado.
- **Valor de negocio:** permite que el [SVAES](#) actúe como un *Quality Gate* bloqueante o informativo dentro de un pipeline de Integración/Despliegue Continuo (CI/CD) corporativo preexistente.

4.10. Necesidades no funcionales

Se presentan las necesidades no funcionales siguiendo el modelo ISO/IEC 25010. En este capítulo los identificadores usan el formato compacto **RNF***N*; en el resto de la memoria se emplea el formato normalizado **RNF-*NN*** con dos dígitos (p. ej. **RNF1** $\equiv$ **RNF-01**).

4.10.1. Rendimiento

ID	Descripción
RNF1	La verificación de una <i>release</i> configurada con hasta 10 reglas deberá completarse en un tiempo máximo de 30 segundos de procesamiento en segundo plano, asumiendo que el tiempo de respuesta de las llamadas a las APIs de los conectores externos sea inferior a 1 segundo por petición.
RNF2	Cualquier vista web debe cargar en un máximo de 3 segundos soportando hasta 20 usuarios concurrentes. Esto asume una conexión del cliente con al menos 10 Mbps de descarga y menos de 50 ms de latencia.
RNF3	El sistema deberá admitir al menos 5 verificaciones concurrentes sin que el tiempo de respuesta de ninguna de ellas supere el doble del tiempo unitario.
RNF4	El endpoint de la API REST correspondiente a la consulta del <i>dashboard</i> de estadísticas deberá responder con código HTTP 200 en un tiempo máximo de 3 segundos al procesar una agregación de hasta 1.000 verificaciones históricas.
RNF5	La API REST deberá responder a consultas simples en menos de 500 milisegundos.

Tabla 5: Requisitos no funcionales de rendimiento.

4.10.2. Disponibilidad

ID	Descripción
RNF6	El sistema deberá estar operativo en menos de 60 segundos desde su arranque.
RNF7	La cola de verificaciones pendientes deberá ser persistente ante reinicios del sistema, de forma que no se pierdan trabajos en curso o en espera.

Tabla 6: Requisitos no funcionales de disponibilidad.

4.10.3. Fiabilidad

ID	Descripción
RNF8	El sistema deberá estar diseñado para soportar operación continua con interrupciones mínimas. En el entorno de despliegue personal del TFG no se establece una garantía contractual de disponibilidad, pero la arquitectura deberá permitir alcanzar una disponibilidad mensual del 99 % si se desplegara en un entorno de producción adecuado.
RNF9	Ante el fallo de un conector externo, el motor de verificación deberá continuar procesando las reglas restantes sin abortar la verificación global.
RNF10	Los errores críticos del sistema deberán quedar registrados de forma automática en el log de la aplicación en un plazo máximo de 5 segundos desde su ocurrencia.
RNF11	La tasa de verificaciones fallidas por causas internas al sistema no deberá superar el 0,1 % del total procesado en condiciones de carga nominal.

Tabla 7: Requisitos no funcionales de fiabilidad.

4.10.4. Seguridad

ID	Descripción
RNF12	Las contraseñas de los usuarios se almacenarán de forma irreversible mediante una función de hash robusta con sal, de acuerdo con las prácticas recomendadas por OWASP.
RNF13	Las credenciales de acceso a los sistemas externos configuradas en los conectores se almacenarán cifradas en reposo; en ningún caso se persistirán en texto claro.
RNF14	El sistema bloqueará temporalmente el acceso de una cuenta tras un número configurable de intentos de autenticación fallidos consecutivos en un intervalo de tiempo determinado.
RNF15	Todas las comunicaciones entre el navegador y el servidor deberán realizarse únicamente mediante protocolo seguro (HTTPS).
RNF16	El acceso a la API REST requerirá autenticación mediante clave de acceso. Las claves no se almacenarán en texto claro en el sistema.
RNF17	El sistema mantendrá un registro de auditoría que recoja, como mínimo, los eventos de autenticación, los cambios de configuración relevantes y las ejecuciones de verificación.
RNF18	Las notificaciones salientes generadas por el sistema no contendrán credenciales ni datos sensibles de los conectores o usuarios.

Tabla 8: Requisitos no funcionales de seguridad.

4.10.5. Usabilidad

ID	Descripción
RNF19	Un usuario estándar (U2) deberá poder lanzar una verificación en un máximo de 5 acciones desde la pantalla principal.
RNF20	Cada acción incorrecta del usuario deberá generar un mensaje de error descriptivo y orientativo que indique la causa y la forma de subsanarla.
RNF21	El <i>dashboard</i> de estadísticas utilizará una codificación visual semántica (p.ej. semáforo de colores) coherente en todas las vistas.
RNF22	La interfaz web deberá ser responsiva y compatible con las versiones actuales de los principales navegadores de escritorio y móvil.
RNF23	El sistema contará con al menos un manual de usuario que describa cómo realizar cada operación.
RNF24	Un usuario con rol estándar (U2), sin experiencia previa en el SVAES y disponiendo únicamente del manual de usuario, deberá ser capaz de completar exitosamente el flujo básico del caso de uso CU-01 (Verificar Release) en un tiempo máximo de 15 minutos medidos desde su primer inicio de sesión.

Tabla 9: Requisitos no funcionales de usabilidad.

4.10.6. Mantenibilidad

ID	Descripción
RNF25	El sistema mantendrá separación de responsabilidades entre sus capas lógicas, de forma que los cambios en una capa no requieran modificaciones en las demás.
RNF26	El registro de un nuevo conector no requerirá modificar el motor de verificación ni la interfaz de usuario.
RNF27	La cobertura de pruebas automáticas del motor de verificación y de los conectores deberá ser igual o superior al 80 %.
RNF28	El código fuente deberá incluir documentación de las funciones y clases públicas, de acuerdo con las convenciones estándar del lenguaje de implementación elegido.

Tabla 10: Requisitos no funcionales de mantenibilidad.

4.10.7. Portabilidad

ID	Descripción
RNF29	El sistema web deberá funcionar correctamente en los cuatro navegadores principales en sus versiones actuales: Google Chrome, Mozilla Firefox, Microsoft Edge y Safari.
RNF30	El sistema deberá funcionar en dispositivos de escritorio, tabletas y dispositivos móviles.
RNF31	El sistema dispondrá de un procedimiento de instalación y despliegue documentado que permita reproducir el entorno de ejecución de forma controlada.

Tabla 11: Requisitos no funcionales de portabilidad.

4.10.8. Extensibilidad

ID	Descripción
RNF32	La interfaz de conectores estará especificada de forma documentada y estable, de modo que terceros puedan implementar nuevos conectores sin modificar el núcleo del sistema.
RNF33	Nuevas reglas de verificación podrán incorporarse al sistema sin necesidad de modificar el motor de verificación.
RNF34	El sistema de notificaciones deberá diseñarse de forma que permita añadir nuevos canales de envío sin modificar la lógica del núcleo.

Tabla 12: Requisitos no funcionales de extensibilidad.

4.10.9. Trazabilidad

ID	Descripción
RNF35	El sistema registrará por cada verificación, y por cada llamada a un conector externo, la fuente consultada, el recurso solicitado, el resultado obtenido y el instante de la llamada.
RNF36	El historial de verificaciones deberá conservar una instantánea inmutable del perfil de verificación y del estado de los artefactos en el momento de cada ejecución.
RNF37	Cada requisito funcional del catálogo deberá estar referenciado en al menos un caso de uso o regla de verificación.

Tabla 13: Requisitos no funcionales de trazabilidad.

4.10.10. Requisitos organizacionales y externos

ID	Descripción
RNF38	El sistema deberá cumplir con el Reglamento General de Protección de Datos (RGPD) en lo que respecta al tratamiento de datos personales de los usuarios.
RNF39	El sistema deberá tener en cuenta las directrices de seguridad de aplicaciones web recogidas en el estándar OWASP Top 10.
RNF40	La API REST pública deberá estar documentada mediante una especificación estándar legible por máquina (p.ej. OpenAPI), de modo que los consumidores externos puedan integrarla sin necesidad de conocimiento previo de la implementación.

Tabla 14: Requisitos no funcionales organizacionales y externos.

4.11. Catálogo de requisitos funcionales

Los requisitos funcionales se organizan siguiendo la misma estructura de épicas definida en la sección 4.9 (Funcionalidades de alto nivel), de modo que cada grupo de requisitos queda vinculado a la funcionalidad de alto nivel que le da origen. Dentro de cada épica, los requisitos se desglosan en hasta tres niveles de profundidad: el nivel superior (p.ej. **ORG1**) expresa la capacidad principal; el nivel intermedio (p.ej. **ORG1.2**) la refina en comportamientos observables; y el nivel hoja (p.ej. **ORG1.2.1**) especifica restricciones, condiciones de error o comportamientos derivados concretos. La priorización sigue la codificación MoSCoW: **M** — *Must have*, **S** — *Should have*, **C** — *Could have*, **W** — *Won't have* (diferido).

A lo largo de todas las sub-secciones de esta sección se emplea un formato uniforme: los requisitos de primer y segundo nivel se presentan en **negrita** precedidos de su identificador y prioridad MoSCoW entre corchetes; los requisitos de tercer nivel (restricciones derivadas) se presentan en *cursiva* sin indicador de prioridad, dado que heredan la prioridad de su requisito padre.

4.11.1. Épica 1 — Arquitectura Multi-inquilino y Seguridad

Esta épica agrupa los requisitos que garantizan el aislamiento entre organizaciones y el control de acceso al sistema. Cubre las funcionalidades FEAT-01 y FEAT-02.

FEAT-01: Estructuración y Aislamiento de Organizaciones

- **ORG1** [M] El sistema almacenará cada organización como una entidad con identificador único, nombre, *slug*, descripción, propietario, lista de miembros, lista de roles, estado y metadatos de fechas.
- **ORG1.1** [M] El sistema asignará a cada organización un identificador UUID v4 generado automáticamente en el momento de la creación.
 - *ORG1.1.1* El sistema no permitirá modificar el identificador de una organización tras su creación.
 - *ORG1.1.2* El sistema utilizará el identificador UUID como clave externa en todas las entidades que referencien a la organización.
- **ORG1.2** [M] El sistema almacenará el nombre de la organización como cadena de texto de entre 3 y 100 caracteres.
 - *ORG1.2.1* El sistema admitirá en el nombre caracteres alfanuméricos Unicode, espacios y los símbolos «- _ . & /», y rechazará cualquier otro carácter con un error de validación.
 - *ORG1.2.2* El sistema rechazará nombres con la misma forma canónica (sin distinción de mayúsculas y minúsculas y sin espacios iniciales o finales) que el de otra organización existente.
- **ORG1.3** [M] El sistema almacenará una *slug* única para cada organización, usada en URLs y en la API.
 - *ORG1.3.1* El sistema generará la *slug* inicial de forma automática a partir del nombre, aplicando transliteración ASCII, conversión a minúsculas y sustitución de espacios por guiones.
 - *ORG1.3.2* El sistema admitirá en la *slug* únicamente caracteres de los rangos a-z, 0-9 y el guion medio, con longitud entre 3 y 40 caracteres.
 - *ORG1.3.3* El sistema permitirá a U3 y al *owner* editar la *slug* tras la creación, preservando la unicidad global.
- **ORG1.4** [S] El sistema almacenará una descripción opcional para cada organización

con un máximo de 500 caracteres en UTF-8.

- **ORG1.5 [M]** El sistema almacenará para cada organización un único usuario propietario (*owner*) que actúa como responsable último de la organización.
 - *ORG1.5.1* El sistema asignará automáticamente como *owner* al usuario que ejecuta la operación de creación de la organización.
 - *ORG1.5.2* El sistema garantizará que toda organización activa tiene en todo momento exactamente un *owner* asignado.
 - *ORG1.5.3* El sistema permitirá al *owner* transferir la propiedad a otro miembro de la organización mediante una operación explícita de transferencia.
- **ORG1.6 [M]** El sistema almacenará el estado de cada organización como uno de los valores del conjunto {ACTIVA, ARCHIVADA}.
- **ORG1.7 [S]** El sistema registrará automáticamente para cada organización la fecha y hora de creación y la fecha y hora de la última modificación, ambas en UTC con precisión de milisegundos.
 - *ORG1.7.1* El sistema actualizará la fecha de última modificación cuando cambien el nombre, la *slug*, la descripción, el *owner*, los miembros, los roles o el estado de la organización.
- **ORG2 [M]** El sistema permitirá a un usuario autenticado crear una nueva organización.
- **ORG2.1 [M]** El sistema mostrará al usuario un formulario de creación que solicitará nombre (obligatorio), *slug* (pre-llenada y editable) y descripción (opcional).
 - *ORG2.1.1* El sistema validará los campos del formulario conforme a las restricciones definidas en ORG1.2, ORG1.3 y ORG1.4 antes de aceptar la creación.
 - *ORG2.1.2* El sistema devolverá al usuario un mensaje de error específico por cada campo que incumpla sus restricciones, sin crear la organización.
- **ORG2.2 [M]** El sistema inicializará toda organización recién creada en estado ACTIVA.
- **ORG2.3 [M]** El sistema permitirá a U3 y al *owner* de la organización modificar el nombre, la *slug* y la descripción de la misma.
 - *ORG2.3.1* El sistema aplicará las mismas reglas de unicidad y formato en la modificación que en la creación.
- **ORG2.4 [S]** El sistema permitirá a U3 y al *owner* archivar una organización en estado ACTIVA.
 - *ORG2.4.1* El sistema solicitará al usuario que escriba el nombre exacto de la organización como confirmación antes de ejecutar el archivado.
 - *ORG2.4.2* El sistema cambiará el estado de la organización a ARCHIVADA y el estado de todos sus proyectos a ARCHIVADO cuando se confirme el archivado.
 - *ORG2.4.3* El sistema conservará íntegramente los datos históricos (*releases*, verificaciones, auditoría) de una organización archivada en modo solo lectura.

- **ORG2.5 [S]** El sistema permitirá a U3 y al *owner* restaurar una organización en estado ARCHIVADA devolviéndola al estado ACTIVA.
- **ORG2.6 [M]** El sistema rechazará la eliminación física de cualquier organización con historial de verificaciones, ofreciendo únicamente la operación de archivado.
 - *ORG2.6.1* El sistema permitirá a U3 eliminar físicamente una organización únicamente cuando ésta no tenga *releases* ni verificaciones asociadas.
- **ORG3 [M]** El sistema mantendrá para cada organización una lista de miembros que identifique a los usuarios con acceso a la misma y el rol asignado a cada uno.
- **ORG3.1 [M]** El sistema dispondrá de un conjunto de cuatro roles fijos no modificables: U1 (invitado), U2 (usuario estándar), U3 (administrador del sistema) y U4 (*manager* de organización).
 - *ORG3.1.1* El sistema asignará el rol U3 únicamente a nivel global, no a nivel de organización concreta.
 - *ORG3.1.2* El sistema permitirá que una misma organización tenga varios miembros con rol U4 simultáneamente.
- **ORG3.2 [S]** El sistema permitirá al *owner* y a los U4 definir roles personalizados adicionales dentro de su organización, distintos de los cuatro roles fijos del sistema (U1–U4). Estos roles personalizados agrupan subconjuntos de permisos atómicos y son independientes de los roles globales.
 - *ORG3.2.1* El sistema exigirá a cada rol personalizado un nombre único dentro de la organización (3-40 caracteres) y una lista explícita de permisos atómicos del catálogo del sistema.
 - *ORG3.2.2* El sistema aislará los roles personalizados al ámbito de su organización, sin que afecten a otras organizaciones.
 - *ORG3.2.3* El sistema rechazará la eliminación de un rol personalizado que tenga miembros asignados, exigiendo reasignar dichos miembros a otro rol previamente.
- **ORG3.3 [M]** El sistema permitirá al *owner* y a los U4 invitar a una organización a un usuario ya registrado mediante su dirección de *email*, indicando el rol a asignar.
 - *ORG3.3.1* El sistema rechazará la invitación cuando el *email* no corresponda a ninguna cuenta existente y activa en el sistema.
 - *ORG3.3.2* El sistema notificará al usuario invitado por *email* cuando el canal de notificaciones por *email* esté configurado en la organización.
- **ORG3.4 [M]** El sistema permitirá al *owner* y a los U4 modificar el rol de cualquier miembro de la organización distinto al *owner*.
 - *ORG3.4.1* El sistema rechazará toda modificación del rol del *owner* que no sea la operación explícita de transferencia de propiedad definida en ORG1.5.3.
- **ORG3.5 [M]** El sistema permitirá al *owner* y a los U4 retirar a un miembro de la organización distinto al *owner*.

- *ORG3.5.1* El sistema mostrará al usuario la lista de *releases* activas de las que el miembro sea responsable antes de confirmar su retirada.
- *ORG3.5.2* El sistema reasignará al *owner* las *releases* activas del miembro retirado cuando no se indique otro responsable.
- *ORG3.5.3* El sistema rechazará que un usuario con rol U4 expulse a otro miembro que también ostente el rol U4 (manager) dentro de la misma organización. Solo el *owner* podrá expulsar a otros *managers*.
- **ORG3.6 [S]** El sistema permitirá que un mismo usuario sea miembro de varias organizaciones simultáneamente, con un rol independiente en cada una.
- **ORG4 [M]** El sistema garantizará que los datos de una organización no son accesibles desde otra organización.
- **ORG4.1 [M]** El sistema rechazará con respuesta HTTP 403 toda petición de un miembro de una organización que intente acceder a proyectos, *releases*, conectores o historial de una organización distinta de la que forma parte, con la excepción del rol U3 (administrador global), que dispone de todos los permisos del rol U4 sobre cualquier organización (AC3.8).
 - *ORG4.1.1* El sistema registrará en el log de auditoría todo intento de acceso cruzado indicando usuario, organización origen, recurso destino, *timestamp* e IP.
- **ORG4.2 [M]** El sistema filtrará toda consulta de datos a nivel de capa de acceso a datos aplicando el identificador de organización del contexto de la petición.
 - *ORG4.2.1* El sistema incluirá el identificador de organización como columna obligatoria en todas las tablas de entidades operativas de la base de datos.
- **ORG4.3 [M]** El sistema mostrará a cada usuario autenticado únicamente la lista de organizaciones a las que pertenece en la pantalla de selección de contexto.
- **ORG5 [M]** El sistema permitirá al *owner* y a los U4 gestionar los recursos de su organización.
- **ORG5.1 [M]** El sistema permitirá al *owner* y a los U4 crear, modificar y archivar proyectos dentro de su organización.
- **ORG5.2 [M]** El sistema permitirá al *owner* y a los U4 registrar y gestionar conectores externos dentro de su organización.
- **ORG5.3 [M]** El sistema permitirá al *owner* y a los U4 crear y gestionar perfiles de verificación dentro de su organización.
- **ORG5.4 [S]** El sistema permitirá al *owner* y a los U4 crear y gestionar plantillas de *release* dentro de su organización.
- **ORG5.5 [S]** El sistema permitirá al *owner* y a los U4 consultar el *dashboard* de estadísticas de su organización completa.

FEAT-02: Control de Acceso y Gestión de Identidad (RBAC)

- **AC1 [M]** El sistema permitirá a todo usuario registrado iniciar sesión mediante credenciales de usuario y contraseña.
- **AC1.1 [M]** El sistema exigirá al usuario los campos obligatorios nombre de usuario o *email* y contraseña en el formulario de inicio de sesión.
- **AC1.2 [M]** El sistema validará las contraseñas comparando el valor introducido con el *hash* almacenado utilizando una función de *hash* robusta e irreversible con sal, de acuerdo con las prácticas recomendadas por OWASP.
- **AC1.3 [M]** El sistema mostrará el mensaje genérico «Credenciales incorrectas» ante cualquier intento de inicio de sesión fallido por contraseña errónea o usuario inexistente, para prevenir ataques de enumeración.
 - *AC1.3.1* El sistema mantendrá un tiempo de respuesta homogéneo ante credenciales incorrectas con independencia de la existencia del usuario, para prevenir ataques de enumeración.
- **AC1.4 [M]** El sistema creará una sesión autenticada y redirigirá al usuario a la pantalla de selección de organización cuando las credenciales sean válidas y la cuenta esté activa.
- **AC1.5 [S]** El sistema mostrará un mensaje específico distinto de «Credenciales incorrectas» cuando la cuenta exista, pero se encuentre inactiva, únicamente si la contraseña introducida es matemáticamente correcta.
- **AC1.6 [S]** El sistema mostrará un mensaje específico distinto de «Credenciales incorrectas» cuando la cuenta exista, pero se encuentre bloqueada por la política de fuerza bruta, únicamente si la contraseña introducida es matemáticamente correcta.
- **AC1.7 [M]** El sistema bloqueará temporalmente una cuenta durante 15 minutos tras 5 intentos fallidos consecutivos dentro de una ventana de 10 minutos.
 - *AC1.7.1* El sistema reiniciará el contador de intentos fallidos de una cuenta tras un inicio de sesión exitoso o al expirar el bloqueo temporal.
 - *AC1.7.2* El sistema registrará cada bloqueo temporal en el log de auditoría con usuario, IP de origen y *timestamp*.
- **AC2 [M]** El sistema gestionará el cierre explícito de sesión y la expiración por inactividad.
- **AC2.1 [M]** El sistema invalidará la sesión en cliente y servidor cuando el usuario ejecute la operación de cierre de sesión.
 - *AC2.1.1* El sistema revocará los *tokens* refrescables asociados a la sesión cuando existan.
- **AC2.2 [M]** El sistema expirará automáticamente una sesión autenticada tras 8 horas de inactividad continuada.
 - *AC2.2.1* El sistema permitirá configurar el tiempo de expiración de sesiones

mediante parámetro de configuración del sistema, con un valor por defecto de 8 horas.

- **AC2.3 [S]** El sistema redirigirá al usuario a la pantalla de inicio de sesión con un mensaje informativo al expirar su sesión por inactividad.
- **AC3 [M]** El sistema aplicará un modelo de control de acceso basado en roles (**RBAC**) con cuatro roles fijos del sistema.
- **AC3.1 [M]** El sistema asignará al rol U1 (no autenticado) acceso a la página de inicio de sesión, la *landing page* pública, el formulario de solicitud de acceso y la documentación pública de la API.
- **AC3.2 [M]** El sistema asignará al rol U2 (usuario estándar) los permisos de creación, lectura y archivado de *releases* en los proyectos a los que esté asignado, así como la modificación de aquellas *releases* de las que sea responsable conforme a GR3.
- **AC3.3 [M]** El sistema asignará al rol U2 los permisos de ejecución de verificaciones sobre las *releases* a las que tenga acceso.
- **AC3.4 [M]** El sistema asignará al rol U2 los permisos de consulta del historial y del *dashboard* limitados a sus proyectos asignados.
- **AC3.5 [S]** El sistema asignará al rol U2 los permisos de gestión de sus propias API *keys* y sus propias suscripciones de notificación.
- **AC3.6 [M]** El sistema asignará al rol U4 (*manager* de organización) todos los permisos del rol U2 dentro de los proyectos de su organización.
- **AC3.7 [M]** El sistema asignará al rol U4 los permisos de gestión de proyectos, conectores, perfiles, plantillas, roles personalizados y miembros dentro de su organización.
- **AC3.8 [M]** El sistema asignará al rol U3 (administrador global) todos los permisos del rol U4 a nivel global del sistema.
- **AC3.9 [M]** El sistema asignará al rol U3 los permisos de gestión global de organizaciones, usuarios globales, configuración del sistema y conectores de ámbito global.
- **AC3.10 [M]** El sistema verificará los permisos de cada petición tanto en la capa de presentación como en la capa de servicio.
 - *AC3.10.1* El sistema rechazará toda petición que supere la verificación de la capa de presentación, pero falle en la capa de servicio, registrando el intento en el log de auditoría.
- **AC4 [M]** El sistema permitirá gestionar las cuentas de usuario globales.
- **AC4.1 [M]** El sistema permitirá a un U3 crear una nueva cuenta de usuario global indicando nombre, *email* único, contraseña inicial y rol global por defecto.
- **AC4.2 [M]** El sistema exigirá a toda contraseña una longitud mínima de 10 caracteres.
 - *AC4.2.1* El sistema exigirá a toda contraseña incluir al menos una letra mayúscula.

- *AC4.2.2* El sistema exigirá a toda contraseña incluir al menos una letra minúscula.
- *AC4.2.3* El sistema exigirá a toda contraseña incluir al menos un dígito.
- *AC4.2.4* El sistema exigirá a toda contraseña incluir al menos un carácter especial del conjunto `!@#$%^&*()_+=[ ]{};:.,<>?/|`.
- *AC4.2.5* El sistema rechazará toda contraseña presente en un diccionario local de contraseñas comprometidas (mínimo 10.000 entradas), cargado en memoria en el arranque para no penalizar el tiempo de respuesta de la autenticación.
- **AC4.3 [M]** El sistema permitirá a un U3 modificar el rol global de cualquier usuario.
- **AC4.4 [M]** El sistema permitirá a un U3 activar o desactivar cualquier cuenta de usuario.
- **AC4.5 [M]** El sistema permitirá a cada usuario autenticado cambiar su propia contraseña previa verificación de la contraseña actual.
- **AC4.6 [S]** El sistema permitirá a cada usuario autenticado modificar su propio nombre de usuario.
- **AC4.7 [S]** El sistema permitirá a cada usuario autenticado modificar su propia dirección de *email* aplicando la restricción de unicidad global.
- **AC4.8 [M]** El sistema permitirá a cada usuario autenticado eliminar su propia cuenta desde la sección de perfil, con la excepción del administrador global (U3), al que no se le mostrará esta opción.
 - *AC4.8.1* El sistema solicitará al usuario la confirmación de su contraseña actual y un segundo factor (cuando esté habilitado) antes de ejecutar la eliminación, y mostrará un aviso explícito de irreversibilidad.
 - *AC4.8.2* El sistema presentará al usuario, antes de la confirmación final, una lista de las organizaciones donde actúa como *owner* junto con la acción prevista para cada una: transferencia de propiedad (cuando existan otros miembros con rol U4) o eliminación de la organización (cuando sea el único miembro).
 - *AC4.8.3* El sistema transferirá automáticamente la propiedad de las organizaciones donde el usuario sea *owner* a otro miembro con rol U4, cuando exista al menos uno. La transferencia se realizará sobre el miembro con mayor antigüedad en la organización.
 - *AC4.8.4* El sistema eliminará junto con la cuenta del usuario las organizaciones donde éste sea el único miembro, previa confirmación explícita por parte del usuario.
 - *AC4.8.5* El sistema conservará en el registro de auditoría la traza de la autoeliminación y de las transferencias de propiedad realizadas como consecuencia de la misma.
- **AC4.9 [M]** El sistema permitirá a cada usuario autenticado exportar todos sus datos personales almacenados en el sistema en un fichero JSON estructurado, garantizando

los derechos de acceso y portabilidad reconocidos en el Reglamento (UE) 2016/679 (RGPD), artículos 15 y 20.

- *AC4.9.1* El sistema expondrá el *endpoint* `GET /api/v1/users/me/export`, accesible exclusivamente al usuario autenticado titular de los datos.
- *AC4.9.2* El sistema incluirá en el fichero JSON de exportación los campos identificador UUID, correo electrónico, nombre de visualización, rol global (U2/U3/U4), estado de la cuenta (`is_active`), fecha de creación, fecha de última actualización, fechas de aceptación de términos y política de privacidad, y lista de organizaciones a las que pertenece el usuario.
- *AC4.9.3* El sistema registrará en el log de auditoría una entrada de tipo `DATA_EXPORT_REQUESTED` por cada solicitud de exportación, indicando usuario, *timestamp* e IP de origen.
- *AC4.9.4* El sistema devolverá el fichero con el nombre de descarga `svaes-user-data.json` y el tipo MIME `application/json`.
- *AC4.9.5* El sistema presentará la funcionalidad en la página de perfil del usuario (`/profile`), en una sección diferenciada bajo el epígrafe «Descargar mis datos», con estilo visual coherente con la sección de eliminación de cuenta.
- **AC5 [S]** El sistema permitirá a cada usuario autenticado gestionar sus propias API *keys*.
 - **AC5.1 [S]** El sistema permitirá al usuario crear una nueva API *key* desde su perfil.
 - *AC5.1.1* El sistema rechazará la creación cuando el usuario ya tenga 5 API *keys* activas.
 - **AC5.2 [S]** El sistema mostrará al usuario el valor completo de una API *key* recién creada una única vez en la pantalla de confirmación.
 - *AC5.2.1* El sistema no permitirá recuperar el valor completo de una API *key* tras la pantalla de confirmación inicial.
 - **AC5.3 [S]** El sistema permitirá al usuario asignar a cada API *key* una descripción opcional de hasta 120 caracteres.
 - **AC5.4 [C]** El sistema permitirá al usuario asignar a cada API *key* una fecha de expiración opcional no anterior a la fecha actual.
 - **AC5.5 [S]** El sistema permitirá al usuario revocar cualquier API *key* propia con efecto inmediato.
 - **AC5.6 [S]** El sistema mostrará en el listado de API *keys* del usuario la descripción, los últimos 8 caracteres del identificador de *key*, la fecha de creación, la fecha de expiración y el estado del conjunto {ACTIVA, REVOCADA, EXPIRADA}.

4.11.2. Épica 2 — Integración y Estandarización de Políticas

Esta épica agrupa los requisitos relativos a la conexión con herramientas externas y a la definición de las políticas de calidad reutilizables. Cubre las funcionalidades FEAT-03 y FEAT-04.

FEAT-03: Integración y Sincronización con Herramientas de Terceros

- **GC1 [M]** El sistema permitirá al *owner* y a los U4 de una organización registrar conectores hacia sistemas externos.
- **GC1.1 [M]** El sistema exigirá a cada conector un nombre único dentro de su organización de entre 3 y 80 caracteres.
- **GC1.2 [M]** El sistema exigirá a cada conector un tipo funcional del conjunto {GESTOR_TAREAS, REPO_CODIGO, SISTEMA_DOCUMENTAL, HERRAMIENTA_PLANIFICACION, GESTION_CAMBIOS}.
- **GC1.3 [M]** El sistema exigirá a cada conector una clase de implementación seleccionada entre las disponibles en el registro de conectores del sistema.
 - *GC1.3.1* El sistema construirá el registro de clases de conector en tiempo de arranque descubriendo automáticamente las subclases concretas de la interfaz `IConnector`.
- **GC1.4 [M]** El sistema exigirá a cada conector una URL base con esquema HTTPS obligatorio en entornos distintos de desarrollo.
- **GC1.5 [M]** El sistema exigirá a cada conector un mecanismo de autenticación del conjunto {BASIC, PAT, TOKEN_BEARER, API_KEY} compatible con la clase seleccionada.
- **GC1.6 [M]** El sistema cifrará las credenciales del conector en reposo mediante cifrado simétrico robusto con clave gestionada por el servicio de secretos de la aplicación.
 - *GC1.6.1* El sistema no incluirá las credenciales del conector en respuestas de la API, en *logs* ni en volcados de error en ninguna circunstancia.
- **GC2 [M]** El sistema definirá una interfaz `IConnector` que toda clase de conector deberá implementar para poder registrarse.
- **GC2.1 [M]** El sistema exigirá a toda clase `IConnector` la implementación del método `test_connection()` que devuelva un objeto con los campos estado $\in \{\text{OK}, \text{ERROR}\}$, `codigo_http` y mensaje opcionales.
- **GC2.2 [M]** El sistema exigirá a toda clase `IConnector` la implementación del método `get_artifact_info(id)` que devuelva los metadatos normalizados {título, tipo, estado, responsable, fecha_modificacion}.
 - *GC2.2.1* El sistema exigirá a la implementación de `get_artifact_info` devolver el valor `None` sin elevar excepción cuando el identificador no exista en la fuente

externa.

- **GC2.3 [M]** El sistema exigirá a toda clase `IConnector` la implementación del método `get_artifact_status(id)` que devuelva el estado del artefacto mapeado al vocabulario estándar del [SVAES](#).
- **GC2.4 [S]** El sistema exigirá a toda clase `IConnector` la implementación del método `search_artifacts(query)` para búsquedas acotadas usadas en la pantalla de importación masiva.

Nota de implementación. Durante el desarrollo, los métodos `get_artifact_info` y `get_artifact_status` se consolidaron en `fetch_artifact(ref, config)`, que devuelve en una sola llamada los metadatos y el estado del artefacto. Análogamente, `search_artifacts` se implementó como `list_artifacts(filter_params, config)` para alinearse con las APIs de paginación de los sistemas externos. Véase el *listing 5* en el capítulo 6.

- **GC3 [M]** El sistema ejecutará automáticamente una prueba de conectividad al registrar o modificar un conector.
- **GC3.1 [M]** El sistema asignará al conector el estado `EN_PRUEBA` durante la ejecución de `test_connection()`.
- **GC3.2 [M]** El sistema asignará al conector el estado `ACTIVO` cuando `test_connection()` devuelva estado `OK`.
- **GC3.3 [M]** El sistema asignará al conector el estado `INACTIVO` cuando `test_connection()` devuelva estado `ERROR` o la llamada supere el *timeout* configurado.
- **GC3.4 [M]** El sistema mostrará al usuario registrador el código HTTP y el mensaje original devueltos por la fuente cuando la prueba falle.
 - *GC3.4.1* El sistema enmascarará las credenciales con el literal «*****» en cualquier mensaje de traza o error devuelto al usuario.
- **GC3.5 [M]** El sistema aplicará a las llamadas de `test_connection()` un *timeout* de 10 segundos (configurable a nivel de variables de entorno o archivo de configuración local del servidor).
- **GC4 [M]** El sistema gestionará el estado operativo de los conectores combinando intervención manual y políticas automáticas.
- **GC4.1 [M]** El sistema permitirá al *owner* y a los U4 activar manualmente un conector en estado `INACTIVO`.
 - *GC4.1.1* El sistema volverá a ejecutar `test_connection()` en cada activación manual y confirmará la activación únicamente si el resultado es `OK`.
- **GC4.2 [M]** El sistema permitirá al *owner* y a los U4 desactivar manualmente un conector en estado `ACTIVO`.
- **GC4.3 [M]** El sistema desactivará automáticamente un conector en estado `ACTIVO` tras 3 fallos de comunicación consecutivos en las últimas 24 horas.

- *GC4.3.1* El sistema reiniciará el contador de fallos consecutivos tras una llamada exitosa al conector o tras una activación manual.
- **GC4.4 [S]** El sistema enviará una notificación a los U4 y al *owner* de la organización afectada cada vez que ocurra una desactivación automática.

Los requisitos GC5 en adelante describen los conectores de referencia incluidos en la implementación inicial del **SVAES** (Jira y GitLab), que sirven como prueba de concepto del puerto IConnector. El catálogo final se amplió durante el desarrollo hasta cubrir veinte implementaciones. El conector Jira ilustra cómo implementar el puerto IConnector para un sistema concreto de gestión de tareas, pero no forma parte del núcleo del sistema: cualquier equipo puede sustituirlo o complementarlo con conectores propios sin modificar ningún componente del sistema base.

- **GC5 [M]** El sistema incluirá una implementación de referencia de IConnector sobre la **API REST** v2 de Jira.
- **GC5.1 [M]** El conector Jira autenticará sus peticiones mediante un *Personal Access Token* (PAT) configurable por conector.
- **GC5.2 [M]** El conector Jira admitirá una URL base de servidor Jira configurable en el parámetro `jira_base_url`.
- **GC5.3 [S]** El conector Jira implementará el método `search_artifacts()` aceptando como entrada una expresión JQL válida.
- **GC5.4 [M]** El conector Jira normalizará los estados nativos (*To Do*, *In Progress*, *Done*, *Closed*) a los estados estándar del vocabulario **SVAES**.
 - *GC5.4.1* El sistema permitirá al usuario redefinir el mapeo de estados Jira→**SVAES** en el parámetro de configuración `status_mapping`.
- **GC5.5 [S]** El conector Jira extraerá en `get_artifact_info` los campos `summary`, `status`, `assignee`, `priority`, `story_points`, `time_tracking`, `subtasks` y `parent`.
- **GC6 [S]** El sistema mantendrá un historial de versiones de configuración de cada conector.
- **GC6.1 [S]** El sistema generará una nueva entrada de historial por cada modificación del conector indicando *timestamp*, usuario ejecutor y lista de campos modificados.
 - *GC6.1.1* El sistema mostrará el valor anterior y el valor nuevo de cada campo modificado salvo para los campos marcados como sensibles.
- **GC6.2 [S]** El sistema permitirá al *owner* y a los U4 consultar el historial de versiones de un conector.
- **GC6.3 [C]** El sistema permitirá al *owner* y a los U4 restaurar una versión anterior de configuración mediante una operación explícita que generará una nueva entrada en el historial.

FEAT-04: Definición de Políticas de Calidad (Perfiles y Plantillas)

- **PV1 [M]** El sistema permitirá al *owner* y a los U4 de una organización gestionar perfiles de verificación dentro de la misma.
- **PV1.1 [M]** El sistema exigirá a cada perfil un nombre único dentro de su organización de entre 3 y 80 caracteres.
- **PV1.2 [S]** El sistema permitirá asociar a cada perfil una descripción opcional de hasta 500 caracteres en UTF-8.
- **PV1.3 [M]** El sistema permitirá incluir en cada perfil una lista ordenada de reglas, cada una con un nivel asignado del conjunto {OBLIGATORIA, OPCIONAL, EXCLUIDA}.
 - *PV1.3.1* El sistema descartará de la evaluación toda regla con nivel EXCLUIDA en la fase de resolución del perfil, antes de enviar el trabajo al motor de verificación. Dichas reglas no se ejecutan ni generan resultado alguno.
- **PV1.4 [S]** El sistema permitirá al *owner* y a los U4 duplicar un perfil existente como punto de partida para uno nuevo.
 - *PV1.4.1* El sistema copiará en la duplicación el conjunto de reglas del perfil origen junto con el nivel asignado a cada una.
- **PV1.5 [M]** El sistema permitirá al *owner* y a los U4 modificar los perfiles existentes en su organización.
- **PV1.6 [M]** El sistema rechazará la eliminación de un perfil referenciado por proyectos o *releases* activas.
 - *PV1.6.1* El sistema mostrará al usuario la lista de proyectos y *releases* que dependen del perfil antes de confirmar la imposibilidad de eliminación.
- **PV1.7 [M]** El sistema no propagará los cambios de un perfil a las verificaciones ya registradas.
 - *PV1.7.1* El sistema almacenará un *snapshot* del perfil en el momento de cada verificación para garantizar la reproducibilidad histórica.
- **PV2 [M]** El sistema incluirá un perfil por defecto no eliminable disponible en todas las organizaciones.
- **PV2.1 [M]** El sistema configurará en el perfil por defecto las reglas RV-01 a RV-10 con los niveles de severidad recomendados: ERROR para RV-01, RV-02, RV-03, RV-04, RV-05, RV-07, RV-08 y RV-10; WARNING para RV-06 y RV-09.
- **PV2.2 [M]** El sistema mostrará el perfil por defecto como seleccionable en todas las organizaciones.
- **PV2.3 [M]** El sistema rechazará toda operación de modificación o eliminación sobre el perfil por defecto.
 - *PV2.3.1* El sistema ofrecerá al usuario la operación de duplicar el perfil por defecto como mecanismo para obtener una variante personalizada.

- **PV3 [M]** El sistema permitirá asociar un perfil de verificación a los proyectos y a las *releases* individuales.
- **PV3.1 [M]** El sistema permitirá al *owner* y a los U4 asignar a cada proyecto un perfil de verificación por defecto.
 - *PV3.1.1* El sistema aplicará el perfil global por defecto como perfil del proyecto cuando ninguno haya sido asignado explícitamente.
- **PV3.2 [M]** El sistema asignará automáticamente a cada *release* recién creada el perfil por defecto del proyecto al que pertenece.
- **PV3.3 [S]** El sistema permitirá al usuario seleccionar un perfil alternativo de entre los disponibles en la organización al crear una *release*.
- **PV3.4 [M]** El sistema permitirá al usuario cambiar el perfil asignado a una *release* únicamente mientras ésta se encuentre en estado BORRADOR.
- **PV4 [S]** El sistema permitirá al *owner* y a los U4 de una organización gestionar plantillas de *release* dentro de la misma.
- **PV4.1 [S]** El sistema exigirá a cada plantilla un nombre único dentro de su organización de entre 3 y 80 caracteres.
- **PV4.2 [C]** El sistema permitirá asociar a cada plantilla una descripción opcional de hasta 500 caracteres en UTF-8.
- **PV4.3 [S]** El sistema permitirá declarar en cada plantilla una lista de artefactos requeridos.
 - *PV4.3.1* El sistema exigirá a cada artefacto requerido los campos tipo (TAREA/CÓDIGO/DOCUMENTO), descripción breve y conector sugerido.
- **PV4.4 [S]** El sistema exigirá a cada plantilla un perfil de verificación asociado seleccionado de entre los de la organización.
- **PV4.5 [C]** El sistema permitirá al *owner* y a los U4 duplicar una plantilla existente como punto de partida para una nueva.
- **PV4.6 [S]** El sistema permitirá al *owner* y a los U4 archivar una plantilla de forma reversible.
- **PV4.7 [S]** El sistema rechazará el archivado de una plantilla referenciada como plantilla por defecto en cualquier proyecto activo.

4.11.3. Épica 3 — Núcleo Operativo (Motor de Verificación)

Esta épica agrupa los requisitos que constituyen el corazón del [SVAES](#): la gestión del ciclo de vida de las *releases* y la ejecución automatizada de las verificaciones. Cubre las funcionalidades FEAT-05 y FEAT-06.

FEAT-05: Orquestación del Ciclo de Vida de Releases

- **GR1 [M]** El sistema permitirá a U2 y U4 crear una nueva *release* asociada a un proyecto al que el usuario esté asignado.
- **GR1.1 [M]** El sistema exigirá al usuario un nombre de *release* de entre 1 y 100 caracteres al crearla.
- **GR1.2 [M]** El sistema exigirá al usuario una versión de *release* conforme al formato Semantic Versioning 2.0.0 (MAJOR.MINOR.PATCH con *pre-release* y *build metadata* opcionales).
 - *GR1.2.1* El sistema rechazará la creación con un error de validación cuando la versión indicada no cumpla el patrón SemVer.
- **GR1.3 [M]** El sistema exigirá al usuario seleccionar un proyecto de entre los proyectos de su organización a los que esté asignado.
- **GR1.4 [S]** El sistema permitirá al usuario introducir una descripción opcional de hasta 1000 caracteres en UTF-8.
- **GR1.5 [S]** El sistema permitirá al usuario indicar una fecha prevista de entrega opcional en formato ISO 8601 no anterior a la fecha actual del servidor.
- **GR1.6 [S]** El sistema permitirá al usuario seleccionar como responsable opcional a uno de los miembros del proyecto.
- **GR1.7 [C]** El sistema permitirá al usuario asociar a la *release* una lista opcional de etiquetas de texto libre para clasificación y filtrado.
 - *GR1.7.1* El sistema limitará cada etiqueta a 30 caracteres y el total de etiquetas por *release* a 20.
- **GR1.8 [S]** El sistema permitirá al usuario seleccionar una plantilla de *release* de entre las plantillas activas del proyecto como origen de la creación.
- **GR1.9 [M]** El sistema rechazará la creación cuando la tupla (nombre, versión) ya exista en el mismo proyecto.
 - *GR1.9.1* El sistema mostrará al usuario un mensaje que identifique la *release* existente y enlace a su ficha cuando se detecte un duplicado.
- **GR1.10 [M]** El sistema inicializará toda *release* recién creada en estado BORRADOR.
- **GR1.11 [M]** El sistema registrará automáticamente en cada *release* el usuario creador, la fecha y hora de creación, la organización y el proyecto.
- **GR2 [M]** El sistema permitirá a los miembros de una organización listar las *releases* de los proyectos a los que están asignados.
- **GR2.1 [M]** El sistema mostrará en el listado por cada *release* el nombre, la versión, el proyecto, el estado, la fecha prevista, el responsable y el resultado de la última verificación.
- **GR2.2 [S]** El sistema permitirá al usuario filtrar el listado por proyecto, estado,

responsable, etiquetas, rango de fechas de creación y rango de fechas previstas.

- **GR2.3 [S]** El sistema permitirá al usuario filtrar el listado por texto libre aplicando coincidencia parcial, insensible a mayúsculas y acentos, sobre los campos nombre y descripción.
- **GR2.4 [S]** El sistema paginará el listado con un tamaño por defecto de 25 elementos por página.
 - *GR2.4.1* El sistema permitirá al usuario seleccionar el tamaño de página entre los valores 10, 25, 50 y 100.
- **GR2.5 [C]** El sistema permitirá al usuario ordenar el listado de forma ascendente o descendente por los campos nombre, versión, estado y fecha prevista.
- **GR3 [M]** El sistema permitirá al responsable de una *release* o a un U4 de su organización modificar los metadatos de la *release*.
- **GR3.1 [M]** El sistema aceptará modificaciones de metadatos únicamente cuando la *release* esté en estado BORRADOR o PENDIENTE.
 - *GR3.1.1* El sistema rechazará toda modificación sobre *releases* en cualquier otro estado devolviendo HTTP 409 *Conflict*.
- **GR3.2 [M]** El sistema permitirá modificar los campos nombre, versión, descripción, fecha prevista, responsable, etiquetas y plantilla origen, y rechazará la modificación del campo proyecto.
- **GR3.3 [M]** El sistema aplicará a las modificaciones las mismas validaciones que a la creación definidas en GR1.1 a GR1.9.
- **GR3.4 [S]** El sistema registrará cada modificación en un log de cambios indicando campo modificado, valor anterior, valor nuevo, usuario y *timestamp*.
 - *GR3.4.1* El sistema hará visible el log de cambios de una *release* a los miembros del proyecto con rol U2, U4, *owner* o U3.
- **GR4 [M]** El sistema gestionará la retirada de *releases* mediante eliminación física o archivado lógico en función de su historial.
- **GR4.1 [M]** El sistema permitirá eliminar físicamente una *release* únicamente cuando se encuentre en estado BORRADOR o PENDIENTE y no tenga ninguna verificación registrada.
- **GR4.2 [M]** El sistema solicitará al usuario que escriba el nombre y la versión exactos de la *release* como confirmación antes de ejecutar la eliminación física.
- **GR4.3 [S]** El sistema permitirá archivar cualquier *release* con historial de verificaciones mediante una operación de archivado lógico.
 - *GR4.3.1* El sistema conservará en el archivado el *snapshot* del perfil utilizado en cada verificación histórica para garantizar la reproducibilidad.
- **GR4.4 [M]** El sistema rechazará toda transición de estado sobre una *release* archivada, salvo la operación explícita de restauración reservada a U3.
- **GR5 [M]** El sistema asociará a cada *release* una lista de artefactos como unidades

de contenido verificable.

- **GR5.1 [M]** El sistema almacenará cada artefacto con tipo (TAREA, CÓDIGO o DOCUMENTO), conector asociado, identificador en la fuente y descripción opcional.
- **GR5.2 [M]** El sistema permitirá al usuario añadir artefactos a una *release* en estado BORRADOR o PENDIENTE de forma individual desde el formulario.
- **GR5.3 [S]** El sistema permitirá al usuario importar artefactos en bloque mediante un fichero CSV.
 - *GR5.3.1* El sistema exigirá al CSV las columnas tipo, conector, identificador y descripción en ese orden.
 - *GR5.3.2* El sistema reportará los errores de formato del CSV línea a línea, permitiendo al usuario corregir antes de confirmar la importación.
- **GR5.4 [M]** El sistema rechazará la adición de un artefacto cuando el conector asociado no exista o no esté en estado ACTIVO. Una vez añadido, si el conector pasa posteriormente a INACTIVO, los artefactos ya asociados se conservan pero las reglas dependientes de dicho conector se marcarán como NO_EVALUADA durante la verificación (MV3.4).
- **GR5.5 [M]** El sistema permitirá al usuario eliminar un artefacto únicamente cuando la *release* esté en estado BORRADOR o PENDIENTE.
- **GR5.6 [S]** El sistema mostrará en la ficha de *release* los artefactos agrupados por tipo con recuento por conector.
- **GR6 [M]** El sistema gestionará el ciclo de vida de las *releases* conforme a la máquina de estados definida.
- **GR6.1 [M]** El sistema reconocerá como estados válidos de *release* los valores BORRADOR, PENDIENTE, EN_VERIFICACION, VALIDA, CON_ADVERTENCIAS, NO_VALIDA y ARCHIVADA.
- **GR6.2 [M]** El sistema aceptará únicamente transiciones de estado válidas y secuenciales.
 - *GR6.2.1* El sistema rechazará toda transición no prevista devolviendo HTTP 409 *Conflict* con un mensaje que indique el estado actual y las transiciones válidas.
- **GR6.3 [S]** El sistema permitirá al responsable de la *release* o a un U4 devolverla del estado PENDIENTE al estado BORRADOR mientras no exista ninguna verificación iniciada.
 - *GR6.3.1* Al devolver una *release* a estado BORRADOR, el sistema conservará los artefactos y metadatos asociados sin alteración.
- **GR7 [M]** El sistema permitirá al *owner* y a los U4 de una organización gestionar los proyectos de esta.
- **GR7.1 [M]** El sistema exigirá a cada proyecto un nombre único dentro de su organización (3-80 caracteres), una clave corta (2-10 caracteres alfanuméricos en

mayúsculas), una descripción opcional y un perfil de verificación por defecto.

- *GR7.1.1* El sistema permitirá al *owner* y a los U4 modificar el nombre, la descripción y el perfil de verificación por defecto de un proyecto existente.
- **GR7.2 [S]** El sistema permitirá archivar un proyecto conservando sus *releases* y su historial de verificaciones en modo solo lectura.
 - *GR7.2.1* El sistema rechazará la creación de nuevas *releases* en proyectos archivados.
 - *GR7.2.2* El sistema rechazará cualquier modificación, transición de estado o re-verificación sobre *releases* pertenecientes a un proyecto archivado.
- **GR8 [S]** El sistema permitirá crear una *release* a partir de una plantilla de *release* precargando su contenido inicial.
- **GR8.1 [S]** El sistema precargará en la nueva *release* los artefactos requeridos declarados en la plantilla seleccionada.
- **GR8.2 [S]** El sistema precargará en la nueva *release* el perfil de verificación asociado a la plantilla seleccionada.
- **GR8.3 [S]** El sistema permitirá al usuario modificar los artefactos precargados antes de guardar la nueva *release*.
- **GR9 [C]** El sistema permitirá clonar una *release* existente como punto de partida para una nueva *release*.
- **GR9.1 [C]** El sistema copiará en la clonación los metadatos modificables, la lista de artefactos y el perfil asignado de la *release* origen.
- **GR9.2 [C]** El sistema excluirá de la clonación el historial de verificaciones de la *release* origen.

FEAT-06: Ejecución Automatizada de Verificaciones (Quality Gate)

- **MV1 [M]** El sistema permitirá al responsable de una *release* y a los U4 de su organización ejecutar verificaciones sobre la misma.
- **MV1.1 [M]** El sistema aceptará la ejecución de verificaciones únicamente sobre *releases* en estado PENDIENTE, VALIDA, NO_VALIDA o CON_ADVERTENCIAS.
- **MV1.2 [M]** El sistema rechazará toda nueva verificación sobre una *release* que tenga otra verificación en estado EN_PROGRESO.
 - *MV1.2.1* El sistema devolverá HTTP 409 *Conflict* con un mensaje que indique el identificador de la verificación en curso.
- **MV1.3 [M]** El sistema encolará cada verificación como una tarea asíncrona en segundo plano y devolverá al usuario el identificador de la tarea inmediatamente.
- **MV1.4 [S]** El sistema mostrará al usuario el progreso de la verificación en curso como la razón reglas completadas / reglas totales, actualizada tras cada regla evaluada.
- **MV1.5 [S]** El sistema enviará una notificación al responsable de la *release* al finalizar la verificación a través de los canales configurados.

- **MV2 [M]** El sistema evaluará las reglas del perfil de forma secuencial e independiente durante cada verificación.
- **MV2.1 [M]** El sistema identificará para cada regla el conector necesario a partir de la declaración de la regla en el perfil.
- **MV2.2 [M]** El sistema obtendrá los datos del artefacto objetivo de la regla invocando el método de `IConnector` correspondiente al tipo de consulta declarado en la regla (`get_artifact_info`, `get_artifact_status` o `search_artifacts`).
- **MV2.3 [M]** El sistema evaluará la condición de la regla sobre los datos obtenidos y registrará el resultado en el informe.
- **MV2.4 [M]** El sistema respetará en la evaluación el orden en que las reglas están declaradas en el perfil.
- **MV2.5 [M]** El sistema consultará cada conector requerido de forma independiente cuando una regla implique información de varios conectores.
- **MV2.6 [M]** El sistema asignará a cada regla evaluada un resultado del conjunto {OK, ERROR, WARNING, NO_EVALUADA}.
- **MV3 [M]** El sistema gestionará los fallos de comunicación con los conectores durante una verificación sin abortarla.
- **MV3.1 [M]** El sistema capturará toda excepción de tipo `ConectorException` lanzada durante la evaluación de una regla.
- **MV3.2 [M]** El sistema marcará como NO_EVALUADA cada regla afectada por un `ConectorException` conservando el detalle del error.
 - *MV3.2.1* El sistema conservará en el detalle el tipo de excepción, el código HTTP cuando aplique y la primera línea del mensaje original.
- **MV3.3 [M]** El sistema aplicará a cada llamada a conector un *timeout* configurable.
- **MV3.4 [M]** El sistema marcará como NO_EVALUADA las reglas cuyo conector asociado esté en estado INACTIVO, sin intentar la comunicación.
- **MV3.5 [M]** El sistema continuará la evaluación de las reglas restantes después de cada regla marcada como NO_EVALUADA.
- **MV3.6 [S]** El sistema registrará en el log de la verificación cada fallo de conector indicando conector afectado, tipo de error, código HTTP y *timestamp*.
- **MV4 [M]** El sistema calculará el resultado global de cada verificación aplicando una política de agregación por severidad.
- **MV4.1 [M]** El sistema asignará el resultado global NO_VALIDA cuando al menos una regla de nivel OBLIGATORIA presente resultado ERROR.
- **MV4.2 [M]** El sistema asignará el resultado global CON_ADVERTENCIAS cuando todas las reglas OBLIGATORIAS sean OK y al menos una regla OPCIONAL presente resultado WARNING.
- **MV4.3 [M]** El sistema asignará el resultado global VALIDA cuando todas las reglas OBLIGATORIAS y OPCIONALES activas sean OK.

- **MV4.4 [M]** El sistema añadirá al resultado global el sufijo `_CON_INCIDENCIAS` cuando exista al menos una regla `NO_EVALUADA` en la verificación.
 - *MV4.4.1* El sistema mostrará el sufijo `_CON_INCIDENCIAS` como un indicador secundario adyacente al resultado principal en la interfaz.
- **MV4.5 [M]** El sistema tratará un resultado `WARNING` en una regla de nivel `OBLIGATORIA` como equivalente a `ERROR` a efectos del cómputo del veredicto global (MV4.1), garantizando que el resultado global sea `NO_VALIDA`.
- **MV5 [M]** El sistema incluirá un catálogo de diez reglas de verificación predefinidas identificadas como `RV-01` a `RV-10`.
- **MV5.1 [M]** El sistema implementará la regla `RV-01` «existencia de artefactos» con severidad `OBLIGATORIA`.
 - *MV5.1.1* La regla `RV-01` resultará `OK` cuando la lista de artefactos recuperados del conector no esté vacía.
 - *MV5.1.2* La regla `RV-01` resultará `NO_EVALUADA` cuando el conector asociado se encuentre `INACTIVO`.
- **MV5.2 [M]** El sistema implementará la regla `RV-02` «trazabilidad entre artefactos» con severidad `OBLIGATORIA`.
 - *MV5.2.1* La regla `RV-02` resultará `OK` cuando todos los artefactos de tipo `source_type` referencien correctamente a artefactos de tipo `target_type` a través del campo `reference_field`.
 - *MV5.2.2* La regla `RV-02` aceptará como parámetros configurables `source_type`, `target_type` y `reference_field`.
- **MV5.3 [M]** El sistema implementará la regla `RV-03` «validación de estados» con severidad `OBLIGATORIA`.
 - *MV5.3.1* La regla `RV-03` resultará `OK` cuando todos los artefactos de tipo `artifact_type` posean un valor perteneciente a la lista `allowed_states` en el campo `status_field`.
- **MV5.4 [M]** El sistema implementará la regla `RV-04` «integridad de campos numéricos» con severidad `OBLIGATORIA`.
 - *MV5.4.1* La regla `RV-04` resultará `OK` cuando los campos listados en `numeric_fields` no sean nulos, contengan valores numéricos y sean mayores o iguales que cero en todos los artefactos del tipo `artifact_type`.
- **MV5.5 [M]** El sistema implementará la regla `RV-05` «disponibilidad de documentos» con severidad `OBLIGATORIA`.
 - *MV5.5.1* La regla `RV-05` resultará `OK` cuando el campo `accessible_field` de cada artefacto del tipo `artifact_type` indique que el documento asociado está accesible.
- **MV5.6 [S]** El sistema implementará la regla `RV-06` «coherencia de atributos» con severidad `OPCIONAL`.

- *MV5.6.1* La regla RV-06 resultará OK cuando el campo `attribute` coincida con `expected_value` en todos los artefactos del tipo `artifact_type`.
- **MV5.7 [M]** El sistema implementará la regla RV-07 «registro externo» con severidad OBLIGATORIA.
 - *MV5.7.1* La regla RV-07 resultará OK cuando el campo `marker_field` esté presente y no vacío en todos los artefactos del tipo `artifact_type`.
- **MV5.8 [M]** El sistema implementará la regla RV-08 «alineación de listas» con severidad OBLIGATORIA.
 - *MV5.8.1* La regla RV-08 resultará OK cuando cada entrada de la lista dependiente de tipo `target_type` tenga correspondencia en la lista maestra identificada por `master_artifact_id` a través del campo `master_field`.
- **MV5.9 [S]** El sistema implementará la regla RV-09 «validación de referencias» con severidad OPCIONAL.
 - *MV5.9.1* La regla RV-09 resultará OK cuando los campos listados en `reference_fields` de cada artefacto del tipo `artifact_type` apunten a recursos cuyo campo `accessible_field` confirma que siguen accesibles.
- **MV5.10 [M]** El sistema implementará la regla RV-10 «aprobación final» con severidad OBLIGATORIA.
 - *MV5.10.1* La regla RV-10 resultará OK cuando el campo `status_field` de cada artefacto del tipo `artifact_type` contenga un valor perteneciente a la lista `approved_states`.
- **MV6 [S]** El sistema soportará reglas de verificación personalizadas definidas en ficheros de configuración estructurados.
- **MV6.1 [S]** El sistema exigirá a cada regla personalizada declarar los campos `id`, `nombre`, `descripcion`, `tipo_artefacto`, `tipo_conector`, `campo_evaluar`, `operador`, `valor_esperado` y `severidad`.
 - *MV6.1.1* El sistema aceptará como operador cualquier valor del conjunto `{equals, not_null, in, regex, gt, lt}`.
- **MV6.2 [S]** El sistema cargará las reglas personalizadas en el momento de arranque de la aplicación.
 - *MV6.2.1* El sistema permitirá recargar las reglas personalizadas sin reiniciar mediante un *endpoint* administrativo protegido por rol U3.
- **MV6.3 [M]** El sistema rechazará el registro de reglas personalizadas cuyo identificador coincida con alguno de RV-01 a RV-10.
- **MV6.4 [S]** El sistema validará la sintaxis de cada fichero de reglas personalizadas en el arranque.
 - *MV6.4.1* El sistema registrará un error en el log y omitirá el fichero de reglas inválido sin interrumpir el arranque de la aplicación.
- **MV7 [M]** El sistema ejecutará todas las verificaciones de forma asíncrona en segundo

plano.

- **MV7.1** [M] El sistema encolará cada verificación con prioridad normal en la cola de procesamiento asíncrono por defecto.
- **MV7.2** [M] El sistema expone al usuario el estado de la tarea como uno de los valores {ENCOLADA, EN_PROGRESO, COMPLETADA, FALLIDA}.
 - *MV7.2.1* El sistema actualizará el estado EN_PROGRESO tras la evaluación de cada regla individual de la verificación.
- **MV7.3** [M] El sistema asignará el resultado global FALLIDA a la verificación cuando el proceso asíncrono falle por un error interno ajeno a los conectores.
 - *MV7.3.1* El sistema enviará una notificación a los U3 con el identificador de la verificación y la traza del error cuando ocurra un fallo interno.
- **MV7.4** [S] El sistema permitirá configurar el número máximo de trabajadores asíncronos concurrentes mediante parámetro de configuración del sistema.

4.11.4. Épica 4 — Observabilidad y Comunicación

Esta épica agrupa los requisitos que permiten auditar, visualizar y comunicar los resultados del sistema. Cubre las funcionalidades FEAT-07, FEAT-08 y FEAT-09.

FEAT-07: Trazabilidad y Auditoría de Resultados

- **RH1** [M] El sistema generará un informe de verificación consultable al finalizar cada ejecución del motor.
- **RH1.1** [M] El sistema mostrará en la cabecera del informe el identificador de la verificación, el nombre y versión de la *release*, el perfil aplicado, la fecha de inicio, la fecha de fin, la duración y el resultado global.
 - *RH1.1.1* El sistema expresará la duración en el formato mm:ss cuando sea inferior a una hora y en el formato hh:mm cuando sea igual o superior a una hora.
- **RH1.2** [M] El sistema mostrará en el informe una tabla con una fila por cada regla evaluada incluyendo identificador, nombre, nivel de obligatoriedad, resultado y detalle de la incidencia cuando proceda.
- **RH1.3** [M] El sistema mostrará en la fila de detalle de cada regla no OK la descripción del fallo, la fuente de datos consultada, el valor esperado, el valor obtenido cuando aplique y el código de error cuando aplique.
 - *RH1.3.1* El sistema incluirá en el detalle de toda regla con resultado derivado de una excepción de conector el tipo de excepción y la primera línea del mensaje original.
- **RH1.4** [S] El sistema mostrará al final del informe una sección de resumen de conectores con error cuando existan reglas NO_EVALUADAS, indicando conector

afectado y tipo de fallo.

- **RH1.5 [M]** El sistema mostrará junto al resultado global del informe un indicador visual coherente (verde para VALIDA, amarillo para CON_ADVERTENCIAS y rojo para NO_VALIDA).
- **RH2 [M]** El sistema persistirá cada verificación ejecutada en la tabla de historial.
- **RH2.1 [M]** El sistema almacenará por cada verificación los campos `id`, `release_id`, `organization_id`, `project_id`, `perfil_snapshot_json`, `usuario_id`, `fecha_inicio`, `fecha_fin`, `duracion_segundos`, `resultado_global`, `detalle_reglas_json` y `log_conectores_json`.
 - *RH2.1.1* El sistema almacenará en el campo `perfil_snapshot_json` una copia inmutable y completa del perfil de verificación vigente en el momento de la ejecución.
- **RH2.2 [M]** El sistema garantizará la inmutabilidad de los registros del historial frente a cualquier cambio posterior en el perfil, las reglas o la *release*.
- **RH2.3 [M]** El sistema permitirá a los usuarios con acceso a un proyecto consultar el historial completo de las *releases* de ese proyecto.
- **RH2.4 [M]** El sistema permitirá a los usuarios con acceso a una *release* consultar el historial restringido a esa *release*.
- **RH2.5 [S]** El sistema ordenará por defecto los registros del historial cronológicamente de más reciente a más antiguo.
- **RH3 [S]** El sistema permitirá al usuario consultar el historial aplicando filtros combinables.
- **RH3.1 [S]** El sistema ofrecerá los filtros organización, proyecto, *release*, usuario ejecutor, rango de fechas y resultado global.
- **RH3.2 [S]** El sistema aplicará todos los filtros activos simultáneamente con lógica AND al calcular el conjunto de resultados.
 - *RH3.2.1* El sistema mostrará los filtros activos como etiquetas eliminables en la interfaz para facilitar el ajuste iterativo de la consulta.
- **RH3.3 [S]** El sistema paginará los resultados del historial con un tamaño por defecto de 25 registros por página.
- **RH4 [S]** El sistema permitirá exportar informes individuales y conjuntos de historial en formatos estándar.
- **RH4.1 [S]** El sistema permitirá exportar un informe de verificación concreto en formato PDF.
 - *RH4.1.1* El sistema incluirá en el PDF exportado todos los campos definidos en RH1.1 a RH1.5.
 - *RH4.1.2* El sistema incluirá en el pie de página del PDF el *timestamp* de generación, el identificador de la verificación y una firma del sistema.
- **RH4.2 [S]** El sistema permitirá exportar el historial filtrado de un proyecto en

formato CSV con una fila por verificación.

- *RH4.2.1* El sistema incluirá como columnas del CSV los campos de cabecera definidos en RH2.1.
- **RH4.3 [S]** El sistema ejecutará la generación del PDF de forma síncrona (al corresponder siempre a un único informe de verificación). La exportación del CSV se ejecutará de forma síncrona cuando el conjunto a exportar contenga menos de 500 registros.
- **RH4.4 [S]** El sistema ejecutará la generación del CSV de forma asíncrona cuando el conjunto a exportar contenga 500 o más registros y notificará al usuario al completarse.

FEAT-08: Inteligencia de Negocio y Métricas de Calidad (Dashboard)

- **DB1 [S]** El sistema mostrará al usuario autenticado una pantalla de *dashboard* con indicadores de calidad del proceso de entrega.
- **DB1.1 [S]** El sistema presentará el *dashboard* como pantalla de destino tras el inicio de sesión y la selección de organización.
- **DB1.2 [S]** El sistema limitará el ámbito de datos del *dashboard* a los proyectos asignados al usuario cuando éste tenga rol U2.
- **DB1.3 [S]** El sistema extenderá el ámbito de datos del *dashboard* a la organización completa cuando el usuario tenga rol U4 o sea el *owner* de la misma.
- **DB1.4 [S]** El sistema permitirá al rol U3 seleccionar una organización concreta o seleccionar la agregación de todas las organizaciones en el *dashboard*.
 - *DB1.4.1* El sistema recargará los indicadores sin navegación adicional cuando el usuario cambie el ámbito seleccionado.
- **DB1.5 [C]** El sistema cargará los datos del *dashboard* de forma asíncrona devolviendo primero el esqueleto de la vista y después cada indicador a medida que esté calculado.
- **DB2 [S]** El sistema permitirá al usuario filtrar el contenido del *dashboard* por criterios combinables.
- **DB2.1 [S]** El sistema permitirá filtrar por organización, por proyecto y por periodo temporal.
- **DB2.2 [S]** El sistema ofrecerá como valores predefinidos de periodo últimos 7 días, últimos 30 días, últimos 90 días, este año y rango personalizado.
 - *DB2.2.1* El sistema exigirá en el rango personalizado dos fechas en formato ISO 8601, rechazando combinaciones en las que la fecha de fin sea anterior a la de inicio.
- **DB2.3 [S]** El sistema mostrará en el selector de proyecto únicamente los proyectos accesibles al usuario según su rol y ámbito.
- **DB2.4 [S]** El sistema combinará todos los filtros activos con lógica AND al calcular los indicadores.

- *DB2.4.1* El sistema indicará al usuario el número de registros cubiertos por la selección actual en un contador visible.
- **DB3 [S]** El sistema mostrará en el *dashboard* un conjunto predefinido de indicadores de calidad.
- **DB3.1 [S]** El sistema calculará y mostrará el indicador *Tasa de éxito de verificaciones* como el porcentaje de verificaciones con resultado VALIDA (incluyendo la variante VALIDA_CON_INCIDENCIAS) sobre el total de verificaciones del periodo seleccionado.
 - *DB3.1.1* El sistema representará el indicador *Tasa de éxito* con color verde cuando el valor sea igual o superior al 80 %, amarillo cuando esté entre el 50 % y el 80 %, y rojo cuando sea inferior al 50 %.
- **DB3.2 [S]** El sistema calculará y mostrará el indicador *Reglas más frecuentemente fallidas* como el listado de las 5 reglas con mayor número de resultados ERROR o WARNING en el periodo seleccionado.
 - *DB3.2.1* El sistema mostrará por cada regla del listado su identificador, su nombre, el número de ocurrencias y el porcentaje sobre el total de evaluaciones de esa regla.
 - *DB3.2.2* El sistema enlazará cada regla del listado a la vista del historial filtrada por esa regla y por el periodo activo.
- **DB3.3 [S]** El sistema calculará y mostrará el indicador *Evolución temporal de verificaciones* como un gráfico con la distribución de resultados por unidad de tiempo en el periodo seleccionado.
 - *DB3.3.1* El sistema utilizará granularidad diaria en el indicador *Evolución temporal* cuando el periodo seleccionado sea menor o igual a 31 días.
 - *DB3.3.2* El sistema utilizará granularidad semanal en el indicador *Evolución temporal* cuando el periodo seleccionado sea mayor a 31 días.
- **DB3.4 [S]** El sistema calculará y mostrará el indicador *Tiempo medio de validación* como la media de duración entre la creación de una *release* y su primer resultado VALIDA en el periodo seleccionado.
 - *DB3.4.1* El sistema excluirá del cálculo del indicador *Tiempo medio de validación* las *releases* que no hayan alcanzado el estado VALIDA.
- **DB3.5 [S]** El sistema calculará y mostrará el indicador *Distribución de releases por estado* como un recuento de *releases* en cada uno de los estados del ciclo de vida.
- **DB3.6 [C]** El sistema calculará y mostrará el indicador *Actividad de conectores* como el listado de conectores de la organización con su estado actual y el número de verificaciones en las que han participado en el periodo.
 - *DB3.6.1* El sistema destacará visualmente en el indicador *Actividad de conectores* aquellos conectores que hayan acumulado 3 o más fallos en el periodo seleccionado.

- **DB4 [C]** El sistema permitirá al usuario exportar los datos del *dashboard* activo en formato CSV.
- **DB4.1 [C]** El sistema ejecutará la exportación de forma síncrona cuando el conjunto a exportar contenga menos de 500 filas.
- **DB4.2 [C]** El sistema ejecutará la exportación de forma asíncrona cuando el conjunto a exportar contenga 500 o más filas.
 - *DB4.2.1* El sistema enviará una notificación al usuario con el enlace de descarga al completarse una exportación asíncrona.
- **DB4.3 [C]** El sistema incluirá en el fichero CSV exportado una cabecera con los filtros aplicados, el periodo seleccionado y una fila por indicador con su valor numérico.

FEAT-09: Sistema de Alertas y Notificaciones Proactivas

- **NF1 [S]** El sistema permitirá al *owner* y a los U4 configurar canales de notificación dentro de su organización.
- **NF1.1 [S]** El sistema soportará los tipos de canal EMAIL_SMTP, SLACK_WEBHOOK y MSTEAMS_WEBHOOK.
- **NF1.2 [M]** El sistema aislará los canales configurados al ámbito de su organización.
 - *NF1.2.1* El sistema impedirá que los canales de una organización sean visibles o utilizables desde otra organización.
- **NF1.3 [S]** El sistema exigirá a cada canal EMAIL_SMTP los parámetros *host*, puerto (1-65535), usuario, contraseña, indicador de uso de TLS o SSL y dirección de remitente.
 - *NF1.3.1* El sistema cifrará en reposo la contraseña de cada canal EMAIL_SMTP mediante cifrado simétrico robusto.
- **NF1.4 [S]** El sistema exigirá a cada canal SLACK_WEBHOOK la URL del *webhook*.
 - *NF1.4.1* El sistema cifrará en reposo la URL de cada canal SLACK_WEBHOOK mediante cifrado simétrico robusto.
- **NF1.5 [S]** El sistema exigirá a cada canal MSTEAMS_WEBHOOK la URL del *webhook*.
 - *NF1.5.1* El sistema cifrará en reposo la URL de cada canal MSTEAMS_WEBHOOK mediante cifrado simétrico robusto.
- **NF1.6 [C]** El sistema permitirá al *owner* y a los U4 ejecutar una prueba de envío contra un canal configurado mediante una operación explícita.
 - *NF1.6.1* El sistema mostrará al usuario el código de respuesta y el primer mensaje de error devueltos por el canal cuando la prueba de envío falle.
- **NF1.7 [C]** El sistema permitirá configurar varios canales del mismo tipo simultáneamente dentro de una organización.
- **NF2 [S]** El sistema generará notificaciones automáticas ante la ocurrencia de un

conjunto predefinido de eventos.

- **NF2.1 [S]** El sistema generará una notificación de tipo *Verificación finalizada* al finalizar cada verificación, dirigida al responsable de la *release* y a los suscriptores del proyecto. El tipo se desglosa en dos subtipos según el veredicto: `RELEASE_VALIDATED` (para los veredictos `VALID` y `VALID_WITH_WARNINGS`) y `RELEASE_INVALIDATED` (para el resto de veredictos).
 - *NF2.1.1* El sistema incluirá en la notificación *Verificación finalizada* el nombre y versión de la *release*, el resultado global, el recuento de reglas por resultado y un enlace al informe.
 - *NF2.1.2* El sistema incluirá en la notificación *Verificación finalizada* un indicador visual coherente con el resultado global (verde/amarillo/rojo).
- **NF2.2 [M]** El sistema generará una notificación de tipo *Conector desactivado automáticamente* cada vez que se ejecute la política de desactivación definida.
 - *NF2.2.1* El sistema dirigirá la notificación *Conector desactivado automáticamente* al *owner* y a los U4 de la organización propietaria del conector.
 - *NF2.2.2* El sistema incluirá en la notificación *Conector desactivado automáticamente* el nombre del conector, su tipo, el motivo del último fallo y el número de proyectos afectados.
- **NF2.3 [C]** El sistema generará una notificación opcional de tipo *Release archivada* al archivar una *release* con historial, dirigida al responsable y al equipo del proyecto.
- **NF2.4 [S]** El sistema generará una notificación de tipo *Usuario invitado a organización* al completar una invitación, dirigida al usuario invitado.
 - *NF2.4.1* El sistema incluirá en la notificación *Usuario invitado a organización* el nombre de la organización y el rol asignado.
- **NF2.5 [C]** El sistema generará una notificación de tipo *Recordatorio de release pendiente* (`release_pending_reminder`) para las *releases* que permanezcan en estado PENDIENTE sin verificar transcurrido un plazo configurable, dirigida al responsable de la *release*.
- **NF2.6 [C]** El sistema generará una notificación de tipo *Resumen semanal* (`weekly_digest`) con las métricas agregadas de la organización (verificaciones realizadas, tasa de éxito, *releases* pendientes), dirigida a los suscriptores de la organización.
- **NF3 [S]** El sistema permitirá a cada usuario gestionar sus suscripciones a notificaciones.
- **NF3.1 [S]** El sistema permitirá al usuario activar o desactivar la recepción de notificaciones para cada tipo de evento (`release_validated`, `release_invalidated`, `release_pending_reminder` y `weekly_digest`) desde la sección de perfil.
 - *NF3.1.1* El sistema suscribirá automáticamente al responsable de cada *release* al evento *Verificación finalizada* de la misma.
 - *NF3.1.2* El sistema presentará cada tipo de notificación como un interruptor

(*toggle*) independiente en la tarjeta «Preferencias de notificación» de la página de perfil. Si el servidor rechaza la actualización, el componente revertirá el *toggle* a su estado anterior y mostrará un mensaje de error.

- **NF3.2** [C] El sistema permitirá al usuario seleccionar el canal de recepción de cada tipo de notificación cuando existan varios canales configurados en su organización.
- **NF3.3** [C] El sistema mostrará la configuración de suscripciones en la sección de perfil del usuario.
- **NF4** [M] El sistema enviará todas las notificaciones de forma asíncrona en segundo plano.
- **NF4.1** [M] El sistema encolará cada notificación como una tarea asíncrona de baja prioridad al ocurrir el evento que la origina.
- **NF4.2** [M] El sistema aislará los fallos de envío de notificación del flujo principal de la operación que los originó.
 - *NF4.2.1* El sistema confirmará al usuario la finalización de la operación origen con independencia del resultado posterior del envío de las notificaciones asociadas.
- **NF4.3** [S] El sistema registrará cada intento de envío de notificación en un log indicando canal, *timestamp* y resultado, con mensaje de error en caso de fallo.
 - *NF4.3.1* El sistema mostrará el log de notificaciones a los U3 globalmente y a los U4 y al *owner* dentro de su propia organización.
- **NF4.4** [S] El sistema reintentará automáticamente el envío de una notificación fallida hasta un máximo de 3 intentos con *backoff* exponencial (30 s, 2 min, 5 min).

4.11.5. Épica 5 — Extensibilidad

Esta épica agrupa los requisitos que permiten al **SVAES** integrarse con flujos automatizados externos. Cubre la funcionalidad FEAT-10.

FEAT-10: Interoperabilidad Programática Automática

- **API1** [M] El sistema expondrá una **API REST** pública documentada mediante una especificación OpenAPI navegable.
- **API1.1** [M] El sistema publicará la especificación en formato OpenAPI 3.x en el *endpoint* público `/api/v1/openapi.json`.
 - *API1.1.1* El sistema incluirá en la especificación un ejemplo de petición y un ejemplo de respuesta por cada *endpoint*.
- **API1.2** [M] El sistema expondrá una interfaz Swagger UI navegable en el *endpoint* `/api/v1/docs`.
- **API1.3** [M] El sistema seguirá las convenciones REST con verbos HTTP semánticos (GET, POST, PATCH, DELETE), respuestas en formato JSON y códigos HTTP

estándar para indicar el resultado.

- **API1.4** [M] El sistema versionará los *endpoints* de la API mediante el prefijo `/api/v{N}/` donde N es un entero correspondiente a la versión mayor.
 - *API1.4.1* El sistema mantendrá cada versión mayor de la API disponible durante un mínimo de 6 meses tras la publicación de su sucesora antes de retirarla.
- **API2** [M] El sistema autenticará las peticiones a la API mediante API *keys*.
- **API2.1** [M] El sistema exigirá a cada petición autenticada incluir la clave de acceso mediante una cabecera HTTP dedicada definida en el contrato OpenAPI.
- **API2.2** [M] El sistema generará cada nueva API *key* con suficiente entropía aleatoria para garantizar su unicidad e impredecibilidad, utilizando un generador criptográficamente seguro.
 - *API2.2.1* El sistema utilizará una fuente de entropía criptográficamente segura proporcionada por el lenguaje o plataforma de implementación elegidos.
- **API2.3** [M] El sistema almacenará cada API *key* únicamente como *hash* en la base de datos, utilizando una función de *hash* robusta y resistente a ataques de fuerza bruta (p. ej. bcrypt o Argon2).
 - *API2.3.1* El sistema no permitirá recuperar el valor en claro de una API *key* tras su generación inicial.
- **API2.4** [S] El sistema limitará a 5 el número máximo de API *keys* activas que un mismo usuario puede tener simultáneamente.
- **API2.5** [C] El sistema permitirá al usuario asignar una descripción de hasta 120 caracteres a cada API *key*.
- **API2.6** [S] El sistema permitirá al usuario revocar una API *key* en cualquier momento con efecto inmediato.
 - *API2.6.1* El sistema responderá HTTP 401 *Unauthorized* a toda petición autenticada con una API *key* revocada.
- **API3** [M] El sistema expondrá a través de la API los *endpoints* funcionales equivalentes a las operaciones clave de la interfaz web.
- **API3.1** [M] El sistema expondrá los *endpoints* de *releases* GET `/api/v1/projects/{project_id}/releases`, POST `/api/v1/projects/{project_id}/releases`, GET `/api/v1/releases/{id}`, PATCH `/api/v1/releases/{id}` y POST `/api/v1/releases/{id}/archive`.
 - *API3.1.1* El sistema admitirá en GET `/api/v1/projects/{project_id}/releases` los mismos filtros definidos para la vista web (proyecto, estado, etiquetas, rangos de fechas y texto libre).
- **API3.2** [M] El sistema expondrá los *endpoints* de artefactos POST `/releases/{id}/artifacts`, GET `/releases/{id}/artifacts` y DELETE `/releases/{id}/artifacts/{artifact_id}`.
- **API3.3** [M] El sistema expondrá los *endpoints* de verificaciones POST

/api/v1/releases/{id}/verify, GET /api/v1/releases/{id}/results y GET /api/v1/releases/{id}/results/{rid}.

- **API3.4** [C] El sistema expondrá el *endpoint* de *dashboard* GET /api/v1/dashboard/metrics aceptando los mismos parámetros de filtro definidos en el *dashboard*.
- **API3.5** [M] El sistema expondrá los *endpoints* de proyectos GET /projects y GET /projects/{id} filtrados por accesibilidad del usuario. El listado de proyectos (GET) se anida bajo el prefijo de organización como GET /api/v1/organizations/{org_id}/projects para reforzar el aislamiento *multi-tenant* a nivel de ruta, mientras que las operaciones sobre un proyecto individual utilizan la ruta plana GET /api/v1/projects/{project_id} con la organización derivada de la sesión autenticada.
- **API3.6** [M] El sistema expondrá los *endpoints* de organizaciones GET /organizations y GET /organizations/{id} filtrados por pertenencia del usuario.
- **API4** [M] El sistema aplicará *rate limiting* a todas las peticiones autenticadas de la API.
- **API4.1** [M] El sistema limitará las peticiones a la API mediante *rate limiting* con tres límites diferenciados según el tipo de operación: 30 peticiones por minuto en los *endpoints* de autenticación, 100 peticiones por minuto en las operaciones de lectura y 20 peticiones por minuto en las operaciones de escritura.
 - **API4.1.1** El sistema contabilizará el uso de cada clave de acceso mediante ventana deslizante en un almacén compartido entre instancias.
 - **API4.1.2** El *endpoint* de solicitud de reseteo de contraseña tendrá un límite específico de 5 peticiones por minuto y dirección IP para prevenir ataques de enumeración.
- **API4.2** [M] El sistema incluirá en cada respuesta de la API las cabeceras X-RateLimit-Limit, X-RateLimit-Remaining y X-RateLimit-Reset.
- **API4.3** [M] El sistema responderá HTTP 429 *Too Many Requests* cuando una API *key* supere su límite de peticiones por minuto.
 - **API4.3.1** El sistema incluirá en la respuesta HTTP 429 la cabecera **Retry-After** indicando los segundos que el cliente debe esperar antes de reintentar.

4.12. Casos de uso

Los casos de uso describen escenarios de interacción entre los actores y el sistema para cumplir los objetivos funcionales del **SVAES**. De entre el conjunto total de casos identificados durante el análisis se han seleccionado para esta sección los dos que constituyen el *núcleo funcional* de la plataforma: **CU-01 Verificar Release**, que materializa el *quality gate* ejecutado sobre una entrega, y **CU-02 Configurar Perfil de Verificación**, que define

las reglas de calidad que dicha verificación aplicará. Ambos casos están relacionados de forma directa: el perfil configurado en CU-02 es el insumo de configuración que CU-01 evalúa contra los artefactos de la *release*.

Cada caso de uso se documenta con su identificación (ID, actores, precondition, disparador), un diagrama UML que ilustra las relaciones de *asociación*, *inclusión* («include») y *extensión* («extend») entre el caso principal y sus sub-escenarios, el flujo básico desglosado por pasos, los flujos alternativos y la postcondición, junto con la trazabilidad a los requisitos funcionales cubiertos.

Esta estructura de documentación no solo facilita la comprensión de la lógica de negocio por parte de los *stakeholders*, sino que también establece una base sólida para las fases posteriores. Al mantener una trazabilidad explícita y detallar cada escenario excepcional, se mitiga el riesgo de ambigüedades durante la implementación, garantizando que el [SVAES](#) evolucione de manera robusta y alineada con los estándares de calidad exigidos.

4.12.1. CU-01: Verificar Release

Este caso de uso constituye la operación central y el principal motor de valor del sistema, actuando como un verdadero mecanismo de control de calidad (*quality gate*) automatizado. A partir de una *release* con artefactos asociados, el sistema evalúa de forma automatizada y exhaustiva el cumplimiento de las reglas de calidad definidas en su perfil. Esta aproximación elimina por completo la validación manual, lo que supone un ahorro significativo de tiempo y coste, a la vez que garantiza la objetividad en la decisión de aprobar o rechazar una entrega. Como resultado, el motor produce un veredicto trazable e inmutable (VALIDA, NO_VALIDA, CON_ADVERTENCIAS o, en presencia de reglas no evaluables por fallos de red, la variante con sufijo _CON_INCIDENCIAS).

Por motivos de rendimiento y usabilidad, la ejecución se divide en dos fases claramente diferenciadas. La primera consiste en una interacción síncrona breve con el usuario, donde el sistema valida las precondiciones, genera un identificador único y encola la verificación como una tarea asíncrona. La segunda fase comprende el procesamiento en segundo plano, diseñado con el fin de no bloquear la interfaz web durante las consultas a los distintos sistemas externos, cuyas latencias pueden ser costosas y variables en el tiempo. Durante esta fase asíncrona, el sistema captura una copia inmutable del perfil para asegurar la reproducibilidad histórica, interroga a las fuentes de datos y consolida el resultado final en el historial de auditoría.

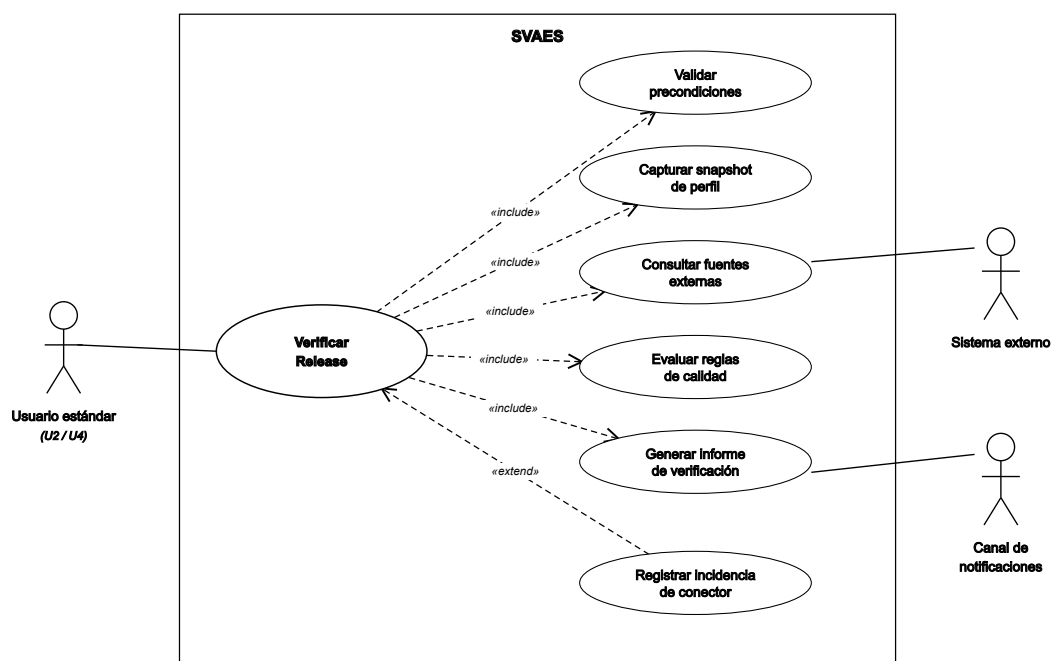


Figura 1: Diagrama de casos de uso CU-01 — Verificar Release.

Identificación del caso de uso.

ID	CU-01
Nombre	Verificar Release
Actor principal	Usuario estándar (U2) o Manager de organización (U4)
Actores secundarios	Sistema externo (fuentes de datos a través de conectores); Canal de notificaciones
Precondición	La <i>release</i> existe en estado PENDIENTE, VALIDA, NO_VALIDA o CON_ADVERTENCIAS; tiene asociados al menos un artefacto y un perfil de verificación
Disparador	El usuario solicita la acción <i>Verificar</i> desde la vista de la <i>release</i> , o un cliente externo invoca <code>POST /api/v1/releases/{id}/verify</code>
Relaciones	«include»: Validar precondiciones, Capturar <i>snapshot</i> de perfil, Consultar fuentes externas, Evaluar reglas de calidad, Generar informe. «extend»: Registrar incidencia de conector

Tabla 15: Identificación del caso de uso CU-01.

Flujo básico.

Fase 1 — Interacción síncrona.

1. El usuario accede a la vista de la *release* y solicita su verificación.
2. El sistema ejecuta *Validar precondiciones* «include»: comprueba estado apto, existencia de artefactos y existencia de perfil asociado.
3. El sistema cambia el estado de la *release* a EN_VERIFICACION, genera un identificador de ejecución, encola la tarea en el *job runner* y devuelve respuesta HTTP 202 con la URL de seguimiento.

Fase 2 — Procesamiento asíncrono.

4. El sistema ejecuta *Capturar snapshot de perfil* «include»: copia la configuración vigente del perfil para garantizar reproducibilidad del resultado aunque el perfil se modifique posteriormente.
5. El sistema ejecuta *Consultar fuentes externas* «include»: invoca en paralelo los conectores definidos en el perfil para recuperar datos de tareas, repositorio, documentación y planificación.
6. El sistema ejecuta *Evaluar reglas de calidad* «include»: aplica el motor de reglas sobre los datos obtenidos, calcula el resultado por regla y el veredicto global.
7. El sistema ejecuta *Generar informe* «include»: persiste el informe en el historial inmutable, actualiza el estado final de la *release* y emite notificación al usuario.

Flujos alternativos.

ID	Escenario	Descripción del flujo
FA-A	Validación de precondiciones fallida	En el paso 2, si la <i>release</i> no tiene artefactos o carece de perfil, el sistema responde HTTP 409, mantiene el estado original y finaliza el caso de uso.
FA-B	Verificación concurrente	En el paso 3, si ya existe una verificación en curso para la misma <i>release</i> , el sistema responde HTTP 409 y no encola una nueva ejecución.
FA-C	Indisponibilidad de fuente externa	En el paso 5, si un conector falla por <i>timeout</i> o error de red, el sistema activa el escenario «extend» <i>Registrar incidencia de conector</i> : marca las reglas dependientes como NO_EVALUADAS, prosigue con el resto de reglas y refleja la incidencia en el informe final añadiendo el sufijo <i>_CON_INCIDENCIAS</i> al resultado global.
FA-D	Fallo crítico no recuperable	Si se produce una excepción no controlada durante la fase 2, el sistema marca la tarea como FALLIDA, restaura la <i>release</i> a su estado anterior, persiste el detalle del fallo y notifica al usuario y al administrador.

ID	Escenario	Descripción del flujo
FA-E	Cancelación del usuario	Si el usuario solicita la cancelación mientras la tarea está encolada o en curso, el sistema interrumpe el procesamiento, descarta el informe parcial y restaura el estado anterior de la <i>release</i> .

Tabla 16: Flujos alternativos del caso de uso CU-01.**Postcondición y trazabilidad.**

Postcondición de éxito	La <i>release</i> queda en estado final (VALIDA, NO_VALIDA o CON_ADVERTENCIAS, eventualmente con sufijo _CON_INCIDENCIAS), se registra un informe inmutable en el historial y se notifica el resultado al usuario.
Postcondición de fallo	La tarea queda en FALLIDA y la <i>release</i> restaura su estado previo; no se genera informe consolidado pero sí entrada de auditoría.
Requisitos cubiertos	GR6.1, GR6.2, MV1.1, MV1.3, MV2, MV3, MV4, MV4.2, MV4.4, MV7, RH1, RH2, NF2.1, API3.3, PV1.7.

Tabla 17: Postcondición y trazabilidad del caso de uso CU-01.

4.12.2. CU-02: Configurar Perfil de Verificación

Este caso de uso describe la creación y configuración de un perfil de verificación, entendido como el conjunto de reglas, niveles (OBLIGATORIA / OPCIONAL / EXCLUIDA) y políticas de asociación que se aplicarán a las *releases* de un proyecto. El perfil constituye el contrato de calidad de la organización: define qué se considera una entrega aceptable y con qué grado de exigencia. Su configuración es responsabilidad del *Manager* de organización (U4); el Técnico (U2) también puede ejercer este caso de uso.

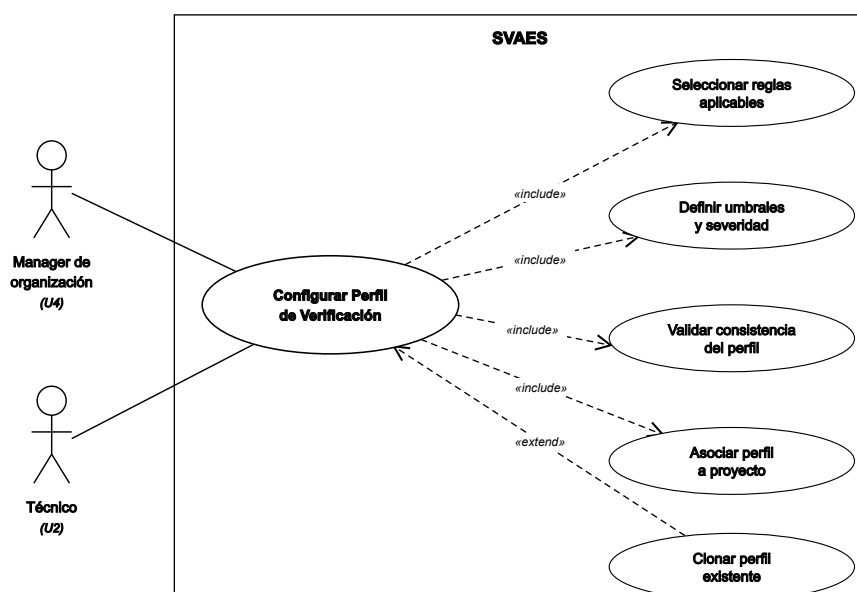


Figura 2: Diagrama de casos de uso CU-02 — Configurar Perfil de Verificación.

Identificación del caso de uso.

ID	CU-02
Nombre	Configurar Perfil de Verificación
Actor principal	Manager de organización (U4); Técnico (U2) también puede ejercer este caso de uso
Actores secundarios	—
Precondición	El usuario tiene permisos de gestión de perfiles sobre la organización; existen reglas de verificación disponibles en el catálogo (RV-01–RV-10) y al menos un proyecto sobre el que asociar el perfil
Disparador	El usuario solicita <i>Nuevo perfil de verificación</i> desde la sección de configuración de la organización

ID	CU-02
Relaciones	«include»: Seleccionar reglas aplicables, Definir umbrales y severidad, Validar consistencia del perfil, Asociar perfil a proyecto. «extend»: Clonar perfil existente

Tabla 18: Identificación del caso de uso CU-02.**Flujo básico.**

1. El usuario accede a la sección *Perfiles de verificación* de la organización y solicita la creación de un perfil nuevo.
2. El sistema muestra el formulario de perfil, precargado con los metadatos básicos (nombre, descripción opcional, organización propietaria).
3. El usuario ejecuta *Seleccionar reglas aplicables* «include»: el sistema lista el catálogo de reglas RV-01–RV-10 con su descripción y el usuario marca las que formarán parte del perfil.
4. El usuario ejecuta *Definir umbrales y severidad* «include»: para cada regla seleccionada asigna un nivel del conjunto {OBLIGATORIA, OPCIONAL, EXCLUIDA} e introduce el valor umbral cuando proceda (porcentaje de cobertura, número máximo de incidencias, etc.).
5. El sistema ejecuta *Validar consistencia del perfil* «include»: comprueba que los umbrales están dentro de rangos válidos, que el nombre es único en la organización y que existe al menos una regla OBLIGATORIA.
6. El usuario ejecuta *Asociar perfil a proyecto* «include»: selecciona uno o varios proyectos de la organización sobre los que el perfil quedará disponible, opcionalmente designándolo como perfil por defecto del proyecto.
7. El sistema persiste el perfil en estado ACTIVO, lo registra en el historial de auditoría y devuelve confirmación al usuario.

Flujos alternativos.

ID	Escenario	Descripción del flujo
FA-A	Clonar perfil existente	Antes del paso 3, el usuario activa el escenario «extend» <i>Clonar perfil existente</i> : selecciona un perfil de la organización o el perfil por defecto del sistema como plantilla; el sistema precarga reglas y niveles asignados y el usuario continúa el flujo desde el paso 3 ajustando lo necesario.
FA-B	Validación de consistencia fallida	En el paso 5, si el sistema detecta umbrales fuera de rango o ausencia de reglas OBLIGATORIA, muestra los errores específicos por regla y devuelve el control al paso 3 sin persistir el perfil.
FA-C	Conflicto de nombre	En el paso 5, si ya existe un perfil con el mismo nombre dentro de la organización, el sistema responde HTTP 409 y solicita al usuario un nombre alternativo.
FA-D	Sin proyectos seleccionados	En el paso 6, si el usuario no selecciona ningún proyecto, el sistema persiste el perfil en estado BORRADOR (no aplicable a verificaciones) y avisa de que deberá asociarlo posteriormente para activarlo.
FA-E	Cancelación	El usuario cancela en cualquier paso anterior al 7; el sistema descarta los cambios sin persistir nada.

Tabla 19: Flujos alternativos del caso de uso CU-02.**Postcondición y trazabilidad.**

Postcondición de éxito	El perfil queda registrado en estado ACTIVO o BORRADOR, asociado a su organización propietaria y, en su caso, a los proyectos seleccionados; está disponible para ser referenciado por nuevas <i>releases</i> y por el caso de uso CU-01.
Postcondición de fallo	No se persiste ningún perfil; se mantiene una entrada de auditoría con el motivo del fallo cuando proceda.
Requisitos cubiertos	ORG5.3, PV1, PV1.1, PV1.2, PV1.3, PV1.4, PV1.5, PV2, PV3, PV3.1, PV3.3.

Tabla 20: Postcondición y trazabilidad del caso de uso CU-02.

4.13. Diagramas del sistema

Junto con los casos de uso, esta sección recoge dos diagramas complementarios que precisan el comportamiento dinámico del sistema en dos áreas críticas: el control de acceso al iniciar sesión (DF-01) y la evolución temporal del concepto central del dominio — la *release* — a lo largo de su ciclo de vida (DF-02). Ambos diagramas sirven de referencia para los casos de prueba de transición de estados descritos en el capítulo 7.

4.13.1. DF-01 — Autenticación

El diagrama DF-01 describe el flujo completo que sigue una solicitud de inicio de sesión desde la recepción de las credenciales hasta la creación efectiva de la sesión autenticada. El sistema localiza al usuario en la base de datos a partir del correo electrónico y encadena tres comprobaciones de carácter discriminatorio antes de proceder a la verificación criptográfica: el estado de bloqueo temporal, el estado activo de la cuenta y la existencia del propio usuario. Esta ordenación es deliberada: permite responder con códigos HTTP precisos (403 Cuenta bloqueada, 403 Cuenta inactiva o 401 Credenciales incorrectas) sin filtrar información sobre la existencia o no de la cuenta en aquellos escenarios en que la diferencia es semánticamente relevante para el usuario legítimo.

La verificación de la contraseña se realiza mediante la comparación del hash calculado sobre el valor introducido contra el almacenado en base de datos. Ante un fallo, el sistema incrementa el contador de intentos asociado al usuario y, al alcanzar el umbral de cinco intentos fallidos, activa un bloqueo temporal de quince minutos que materializa la política de fuerza bruta descrita en los requisitos no funcionales de seguridad. Cuando la verificación es exitosa, el contador de fallos se reinicia, se crea la sesión autenticada y el usuario es redirigido a la selección de organización, que constituye el punto de entrada al espacio de trabajo multi-tenant del SVAES. Las transiciones reflejadas en este diagrama se ejercitan en las pruebas de seguridad y de aceptación del capítulo 7 para garantizar la robustez del control de acceso ante credenciales inválidas, cuentas inactivas y escenarios de bloqueo.

Desde el punto de vista de la implementación, este flujo se apoya en los requisitos AC1 a AC5 del catálogo funcional, que definen con precisión cada condición de guarda y cada respuesta del sistema. En particular, el requisito AC1.3 establece que ante cualquier intento fallido el sistema devuelva exclusivamente el mensaje genérico «Credenciales incorrectas», ocultando si el fallo se debe a un usuario inexistente o a una contraseña errónea. Para reforzar esta protección, el requisito AC1.3.1 exige que el tiempo de respuesta del endpoint de autenticación sea homogéneo con independencia de la causa del fallo, neutralizando así posibles ataques de enumeración de usuarios por análisis de latencia. La sesión creada al autenticarse incorpora el contexto del usuario — identificador, organización activa y rol global — y es verificada en cada petición subsiguiente tanto en la capa de presentación

como en la de servicio (AC3.10), garantizando que el control de acceso no dependa exclusivamente de comprobaciones en el frontend. Adicionalmente, el sistema registra en el *log* de auditoría cada evento de autenticación, éxito o fallo, incluyendo la dirección IP de origen y el instante UTC con precisión de milisegundos (AC1.7.2), lo que proporciona la trazabilidad necesaria para detectar patrones de ataque y para cumplir con los requisitos organizacionales de seguridad recogidos en el apartado 4.10.10.

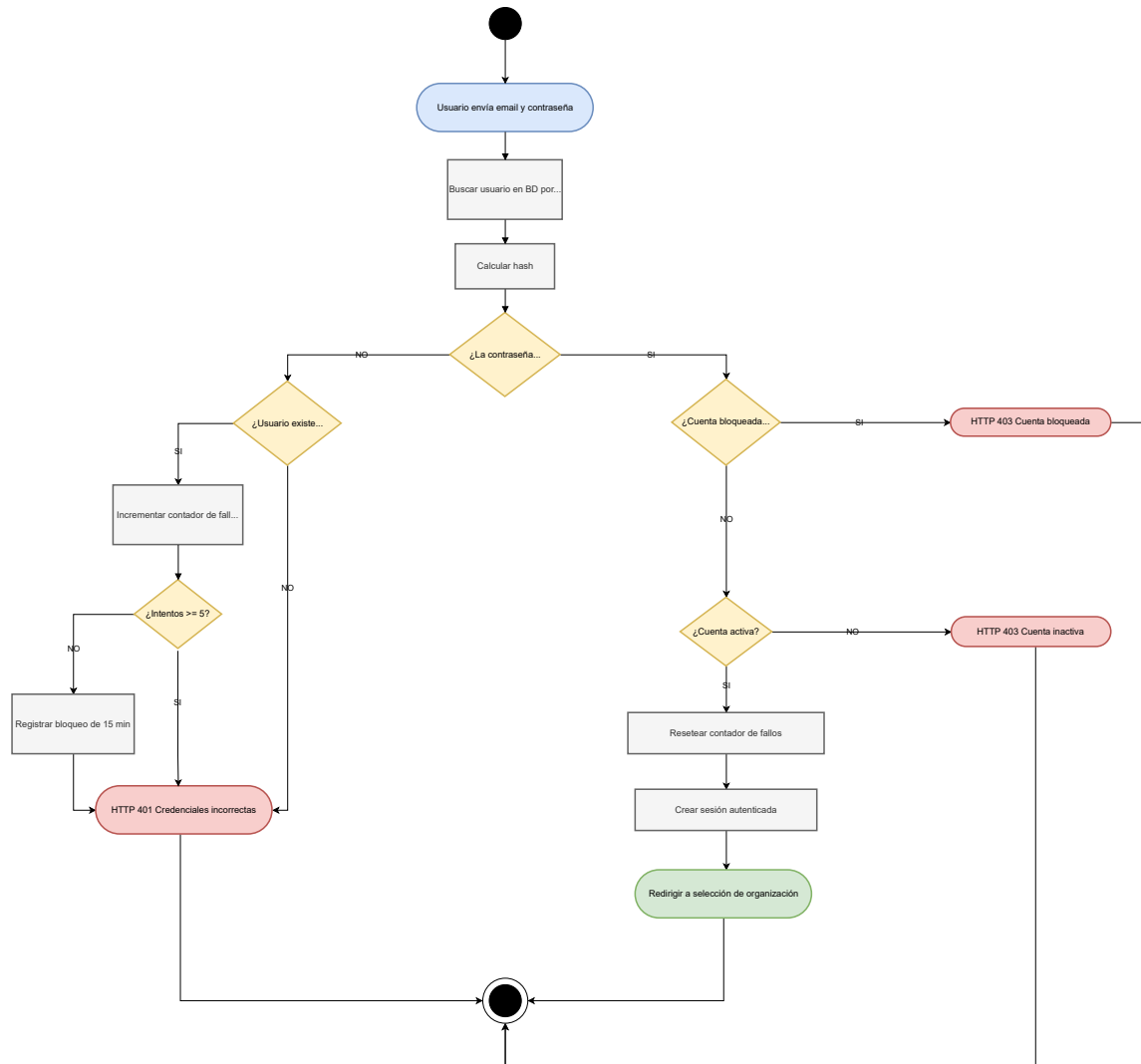


Figura 3: DF-01. Flujo de autenticación.

4.13.2. DF-02 — Ciclo de vida de release

El diagrama DF-02 modela el ciclo de vida de una *release* como un autómata de estados finitos que recoge los siete estados reconocidos por el sistema (requisito GR6.1): BORRADOR (estado inicial, asignado al crear la *release*); PENDIENTE (cuando se le han asociado uno o más artefactos y está lista para evaluarse); EN_VERIFICACION (estado activo durante el procesamiento asíncrono del motor, con la actividad `do / ejecutar_motor_asincrono()`); los tres estados terminales de evaluación VALIDA,

CON_ADVERTENCIAS y NO_VALIDA, que reflejan el veredicto global obtenido conforme a los criterios de MV4; y ARCHIVADA, estado de solo lectura al que puede transitarse desde cualquier estado terminal con historial asociado (acción de entrada `entry` / `establecer_solo_lectura()`).

Las transiciones quedan etiquetadas con el evento que las dispara, la acción de soporte y la condición de guarda cuando procede: `asociar_artefactos()` [`artefactos_count > 0`] habilita el paso de BORRADOR a PENDIENTE; `solicitar_verificacion()` / `encolar_tarea()` desencadena la entrada en EN_VERIFICACION desde cualquiera de los estados aptos; las tres variantes de `finalizar_verificacion()` discriminan por su guarda (`errores == 0 && warnings == 0`, `errores == 0 && warnings > 0` y `errores > 0`) el estado terminal al que se transita; `archivar_release()` [`tiene_historial == true`] permite el paso a ARCHIVADA; y `eliminar_fisicamente()` [`verificaciones == 0`] contempla la única vía de eliminación destructiva sobre *releases* sin historial. La importancia de este diagrama es doble: por un lado, define el contrato de comportamiento dinámico que el backend debe respetar; por otro, constituye la base sobre la que se diseña la batería de pruebas de transición de estados del capítulo 7 (casos TC-INT-EST-01 a TC-INT-EST-08), que cubre las siete transiciones T1–T6 más la restauración, así como las transiciones ilegales que el sistema debe rechazar.

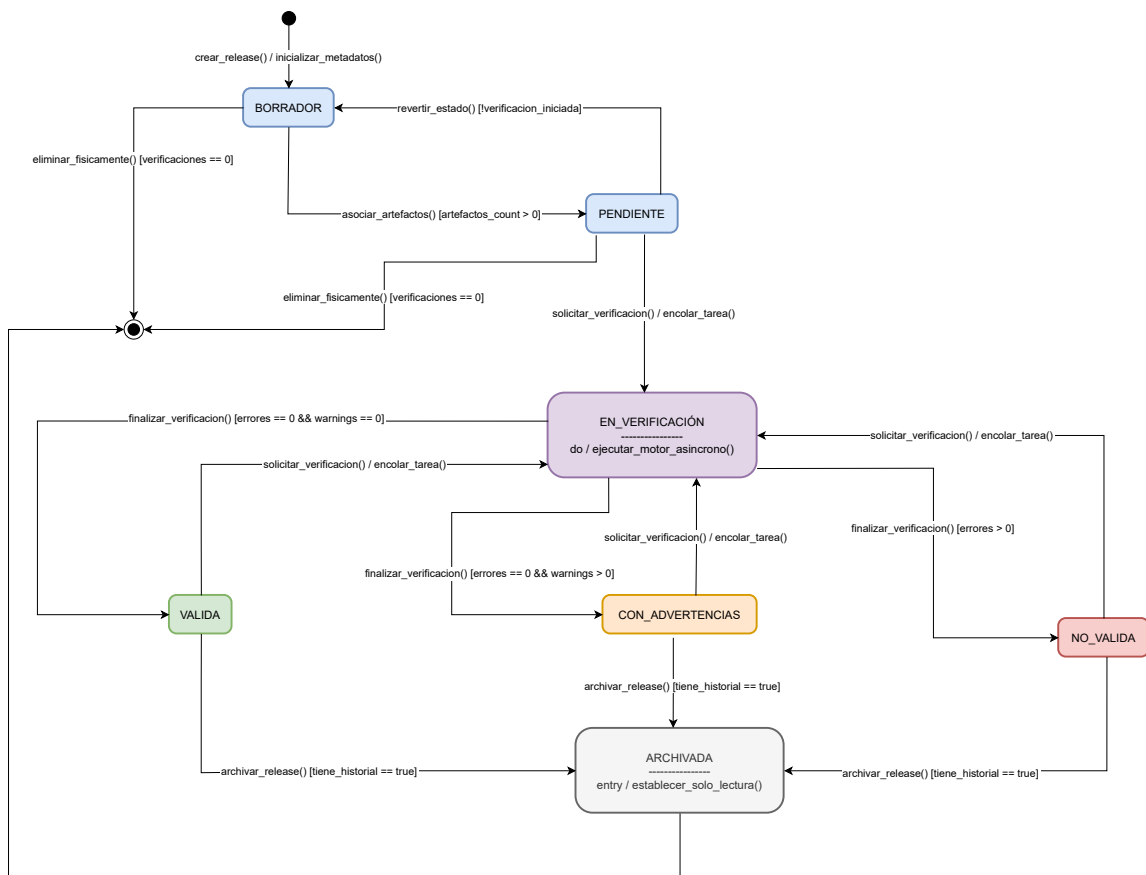


Figura 4: DF-02. Ciclo de vida de una *release*.

5 Diseño del Sistema

El presente capítulo describe la arquitectura y el diseño del Sistema de Verificación Automática de Entregas de Software (SVAES). Partiendo del catálogo de requisitos definido en el capítulo anterior, se establecen las decisiones arquitectónicas fundamentales, la descomposición del sistema en capas y componentes, el modelo de datos, el diseño de la interfaz de usuario y los mecanismos de seguridad incorporados al diseño. Toda decisión adoptada en este capítulo es trazable a uno o varios requisitos del capítulo 4.

El diseño sigue el marco metodológico de la arquitectura C4 (Context, Container, Component, Code) para la documentación de la arquitectura de software, combinado con la notación UML 2.5 para los diagramas de clases y de secuencia. Las decisiones tecnológicas se fundamentan en el análisis de alternativas recogido en la sección 2.10 del capítulo 2.

Un principio rector del diseño es la **genericidad**: el SVAES no está acoplado a ninguna herramienta externa concreta. Cualquier sistema externo puede integrarse implementando el puerto *ICconnector* sin modificar el núcleo del sistema. Este principio impregna todas las decisiones de diseño documentadas en este capítulo.

5.1. Visión Arquitectónica y Decisiones de Diseño

El SVAES adopta una arquitectura híbrida que combina los principios de la **arquitectura hexagonal** (Ports and Adapters, Cockburn 2005) [2] con las reglas de dependencia de la **Clean Architecture** (Martin 2017) [6]. Esta elección responde a la naturaleza genérica del sistema: el motor de verificación y las reglas de negocio deben permanecer completamente independientes de cualquier herramienta externa concreta, de forma que la adición o sustitución de un conector no requiera modificar el núcleo del sistema.

La regla de dependencia fundamental establece que las dependencias de código fuente únicamente pueden apuntar hacia el interior del sistema: los adaptadores conocen el dominio, pero el dominio no conoce a los adaptadores. Esta restricción es verificable mediante análisis estático de importaciones y se refuerza mediante la definición explícita de puertos (interfaces abstractas) en la capa de aplicación.

Adicionalmente, la lógica de verificación —componente más crítico del sistema en términos de corrección y rendimiento— se implementa en **Rust** como proceso independiente. Esta decisión aísla el motor del resto del sistema, permite su prueba y evolución autónoma, y garantiza las propiedades de seguridad en memoria y rendimiento predecible sin recolector de basura requeridas por el requisito no funcional RNF-01 (rendimiento en tiempo de verificación).

5.1.1. Tabla de Decisiones Arquitectónicas

La Tabla 21 recoge las decisiones arquitectónicas fundamentales adoptadas, junto con su justificación e implicaciones de diseño.

Decisión	Justificación	Implicaciones
Arquitectura hexagonal + Clean Architecture	Aísla la lógica de negocio de la infraestructura. La regla de dependencia es verificable mediante análisis estático de importaciones.	Se definen puertos (interfaces) para cada punto de entrada/salida. Los adaptadores implementan los puertos sin conocer el dominio.
Frontend Angular (SPA)	Framework maduro con TypeScript nativo, inyección de dependencias y generación automática de clientes REST desde el contrato OpenAPI.	Comunicación con el backend exclusivamente mediante la API REST . El cliente se genera desde la especificación OpenAPI 3.x.
Motor de verificación en Rust	Rust ofrece paralelismo de datos seguro (Rayon), garantías de seguridad en memoria y latencias predecibles sin GC. Cubre RNF-01.	El motor se despliega como proceso independiente. No realiza llamadas de red: opera exclusivamente sobre datos en memoria.
Backend principal en Python (FastAPI)	Ecosistema maduro para integración REST, validación con Pydantic y generación automática de especificación OpenAPI.	El backend orquesta el ciclo de vida de releases y delega la verificación al motor Rust. No contiene lógica de verificación.
Base de datos relacional (PostgreSQL 17)	Modelo de datos inherentemente relacional. Soporte JSONB para configuraciones de conectores, transacciones ACID y tipos enumerados nativos.	ORM SQLAlchemy 2.x con migraciones Alembic. Credenciales de conector cifradas con Fernet (AES-128-CBC + HMAC-SHA256).
Ejecución asíncrona (Celery + Redis)	Las verificaciones implican llamadas a APIs externas con latencias variables. La ejecución síncrona produciría timeouts y degradaría la experiencia de usuario.	Cola Celery con broker Redis. El backend responde inmediatamente con 202 Accepted. El frontend consulta el estado mediante polling.

Tabla 21: Decisiones arquitectónicas del [SVAES](#).

5.1.2. Stack Tecnológico

La Tabla 22 resume las tecnologías seleccionadas para cada capa del sistema. La selección detallada y el análisis de alternativas descartadas se recogen en la sección 2.10 del capítulo 2. Todas las tecnologías seleccionadas son software libre o de código abierto, con licencias compatibles con un proyecto académico.

Capa	Tecnología	Versión mín.	Función
Frontend	Angular + TypeScript	21.2.0/5.9.2	SPA de administración y operación
Frontend	@hey-api/openapi-ts + @hey-api/client-fetch	0.97.3/0.13.1	Generación del cliente REST tipado desde el contrato OpenAPI
API Gateway	Python/FastAPI	0.115.0	Enrutado HTTP, autenticación JWT , RBAC , documentación automática
Dominio	Python (core)	3.11	Casos de uso, entidades, puertos, validaciones Pydantic
Motor verificación	Rust (Actix-web 4.4)	1.77	Ejecución paralela de reglas con Rayon, sin GC
Cola de tareas	Celery + Redis	5.6.3/7	Ejecución asíncrona y desacoplada de verificaciones
Base de datos	PostgreSQL	17	Almacén relacional con soporte JSONB y tipos enumerados
ORM/migraciones	SQLAlchemy + Alembic	2.0.51/1.18.4	Mapeo objeto-relacional y gestión versionada del esquema
Conectores	httpx + pydantic	0.28.1/2.14.2	Cientes HTTP asíncronos tipados para APIs externas
Autenticación	JWT (PyJWT)	2.13.0	Tokens de sesión stateless con refresh token en base de datos
Despliegue	Docker + Docker Compose	17 (PG), 7 (Redis)	Contenerización y orquestación del entorno local y CI
Contrato API	OpenAPI 3.x	3.1	Especificación REST pública del SVAES , autodocumentada

Tabla 22: Stack tecnológico del [SVAES](#).

5.2. Arquitectura del Sistema

Esta sección describe la arquitectura del **SVAES** mediante el marco de documentación C4, que organiza la descripción arquitectónica en cuatro niveles de abstracción progresivamente más detallados: contexto, contenedores, componentes y código. Este capítulo cubre los tres primeros niveles; el nivel de código se documentará en el capítulo de implementación.

5.2.1. Diagrama de Contexto (C4 Nivel 1)

El diagrama de contexto sitúa al **SVAES** en su entorno operativo, identificando los actores humanos y las categorías de sistemas externos con los que interactúa. En este nivel de abstracción se omiten deliberadamente los nombres de herramientas concretas: el **SVAES** es un sistema genérico cuya integración con sistemas externos depende de los conectores que el usuario configure.

Los actores identificados son el **Usuario Estándar**, que opera las releases y lanza verificaciones, y el **Manager/Admin**, que configura la organización, los conectores y los perfiles de calidad. Los sistemas externos se clasifican en cuatro categorías funcionales: herramientas de gestión de tareas, repositorios de código, herramientas de documentación y canales de notificación.

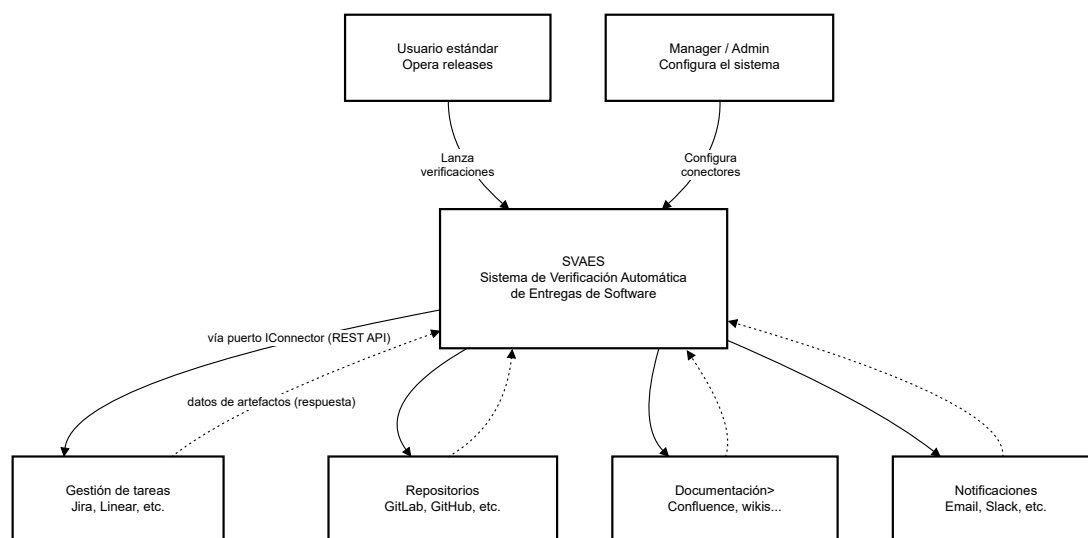


Figura 5: Diagrama de contexto del **SVAES** (C4 Nivel 1).

5.2.2. Diagrama de Contenedores (C4 Nivel 2)

El diagrama de contenedores descompone el **SVAES** en sus bloques desplegable independientes. Se identifican los siguientes contenedores:

- **Frontend Angular (SPA)**: aplicación web de página única. El cliente REST se genera automáticamente desde la especificación OpenAPI del backend.
- **API Backend (FastAPI / Python)**: núcleo de la lógica de aplicación y orquestador del ciclo de vida de releases. Expone la API REST documentada mediante OpenAPI 3.x, aplica autenticación **JWT** y control de acceso **RBAC**.
- **Motor de Verificación (Rust)**: proceso independiente que recibe trabajos de verificación a través de la cola de tareas y ejecuta las reglas configuradas. No realiza llamadas de red.
- **Cola de Tareas (Celery + Redis)**: gestiona la ejecución asíncrona de las verificaciones, desacoplando el tiempo de respuesta HTTP del tiempo de ejecución del motor.
- **Base de Datos (PostgreSQL 17)**: almacén persistente con soporte JSONB y tipos enumerados nativos para los estados del dominio.
- **Capa de Conectores**: conjunto de adaptadores que implementan el puerto IConnector. La adición de un nuevo conector no requiere modificar ningún otro contenedor.

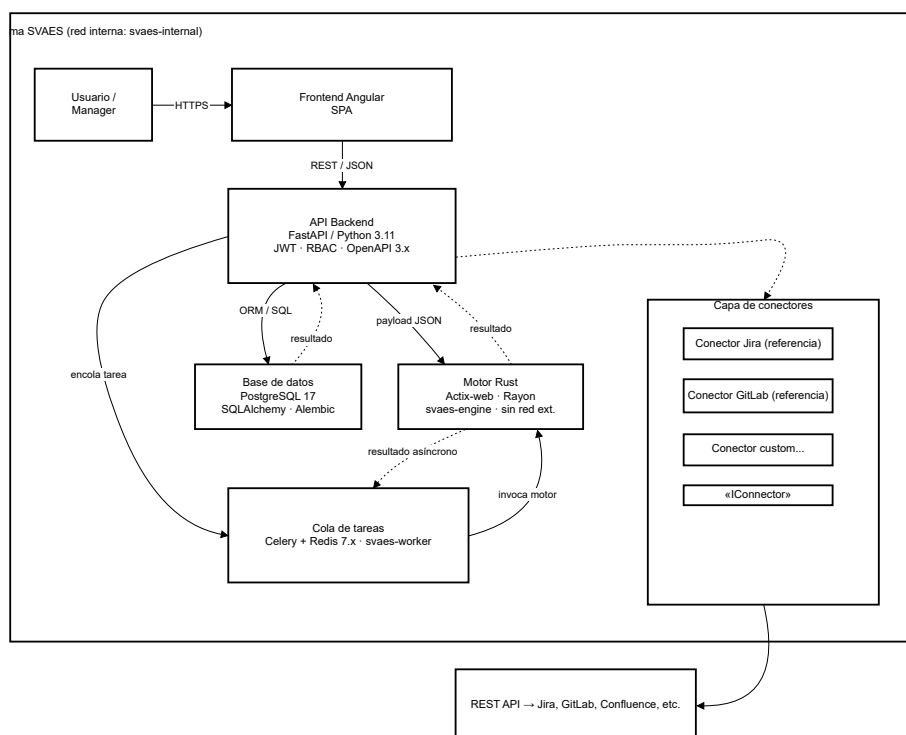


Figura 6: Diagrama de contenedores del **SVAES** (C4 Nivel 2).

5.2.3. Diagrama de Componentes: Backend (C4 Nivel 3)

La Figura 7 detalla la estructura interna del contenedor Backend siguiendo el modelo de capas de la arquitectura hexagonal. El diagrama expone cuatro capas diferenciadas: los adaptadores primarios (entrada), la capa de aplicación (casos de uso y puertos), la capa de dominio (núcleo puro) y los adaptadores secundarios (salida e infraestructura).

La regla de dependencia se visualiza en el diagrama mediante la dirección de las flechas: todas apuntan hacia el interior, hacia el dominio. Los puertos de salida (`IReleaseRepository`, `IConnectorRepository`, `IVerificationEngine`, `ITaskQueue`) constituyen el contrato entre la capa de aplicación y la infraestructura. En tiempo de ejecución, la inyección de dependencias sustituye estas interfaces por sus implementaciones concretas.

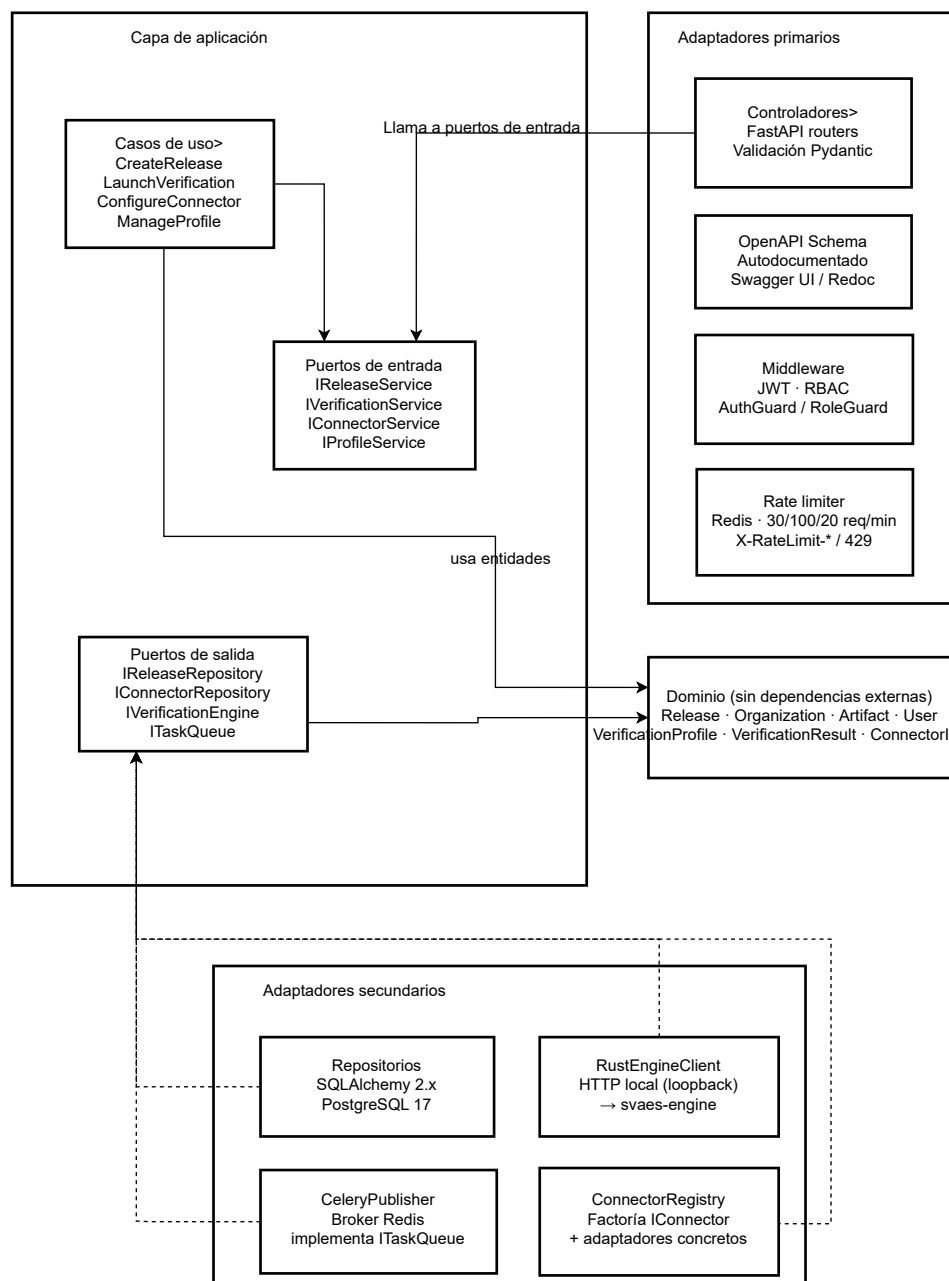


Figura 7: Diagrama de componentes del SVAES (C4 Nivel 3).

5.2.4. Diagrama de Despliegue

El sistema se despliega mediante Docker Compose. Cada contenedor lógico del modelo C4 se corresponde con un servicio definido en el fichero de composición. Los servicios son: **svaes-frontend**, **svaes-backend**, **svaes-engine**, **svaes-worker**, **postgres** y **redis**. Todos los servicios residen en una red bridge interna denominada **svaes-internal**. Únicamente el servicio **svaes-frontend** expone un puerto al host (80 → 8080), siendo el único punto de entrada externo. El motor Rust no tiene acceso a Internet externo. Las variables de entorno sensibles (**DATABASE_URL**, **REDIS_URL**, **SECRET_KEY**) se gestionan mediante un fichero **.env** excluido del control de versiones.

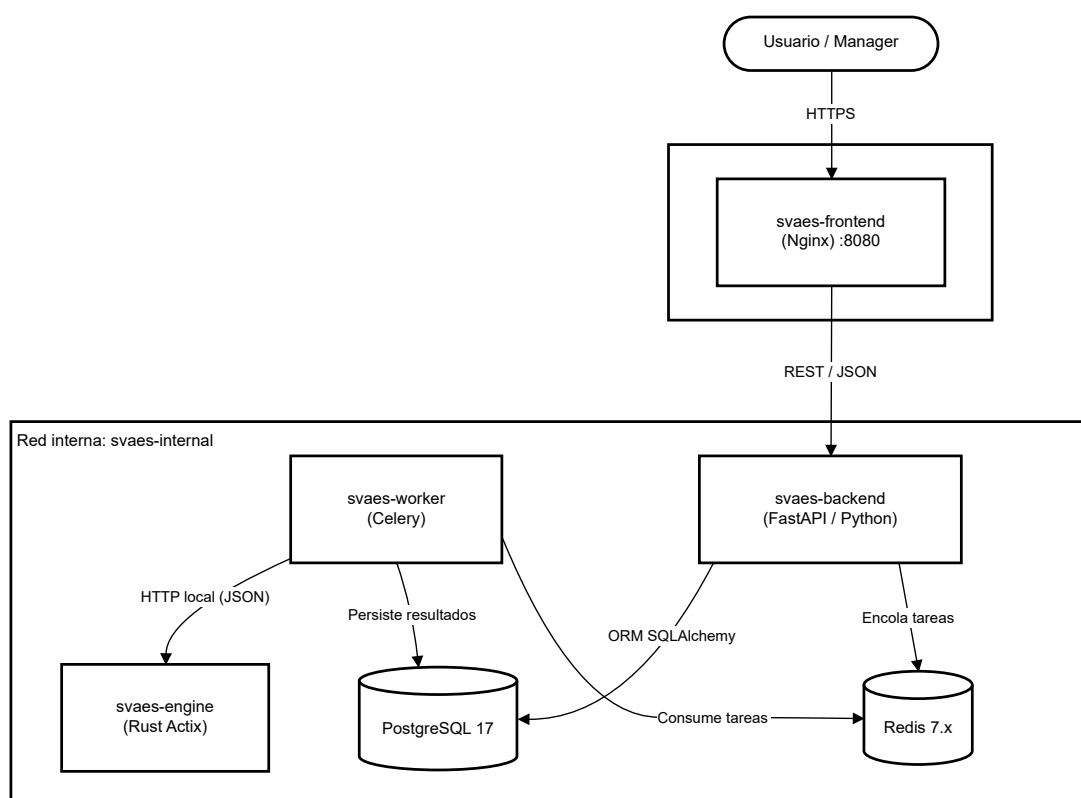


Figura 8: Diagrama de despliegue del **SVAES** mediante Docker Compose.

5.2.5. Soporte Multi-organización

El **SVAES** adopta el patrón habitual en SaaS B2B multi-tenant (GitHub, Slack, Notion): una sesión (**JWT**) está siempre **anclada a una organización activa**, pero un usuario puede tener **múltiples membresías**, cada una con su propio rol independiente. Cambiar de organización activa reemite el **JWT** con el **role** y **organization_id** correctos para esa organización.

La relación entre las entidades es la siguiente:

- **user** (entidad): **organization_id** y **role** son campos denormalizados que representan la organización activa de la sesión actual. Se mantienen para no romper las ~10 dependencias de autorización en `core/dependencies.py`.
- **user_membership** (tabla de pertenencia): **user_id**, **organization_id**, **role** (independiente por fila), **created_at**. La restricción `unique(organization_id, user_id)` garantiza que no existe duplicación.

Un mismo usuario puede tener una fila en **user_membership** por cada organización a la que pertenece, cada una con su propio rol.

La tabla **user_membership** ya existía en el esquema desde la migración inicial (`2fd6efcfd6c9_initial_schema.py`) pero permanecía sin uso en el código de aplicación. Se reactivó con su estructura original: **id**, **organization_id**, **user_id**, **role**, **created_at**, con la restricción `unique(organization_id, user_id)`.

El puerto **IUserMembershipRepository** define las operaciones de persistencia: **create**, **get**, **list_by_user**, **list_by_organization**, **update_role**, **delete** y **delete_all_for_user**.

La función **require_org_access()** se extendió para aceptar acceso de lectura a una organización en la que el usuario tiene membresía aunque no sea su organización activa (además del *owner* y el rol ADMIN global). Las operaciones de escritura (*releases*, proyectos, conectores) siguen requiriendo que la organización sea la activa — cambio deliberadamente acotado para no expandir el alcance de esta iteración.

5.3. Diseño de la Capa de Dominio y Aplicación

Esta sección describe la estructura interna del núcleo del sistema siguiendo los principios de la Clean Architecture. El objetivo es garantizar que la lógica de negocio no presente dependencias hacia la infraestructura, haciendo posible su prueba unitaria en completo aislamiento.

5.3.1. Organización en Capas

La Tabla 23 describe la organización en capas del [SVAES](#) y los elementos concretos que las componen.

Capa	Elemento en SVAES	Responsabilidad
Entidades (Domain)	Release, Organization, Rule, Artifact, VerificationResult, ConnectorInstance, VerificationProfile, VerificationRule	Objetos de negocio puros, sin dependencias externas. Contienen invariantes, identidad y comportamiento de dominio.
Casos de uso (Application)	CreateRelease, LaunchVerification, ConfigureConnector, ManageProfile, GetVerificationHistory	Orquestan las entidades y los puertos. Contienen la lógica de aplicación: flujos, validaciones de estado, composición de servicios.
Puertos de entrada	IReleaseService, IVerificationService, IConnectorService, IProfileService	Contratos que el adaptador HTTP (FastAPI) invoca. Permiten cambiar el mecanismo de entrada sin modificar los casos de uso.
Puertos de salida	IReleaseRepository, IConnectorRepository, IVerificationEngine, ITaskQueue, IProfileRepository, IUserMembershipRepository	Abstracciones que la capa de aplicación usa sin conocer la implementación concreta.
Adaptadores primarios	Controladores FastAPI, middleware JWT/RBAC , esquema OpenAPI, CLI de administración	Traducen peticiones HTTP a llamadas a los casos de uso. Realizan la validación de esquema y la autenticación.

Capa	Elemento en SVAES	Responsabilidad
Adaptadores secundarios	Repositorios SQLAl- chemy, RustEngineClient, CeleryPublisher, ConnectorRegistry e implementa- ciones IConnector	Implementan los puertos de salida. Son sustituibles sin modificar el dominio.

Tabla 23: Organización en capas de la arquitectura del SVAES.

5.3.2. Diagrama de Clases del Dominio

Todas las entidades tienen clave primaria de tipo UUID generada en el servidor, un campo `created_at` y un campo `updated_at` gestionado mediante trigger. Ninguna entidad tiene dependencias hacia el ORM, el framework web o cualquier biblioteca de infraestructura. La relación entre `Release` y `VerificationProfile` es de uso, no de composición. El resultado de verificación (`VerificationResult`) almacena una instantánea del perfil y de los artefactos evaluados en el momento de la ejecución, garantizando la trazabilidad histórica.

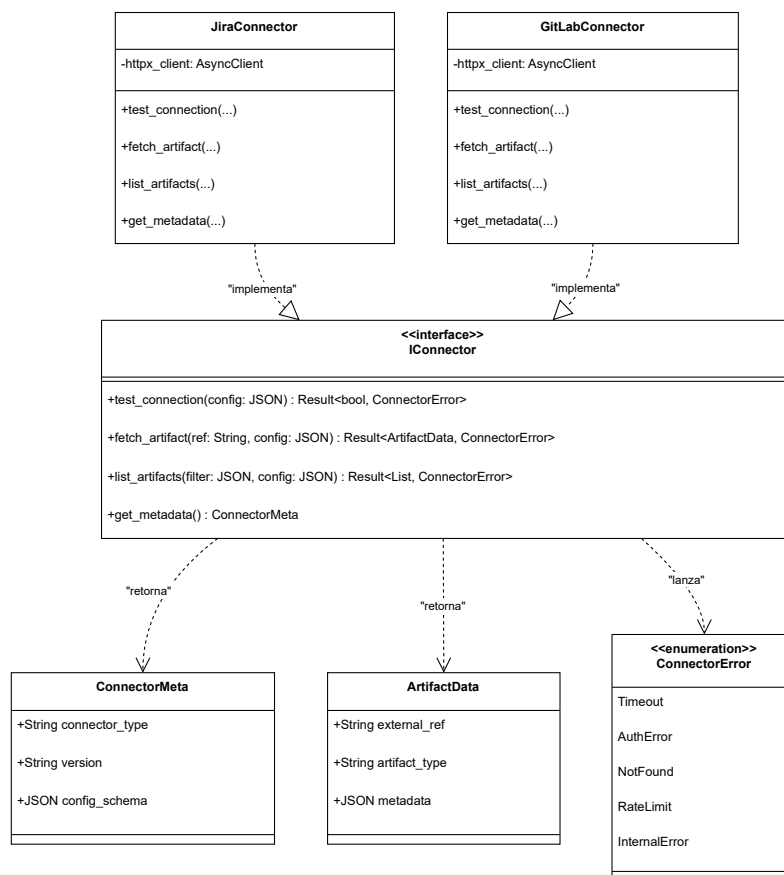


Figura 9: Diagrama de clases del subsistema de conectores del SVAES.

5.3.3. Diagramas de Secuencia

Lanzamiento de una Verificación (DS-01)

La Figura 10 muestra la secuencia completa desde que el usuario solicita la verificación de una release hasta que el motor Rust devuelve el resultado. El flujo se divide en dos fases:

- **Fase síncrona:** el backend valida el estado de la release, actualiza su estado a `EN_VERIFICACION` y encola la tarea, respondiendo al frontend con un `202 Accepted` inmediato (<200 ms, cubre RNF-05).
- **Fase asíncrona:** el worker Celery recupera los datos de los artefactos mediante los conectores, invoca al motor Rust y persiste el resultado. El frontend consulta el estado mediante polling al endpoint `GET /api/v1/tasks/{task_id}`.

En el flujo alternativo FA-C, si un conector falla durante la recuperación de datos, la regla correspondiente se marca como `NO_EVALUADA` en lugar de propagar el error a toda la verificación.

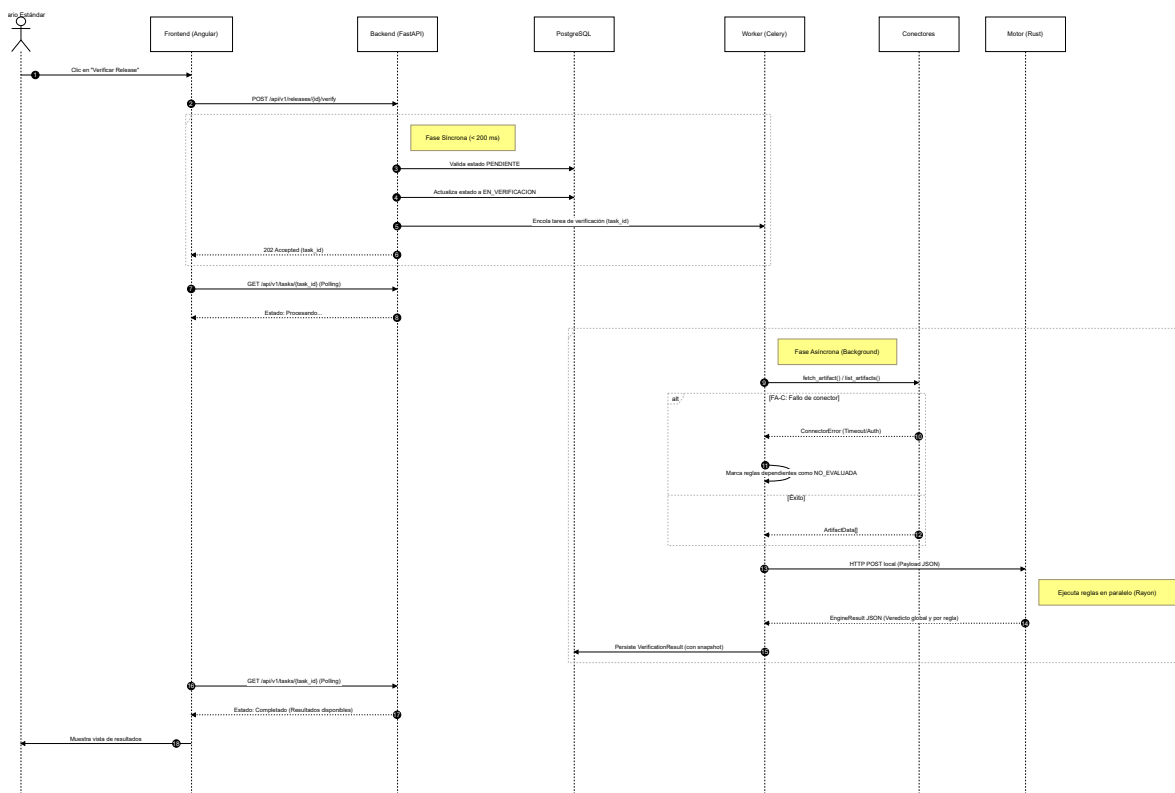


Figura 10: Diagrama de secuencia DS-01: lanzamiento de una verificación.

Registro y Activación de un Conector (DS-02)

La Figura 11 detalla la secuencia correspondiente al caso de uso CU-02, desde la introducción de los datos de configuración del conector hasta la validación de conectividad y la persistencia en estado `ACTIVO`. El flujo incluye: FA-A (error de validación Pydantic → HTTP 422), FA-B (fallo de conectividad → conector en estado `INACTIVO`) y FA-C

(cancelación del usuario).

Las credenciales se cifran mediante Fernet (AES-128-CBC + HMAC-SHA256) antes de la persistencia. La clave de cifrado reside exclusivamente en variables de entorno del backend. El campo *associated data* incluye el identificador de la instancia, impidiendo la reutilización del ciphertext. Cubre RNF-13.

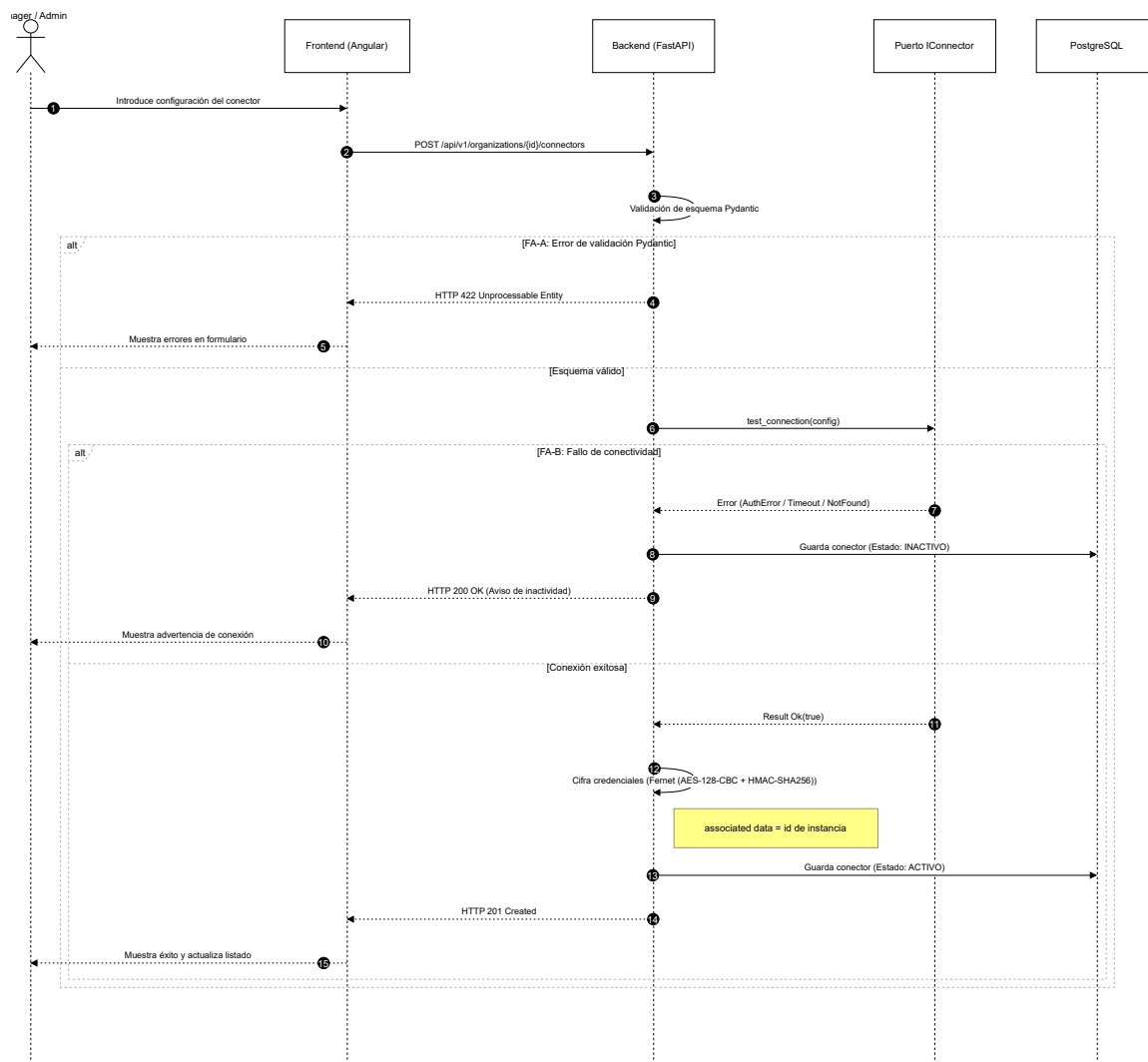


Figura 11: Diagrama de secuencia DS-02: registro y activación de un conector.

5.4. Diseño del Motor de Verificación (Rust)

El motor de verificación es el componente central del [SVAES](#) y el único implementado en Rust. Su responsabilidad es recibir un trabajo de verificación —compuesto por la definición del perfil activo y los datos de los artefactos ya recuperados por los conectores— y evaluar cada una de las reglas aplicables, produciendo un resultado estructurado con el veredicto global y el resultado por regla.

La separación entre la recuperación de datos (responsabilidad de los conectores en Python) y la evaluación de las reglas (responsabilidad del motor en Rust) garantiza que el motor no realiza llamadas de red y opera exclusivamente sobre datos en memoria, simplificando su modelo de prueba y garantizando latencias predecibles.

5.4.1. Interfaz de Comunicación con el Backend

El motor Rust expone una interfaz de comunicación con el backend Python mediante **HTTP local sobre Unix socket o loopback**. El motor actúa como servidor HTTP minimalista implementado con Actix-web. El backend Python le envía un payload JSON con los datos de verificación serializado mediante `serde` y recibe el resultado estructurado en la respuesta HTTP.

Este mecanismo facilita el despliegue independiente del motor como servicio Docker separado (`svaes-engine`) y su prueba mediante herramientas estándar. El contrato de datos (esquema de entrada `VerificationPayload` y esquema de salida `EngineResult`) se define mediante JSON Schema compartido entre el motor y el backend. El backend no conoce los detalles internos del motor; únicamente consume su interfaz pública.

5.4.2. Modelo Interno del Motor

Internamente, el motor sigue una **arquitectura de pipeline sin estado**. Para cada trabajo de verificación recibido, el motor ejecuta las siguientes etapas en orden:

1. **Deserialización y validación** del payload de entrada mediante `serde_json`.
2. **Resolución del perfil**: carga las definiciones de las reglas aplicables desde el campo `profile_config` del payload. No accede a la base de datos durante la ejecución.
3. **Ejecución paralela de las reglas** sobre los artefactos mediante el crate `Rayon` (work-stealing scheduling). Cada regla se evalúa de forma independiente.
4. **Agregación de resultados**: el componente `Aggregator` calcula el veredicto global (`VALID`, `VALID_WITH_WARNINGS` o `INVALID`). Las reglas `NO_EVALUADA` no computan a efectos del veredicto.
5. **Serialización del resultado** y retorno al backend.

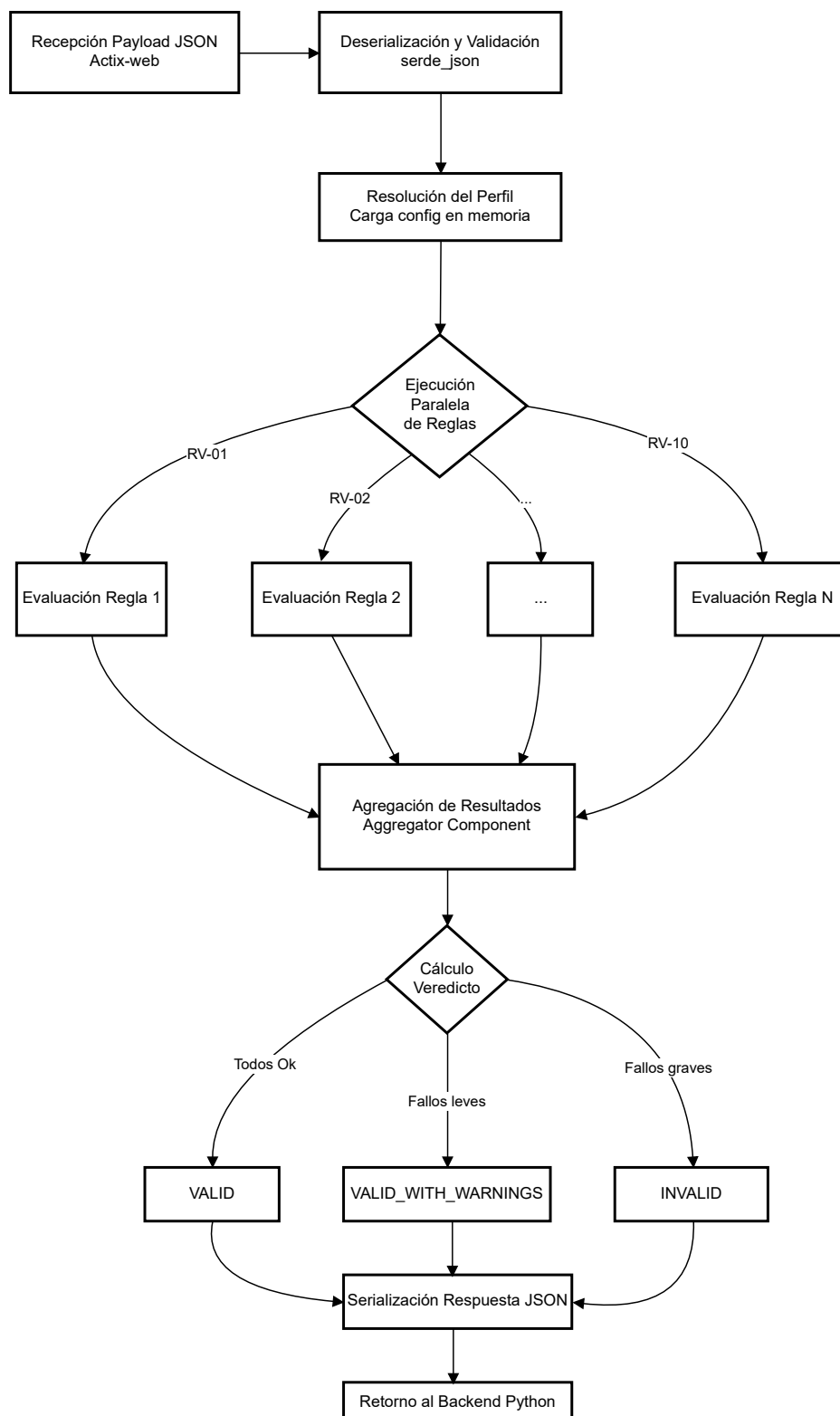


Figura 12: Estructura interna del motor de verificación del [SVAES](#).

5.4.3. Catálogo de Reglas de Verificación (RV-01 a RV-10)

El motor implementa diez plantillas de regla predefinidas que pueden instanciarse y configurarse en cualquier perfil de verificación. La Tabla 24 define cada regla, su comportamiento

y los parámetros que requiere para su configuración.

ID	Nombre	Descripción	Parámetros requeridos
RV-01	Existencia de artefactos	Verifica que la lista de artefactos recuperados no está vacía. Si el <i>payload</i> no contiene ningún artefacto, la regla falla. Severidad OBLIGATORIA. Cubre MV5.1.	Sin parámetros
RV-02	Trazabilidad entre artefactos	Verifica que los artefactos de tipo <code>source_type</code> referencian correctamente a artefactos de tipo <code>target_type</code> a través del campo <code>reference_field</code> . Si una referencia está ausente, vacía o apunta a un artefacto inexistente, la regla falla. Severidad OBLIGATORIA.	<code>source_type</code> , <code>target_type</code> , <code>reference_field</code>
RV-03	Validación de estados	Verifica que todos los artefactos del tipo <code>artifact_type</code> poseen un valor perteneciente a la lista <code>allowed_states</code> en el campo <code>status_field</code> . Si algún artefacto está en un estado no permitido, la regla falla. Severidad OBLIGATORIA.	<code>artifact_type</code> , <code>allowed_states</code> , <code>status_field</code>
RV-04	Integridad de campos numéricos	Verifica que, en todos los artefactos del tipo <code>artifact_type</code> , los campos listados en <code>numeric_fields</code> no son nulos, contienen valores numéricos y son mayores o iguales que cero. Si algún campo incumple alguna de estas condiciones, la regla falla. Severidad OBLIGATORIA.	<code>artifact_type</code> , <code>numeric_fields</code>

ID	Nombre	Descripción	Parámetros requeridos
RV-05	Disponibilidad de documentos	Verifica que, para cada artefacto del tipo <code>artifact_type</code> , el campo <code>accessible_field</code> indica que el documento asociado está accesible (no ha sido eliminado, movido o restringido). Si algún documento no está disponible, la regla falla. Severidad OBLIGATORIA.	<code>artifact_type</code> , <code>accessible_field</code>
RV-06	Coherencia de atributos	Verifica que, en todos los artefactos del tipo <code>artifact_type</code> , el campo <code>attribute</code> coincide con el <code>expected_value</code> . Permite validar propiedades como la rama destino, la prioridad o el tipo de incidencia esperados. Si algún artefacto difiere, la regla genera un aviso. Severidad OPCIONAL.	<code>artifact_type</code> , <code>attribute</code> , <code>expected_value</code>
RV-07	Registro externo	Verifica que todos los artefactos del tipo <code>artifact_type</code> tienen presente y no vacío el campo <code>marker_field</code> , que actúa como confirmación de que el artefacto ha sido registrado en un sistema externo (p. ej. un número de <i>ticket</i> o un código de trazabilidad). Si algún artefacto carece de este marcador, la regla falla. Severidad OBLIGATORIA.	<code>artifact_type</code> , <code>marker_field</code>

ID	Nombre	Descripción	Parámetros requeridos
RV-08	Alineación de listas	Verifica que, para cada artefacto del tipo <code>target_type</code> , el valor de su campo <code>master_field</code> coincide con algún valor del campo equivalente del artefacto identificado por <code>master_artifact_id</code> . Garantiza la coherencia entre conjuntos de datos relacionados. Si alguna entrada de la lista dependiente no tiene correspondencia en la maestra, la regla falla. Severidad OBLIGATORIA.	<code>master_artifact_id</code> , <code>master_field</code> , <code>target_type</code>
RV-09	Validación de referencias	Verifica que, en todos los artefactos del tipo <code>artifact_type</code> , los campos listados en <code>reference_fields</code> contienen referencias válidas a recursos cuyo campo <code>accessible_field</code> confirma que siguen activos. Si alguna referencia apunta a un recurso inaccesible, la regla genera un aviso. Severidad OPCIONAL.	<code>artifact_type</code> , <code>reference_fields</code> , <code>accessible_field</code>
RV-10	Aprobación final	Verifica que todos los artefactos del tipo <code>artifact_type</code> poseen, en el campo <code>status_field</code> , un valor perteneciente a la lista de estados aprobados <code>approved_states</code> . Si algún artefacto no ha alcanzado un estado de aprobación, la regla falla. Severidad OBLIGATORIA. Cubre MV5.10.	<code>artifact_type</code> , <code>status_field</code> , <code>approved_states</code>

Tabla 24: Catálogo de reglas de verificación del motor [SVAES](#).

Además del catálogo predefinido (RV-01 a RV-10), el motor implementa un conjunto de **reglas adicionales** que permiten verificar métricas de otro tipo: ausencia de código duplicado (`has_duplicated_code`), ausencia de vulnerabilidades de alta y crítica severidad

(`has_high_severity_vulnerabilities`, `has_critical_vulnerabilities`), control de *code smells* (`has_code_smells`), ausencia de *security hotspots* (`has_security_hotspots`), cobertura mínima de pruebas (`meets_minimum_test_coverage`) y complejidad máxima (`meets_maximum_complexity`), entre otras. Estas reglas de calidad son opcionales y se activan únicamente en los perfiles de verificación que integran SonarCloud como fuente de artefactos.

El motor soporta también reglas dinámicas definidas mediante expresiones JSONPath o JMESPath configuradas por el usuario, conforme al requisito MV6.1. Estas reglas personalizadas se cargan en el arranque de la aplicación y pueden recargarse en caliente mediante el endpoint `POST /api/v1/admin/rules/reload` (MV6.2.1).

5.5. Diseño de la Integración con Herramientas Externas

El [SVAES](#) no se acopla a ninguna herramienta externa concreta. La genericidad se materializa en el patrón de **puerto IConnector**, que actúa como contrato único para toda integración con sistemas externos.

5.5.1. Puerto IConnector

La Tabla 25 define los métodos que toda implementación de conector debe proporcionar. Esta interfaz constituye el único contrato entre la capa de dominio/aplicación y los sistemas externos.

Método del puerto IConnector	Descripción
<code>test_connection(config)</code> → <code>Result<bool, ConnectorError></code>	Comprueba que las credenciales y URL configuradas permiten establecer comunicación. Retorna error tipado en caso de fallo de autenticación, timeout o URL inaccesible. Se invoca durante el registro del conector.
<code>fetch_artifact(ref, config)</code> → <code>Result<ArtifactData, ConnectorError></code>	Recupera los datos de un artefacto concreto. El contenido devuelto se normaliza en la estructura <code>ArtifactData</code> , independientemente del formato nativo del sistema externo.

Método del puerto IConnector	Descripción
<code>list_artifacts(filter, config)</code> <code>→ Result<Vec<ArtifactData>, ConnectorError></code>	Recupera el conjunto de artefactos que corresponden a los filtros de la release (versión, sprint, rama, fecha). Los filtros dependen del tipo de conector; el puerto normaliza el resultado.
<code>get_metadata()</code> <code>→ ConnectorMeta</code>	Retorna el identificador de tipo, la versión del conector y el esquema JSON de configuración esperado. El backend lo utiliza para validar la configuración antes de persistirla.

Tabla 25: Métodos del puerto IConnector.

El tipo `ConnectorError` es una enumeración tipada con variantes para cada categoría de error posible: `Timeout`, `AuthError`, `NotFound`, `RateLimit`, `InternalError`. Esta tipificación permite al motor distinguir entre errores recuperables (`Timeout`, `RateLimit`) y errores definitivos (`AuthError`, `NotFound`).

5.5.2. Conectores de Referencia

El alcance del proyecto contempla dos conectores de referencia (Jira y GitLab) como prueba de concepto del puerto IConnector genérico. Durante la fase de implementación, como demostración adicional de la extensibilidad del sistema, el catálogo se amplió hasta un total de veinte conectores que cubren los cinco tipos funcionales descritos en la sección 6.6.2 del capítulo 6. Los conectores de referencia constituyen el alcance exigido del proyecto; los dieciocho conectores adicionales son **ilustraciones de la extensibilidad** del diseño: cualquier equipo que adopte el [SVAES](#) puede desarrollar sus propios conectores sin modificar el sistema base.

- **Conector de gestión de tareas de referencia:** integración con la [API REST v2](#) de Jira Cloud mediante autenticación PAT. Implementa todos los métodos del puerto IConnector para recuperar tareas, estados, sprints y versiones asociadas a una release.
- **Conector de repositorio de código de referencia:** integración con la [API REST v4](#) de GitLab. Recupera commits, merge requests y etiquetas de versión asociados a una rama o referencia de versión.

La elección de Jira y GitLab es exclusivamente una decisión de implementación: el sistema no requiere ninguna de estas herramientas para funcionar. Un equipo que use GitHub y Linear puede implementar sus propios conectores siguiendo la misma interfaz IConnector.

5.6. Diseño de la Base de Datos

5.6.1. Sistema Gestor de Base de Datos

Se utiliza **PostgreSQL 17** como sistema gestor de base de datos relacional. La elección responde a los siguientes factores: modelo de datos inherentemente relacional con claves foráneas bien definidas; soporte transaccional ACID; disponibilidad del tipo de columna JSONB para almacenar configuraciones de conectores y resultados de verificación; tipos enumerados nativos para los estados del dominio; e índices GIN sobre columnas JSONB para búsquedas eficientes.

El acceso se realiza exclusivamente a través del ORM SQLAlchemy 2.x, con gestión de esquema mediante Alembic. No se permiten modificaciones ad hoc del esquema fuera del proceso de migración versionado.

5.6.2. Diagrama Entidad-Relación

El diagrama entidad-relación del **SVAES** utiliza notación Crow's Foot. Las relaciones principales son:

- **user** — **user_membership** — **organization**: un usuario puede pertenecer a múltiples organizaciones con distintos roles (N:M implementada con tabla intermedia).
- **organization** — **project**: una organización contiene múltiples proyectos (1:N).
- **organization** — **connector_instance**: una organización configura uno o más conectores (1:N).
- **organization** — **verification_profile**: una organización define uno o más perfiles (1:N).
- **project** — **release**: un proyecto agrupa múltiples releases (1:N). El par (`project_id`, `version`) tiene restricción UNIQUE.
- **release** — **artifact**: una release está compuesta por múltiples artefactos (1:N).
- **release** — **verification_result**: una release puede tener múltiples resultados históricos (1:N).
- **verification_profile** — **verification_rule**: un perfil contiene múltiples reglas instanciadas de las plantillas RV-01 a RV-10 (1:N).

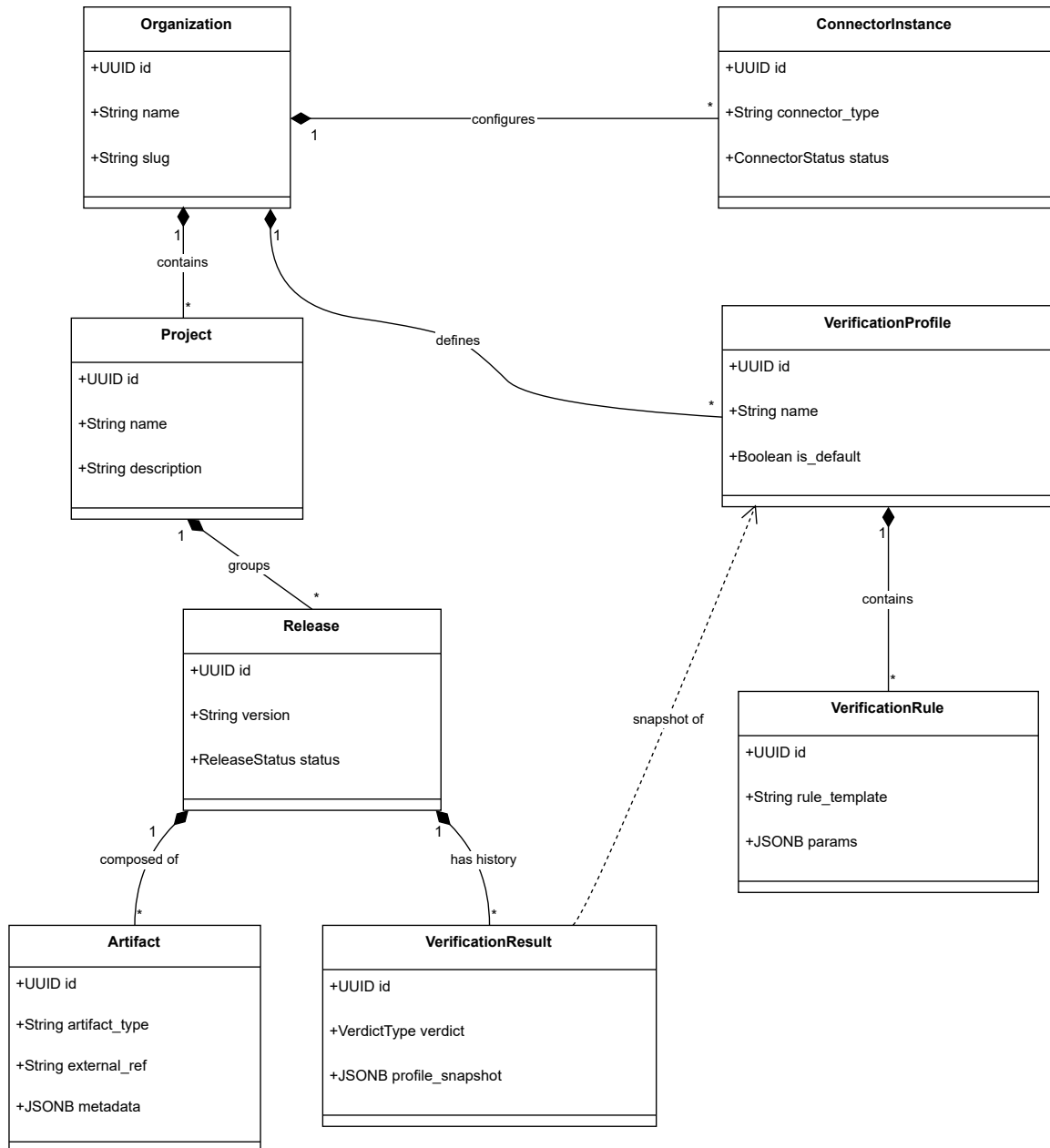


Figura 13: Diagrama entidad-relación del **SVAES** (notación Crow’s Foot). Se muestran las entidades del núcleo operativo del modelo; por claridad se omiten **user** y **user_membership**, que forman parte del núcleo pero cuyas relaciones con el resto del esquema son transversales. Las tablas **user**, **user_membership**, **api_key**, **custom_role**, **access_request**, **audit_log**, **notification_channel**, **notification_subscription** y **template** también forman parte del esquema (descritas en la Tabla 26) pero se omiten del diagrama por claridad.

5.6.3. Descripción de Tablas del Esquema

La Tabla 26 describe cada tabla del esquema relacional del **SVAES**, sus columnas principales, índices y propósito. La tabla utiliza el entorno **longtable** para fluir naturalmente entre páginas sin cortes forzados.

Tabla	Columnas principales	Índices	Descripción
user	id UUID PK, email UNIQUE, hashed_password, display_name, role, is_active, failed_login_attempts, locked_until, created_at, updated_at	email: UNIQUE; Trigger: updated_at	Usuarios del sistema. Las credenciales se almacenan como hash bcrypt. El campo role codifica el rol global del sistema (U2–U3). El rol U1 (invitado) corresponde a usuarios no autenticados y no se almacena en base de datos. El rol U4, por ser de ámbito de organización, se asigna exclusivamente en la tabla user_membership. Los campos failed_login_attempts y locked_until soportan la política de bloqueo por fuerza bruta (AC1.7).
organization	id UUID PK, name, slug UNIQUE, description, owner_id FK, is_active, created_at, updated_at	slug: UNIQUE; Trigger: updated_at	Entidad raíz del modelo multi-tenant. Toda consulta filtra por su id. owner_id referencia al usuario propietario de la organización. description es un campo de texto libre de hasta 500 caracteres (ORG1.4).
user_membership	id UUID PK, organization_id FK, user_id FK, role, created_at	UNIQUE (org_id, user_id)	Pertenencia de un usuario a una organización con su rol asignado.

Tabla	Columnas principales	Índices	Descripción
project	id UUID PK, organization_id FK, name, short_ key, description, profile_id FK, is_ archived, created_ at, updated_at	UNIQUE (org_id, name); UNIQUE (org_id, short_key)	Agrupar releases de un mismo producto dentro de una organización. short_key es una clave alfanumérica corta (2–10 caracteres, mayúsculas). profile_id asocia el perfil de verificación por defecto. is_archived soporta el archivado lógico sin borrado físico (GR7.2).
release	id UUID PK, project_id FK, profile_id FK, name, version, status, description, due_ date, assignee_ id FK, tags TEXT[], created_ by, created_at, updated_at	UNIQUE (project_id, version); idx: status, due_date; GIN: tags	Unidad principal de verificación. El campo status sigue la máquina de estados del dominio. due_date es la fecha prevista de entrega opcional, assignee_id referencia al miembro responsable y tags contiene las etiquetas de texto libre para clasificación.
artifact	id UUID PK, release_id FK, connector_ instance_id FK, artifact_type, external_ref, metadata JSONB, created_at	idx: release_id, artifact_type; GIN: metadata	Artefacto externo (tarea, commit, documento) asociado a una release. La implementación del conector se obtiene a través de la FK a connector_instance.

Tabla	Columnas principales	Índices	Descripción
connector_instance	id UUID PK, organization_id FK, connector_type, name, encrypted_ credentials BYTEA, status, last_tested_at, created_at	idx: org_id, status	Instancia de conector. Credenciales cifradas con Fernet.
verification_ profile	id UUID PK, organization_ id FK, name, description, is_ default, created_ at, updated_at	UNIQUE (org_id, na- me)	Conjunto de reglas de calidad aplicable a una release.
verification_rule	id UUID PK, profile_id FK, connector_ instance_id FK, rule_template, params JSONB, severity, display_ order, is_active, created_at	idx: profi- le_id; GIN: params	Instancia configurada de una plantilla de regla (RV-01 ... RV-10).
verification_ result	id UUID PK, release_id FK, verdict, rule_ results JSONB, profile_snapshot JSONB, executed_at, duration_ms	idx: relea- se_id, execu- ted_at; GIN: ru- le_results	Resultado inmutable de una ejecución del motor. El snapshot garantiza trazabilidad histórica.

Tabla	Columnas principales	Índices	Descripción
api_key	id UUID PK, user_id FK, organization_id FK, name, key_ hash UNIQUE, prefix, is_ active, expires_at, created_at, last_ used_at	UNIQUE: key_hash; idx: user_id	Clave de acceso programático a la API . El hash SHA-256 se almacena; el valor original solo se muestra en el momento de creación (AC5.2).
custom_role	id UUID PK, organization_id FK, name, permissions[], is_ active, created_at, updated_at	idx: org_id	Rol personalizado definido por un manager (U4) que agrupa un subconjunto de permisos del sistema. Soporta el requisito ORG3.2 de roles extensibles.
access_request	id UUID PK, organization_id FK, requested_email, requested_role, status, requested_ at, resolved_at, resolved_by FK	idx: organization_id, status	Solicitud de acceso de un usuario externo a una organización. Soporta los flujos U4 de aprobación/rechazo de altas (RF-ORG-06).
audit_log	event_id UUID PK, timestamp, user_id (pseudonimizado), organization_id FK, resource, action, payload_digest SHA-256	idx: timestamp, organization_id; GIN: resource	Registro inmutable de auditoría. Cada operación con efecto sobre el estado del sistema genera una fila (RNF-38).
notification_channel	id UUID PK, organization_id FK, channel_type, config JSONB, is_ active, created_at	idx: organization_id	Canal de salida configurado por una organización (email, webhook). Las credenciales se cifran con Fernet.

Tabla	Columnas principales	Índices	Descripción
notification_subscription	id UUID PK, channel_id FK, user_id FK, event_type, is_active	idx: channel_id, user_id	Suscripción de un usuario a un tipo de evento sobre un canal. Soporta el requisito de notificaciones selectivas.
template	id UUID PK, organization_id FK, name, kind, body JSONB, is_active, created_at, updated_at	idx: organization_id, kind	Plantilla reutilizable (perfil de verificación predefinido, formato de informe, etc.) gestionada por U4.

Tabla 26: Descripción de las diecisiete tablas del esquema relacional del [SVAES](#).

5.6.4. Consideraciones de Diseño del Esquema

Se adoptan las siguientes convenciones de diseño transversales a todo el esquema:

- Todas las tablas disponen de clave primaria UUID generada en el servidor mediante `gen_random_uuid()`. Se evitan claves enteras secuenciales para facilitar el aislamiento multi-tenant.
- Las tablas principales del dominio incluyen columnas de auditoría `created_at` y `updated_at` de tipo `TIMESTAMPTZ` como convención general. La columna `updated_at` se actualiza automáticamente mediante un trigger genérico en aquellas tablas que la incluyen. Las tablas de naturaleza inmutable (resultados de verificación, registros de auditoría, solicitudes de acceso) emplean nomenclatura específica para su marca temporal (`executed_at`, `timestamp`, `requested_at`).
- Las transiciones de estado de release se validan en la capa de aplicación mediante los casos de uso, no en la base de datos, para mantener la lógica de negocio en el dominio.
- Las credenciales de cada `connector_instance` se almacenan cifradas con Fernet en un campo `encrypted_credentials` de tipo `BYTEA`. La clave de cifrado reside en variables de entorno del backend.
- Los `verification_result` almacenan una instantánea (snapshot) del perfil y los artefactos en columnas `JSONB`, garantizando trazabilidad histórica aunque el perfil cambie posteriormente.
- Tipos enumerados nativos de PostgreSQL (`CREATE TYPE`) para: `release_status`,

`verdict_type`, `connector_status`, `severity_type` y `user_role`.

5.7. Diseño de la Interfaz de Usuario

El frontend del [SVAES](#) es una aplicación Angular de página única (SPA). Su diseño sigue las directrices de usabilidad recogidas en los requisitos no funcionales RNF-01 y RNF-02, con énfasis en la claridad de los resultados de verificación. El objetivo es que un usuario identifique inmediatamente qué reglas han fallado y por qué, sin necesidad de consultar documentación adicional.

5.7.1. Mapa de Navegación

Todas las rutas excepto `/auth/login` están protegidas por el **AuthGuard**. Las áreas funcionales son:

- **Autenticación** (`/auth/login`): inicio de sesión con formulario de credenciales y selección de organización activa.
- **Dashboard** (`/dashboard`): tasa de éxito de releases, tiempo medio de verificación, evolución temporal y listado de releases recientes.
- **Releases** (`/releases`, `/releases/:id`): listado paginado y filtrable, creación de releases y vista de detalle con veredicto global, resultado regla a regla con evidencias e historial.
- **Conectores** (`/connectors`): alta, prueba, edición y desactivación de conectores. Protegido por RoleGuard (MANAGER o superior).
- **Perfiles de verificación** (`/profiles`, `/profiles/:id`): creación y edición de conjuntos de reglas y asignación del perfil por defecto.
- **Administración** (`/admin`): gestión de usuarios y organizaciones. Solo para rol ADMIN.

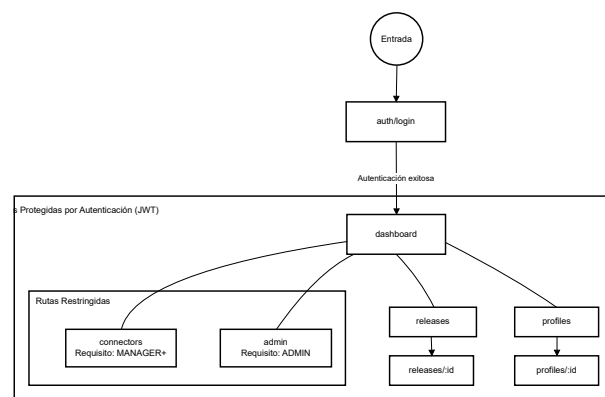


Figura 14: Mapa de navegación del frontend del [SVAES](#).

5.7.2. Wireframes de las Vistas Principales

Los wireframes de baja fidelidad validan la distribución funcional de los elementos en pantalla y los flujos de interacción. Los colores, tipografía y espaciado precisos se definirán durante la implementación del frontend.

Vista de detalle de release. Muestra el veredicto global con código de color inequívoco, un resumen de contadores por estado de regla y una tabla expandible con el resultado de cada regla individual. Cada fila incluye un enlace a las evidencias que originaron el resultado.

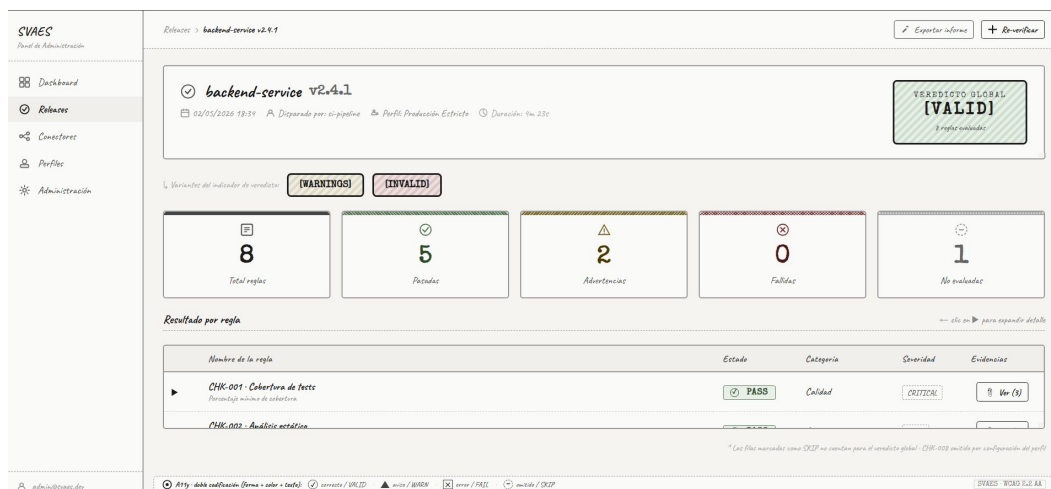


Figura 15: Wireframe de la vista de detalle de release.

Vista de gestión de conectores. Permite al Manager dar de alta nuevos conectores mediante un panel modal. El formulario muestra únicamente los campos requeridos por el tipo de conector seleccionado, derivados del esquema JSON retornado por `get_metadata()`.

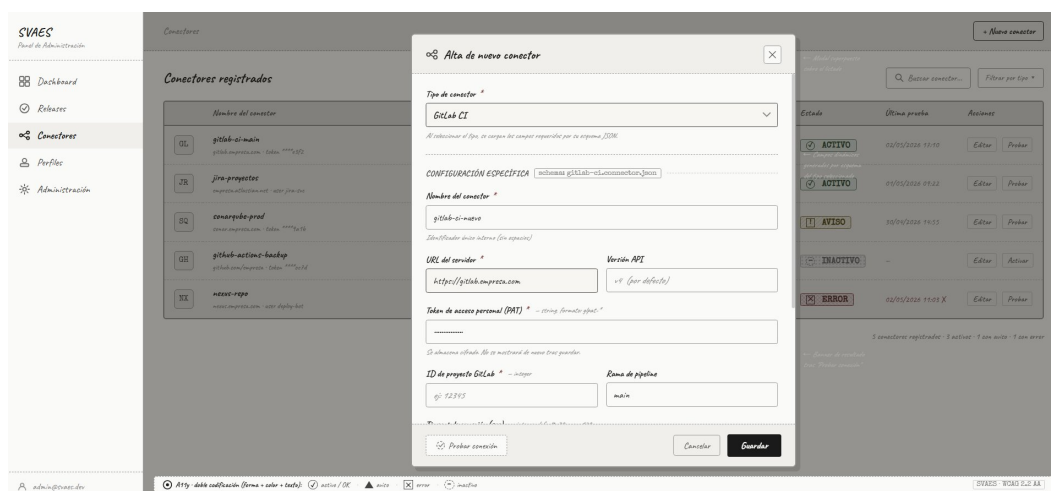


Figura 16: Wireframe de la vista de gestión de conectores.

Vista de edición de perfil. Sigue un diseño de dos paneles: el izquierdo lista las reglas del perfil; el derecho muestra el editor de la regla seleccionada con los campos derivados de la plantilla (RV-01 a RV-10).

Figura 17: Wireframe de la vista de edición de perfil de verificación.

Dashboard. Presenta cuatro KPI cards (tasa de releases válidas, total de releases, tiempo medio de verificación, reglas más fallidas), un gráfico de evolución temporal y un listado de releases recientes.

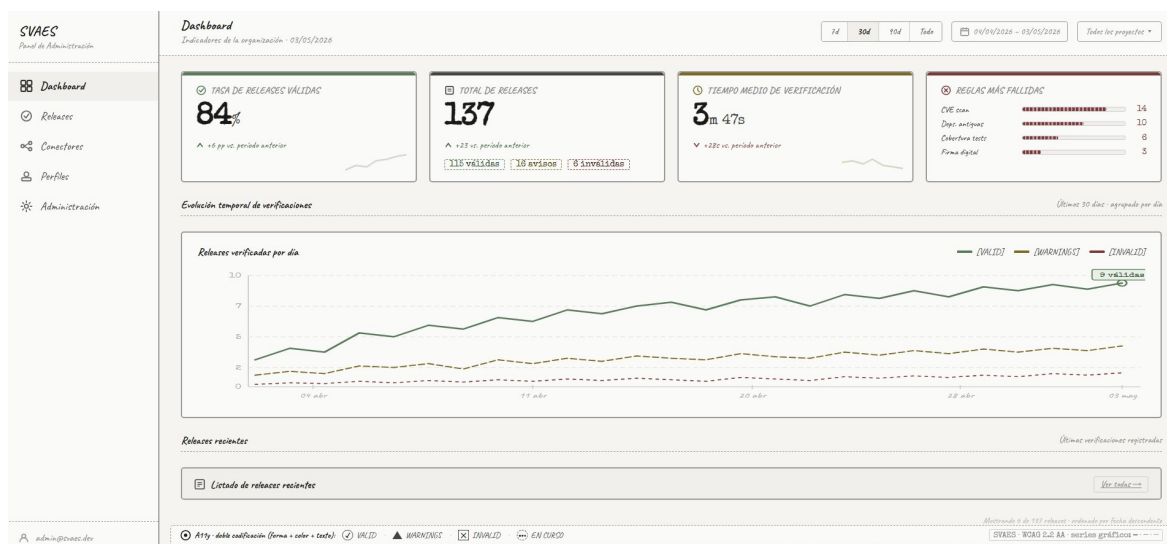


Figura 18: Wireframe de la vista Dashboard.

Preferencias de notificación en el perfil de usuario. La página de perfil del usuario incluye una tarjeta «Preferencias de notificación» con cuatro interruptores (*toggle*s) que gobiernan de forma independiente la recepción de correos electrónicos para los eventos `release_validated`, `release_invalidated`, `release_pending_reminder` y `weekly_digest`. Cada cambio de estado de un *toggle* envía una petición PATCH

`/api/v1/notifications/preferences`; si falla, el *toggle* revierte a su valor anterior (mecanismo de *rollback*). La carga inicial se realiza mediante GET al mismo *endpoint* al montar el componente.

5.7.3. Diseño de la API REST Pública

La API REST del SVAES se especifica mediante OpenAPI 3.1 y sigue las convenciones REST estándar. La especificación se genera automáticamente por FastAPI a partir de las anotaciones de tipo de los controladores y los esquemas Pydantic.

La API distingue tres mecanismos de autenticación según el endpoint:

- **Sin credenciales previas:** únicamente `/api/v1/auth/login`, que recibe usuario y contraseña (más, en su caso, el código TOTP) y emite el par de tokens.
- **Refresh token en cookie HttpOnly:** `/api/v1/auth/refresh` autentica al cliente mediante el *refresh token* transmitido en una *cookie* `HttpOnly`; `Secure`; `SameSite=Strict`, verifica su validez contra la base de datos (incluida la *blacklist* de revocación) y emite un nuevo JWT de acceso. La política `SameSite=Strict` mitiga CSRF y la marca `HttpOnly` impide la exfiltración mediante XSS.
- **Bearer Token (JWT):** el resto de los *endpoints* exigen la cabecera `Authorization: Bearer <JWT>`, con expiración de 30 minutos (configurable mediante `JWT_EXPIRE_MINUTES`) y validación de firma HS256.

Los errores de autenticación retornan HTTP 401; los de autorización, HTTP 403; los de validación de esquema, HTTP 422. El *rate limiting* (RNF-05, requisito API4) se implementa en el adaptador HTTP mediante middleware Redis con ventana deslizante y tres límites diferenciados por tipo de operación: 30 peticiones/minuto para endpoints de autenticación, 100 peticiones/minuto para operaciones de lectura y 20 peticiones/minuto para operaciones de escritura, con cabeceras `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` en todas las respuestas y HTTP 429 al superar el límite.

Endpoint	Método(s)	Descripción
<code>/api/v1/auth/login</code>	POST	Autenticación de usuario y emisión de JWT de acceso (30 min, configurable) y refresh token.
<code>/api/v1/auth/refresh</code>	POST	Renovación del JWT de acceso. Autenticado mediante refresh token en cookie <code>HttpOnly</code> .

Endpoint	Método(s)	Descripción
/api/v1/organizations	GET, POST, DELETE	Listado, creación y eliminación de organizaciones. DELETE requiere rol U3 y que la organización no tenga releases ni verificaciones asociadas (ORG2.6.1).
/api/v1/organizations/{org_id}/projects	GET, POST	Proyectos de una organización. Requiere pertenencia a la organización.
/api/v1/projects/{project_id}	GET, PATCH	Consulta y actualización de un proyecto (nombre, descripción y perfil de verificación por defecto, GR7.1.1). GET requiere acceso a la organización. PATCH requiere UPDATE_PROJECT.
/api/v1/projects/{project_id}/releases	GET, POST	Releases de un proyecto. POST requiere rol U2 o superior y que el proyecto no esté archivado (GR7.2.1).
/api/v1/releases/{release_id}	GET, PATCH, DELETE	Consulta y modificación de una release. DELETE solo en estado BORRADOR o PENDIENTE y sin verificaciones.
/api/v1/releases/{release_id}/verify	POST	Lanzamiento de verificación. Responde 202 Accepted con task_id. Requiere estado PENDIENTE.
/api/v1/releases/{release_id}/results	GET	Historial de resultados de verificación de la release, paginado.
/api/v1/releases/{release_id}/results/{result_id}	GET	Resultado individual de verificación con detalle por regla y evidencias.
/api/v1/organizations/{org_id}/connectors	GET, POST	Gestión de conectores de la organización. Requiere rol U4.
/api/v1/organizations/{org_id}/connectors/{connector_id}	PATCH, DELETE	Modificación y eliminación de un conector. Requiere rol U4.
/api/v1/organizations/{org_id}/connectors/{connector_id}/test	POST	Prueba de conectividad de un conector existente. Retorna resultado y latencia.

Endpoint	Método(s)	Descripción
/api/v1/organizations/{org_id}/profiles	GET, POST	Perfiles de verificación de una organización. POST requiere rol U4 o superior.
/api/v1/profiles/{profile_id}	PATCH, DELETE	Modificación y eliminación de un perfil de verificación. Requiere rol U4 o superior.
/api/v1/dashboard/metrics	GET	Métricas agregadas del dashboard: tasa de éxito, tiempo medio y tendencias.
/api/v1/organizations/{org_id}/roles	GET, POST	Gestión de roles personalizados de la organización. Requiere rol U4 o superior.
/api/v1/roles/{role_id}	PATCH, DELETE	Modificación y eliminación de un rol personalizado. Requiere pertenencia a la organización y rol U4 o superior.
/api/v1/templates	GET, POST	Listado y creación de plantillas de perfil de verificación. Requiere rol U3.
/api/v1/templates/{template_id}	GET, PATCH	Consulta y modificación de una plantilla.
/api/v1/templates/{template_id}/archive	POST	Archivado reversible de una plantilla (PV4.6).
/api/v1/templates/{template_id}/clone	POST	Clonado de una plantilla a la organización activa.
/api/v1/notifications/channels	GET, POST	Listado y creación de canales de notificación. Requiere rol U3.
/api/v1/notifications/channels/{channel_id}	PATCH, DELETE	Modificación y eliminación de un canal de notificación.

Endpoint	Método(s)	Descripción
/api/v1/notifications/preferences	GET, PATCH	Consulta y modificación de las preferencias de notificación del usuario autenticado. Cada preferencia se gobierna mediante un <i>toggle</i> independiente (<code>release_validated</code> , <code>release_invalidated</code> , <code>release_pending_reminder</code> , <code>weekly_digest</code>).
/api/v1/users/{user_id}/api-keys	GET, POST, DELETE	Gestión de API keys de un usuario.
/api/v1/tasks/{task_id}	GET	Consulta del estado de una tarea asíncrona. Retorna estado y progreso.
/api/v1/admin/rules/reload	POST	Recarga en caliente de reglas personalizadas (MV6.2.1). Requiere rol U3.
/api/v1/users/me	GET, PATCH, DELETE	Consulta y modificación del perfil del usuario autenticado (AC4.6, AC4.7). DELETE permite la autoeliminación de la cuenta con transferencia de propiedad de organizaciones cuando proceda (AC4.8).
<i>Nota:</i> la tabla recoge los endpoints principales de la API pública. El catálogo completo consta de 16 <i>routers</i> y 101 <i>endpoints</i> que incluyen, además de los aquí mostrados, los <i>routers</i> de roles personalizados (<code>/api/v1/organizations/{org_id}/roles</code>), claves de API (<code>/api/v1/users/{user_id}/api-keys</code>), solicitudes de acceso (<code>/api/v1/access-requests</code>), auditoría (<code>/api/v1/audit</code>), notificaciones (<code>/api/v1/notifications</code>) y retroalimentación (<code>/api/v1/feedback</code>).		

Tabla 27: Endpoints de la [API REST](#) pública del [SVAES](#).

5.8. Seguridad en el Diseño

Seguridad transversal desde el diseño siguiendo las directrices de OWASP Top 10 (2021). Integración de controles de seguridad basados en los requisitos RNF-12 a RNF-18.

- **Autenticación stateless mediante JWT:** tokens de acceso con expiración de 30 minutos (configurable mediante `JWT_EXPIRE_MINUTES`), complementados con refresh token de 30 días almacenado en la tabla `user` con capacidad de revocación individual.

El endpoint de renovación valida el refresh token contra la base de datos antes de emitir un nuevo JWT, soportando revocación inmediata. Para satisfacer AC2.2 (expiración por inactividad de 8 h), la tabla `user` incluye una columna `last_activity` que se actualiza en cada emisión o renovación exitosa de tokens; si la diferencia entre la marca temporal actual y `last_activity` supera las 8 horas, el *endpoint* de refresco rechaza la petición y obliga al usuario a iniciar sesión de nuevo.

- **Aislamiento multi-tenant a nivel de aplicación:** toda consulta incluye un filtro obligatorio por `organization_id`, inyectado como evento de escucha de SQLAlchemy.
- **Almacenamiento cifrado de credenciales de conectores:** Fernet (AES-128-CBC + HMAC-SHA256). La clave maestra se carga desde la variable de entorno `ENCRYPTION_KEY`.
- **Validación estricta de entrada (allowlist validation):** esquemas Pydantic con `extra='forbid'` en todos los endpoints. Validación de longitud máxima, tipo, formato y valores permitidos.
- **Registro de auditoría estructurado:** log append-only en formato JSON por cada operación con efecto sobre el estado del sistema, incluyendo usuario, organización, timestamp ISO 8601, recurso y acción.
- **Cabeceras de seguridad HTTP:** `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security` y `Referrer-Policy` configuradas en el middleware FastAPI.

5.9. Trazabilidad Diseño–Requisitos

La Tabla 28 recoge la trazabilidad entre los principales elementos de diseño de este capítulo y los requisitos del capítulo 4 que los originan. Esta matriz garantiza que todas las decisiones de diseño tienen una justificación explícita en los requisitos y que los requisitos más críticos están cubiertos por al menos un elemento de diseño.

Elemento de diseño	Requisitos cubiertos
Puerto IConnector + arquitectura hexagonal	GC1–GC6
Motor de verificación Rust (proceso independiente)	MV5.1, MV5.10; RNF-01, RNF-05, RNF-27
Cola de tareas asíncrona (Celery + Redis)	GR1–GR9; RNF-01, RNF-08
Multi-tenancy (filtro obligatorio por <code>organization_id</code>)	ORG1–ORG5; RNF-38

Elemento de diseño	Requisitos cubiertos
RBAC en adaptador HTTP (roles U1/U2/U3/U4)	AC1–AC5; RNF-38
Snapshot en VerificationResult (perfil + artefactos)	GR8; RNF-35, RNF-36
Cifrado Fernet de credenciales de conector	RNF-13
OpenAPI 3.x + generación de cliente Angular	GR9; RNF-40
PostgreSQL JSONB para configuración de conectores	GC1–GC3; RNF-32
JWT stateless con refresh token en BD	AC1–AC5; RNF-12, RNF-16
Docker Compose con red bridge interna	GR1; RNF-08, RNF-31
Tabla user + user_membership con roles	AC1–AC5; RNF-12, RNF-38
Rate limiting en adaptador HTTP (Redis, ventana deslizante)	AC4; RNF-05
Reglas de verificación predefinidas (RV-01 a RV-10)	MV5.1, MV5.10.
Motor de reglas personalizables (JSON-Path/JMESPath)	MV6.1–MV6.4

Tabla 28: Matriz de trazabilidad diseño–requisitos.

5.10. Resumen del Capítulo

Este capítulo ha definido la arquitectura completa del **SVAES** mediante el marco C4 y los principios de la Clean Architecture y la arquitectura hexagonal. Las decisiones fundamentales adoptadas son:

- **Arquitectura hexagonal con Clean Architecture:** el dominio es el único componente sin dependencias externas.
- **Motor de verificación en Rust** con diez reglas predefinidas (RV-01 a RV-10), desplegado como servicio Docker independiente con interfaz HTTP local (Actix-web).
- **Ejecución asíncrona de verificaciones** mediante Celery y Redis, con respuesta HTTP inmediata (202 Accepted) y consulta de estado mediante polling.

- **Puerto IConnector** como contrato genérico para toda integración. Dos conectores de referencia (Jira y GitLab) ilustran la implementación sin restringir el sistema a ninguna herramienta concreta.
- **Frontend Angular** con cliente REST generado automáticamente desde el contrato OpenAPI.
- **PostgreSQL 17** con esquema relacional completo incluyendo la tabla `user`, cifrado Fernet de credenciales, snapshots de resultados y tipos enumerados nativos.
- **Despliegue mediante Docker Compose** con red bridge interna `svaes-internal` y rate limiting mediante Redis.

Los elementos de diseño descritos sirven de base directa para la implementación descrita en el capítulo siguiente. La trazabilidad entre las decisiones de diseño y los requisitos del capítulo 4 garantiza que el diseño es completo y consistente con la especificación.

6 Implementación

Este capítulo describe las decisiones de implementación del sistema [SVAES](#) y justifica cómo el código fuente materializa la arquitectura definida en el capítulo 5. La exposición sigue el mismo orden que la descomposición en capas del capítulo anterior: dominio, aplicación, adaptadores primarios ([API REST](#)), adaptadores secundarios (persistencia y cola), motor de verificación, sistema de conectores, frontend y despliegue. Se documentan además los problemas técnicos encontrados durante el desarrollo y las soluciones adoptadas.

6.1. Estándares y Normas Seguidas

Los estándares y normas que rigen la implementación son los definidos en la sección 4.8 del capítulo 4. A nivel de código fuente se aplican adicionalmente las siguientes guías de estilo específicas de cada tecnología:

- **PEP 8** y **PEP 20**: convenciones de estilo y filosofía de diseño para todo el código Python del backend. El formateador `black` aplica automáticamente el estilo en el precommit.
- **Rust API Guidelines** (Rust Foundation, 2024): nomenclatura de tipos, rasgos y funciones del motor de verificación.
- **Angular Style Guide** (Google, 2024): estructura de módulos, convenciones de nombres de componentes, servicios y directivas del frontend.
- **OWASP Secure Coding Practices** [9]: controles de seguridad implementados en el adaptador HTTP y en el módulo de autenticación.

6.2. Lenguajes de Programación

La Tabla 29 recoge los lenguajes utilizados, sus versiones mínimas requeridas y la función que desempeñan en el sistema.

Lenguaje	Versión	Descripción
Python	3.11	Backend (API REST FastAPI y capa de dominio): casos de uso, entidades de dominio, conectores e infraestructura de persistencia.
Rust	1.77 (Actix-web 4.4)	Motor de verificación: evaluación paralela de las diez reglas RV-01–RV-10.
TypeScript	~5.9.2	Aplicación frontend (Angular 21): componentes, servicios, guardas y cliente REST tipado.

SQL (PostgreSQL 17)	—	Definición del esquema relacional y consultas de bajo nivel no cubiertas por el ORM.
---------------------	---	--

Tabla 29: Lenguajes de programación del [SVAES](#).

La elección de Python y Rust como lenguajes del backend y del motor respectivamente está justificada en la sección 2.10 del capítulo 2. TypeScript es la lengua nativa de Angular y aporta verificación de tipos en tiempo de compilación para el cliente REST generado desde el contrato OpenAPI.

6.3. Tecnologías y Herramientas Utilizadas

El *stack* tecnológico se corresponde con el definido en la Tabla 22 del capítulo 5. A las tecnologías allí recogidas se añaden las siguientes herramientas de calidad y análisis estático, activas en el repositorio durante todo el desarrollo:

- **SonarCloud**: análisis estático continuo con umbral de cobertura mínima del 80 %. Los informes se integran en cada *pull request* mediante GitHub Actions.
- **CodeQL**: análisis de seguridad estático sobre el código Python y TypeScript, ejecutado en cada *push* a la rama principal.
- **Dependabot**: detección y propuesta automática de actualizaciones de dependencias con parches de seguridad.
- **Locust**: generación de carga concurrente para las pruebas de rendimiento descritas en la sección 7.5 del capítulo 7.
- **OWASP ZAP**: análisis dinámico de vulnerabilidades de la [API REST](#), empleado en las pruebas de seguridad del mismo capítulo.

6.4. Implementación del Backend (FastAPI)

6.4.1. Organización del Código Fuente

El directorio `api/src/` implementa la organización en capas definida en la Tabla 23 del capítulo 5. La estructura de paquetes refleja directamente las cuatro capas de la Clean Architecture, lo que permite verificar el cumplimiento de la regla de dependencia mediante análisis estático de importaciones:

```

1 api/src/
2 +-- domain/           # Capa de dominio: cero dependencias externas
3   +-- entities/       # Entidades y objetos de valor
4   +-- enums.py        # Tipos enumerados del dominio
5 +-- application/      # Capa de aplicación

```

```

6   +-- ports/
7       +-- input/      # Puertos de entrada (interfaces de servicio)
8       +-- output/     # Puertos de salida (interfaces de repositorio)
9   +-- use_cases/      # Implementaciones de casos de uso
10  +-- infrastructure/  # Adaptadores (primarios y secundarios)
11      +-- primary/
12          +-- routers/ # Controladores FastAPI (v1)
13          +-- middleware/ # JWT, rate limiting, RBAC
14      +-- secondary/
15          +-- database/ # Modelos SQLAlchemy y repositorios
16          +-- queue/    # Worker Celery + Redis
17          +-- connectors/ # 20 implementaciones IConnector
18  +-- core/            # Configuración, inyección de dependencias

```

Listing 1: Estructura de directorios del backend [SVAES](#).

La separación entre `domain/` y `infrastructure/` es la restricción más importante del diseño: ningún módulo bajo `domain/` importa nada de `infrastructure/`, `application/` o cualquier librería de terceros. Esta propiedad se verifica automáticamente mediante la configuración de `import-linter` en el pipeline de integración continua.

6.4.2. Capa de Dominio

Las entidades del dominio (Organization, User, UserMembership, Project, Release, Artifact, ConnectorInstance, VerificationProfile, VerificationRule y VerificationResult) se implementan como *dataclasses* de Python con campos tipados. Todas las claves primarias son de tipo `uuid.UUID`, generadas en el servidor mediante `uuid.uuid4()`.

Ninguna entidad importa SQLAlchemy, Pydantic ni ningún otro framework. Los invariantes del dominio se protegen mediante propiedades con lógica de validación en los *setters*. Por ejemplo, la entidad `Release` rechaza cualquier transición de estado que no esté contemplada en la Figura 4 del capítulo 4, lanzando una excepción de dominio (`InvalidStateTransitionError`) que los casos de uso capturan y transforman en respuestas HTTP adecuadas. De forma análoga, la entidad `Project` expone un método `ensure_not_archived()` que el caso de uso `CreateRelease` invoca antes de instanciar una nueva `Release`, impidiendo su creación en proyectos archivados (GR7.2.1); los campos `name`, `description` y `default_profile_id` sí son mutables mediante *setters* dedicados, empleados por el caso de uso `UpdateProject` (GR7.1.1).

El fichero `domain/enums.py` define los tipos enumerados compartidos por todo el sistema: `ReleaseStatus`, `VerificationVerdict`, `UserRole`, `ConnectorType` y `ConnectorImplementation`. Este último combina los veinte identificadores de las implementaciones de conectores disponibles, siendo el único punto del sistema donde se nombran

explícitamente las herramientas externas concretas. Esta es una concesión pragmática al principio de inversión de dependencias: los valores del enumerado son identificadores abstractos que el `ConnectorRegistry` (en la capa de infraestructura) utiliza como claves de búsqueda para instanciar la clase concreta correspondiente. El dominio desconoce por completo la implementación de cada conector; únicamente conoce su identificador.

6.4.3. Capa de Aplicación: Puertos y Casos de Uso

Los puertos de salida se definen como clases abstractas (ABC) en el paquete `application/ports/output/`. Los puertos de entrada se implementan como `Protocol` de la librería estándar de Python, que ofrece tipado estructural sin herencia explícita. La distinción es deliberada: los puertos de entrada son contratos de comportamiento que el adaptador HTTP invoca, mientras que los puertos de salida son contratos que los repositorios deben satisfacer en tiempo de ejecución.

Cada caso de uso recibe sus dependencias por el constructor (*constructor injection*), lo que facilita su prueba unitaria con repositorios simulados. El fragmento de código 2 ilustra la estructura del caso de uso `LaunchVerification`, que orquesta el lanzamiento de una verificación asíncrona:

```
1 class LaunchVerification:
2     def __init__(
3         self,
4         release_repo: IReleaseRepository,
5         task_queue: ITaskQueue,
6         profile_repo: IProfileRepository,
7     ) -> None:
8         self._releases = release_repo
9         self._queue = task_queue
10        self._profiles = profile_repo
11
12    async def execute(self, release_id: UUID, user: User) -> TaskReference:
13        release = await self._releases.get_by_id(release_id)
14        release.request_verification()      # valida transición de estado
15        profile = await self._profiles.get_for_project(release.project_id)
16        task_ref = await self._queue.enqueue_verification(release, profile)
17        await self._releases.save(release)
18        return task_ref
```

Listing 2: Estructura del caso de uso `LaunchVerification`.

El caso de uso no conoce ni la base de datos ni la cola de mensajes: opera exclusivamente sobre los puertos abstractos. Los adaptadores secundarios proporcionan las implementaciones concretas en tiempo de ejecución mediante el contenedor de dependencias de

FastAPI.

6.4.4. Adaptadores Primarios: **API REST** y Middleware

Los controladores FastAPI residen en `infrastructure/primary/routers/` y se organizan por recurso en dieciséis módulos: `auth.py` y `users.py` (autenticación y gestión de usuarios); `organizations.py`, `projects.py` y `releases.py` (entidades del dominio); `connectors.py`, `profiles.py` y `templates.py` (integración y políticas de calidad); `admin.py`, `audit.py`, `tasks.py`, `access_requests.py`, `api_keys.py`, `custom_roles.py` y `notifications.py` (administración, trazabilidad y comunicaciones); `dashboard.py` (métricas); y `feedback.py` (canal de retroalimentación de usuarios). Todos los *routers* se registran bajo el prefijo `/api/v1/`. El *router* de usuarios incluye, además de las operaciones de administración, el *endpoint* `GET /api/v1/users/me/export`, que devuelve un fichero JSON con todos los datos personales del usuario autenticado y registra una entrada de auditoría `DATA_EXPORT_REQUESTED`, garantizando los derechos de acceso y portabilidad del RGPD (artículos 15 y 20).

Soporte multi-organización. El módulo `users.py` incorpora dos *endpoints* adicionales para la gestión de membresías múltiples:

- `GET /api/v1/users/me/organizations`: devuelve la lista de organizaciones del usuario autenticado con su rol en cada una y cuál es la organización activa.
- `POST /api/v1/users/me/switch-organization`: cambia la organización activa (requiere membresía existente) y reemite `access_token` y `refresh_token` con el `organization_id` y `role` actualizados.

El *endpoint* `POST /api/v1/organizations/{org_id}/users/invite` se amplió para permitir invitar a un usuario ya miembro de otra organización; si el usuario invitado no tiene organización activa, la nueva pasa a serlo. El *endpoint* `GET /api/v1/organizations/{org_id}/users` es accesible ahora también a los miembros de esa organización aunque no sea su organización activa.

Los esquemas Pydantic de entrada y salida residen exclusivamente en los módulos de los *routers* y nunca se utilizan fuera de la capa de infraestructura. Los esquemas de entrada definen `model_config = ConfigDict(extra="forbid")`, lo que rechaza automáticamente con `HTTP 422` cualquier campo no declarado, implementando la validación de lista blanca (*allowlist validation*) requerida por RNF-05.

La autenticación se implementa como una dependencia reutilizable de FastAPI:

- `get_current_user`: decodifica y valida el **JWT** de la cabecera `Authorization: Bearer <token>`. Retorna `HTTP 401` si el token es inválido o ha expirado.
- `require_role(minimum_role)`: dependencia paramétrica que comprueba que el rol del usuario autenticado es al menos el mínimo requerido. Retorna `HTTP 403` en caso contrario.

El módulo de *rate limiting* usa `slowapi` con Redis como *backend* de contadores. Se definen tres límites distintos: 30 peticiones por minuto en los *endpoints* de autenticación, 100 peticiones por minuto en las operaciones de lectura y 20 peticiones por minuto en las operaciones de escritura. Los contadores se almacenan en Redis con expiración automática, lo que garantiza el reinicio de las ventanas sin limpiezas periódicas.

Las cabeceras de seguridad `HTTP` (`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security` y `Referrer-Policy`) se inyectan en todas las respuestas mediante un *middleware* propio que envuelve la aplicación FastAPI.

6.4.5. Adaptadores Secundarios: Persistencia y Migraciones

Los modelos SQLAlchemy residen en `infrastructure/secondary/database/models/` y son completamente independientes de las entidades del dominio. El mapeo entre modelo ORM y entidad de dominio se realiza en los repositorios, que implementan los puertos de salida (`IReleaseRepository`, `IUserRepository`, etc.).

La configuración de la sesión asíncrona utiliza `AsyncSession` de SQLAlchemy ^2.0.51 con `asyncpg` como driver PostgreSQL. El contexto de sesión se gestiona mediante una dependencia FastAPI que garantiza el cierre de la transacción al finalizar cada petición, incluso ante excepciones no controladas.

Las migraciones del esquema se gestionan con Alembic. El directorio `alembic/versions/` contiene los scripts de migración en orden cronológico. La migración inicial (`2fd6efcfd6c9_initial_schema.py`) crea las diez tablas del núcleo del dominio descritas en la sección 5.6 del capítulo 5: `organization`, `user`, `user_membership`, `project`, `release`, `artifact`, `connector_instance`, `verification_profile`, `verification_rule` y `verification_result`. Migraciones posteriores incorporan las siete tablas restantes del esquema final: `api_key` y `custom_role` (gestión de acceso programático y roles personalizados), `access_request` (flujo de altas), `audit_log` (trazabilidad RNF-38), `notification_channel` y `notification_subscription` (notificaciones) y `template` (plantillas reutilizables). El estado consolidado tras todas las migraciones coincide con las diecisiete tablas de la Tabla 26.

La tabla `user_membership` (creada en la migración inicial pero no utilizada desde el código de aplicación) se reactivó para soportar el patrón multi-organización. Una migración de solo datos (`u9v0w1x2y3z4_backfill_user_membership.py`) crea, para cada usuario con `organization_id` no nulo, una fila de membership con su rol actual, convirtiendo la relación single-org existente en su primera membership. El modelo SQLAlchemy correspondiente reside en `infrastructure/secondary/database/models/user_membership_model.py` y el repositorio en `infrastructure/secondary/database/repositories/user_membership_repository.py`.

6.4.6. Seguridad Transversal

Autenticación JWT y doble factor (2FA). El módulo `auth.py` del *router* gestiona dos flujos de autenticación. En el flujo estándar, un `POST /auth/login` con credenciales válidas emite un `JWT` de acceso con expiración de 30 minutos (configurable mediante `JWT_EXPIRE_MINUTES`) firmado con `HS256`. Si el usuario tiene el segundo factor activado, el mismo *endpoint* retorna únicamente un *pre-token* de corta vida. El código TOTP de seis dígitos generado por la aplicación de autenticación del usuario se valida en `POST /auth/2fa/verify` mediante la librería `pyotp`, que implementa el algoritmo TOTP definido en la RFC 6238.

El secreto TOTP de cada usuario se almacena cifrado con Fernet en la base de datos. Los códigos QR para el registro del segundo factor se generan con `segno` y se sirven *in-line* en la respuesta de activación.

Cifrado de credenciales de conectores. Las credenciales que los gestores de organización registran para cada conector externo (claves API, tokens OAuth, contraseñas) se cifran con `cryptography.fernet.Fernet` antes de persistirse. Fernet proporciona cifrado simétrico autenticado (AES-128-CBC + HMAC-SHA256), satisfaciendo el requisito RNF-13 de confidencialidad e integridad de las credenciales. La clave Fernet se carga desde la variable de entorno `ENCRYPTION_KEY` al inicio de la aplicación; si la variable no está presente, el proceso no arranca.

Protección contra fuerza bruta. El módulo de autenticación lleva un contador de intentos fallidos por usuario almacenado en Redis. Tras cinco intentos consecutivos fallidos, la cuenta queda bloqueada durante 15 minutos (`HTTP 403` con cabecera `Retry-After`). El contador se reinicia automáticamente ante un inicio de sesión exitoso.

6.5. Motor de Verificación en Rust

6.5.1. Estructura del Proyecto y Servidor HTTP

El motor de verificación reside en el directorio `engine/` como un *workspace* Cargo independiente del backend Python. Esta separación de repositorio garantiza que ningún cambio en las dependencias del backend puede afectar la compilación del motor, y permite que ambos componentes evolucionen con ciclos de *release* distintos.

El punto de entrada del motor es `engine/src/main.rs`, que lanza un servidor Actix-web en el puerto 8081 (solo accesible en la red interna `svaes-internal`). El servidor expone un único *endpoint*:

```
1 POST /verify
```

```

2 Content-Type: application/json
3 Body: VerificationPayload (definido en engine/src/models.rs)
4 Response: EngineResult      (definido en engine/src/models.rs)

```

Listing 3: *Endpoint* del motor de verificación.

El contrato de datos se define mediante JSON Schema compartido entre el motor y el backend. El backend serializa el *payload* con la librería `httpx` (petición síncrona bloqueante desde el *worker* Celery) y deserializa la respuesta. El motor no realiza llamadas de red: opera exclusivamente sobre los datos incluidos en el *payload*.

6.5.2. Pipeline de Verificación y Ejecución Paralela

Cada solicitud de verificación desencadena el siguiente pipeline, tal y como se describió en la sección 5.4 del capítulo 5:

1. **Deserialización:** `serde_json` convierte el cuerpo `HTTP` en la estructura `VerificationPayload`, que incluye los metadatos de la release, el perfil de verificación y los artefactos externos ya recuperados por el *worker*.
2. **Resolución de reglas:** el componente `RuleResolver` lee el campo `profile_config` del *payload* y construye el conjunto de instancias de regla aplicables con sus parámetros de configuración.
3. **Ejecución paralela:** el `crate rayon` distribuye la evaluación de las reglas entre los hilos del *pool* del sistema mediante *work-stealing scheduling*. Cada regla implementa el rasgo `Rule` (véase fragmento 4) y produce un `RuleOutcome` independiente.
4. **Agregación:** el componente `Aggregator` calcula el veredicto global. Si alguna regla obligatoria produce `ERROR`, el veredicto es `INVALID`. Si solo hay `WARNINGS` en reglas opcionales, el veredicto es `VALID_WITH_WARNINGS`. En ausencia de incidencias, es `VALID`. El sufijo `_WITH_INCIDENTS` se añade si al menos una regla quedó en estado `NOT_EVALUATED` por un fallo del conector.
5. **Serialización:** `serde_json` convierte el `EngineResult` en `JSON` y Actix-web lo retorna con `HTTP 200`.

```

1 pub trait Rule: Send + Sync {
2     fn id(&self) -> &str;
3     fn evaluate(
4         &self,
5         artifacts: &[ArtifactData],
6         config: &RuleConfig,
7     ) -> RuleOutcome;
8 }

```

Listing 4: Rasgo `Rule` del motor de verificación.

La implementación del rasgo **Send + Sync** es un requisito de Rayon: solo los tipos que cumplen estas propiedades pueden participar en el *scheduling* de hilos. Dado que ninguna regla mantiene estado mutable tras su construcción, todas las implementaciones satisfacen automáticamente estos requisitos.

6.5.3. Implementación de las Reglas RV-01 a RV-10

La Tabla 30 resume la implementación de las diez reglas del catálogo, indicando el módulo fuente donde reside cada una.

ID	Módulo	Descripción
RV-01	engine/src/rules/rv01.rs	Verifica que la lista de artefactos no está vacía.
RV-02	engine/src/rules/rv02.rs	Verifica la trazabilidad entre artefactos de tipos <code>source_type</code> y <code>target_type</code> mediante el campo <code>reference_field</code> .
RV-03	engine/src/rules/rv03.rs	Valida que los artefactos del tipo configurado se encuentren en uno de los estados permitidos.
RV-04	engine/src/rules/rv04.rs	Comprueba la integridad de los campos numéricos configurados (no nulos, numéricos, ≥ 0).
RV-05	engine/src/rules/rv05.rs	Verifica que los documentos asociados a los artefactos están accesibles.
RV-06	engine/src/rules/rv06.rs	Comprueba la coherencia de un atributo configurado contra un valor esperado.
RV-07	engine/src/rules/rv07.rs	Verifica la presencia del campo marcador de registro externo en los artefactos.
RV-08	engine/src/rules/rv08.rs	Alinea listas de artefactos comprobando la correspondencia entre campos maestro y dependiente.
RV-09	engine/src/rules/rv09.rs	Valida que los campos de referencia de los artefactos apuntan a recursos accesibles.
RV-10	engine/src/rules/rv10.rs	Verifica que los artefactos han alcanzado un estado de aprobación final.

Tabla 30: Implementación de las reglas de verificación en Rust.

Todas las reglas comparten la misma firma: `pub fn evaluate(artifacts: &[ArtifactData], rule_config: &RuleConfig) -> RuleEvaluation`. Cada módulo bajo `engine/src/rules/` implementa esta función de forma independiente. Las reglas son *stateless*: la función recibe únicamente los datos de entrada (artefactos y configuración

de la regla), sin mantener estado mutable entre invocaciones.

6.6. Sistema de Conectores

6.6.1. Puerto IConnector

El puerto IConnector se implementa como Protocol de Python (tipado estructural) en `application/ports/output/i_connector.py`. El fragmento 5 reproduce la interfaz completa:

```
1 class IConnector(Protocol):
2     @property
3     def connector_type(self) -> str: ...
4
5     @property
6     def connector_implementation(self) -> str: ...
7
8     async def test_connection(
9         self, config: Dict[str, Any]
10    ) -> bool: ...
11
12    async def fetch_artifact(
13        self, ref: str, config: Dict[str, Any]
14    ) -> Dict[str, Any]: ...
15
16    async def list_artifacts(
17        self, filter_params: Dict[str, Any], config: Dict[str, Any]
18    ) -> List[Dict[str, Any]]: ...
19
20    def get_metadata(self) -> Dict[str, Any]: ...
```

Listing 5: Puerto IConnector.

El uso de Protocol en lugar de ABC permite que cualquier clase que satisfaga estructuralmente la interfaz sea tratada como conector sin necesidad de herencia explícita. Esto facilita la adición de nuevos conectores por parte de terceros sin modificar el paquete `application/`.

6.6.2. Catálogo de Implementaciones

El sistema incluye veinte implementaciones de `IConnector`, distribuidas en cinco tipos funcionales. La Tabla 31 detalla las implementaciones disponibles.

Tipo funcional	Implementación	Protocolo API	Clase Python
GESTOR_TAREAS	Jira	REST v2/v3	<code>JiraConnector</code>
	Linear	GraphQL	<code>LinearConnector</code>
	Trello	REST	<code>TrelloConnector</code>
	Asana	REST	<code>AsanaConnector</code>
REPO_CODIGO	GitLab	REST v4	<code>GitLabConnector</code>
	GitHub	REST	<code>GitHubConnector</code>
	Bitbucket	REST	<code>BitbucketConnector</code>
	Gitea	REST	<code>GiteaConnector</code>
SISTEMA_DOCUMENTAL	Confluence	REST	<code>ConfluenceConnector</code>
	Notion	REST	<code>NotionConnector</code>
	Wiki.js	GraphQL	<code>WikijsConnector</code>
	BookStack	REST	<code>BookStackConnector</code>
HERRAMIENTA_PLANIF.	ClickUp	REST	<code>ClickUpConnector</code>
	Taiga	REST	<code>TaigaConnector</code>
	Plane	REST	<code>PlaneConnector</code>
	Miro	REST	<code>MiroConnector</code>
GESTION_CAMBIOS	Jira SM	REST	<code>JiraSMConnector</code>
	GLPI	REST	<code>GLPIConnector</code>
	Zammad	REST	<code>ZammadConnector</code>
	Redmine	REST/XML	<code>RedmineConnector</code>

Tabla 31: Catálogo de implementaciones de conectores del [SVAES](#).

Todos los conectores que usan GraphQL delegan la construcción de las consultas en un módulo de utilidades compartido (`infrastructure/secondary/connectors/graphql_utils.py`), evitando la duplicación de la lógica de *batching* y manejo de *cursors* de paginación.

Las peticiones HTTP salientes utilizan `httpx.AsyncClient` con tiempo de espera

configurado en 10 segundos. Si el conector lanza una excepción (`ConnectorTimeout`, `ConnectorAuthError`, `ConnectorNotFound` o `ConnectorRateLimit`), el *worker* la captura y marca la regla afectada como `NOT_EVALUATED`, en lugar de abortar la verificación completa.

6.6.3. ConnectorRegistry y Cifrado de Credenciales

El `ConnectorRegistry` (puerto de salida `IConnectorRegistry`) mapea cada valor de `ConnectorImplementation` a la clase Python correspondiente. La instanciación es *on-demand*: el *worker* pide al registro el conector adecuado para cada `ConnectorInstance` justo antes de llamar a `list_artifacts`.

Las credenciales se descifran en memoria únicamente durante el tiempo de ejecución de la verificación. El ciclo de vida de la clave en memoria es:

1. El *worker* carga la `ConnectorInstance` de la base de datos (credenciales en formato cifrado).
2. Descifra el *blob* con `Fernet.decrypt()` usando la clave cargada al inicio del proceso desde `ENCRYPTION_KEY`.
3. Pasa las credenciales en texto plano al conector en forma de `Dict[str, Any]`. El diccionario se destruye al finalizar el *scope* de la función.

Este patrón garantiza que las credenciales en texto plano nunca se persisten ni se registran en los *logs*.

6.7. Cola de Tareas Asíncrona (Celery + Redis)

El fichero `infrastructure/secondary/queue/verification_worker.py` define la tarea Celery `run_verification`. La tarea recibe el identificador de la release y el identificador del perfil, recupera los datos necesarios de la base de datos (relación N+1 evitada mediante carga *eager*), construye el `VerificationPayload` y realiza la llamada HTTP al motor Rust.

El ciclo de vida de una tarea de verificación expone cuatro estados al usuario: `ENCOLADA`, `EN_PROGRESO`, `COMPLETADA` y `FALLIDA`. La actualización de estado a `EN_PROGRESO` se produce antes de la llamada al motor; la actualización a `COMPLETADA` o `FALLIDA` tras recibir la respuesta. El frontend realiza *polling* al *endpoint* `GET /api/v1/tasks/{task_id}` cada tres segundos hasta recibir uno de los dos estados terminales.

El número de procesos hijo concurrentes se configura mediante la variable de entorno `CELERY_CONCURRENCY` (valor por defecto: 4). El modelo de concurrencia *prefork* de Celery genera múltiples procesos hijo a partir de un proceso maestro, de modo que cada proceso hijo procesa una verificación de forma independiente y bloqueante, permitiendo así la ejecución en paralelo de múltiples verificaciones.

Además de `run_verification`, el sistema registra tareas adicionales en Celery para operaciones asíncronas de larga duración, entre ellas `export_csv` (generación de archivos CSV cuando el conjunto a exportar supera los 500 registros, conforme a RH4.4 y DB4.2) y `export_pdf` (generación de informes en segundo plano cuando se solicita desde el historial). Ambas tareas notifican al usuario mediante el sistema de notificaciones al completarse. La función `_notify_user` del *worker* de verificación se ha ampliado con una consulta previa a la tabla `notification_subscription` antes de enviar el correo electrónico de resultado. El mapeo de veredictos a tipos de notificación sigue la siguiente correspondencia: los veredictos `VALID` y `VALID_WITH_WARNINGS` se traducen al tipo `RELEASE_VALIDATED`, mientras que el resto de veredictos (`INVALID`, `INVALID_WITH_INCIDENTS`) se asocian al tipo `RELEASE_INVALIDATED`. Si el usuario carece de fila en la tabla de suscripción, el sistema asume el envío activo por defecto; si la fila existe y la suscripción correspondiente está desactivada, el envío se omite. Esta funcionalidad permite a cada usuario controlar qué correos de notificación desea recibir sin renuncia al seguimiento de las verificaciones a través de la interfaz web.

6.7.1. Servicios de Aplicación: Gestión Multi-organización

Los servicios `UserService` y `OrganizationService` reciben `user_membership_repository` como dependencia **opcional** (`None` por defecto) para no romper la firma en código y tests existentes; en producción `core/dependencies.py` siempre la inyecta.

`UserService` amplió los siguientes métodos para soportar membresías múltiples:

- `invite_user`: si el usuario ya existe y **no** es miembro de la organización destino, se crea una membership adicional con el rol indicado; si ya es miembro de esa organización, lanza `DuplicateEntityError`. Si el usuario invitado no tenía organización activa, la nueva organización pasa a serlo.
- `update_user_role` y `remove_user_from_organization`: operan sobre la membership de la organización indicada, no sobre la org activa del usuario. Si la organización afectada es la activa, se sincroniza también `user.role`; si no, solo se actualiza o borra la membership (la sesión activa no se ve afectada). Al eliminar la membership activa, si quedan otras, la primera pasa a ser la nueva organización activa.
- `list_organization_users`: lista por membership, devolviendo a cada miembro con **su rol en esa organización concreta**.
- `list_user_organizations(user_id)`: devuelve todas las organizaciones del usuario con su rol en cada una.
- `switch_active_organization(user_id, organization_id)`: valida que exista membership y actualiza `user.organization_id` y `user.role`.
- `delete_user_account`: la comprobación de «eres *owner* de una organización con

otros miembros» ahora recorre **todas** las membership del usuario, no solo la activa.

OrganizationService amplió:

- **create_organization**: crea la membership del *owner* junto con el registro de organización.
- **transfer_ownership**: sincroniza el rol de la membership del *owner* saliente y entrante, además del rol denormalizado en *user*.

El puerto IUserMembershipRepository define los métodos: **create**, **get**, **list_by_user**, **list_by_organization**, **update_role**, **delete** y **delete_all_for_user**.

6.8. Frontend Angular

6.8.1. Estructura de la Aplicación

La aplicación frontend se desarrolló con Angular 21 y TypeScript ^{~5.9.2}, empleando exclusivamente componentes *standalone* (sin NgModule), detección de cambios *zoneless* (`provideZonelessChangeDetection()`) y Signals para la reactividad. El proyecto se organiza en las siguientes áreas funcionales bajo `web/src/app/`:

- **features/auth/**: formularios de *login*, registro, verificación de 2FA y recuperación de contraseña.
- **features/dashboard/**: vista de resumen con métricas agregadas de la organización seleccionada.
- **features/releases/**: gestión completa del ciclo de vida de releases: creación, edición de artefactos, lanzamiento de verificación y consulta del historial de resultados.
- **features/connectors/**: alta, edición y prueba de conectividad de instancias de conectores.
- **features/profiles/**: definición y edición de perfiles de verificación y sus reglas.
- **features/admin/**: gestión de usuarios, organizaciones y plantillas (roles U3 y superiores).
- **core/**: servicios transversales (`AuthService`, `ToastService`, `TranslationService`, `DashboardService`), guardas funcionales (`CanActivateFn`) e interceptores (`jwtInterceptor`, `errorInterceptor`, `timeoutInterceptor`).
- **shared/**: componentes reutilizables: semáforo de verificación, tabla de resultados, formulario genérico de conector y modal de confirmación.

El enrutado se define en `appConfig` mediante *lazy loading* con `loadComponent`, sin referencias a módulos. Las pruebas unitarias y de componente utilizan **Vitest 4.x** como ejecutor, en sustitución de Jasmine/Karma.

El cliente REST tipado se genera desde la especificación OpenAPI 3.1 del backend mediante `@hey-api/openapi-ts` ^{~0.97.3}, con `@hey-api/client-fetch` ^{~0.13.1} como *runtime*. El código generado reside en `web/src/app/api/` y no debe editarse manualmente. Los

componentes no utilizan el SDK generado directamente: emplean `HttpClient` de Angular para todas las llamadas a la [API](#), aprovechando los tipos exportados del SDK como contratos estáticos que garantizan la coherencia con el contrato OpenAPI sin añadir una capa de abstracción adicional en tiempo de ejecución.

6.8.2. Autenticación, 2FA y RBAC en Cliente

El servicio `AuthService` gestiona el ciclo de vida de los tokens [JWT](#): almacena el *access token* en memoria (no en `localStorage`) y el *refresh token* en una *cookie* `HttpOnly; Secure; SameSite=Strict`. Esta elección protege el *refresh token* frente a ataques XSS.

Soporte multi-organización. El servicio `AuthService` incorpora dos métodos adicionales:

- `getMyOrganizations()`: consulta `GET /api/v1/users/me/organizations` y devuelve la lista de organizaciones del usuario con su rol en cada una.
- `switchOrganization(org_id)`: invoca `POST /api/v1/users/me/switch-organization`, persiste los nuevos tokens en la *cookie* y actualiza el usuario en `localStorage`.

El componente `ShellComponent` (`features/layout/shell/shell.component.*`) muestra la organización activa en la *topbar*. Cuando el usuario pertenece a más de una organización, la *topbar* muestra un desplegable con la opción «Cambiar de organización»; seleccionar otra llama a `switchOrganization` y recarga la aplicación para refrescar todo el estado anclado a la organización anterior. Las claves i18n nuevas (`shell.switch_org_label`, `shell.switch_org_current`, `shell.switch_org_error`) se añadieron a `en.json` y `es.json`.

El flujo de doble factor se implementa como una secuencia de dos pasos en el *router* Angular:

1. El formulario de *login* envía credenciales. Si el servidor retorna un *pre-token*, el usuario se redirige automáticamente a la pantalla de código TOTP.
2. El formulario de código TOTP envía el código de seis dígitos y el *pre-token*. Si la validación es correcta, el servidor emite el [JWT](#) completo y la sesión comienza.

La página de perfil (`ProfileComponent`) incluye, junto a la gestión de credenciales y la eliminación de cuenta, una sección de exportación de datos personales (RGPD). El usuario pulsa el botón «Descargar datos», el frontend invoca `GET /api/v1/users/me/export` y el navegador descarga el fichero `svaes-user-data.json` con todos los datos que el sistema almacena sobre el usuario. Las claves de internacionalización (`profile_page.export_data_*`) residen en los ficheros `es.json` y `en.json` para español e inglés respectivamente.

La sección de preferencias de notificación, incorporada a la misma página de perfil, presenta al usuario una tarjeta con cuatro interruptores (*toggles*) que gobiernan las categorías de correo electrónico: `release_validated`, `release_invalidated`, `release_pending_reminder` y `weekly_digest`. Cada cambio de estado de un *toggle* dis-

para una petición `PATCH /api/v1/notifications/preferences` con el nuevo valor; si el servidor rechaza la actualización, el componente revierte el *toggle* a su estado anterior mediante un mecanismo de *rollback* y presenta un mensaje de error descriptivo. La carga inicial de preferencias se realiza al montar el componente mediante una petición `GET` al mismo *endpoint*.

Las guardas de ruta se implementan como funciones `CanActivateFn` (*functional guards*) aplicadas mediante `appConfig`:

- `authGuard`: verifica la presencia de un token válido en memoria y redirige al *login* si la comprobación falla.
- `roleGuard(requiredRole)`: guarda paramétrica que comprueba que el rol del usuario autenticado satisface el mínimo requerido por la ruta. Retorna `false` y redirige a una vista de acceso denegado en caso contrario.
- `businessRouteGuard`: validaciones específicas de negocio (p.ej. pertenencia del usuario a la organización del recurso solicitado), aplicables a rutas que requieren contexto de organización o proyecto.

Las peticiones HTTP salientes son interceptadas por tres interceptores funcionales registrados con `withInterceptors()`:

- `jwtInterceptor`: adjunta el `JWT` de acceso en la cabecera `Authorization` de toda petición autenticada y gestiona la renovación automática del token ante una respuesta `HTTP 401`.
- `errorInterceptor`: captura errores de red y de servidor, presentando notificaciones *toast* descriptivas al usuario.
- `timeoutInterceptor`: aplica un tiempo máximo de espera configurable a cada petición, abortando aquellas que superen el umbral.

6.8.3. Módulos de Gestión de Releases y Conectores

La vista de detalle de release (`ReleaseDetailComponent`) centraliza el flujo de trabajo del usuario tipo U2 (Operador). Muestra los artefactos asociados, el estado actual de la release y el botón de lanzamiento de verificación. Durante la verificación, el componente activa el *polling* al *endpoint* de tareas y actualiza la vista en tiempo real conforme avanza el procesamiento.

El componente de resultados (`VerificationResultComponent`) renderiza el informe completo: cabecera con veredicto global y duración, tabla de reglas con semáforo de color (verde/naranja/rojo), detalle expandible por regla con la fuente de datos consultada, el valor esperado y el valor obtenido. Los resultados históricos se muestran paginados, preservando el perfil de verificación vigente en el momento de cada ejecución gracias al *snapshot* almacenado en la tabla `verification_result`.

6.8.4. Internacionalización

La aplicación soporta dos idiomas: español (predeterminado) e inglés. La internacionalización se implementa con el módulo `@ngx-translate/core`, que carga los ficheros JSON de traducciones de forma *lazy* según el idioma activo. El selector de idioma en la cabecera persiste la preferencia del usuario en `localStorage`. Todos los textos visibles de la interfaz, incluidos los mensajes de error de validación y los estados del semáforo, están externalizados en los ficheros de traducción.

6.9. Infraestructura de Despliegue

6.9.1. Configuración Docker Compose

El sistema se despliega con Docker Compose mediante dos ficheros de composición: `docker-compose.yml` (base, empleado en desarrollo) y `docker-compose.prod.yml` (*override* para producción). Los seis servicios y su configuración se recogen en la Tabla 32.

Servicio	Imagen	Descripción
svaes-frontend	node:21-alpine (build) + nginx:alpine	SPA Angular servida por Nginx. Puerto host 8080 → contenedor 80 (HTTP, desarrollo). En producción, un proxy inverso externo con terminación TLS garantiza el cumplimiento de RNF15 (HTTPS).
svaes-backend	python:3.11-slim	API REST FastAPI con Uvicorn. Puerto interno 8000.
svaes-engine	rust:1.77-slim (build) + debian:bookworm-slim	Motor de verificación Actix-web. Puerto interno 8081.
svaes-worker	misma imagen que backend	Worker Celery. Sin puertos expuestos.
postgres	postgres:17-alpine	Base de datos. Puerto interno 5432.
redis	redis:8-alpine	<i>Broker</i> Celery y backend de contadores. Puerto interno 6379.

Tabla 32: Servicios Docker Compose del [SVAES](#).

Todos los servicios comparten la red *bridge* interna `svaes-internal`. Únicamente `svaes-frontend` expone un puerto al *host* en todas las configuraciones; el puerto de PostgreSQL (5432) se publica al host únicamente en el entorno de desarrollo para facilitar las

tareas de depuración. Las variables de entorno sensibles (`DATABASE_URL`, `JWT_SECRET_KEY`, `ENCRYPTION_KEY`) se gestionan mediante un fichero `.env` excluido del control de versiones. En el entorno de producción, los servicios `svaes-backend` y `svaes-worker` se despliegan en un servidor Oracle Cloud Infrastructure Ampere A1 (arquitectura AArch64, 4 vCPU, 24 GB de RAM). Las imágenes Docker se construyen con soporte *multi-platform* (`linux/arm64`) mediante `docker buildx`.

6.9.2. Análisis Estático y Calidad Continua del Código

El repositorio `adrianmfuentes/SVAES` en GitHub mantiene activos tres mecanismos de análisis automático:

- **SonarCloud**: análisis de calidad y cobertura en cada *push*. El umbral de cobertura global es del 80 %. La cobertura del módulo del motor Rust se mide con `llvm-cov` y se exporta en formato LCOV. La cobertura del backend Python se mide con `pytest-cov` y se exporta en formato `coverage.xml`.
- **CodeQL**: análisis de seguridad estático en Python y TypeScript configurado con las consultas `security-extended`. Los resultados se integran en la pestaña de seguridad del repositorio.
- **Dependabot**: monitorización de vulnerabilidades conocidas en las dependencias de `requirements.txt`, `Cargo.toml` y `package.json`. Las actualizaciones se proponen automáticamente como *pull requests*.

La automatización de mensajes de confirmación (*commit*) se implementa mediante un *hook* Git `prepare-commit-msg` que invoca el Claude Code CLI para generar el mensaje en formato Conventional Commits a partir del resumen del `diff` en curso.

6.10. Problemas Encontrados y Soluciones Adoptadas

Durante el desarrollo del sistema se presentaron los siguientes problemas técnicos de interés:

P1 — Aislamiento multi-tenant en SQLAlchemy con sesiones asíncronas. Al migrar de sesiones síncronas a `AsyncSession`, el evento de escucha `do_orm_execute` de SQLAlchemy, utilizado para inyectar el filtro `organization_id` en todas las consultas, no se propagaba correctamente en las consultas anidadas emitidas por las relaciones *lazy*. La solución fue sustituir las relaciones *lazy* por carga *selectin* en todos los modelos ORM que participan en consultas multi-tenant, eliminando así las consultas anidadas y haciendo explícita la estrategia de carga en el código.

P2 — Compatibilidad AArch64 del motor Rust. La imagen Docker oficial de Rust 1.77 no incluye la cadena de compilación para `linux/arm64` en su variante `slim`. La compilación cruzada en el entorno de CI (AMD64) hacia el servidor de producción (Ampere A1, AArch64) fallaba por la ausencia del *linker* `aarch64-linux-gnu-gcc`. La solución fue utilizar `cross-rs` como herramienta de compilación cruzada en el *step* de GitHub Actions, que gestiona automáticamente los contenedores de compilación con las cadenas de herramientas necesarias.

P3 — Paginación con *cursors* en la API GraphQL de Linear. La API GraphQL de Linear implementa paginación basada en *cursors* opacos en lugar de *offsets*. Las primeras implementaciones del conector recuperaban únicamente la primera página (100 elementos). Para colecciones de tareas grandes, esto provocaba que la regla RV-02 (estado final de artefactos) evaluara solo un subconjunto de las tareas reales. La solución fue implementar un iterador de paginación en `graphql_utils.py` que itera hasta recibir `pageInfo.hasNextPage = false`, acumulando todos los nodos en una lista antes de retornar el resultado al *worker*.

P4 — Renderizado de SVG con etiquetas HTML en $\text{\LaTeX}$. Los diagramas exportados desde draw.io en formato SVG con la opción de etiquetas HTML activada incluían bloques `<foreignObject>` con HTML interno que `pdflatex` no puede procesar mediante el paquete `svg`. La solución fue exportar todos los diagramas con la opción `html=0` en draw.io, generando SVG puro sin `<foreignObject>`, compatible con la compilación pdfLaTeX.

P5 — Conflicto de versiones entre `httpx` y `respx` en las pruebas del conector. La actualización de `httpx` a la versión 0.27 rompió la interceptación HTTP de `respx` 0.21 en las pruebas unitarias de los conectores, ya que `respx` dependía de APIs internas de `httpx` que cambiaron en esa versión. La solución fue fijar la versión de `httpx` en `requirements.txt` mediante un *pin* de rango (`httpx>=0.28.1`) hasta que `respx` publicó la versión 0.22 con soporte para la nueva API.

6.11. Descripción de la Estructura del Código Fuente

La descripción detallada de la estructura de clases, módulos y sus responsabilidades se recoge en el Apéndice 12.2, que incluye el árbol completo de directorios del repositorio con una descripción por módulo.

A modo de resumen, el sistema consta de los siguientes elementos de código cuantificables:

- **Backend Python:** 16 entidades de dominio, 23 casos de uso, 35 puertos (14 de entrada y 21 de salida), 16 *routers* FastAPI, 17 modelos SQLAlchemy, 14 repositorios

y 20 implementaciones de conector.

- **Motor Rust:** 1 servidor Actix-web, 10 funciones `evaluate` (una por regla RV-01 a RV-10, en los módulos `engine/src/rules/rvNN.rs`), reglas de calidad SonarQube adicionales y 1 componente `Aggregator`.
- **Frontend Angular 21:** siete áreas funcionales bajo `features/*` (`auth`, `dashboard`, `releases`, `connectors`, `profiles`, `admin`) más `shared`; 30 componentes *standalone*, 5 servicios (`AuthService`, `ToastService`, `TranslationService`, `DashboardService`, `FeedbackService`), 3 guardas (`authGuard`, `roleGuard`, `businessRouteGuard`) y 2 ficheros de traducción (es/en, ~1 075 claves cada uno).

6.12. Implementación de la Suite de Pruebas

Esta sección documenta la implementación de la suite de pruebas automatizadas del SVAES, constituyendo la contraparte ejecutada del plan de pruebas diseñado en el capítulo 7. Se describe la organización de los ficheros de test, la configuración de cobertura aplicada en cada componente y los resultados cuantitativos obtenidos.

6.12.1. Estructura de la Suite

El directorio `tests/` del repositorio organiza los ficheros de prueba por componente y por técnica de diseño aplicada. La Tabla 33 resume los ficheros de test del backend Python.

Grupo lógico	Criterio	Pruebas	Cobertura
Endpoints REST (Base Choice)	Base Choice sobre <i>end-points</i> REST	8	32
Conectores (CE+VL)	CE+VL sobre conectores	6	24
Ciclo de vida de <i>release</i> (Transiciones)	Transición de estados (Docker Compose)	8	18
Resiliencia de flujo (Docker Compose)	Resiliencia de flujo (Docker Compose)	8	14
Seguridad (OWASP)	Vectores OWASP (TC-SEC)	5	15

<i>Aggregator</i> (Branch Coverage)	Branch Coverage (<i>Aggregator</i>)	9	9
Casos de uso (Branch Coverage)	Branch Coverage (casos de uso)	6	6
Conectores (Branch Coverage)	Branch Coverage (<i>Connector</i>)	9	9
Fixtures compartidos	Fixtures compartidos y configuración	—	—
Autenticación	Autenticación JWT y 2FA	4	20
Perfiles de verificación	CRUD de perfiles de verificación	5	15
Organizaciones	Gestión multi-tenant	5	12

Tabla 33: Agrupación lógica de pruebas del backend Python por técnica de diseño.

La Tabla 33 presenta una agrupación lógica por técnica de diseño de pruebas (*Base Choice*, *CE+VL*, *Docker Compose*, *OWASP* y *Branch Coverage*) que ilustra la distribución de 73 casos representativos entre los niveles unitario, de integración y de seguridad definidos en el plan. La estructura física real de la suite del backend, que totaliza 1 216 tests, se organiza en 26 archivos distribuidos en cinco directorios bajo `tests/` (`unit/`, `integration/`, `security/`, `performance/` y `acceptance/`), cuya relación detallada se recoge en la sección 7.2 del capítulo 7. Ambas vistas describen el mismo esfuerzo de prueba desde perspectivas complementarias: la primera agrupa lógicamente los casos por la técnica de diseño que ejercitan, y la segunda refleja la organización física de los ficheros en el repositorio por el estrato arquitectónico que validan.

Los casos de *Base Choice* en `test_releases_endpoint.py` (TC-UNI-API-00 a TC-UNI-API-07) ejercitan el *router* de releases combinando un caso base (petición válida con todos los campos correctos) con siete variantes en las que se modifica un único parámetro cada vez. Los casos de *CE+VL* en `test_connectors_optimized.py` (TC-UNI-CON-01 a TC-UNI-CON-06) combinan clases de equivalencia y valores límite sobre los parámetros de configuración del conector, cubriendo tanto el puerto *Connector* como

el `ConnectorRegistry`.

La integración con Docker Compose se materializa en dos ficheros: `test_release_lifecycle.py` y `test_flow_resilience.py`. Entre ambos reúnen 16 casos de prueba que levantan los contenedores `svaes-backend`, `svaes-engine`, `svaes-worker`, `postgres` y `redis`, ejercitando el ciclo de vida completo de una release y las condiciones de fallo de la infraestructura (*engine* inaccesible, *broker* ausente, *timeout* de red).

Los 5 vectores OWASP de `test_security_suite.py` (TC-SEC-AUT-01, TC-SEC-AUT-02, TC-SEC-INY-01, TC-SEC-INY-02 y TC-SEC-CIF-01) verifican inyección SQL, exposición de credenciales en respuestas, caducidad de token `JWT`, omisión de cabeceras de seguridad y protección contra enumeración de usuarios.

Los casos de *Branch Coverage*, identificados en la Tabla 33 bajo los grupos lógicos *Aggregator* (Branch Coverage), Casos de uso (Branch Coverage) y Conectores (Branch Coverage) como parte de la agrupación por técnica, contienen los métodos adicionales derivados iterativamente a partir del informe `pytest-cov --show-missing`, con el fin de alcanzar el umbral de cobertura exigido por RNF-27 y superar el *Quality Gate* de SonarCloud. La justificación de estos casos se detalla en la sección 7.2 del capítulo 7, donde se describe la técnica de *Branch Coverage* y su aplicación iterativa a partir del informe de cobertura para alcanzar el umbral de RNF-27.

En cuanto al motor Rust, la suite se organiza de la siguiente manera:

- **Tests unitarios del Aggregator:** 8 tests en `engine/src/aggregator.rs` que reproducen los casos TC-UNI-AGG-01 a TC-UNI-AGG-08 del plan de pruebas, aplicando clases de equivalencia y valores límite sobre las combinaciones de veredictos de regla.
- **Tests inline por regla:** cada módulo bajo `engine/src/rules/` incluye un bloque `#[cfg(test)] mod tests` con tests inline por módulo de regla (RV-01 a RV-10) verificando el comportamiento de cada regla ante entradas válidas, inválidas y casos límite. Los módulos de reglas de calidad SonarQube siguen el mismo patrón.
- **Benchmarks:** 3 benchmarks en `engine/benches/verification_benchmark.rs` que miden el rendimiento del *pipeline* de verificación completo bajo cargas de 10, 50 y 100 artefactos concurrentes, verificando el cumplimiento de RNF-05 (tiempo máximo de 500 ms para respuesta de [API](#)).

En total, el motor Rust suma 103 tests: 8 del Aggregator, 89 de las reglas RV-01 a RV-10 y 6 de las reglas de calidad SonarQube.

6.12.2. Configuración de Cobertura

La medición de cobertura se configura de forma independiente para cada componente del sistema. En el backend Python, el fichero `pyproject.toml` incluye el siguiente bloque de configuración:

```
1 [tool.coverage.run]
2 source = ["api/src"]
3 omit = [
4     "*/migrations/*",
5     "*/main.py",
6     "*/config.py",
7     "*/__init__.py",
8 ]
9
10 [tool.coverage.report]
11 exclude_lines = [
12     "pragma: no cover",
13     "if TYPE_CHECKING:",
14     "raise NotImplementedError",
15 ]
```

Listing 6: Configuración de cobertura en `pyproject.toml` del backend Python.

Las exclusiones son legítimas y se justifican de la siguiente manera: los ficheros de migración de Alembic no contienen lógica de negocio; `main.py` es el punto de entrada de la aplicación y consta únicamente de la instanciación del servidor; `config.py` contiene carga de variables de entorno sin flujo de control; los `__init__.py` vacíos no aportan código ejecutable. Las líneas marcadas con `pragma: no cover`, los bloques `TYPE_CHECKING` y los métodos abstractos se excluyen del informe por no ser ejecutables en tiempo de prueba.

El fichero `sonar-project.properties` complementa la configuración anterior declarando las mismas exclusiones para el análisis de SonarCloud:

```
1 sonar.coverage.exclusions=\
2   api/src/**/migrations/**,\
3   api/src/**/main.py,\
4   api/src/**/config.py,\
5   api/src/**/__init__.py
```

Listing 7: Exclusiones de cobertura en `sonar-project.properties`.

Para el motor Rust, la cobertura se obtiene mediante `cargo llvm-cov` con el siguiente comando, que excluye los ficheros de test y los módulos generados automáticamente:

```
1 cargo llvm-cov --all-features --workspace --lcov \
2   --output-path coverage/rust.lcov \
3   --ignore-filename-regex "tests/benches/target/"
```

Listing 8: Comando de cobertura del motor Rust con `cargo llvm-cov`.

El *flag* `--ignore-filename-regex` excluye del informe los directorios `tests/` (tests de integración), `benches/` (benchmarks) y `target/` (código generado por la compilación), de modo que el porcentaje refleja exclusivamente la cobertura del código de producción.

6.12.3. Resultados de Cobertura

El backend Python alcanza una cobertura de líneas del $\geq 80\%$, cumpliendo el umbral mínimo del 80 % fijado por RNF-27 y validado por el *Quality Gate* de SonarCloud. La cobertura de ramas (*Branch Coverage*) se sitúa en el 58 %, resultado de la aplicación iterativa de la técnica descrita en la sección 7.2 del capítulo 7. La diferencia entre la cobertura de líneas y la de ramas es esperable en una arquitectura de capas con numerosas guardas de validación y caminos de error, cuyo ejercicio completo requiere un volumen de casos muy superior al mínimo necesario para RNF-27.

La cobertura del motor Rust se ha integrado en el pipeline de CI mediante el *script* `run_rust_coverage.ps1`, que invoca `cargo llvm-cov` con los parámetros indicados en el fragmento 8. El informe LCov resultante se fusiona con el del backend Python en SonarCloud para obtener la cobertura global del sistema, superando el umbral del 80 % exigido por el *Quality Gate*.

6.12.4. Referencias Cruzadas con el Plan de Pruebas

La implementación de la suite de pruebas se corresponde directamente con el plan diseñado en el capítulo 7:

- Las técnicas de diseño (*Base Choice*, *CE+VL*, transición de estados y *Branch Coverage*) aplicadas en los ficheros de test del backend Python siguen la metodología definida en la sección 7.1 del capítulo 7.
- Los casos de prueba de *Branch Coverage* (TC-UNI-AGG-01 a TC-UNI-AGG-08, TC-UNI-CON-01 a TC-UNI-CON-06 y los tests de casos de uso) se implementan en los tres ficheros `test_*_branch_coverage.py` del backend, con métodos adicionales derivados del informe de cobertura tal como prescribe el plan de pruebas unitarias (sección 7.2 del capítulo 7).
- Los tests del motor Rust (`aggregator.rs` e inline por regla) reproducen fielmente los conjuntos de casos definidos en las secciones 7.2.1 y 7.2.2 del capítulo 7.
- Los 5 vectores OWASP de seguridad (TC-SEC-AUT-01, TC-SEC-AUT-02, TC-SEC-INY-01, TC-SEC-INY-02 y TC-SEC-CIF-01) y las pruebas de rendimiento con Locust se corresponden con las secciones 7.6 y 7.5 del capítulo 7, respectivamente.

6.13. Resumen del Capítulo

Este capítulo ha documentado la implementación completa del sistema SVAES, trazando cada componente hasta las decisiones de diseño del capítulo 5 y los requisitos del capítulo 4. Los aspectos más relevantes de la implementación son:

- La **Clean Architecture** se ha aplicado de forma estricta y verificable: ningún módulo del dominio importa código de infraestructura.
- El **motor de verificación en Rust** evalúa las diez reglas en paralelo mediante Rayon y retorna el veredicto en menos de 500 ms bajo carga normal (RNF-05).
- Los **veinte conectores** implementan un único puerto **Connector** y cubren cinco tipos funcionales, haciendo el sistema genérico e independiente de cualquier herramienta externa concreta.
- La **seguridad** se ha integrado en todas las capas: **JWT** con 2FA TOTP, cifrado Fernet de credenciales, protección contra fuerza bruta, *rate limiting* por Redis y cabeceras de seguridad **HTTP** en todas las respuestas.
- El **frontend Angular 21** ofrece una SPA completamente funcional con internacionalización ES/EN, **RBAC** en cliente y generación automática del cliente REST desde el contrato OpenAPI.
- La **calidad del código** se mantiene de forma continua mediante SonarCloud (cobertura $\geq 80\%$), CodeQL y Dependabot.

El capítulo siguiente describe el plan de pruebas del sistema y los resultados obtenidos en cada nivel de verificación.

7 Plan y ejecución de pruebas del sistema

El plan de pruebas del [SVAES](#) persigue dos objetivos complementarios. El primero, comprobar que el sistema satisface los requisitos especificados en el capítulo 4 y, en particular, que las reglas del motor de verificación se comportan según lo definido en la sección 5.4.3. El segundo, evidenciar que los atributos de calidad establecidos en los requisitos no funcionales (sección 4.10) se cumplen dentro de los umbrales medibles fijados, especialmente en lo relativo a tiempo de respuesta (RNF-01, RNF-05), cobertura mínima de pruebas (RNF-27), seguridad de credenciales (RNF-13, RNF-14) y experiencia de usuario (RNF-19 a RNF-30).

La estrategia adoptada se apoya en el estándar *ISO/IEC/IEEE 29119* [4], concretamente en su parte 4 dedicada a las técnicas de diseño de pruebas. Cada caso de prueba se deriva de manera trazable a partir de condiciones de prueba identificadas en el [SRS](#) y de las situaciones a cubrir resultantes de aplicar técnicas sistemáticas de diseño.

A lo largo del capítulo se distinguen dos planos. El plano de **diseño**, recogido en las secciones 7.1 a 7.6, describe las técnicas aplicadas, las condiciones de prueba seleccionadas y los casos representativos que estructuran cada suite. El plano de **ejecución**, sintetizado en la sección 7.8, recoge los resultados obtenidos al ejecutar la suite completa sobre la implementación real del sistema, incluyendo recuentos exactos por nivel, cobertura medida y estado del *Quality Gate* de SonarCloud. El último apartado (sección 7.9) cierra el ciclo enlazando cada requisito del [SRS](#) con los casos que lo validan.

La implementación final comprende **91 archivos de prueba** y aproximadamente **2083 funciones de test** repartidas entre el *backend* Python (`pytest`, 1 216 tests en 26 archivos organizados en cinco directorios: `unit/`, `integration/`, `security/`, `performance/` y `acceptance/`), el motor Rust (`cargo test`, 103 tests en 21 archivos con módulos `#[cfg(test)]` inline — cobertura completa de camino feliz y ramificaciones negativas para las reglas RV-01 a RV-10 y las reglas de calidad SonarQube), el *frontend* Angular (`Vitest` 4.x, 721 tests en 43 archivos `spec.ts`), y el *frontend* Angular Cypress E2E (43 tests en 1 archivo). Esta cifra supera ampliamente el catálogo de casos de diseño documentados en las secciones previas porque cada caso del plan se materializa habitualmente en varias funciones de `pytest` o de `Vitest` que cubren el camino positivo y las distintas ramificaciones negativas necesarias para alcanzar el umbral de cobertura estructural.

7.1. Técnicas de diseño de pruebas

El diseño de pruebas atraviesa tres etapas claramente diferenciadas. En primer lugar, se identifican las *condiciones de prueba* (*test conditions*), entendidas como aquello sobre lo

que se quiere probar el sistema: una entrada, una salida, un estado, un evento o cualquier elemento observable derivado de la especificación. A continuación, se aplican técnicas para obtener las *situaciones a cubrir* (*test coverage items*), que son las particiones, valores límite o transiciones que el conjunto de casos de prueba debe ejercitar. Finalmente se decide una estrategia de combinación y se derivan los casos de prueba concretos.

Las técnicas seleccionadas se ajustan a la naturaleza de cada componente del [SVAES](#). El motor de verificación tiene una lógica algorítmica con salidas discretas y bien delimitadas, por lo que admite particionado mediante clases de equivalencia y valores límite. El catálogo de reglas RV-01 a RV-10 contiene múltiples valores categóricos cuya combinación exhaustiva resulta inviable; la estrategia minimizada *Each Choice* ofrece una cobertura suficiente con un coste muy moderado. Los *endpoints* del *backend* presentan parámetros relativamente independientes en los que interesa probar la sensibilidad ante pequeños cambios, por lo que se aplica *Base Choice*. Las entidades persistentes tienen un ciclo de vida claro descrito mediante diagramas de estado, lo que motiva el uso de pruebas de transición de estados.

7.1.1. Clases de equivalencia y valores límite

Esta técnica particiona el dominio de cada entrada en clases válidas e inválidas tales que cualquier valor dentro de una clase produce el mismo comportamiento del sistema. A continuación se identifican los valores límite exactos en las fronteras entre clases, donde tradicionalmente se concentra una proporción elevada de defectos. En el [SVAES](#) esta técnica se aplica de forma intensiva al *aggregator* del motor, donde las fronteras entre veredictos (VALID, INVALID, VALID_WITH_WARNINGS y los sufijos _WITH_INCIDENTS) son discretas y exactas, y a las implementaciones de *IConnector*, en las que el *timeout* configurado define una frontera numérica precisa.

7.1.2. Each Choice

Estrategia de combinación minimizada: cada valor de cada clasificación debe quedar ejercitado al menos una vez por algún caso, sin pretender combinarlo con todos los valores de las demás clasificaciones. El número de casos necesario equivale al máximo del número de valores entre todas las clasificaciones. En este [Trabajo Fin de Grado \(TFG\)](#) se emplea para el catálogo de reglas RV-01–RV-10, en el que una combinación exhaustiva generaría 160 casos mientras que *Each Choice* permite cubrir cada valor con un número de tests sensiblemente menor.

7.1.3. Base Choice

Fija una combinación base —la más representativa del uso real— y genera el resto de los casos variando exactamente un parámetro respecto a la base. Como ilustración, la

tabla 34 muestra los ocho casos derivados para `POST /releases` con la combinación base `{OPERATOR, token_válido, propia, completo}`:¹

Caso	Rol	Autenticación	org_context	body	HTTP
TC-BC-00 (base)	OPERATOR	token_válido	propia	completo	201
TC-BC-01	ADMIN	token_válido	propia	completo	201
TC-BC-02	U1	sin_token	propia	completo	401
TC-BC-03	OPERATOR	caducado	propia	completo	401
TC-BC-04	OPERATOR	sin_token	propia	completo	401
TC-BC-05	OPERATOR	token_válido	ajena	completo	404
TC-BC-06	OPERATOR	token_válido	propia	campo_falta	422
TC-BC-07	OPERATOR	token_válido	propia	tipo_incorr.	422

Tabla 34: Derivación *Base Choice* para `POST /releases`.

7.1.4. Transición de estados

Cuando el comportamiento de una entidad depende del estado en que se encuentra, las pruebas de transición de estados resultan más expresivas que las técnicas anteriores. Las situaciones a cubrir son las transiciones del diagrama, y los casos se construyen como secuencias de eventos que las recorren. Se diseñan también pruebas negativas que intentan provocar transiciones no declaradas en el modelo, comprobando que el sistema las rechaza con el código de error apropiado y sin alterar el estado persistido.

Resumen de técnicas. **CE+VL:** clases de equivalencia y valores límite. **EC:** *Each Choice* (estrategia minimizada). **BC:** *Base Choice* (variación de un parámetro). **TE:** transición de estados. La técnica se refleja en el identificador del caso (p. ej. TC-UNI-AGG-01).

7.2. Pruebas unitarias

Las pruebas unitarias verifican el comportamiento aislado de las unidades de código del sistema. La implementación final agrupa las suites unitarias en tres grandes bloques que se corresponden con los tres componentes desplegables del **SVAES**: el motor Rust, el *backend* Python (FastAPI + SQLAlchemy) y el *frontend* Angular. El objetivo de cobertura mínima establecido por el requisito RNF-27 es del 80% de líneas; en la práctica, el *Quality Gate*

¹Notación de roles en las pruebas: **OPERATOR** = U2 (usuario estándar), **ADMIN** = U3 (administrador global), **U1** = invitado (sin autenticar). Esta correspondencia se define en el glosario del Apéndice 12.10.

configurado en SonarCloud se ha fijado en el 80% para los tres formatos de cobertura agregados (`llvm-cov` para Rust, `coverage.xml` para Python y `lcov.info` para Angular), con lo que el umbral aplicado supera de forma estricta lo exigido por el [SRS](#).

Cada subsección que sigue presenta primero las condiciones de prueba y la técnica seleccionada, describe a continuación los casos representativos derivados del diseño y finaliza, cuando procede, con una nota sobre la materialización efectiva en la base de código. Por motivos de extensión, los listados nominales no enumeran las cerca de mil funciones `pytest` ni los más de trescientos tests de `Vitest`; en su lugar se documentan los casos sistemáticos derivados con técnica formal y se resumen agregadamente los grupos restantes, que actúan como complemento estructural orientado a cubrir las ramas necesarias para alcanzar el umbral de SonarCloud.

7.2.1. Agregator del motor de verificación

El *aggregator* recibe una lista de resultados parciales emitidos por las reglas RV-01–RV-10 y produce un veredicto global. Sus entradas se modelan en dos condiciones: el estado de cada regla (`{OK, ERROR, WARNING, NOT_EVALUATED}`) y la severidad (`{OBLIGATORIA, OPCIONAL, EXCLUIDA}`). Dos transiciones críticas motivan los valores límite: el paso de 0 a 1 reglas OBLIGATORIAS con ERROR (frontera VALID/INVALID) y la presencia de 0 a 1 reglas NOT_EVALUATED (sufijo `_WITH_INCIDENTS`).

Condiciones de prueba: C_1 : `estado_regla` $\in$ `{OK, ERROR, WARNING, NOT_EVALUATED}`. C_2 : `severidad` $\in$ `{OBLIGATORIA, OPCIONAL, EXCLUIDA}`. Técnica: **CE+VL**.

Implementación. El módulo `engine/src/aggregator.rs` contiene ocho funciones de prueba inline mediante el atributo `#[cfg(test)]`, una por cada caso del diseño:

- TC-UNI-AGG-01 (CE+VL). Todas las reglas obligatorias y opcionales devuelven OK y ninguna regla queda sin evaluar. Veredicto esperado: **Valida**.
- TC-UNI-AGG-02 (CE). Al menos una regla obligatoria devuelve ERROR, con mezcla de resultados en las opcionales. Veredicto esperado: **NoValida**.
- TC-UNI-AGG-03 (CE). Todas las reglas obligatorias devuelven OK, al menos una opcional devuelve WARNING y ninguna regla queda sin evaluar. Veredicto esperado: **ConAdvertencias**.
- TC-UNI-AGG-04 (VL). Transición $0 \rightarrow 1$ reglas obligatorias con ERROR: se verifica que el paso desde un escenario completamente válido a uno con un único ERROR obligatorio bascula efectivamente el veredicto global hacia **NoValida**.
- TC-UNI-AGG-05 (CE). Las reglas marcadas con severidad EXCLUIDA son descartadas en la fase de resolución del perfil antes de llegar al motor, por lo que no aparecen en la lista de resultados ni afectan al veredicto global. Este test valida que el *aggregator* no recibe ni procesa reglas excluidas. (PV1.3).
- TC-UNI-AGG-06 (VL). El ERROR en una regla obligatoria prevalece sobre cualquier

número de **WARNING** en reglas opcionales. Frontera de precedencia entre **NoValida** y **ConAdvertencias**.

- **TC-UNI-AGG-07 (CE)**. Múltiples errores obligatorios simultáneos: el veredicto sigue siendo **NoValida** sin degeneración hacia otros estados.
- **TC-UNI-AGG-08 (CE)**. Una regla obligatoria devuelve **ERROR** y otra opcional también devuelve **ERROR**. El veredicto global es **ConAdvertencias** (el error opcional no invalida la entrega, pero genera advertencia).

La elección de **CE+VL** sobre *Each Choice* se justifica porque el *aggregator* es el punto en el que las clases de equivalencia se acumulan: cualquier defecto en su lógica compromete la totalidad del sistema. La estrategia minimizada sacrificaría la capacidad de detectar fronteras concretas, por lo que se ha preferido la combinación de particionado equivalente con pruebas explícitas en cada frontera.

7.2.2. Catálogo de reglas RV-01 a RV-10

El motor implementa diez reglas que consultan conectores y emiten un veredicto parcial. Se modela un árbol de clasificación (C_1 : estado conector; C_2 : resultado artefacto) y se aplica *Each Choice*: la combinación exhaustiva daría $2 \times 4 \times 2 = 16$ casos por regla (160 en total); la estrategia minimizada cubre cada valor con un único caso por regla y eje. La materialización final ha ampliado la cobertura para incluir ramificaciones negativas en cada regla: el camino feliz (**OK**) se complementa con casos **ERROR**, **WARNING** y **NOT_EVALUATED** derivados de estados inválidos, campos ausentes, valores nulos y condiciones de frontera. **RV-09** concentra nueve tests por la riqueza de sus ramificaciones; el resto de reglas oscilan entre tres y cinco tests cubriendo los escenarios de error más habituales. En total, las 103 funciones de test del motor (89 de reglas RV-01 a RV-10, 6 de reglas de calidad SonarQube y 8 del *aggregator*) sobrepasan el umbral de cobertura estructural del 80%.

Condiciones de prueba: C_1 : estado_conector $\in \{\text{ACTIVO}, \text{INACTIVO}\}$. C_2 : resultado $\in \{\text{OK}, \text{WARNING}, \text{ERROR}, \text{NOT_FOUND}\}$. Técnica: **Each Choice**.

Implementación. Cada regla reside en un fichero independiente del módulo `engine/src/rules/`, con módulos `#[cfg(test)]` inline. La distribución real de tests es la siguiente:

Regla	Fichero	Tests	Cobertura del test
RV-01	rules/rv01.rs	2	Lista de artefactos no vacía / vacía
RV-02	rules/rv02.rs	3	Estado final OK; sin task_id (diseño); sin artefactos CODIGO
RV-03	rules/rv03.rs	3	Cobertura completa; estado inválido; campo ausente; sin TAREAs

Regla	Fichero	Tests	Cobertura del test
RV-04	rules/rv04.rs	4	Float válido; valor negativo; campo faltante; null
RV-05	rules/rv05.rs	3	Documentos OK; sin documentos; documento inaccesible; campo ausente
RV-06	rules/rv06.rs	3	Versión OK; versión distinta; atributo ausente; sin DOCUMENTOs
RV-07	rules/rv07.rs	3	Back-reference OK; sin <code>artifact_type</code> ; artefacto no encontrado; campo falso
RV-08	rules/rv08.rs	4	Umbral OK; sin <code>master_artifact_id</code> ; maestro no encontrado; tarea declarada faltante; tarea extra no declarada
RV-09	rules/rv09.rs	9	Identificador externo duplicado en el mismo conector
RV-10	rules/rv10.rs	5	Estado VALIDADO; sin documentos; estado incorrecto; campo ausente; estados personalizados
Total motor	21 archivos	103	8 aggregator + 89 reglas RV + 6 SonarQube

Tabla 35: Distribución de tests unitarios del motor Rust por regla.

A continuación se enumeran, a modo de catálogo nominal, los casos de diseño sistemáticos derivados por *Each Choice*:

- TC-UNI-MOT-01 (EC · RV-01). Conector activo y artefacto con resultado OK. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-02 (EC · RV-01). Conector inactivo. Se espera `RuleResult{status: NOT_EVALUATED}`.
- TC-UNI-MOT-03 (EC · RV-02). Todos los artefactos en estado final permitido (p. ej. DONE). Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-04 (EC · RV-02). Un artefacto en estado intermedio (IN_PROGRESS). Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-05 (EC · RV-02). Artefacto CODIGO sin `task_id` (comportamiento por diseño). Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-06 (EC · RV-02). Ausencia total de artefactos tipo CODIGO. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-07 (EC · RV-03). Cobertura documental completa: todos los tipos documentales obligatorios tienen al menos un documento. Se espera

`RuleResult{status: OK}`.

- TC-UNI-MOT-08 (CE · RV-03). Estado de documento inválido (INACTIVE). Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-09 (CE · RV-03). Campo documental ausente. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-10 (CE · RV-03). Sin tareas en GESTOR_TAREAS. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-11 (EC · RV-04). Versión coherente (float) en todos los artefactos. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-12 (CE · RV-04). Valor de versión negativo. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-13 (CE · RV-04). Campo de versión faltante en un artefacto. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-14 (CE · RV-04). Valor de versión null. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-15 (EC · RV-05). Todos los campos obligatorios informados y no vacíos. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-16 (CE · RV-05). Sin documentos en la release. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-17 (CE · RV-05). Documento inaccesible (conector devuelve error). Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-18 (CE · RV-05). Campo obligatorio ausente en el artefacto. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-19 (EC · RV-06). Todas las tareas de GESTOR_TAREAS presentes y en estado COMPLETE. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-20 (CE · RV-06). Versión de atributo distinta entre artefactos. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-21 (CE · RV-06). Atributo de versión ausente en un artefacto. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-22 (CE · RV-06). Sin DOCUMENTOs en la release. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-23 (EC · RV-07). La *back-reference* del artefacto apunta correctamente a la *release* en el sistema externo. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-24 (CE · RV-07). Sin parámetro `artifact_type`. Se espera `RuleResult{status: NOT_EVALUATED}`.
- TC-UNI-MOT-25 (CE · RV-07). Artefacto referenciado no encontrado. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-26 (CE · RV-07). Campo de back-reference con valor falso. Se espera `RuleResult{status: ERROR}`.

- TC-UNI-MOT-27 (EC · RV-08). Antigüedad dentro del umbral. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-28 (CE · RV-08). Sin `master_artifact_id`. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-29 (CE · RV-08). Artefacto maestro no encontrado. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-30 (CE · RV-08). Tarea declarada faltante en el artefacto. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-31 (CE · RV-08). Tarea extra no declarada en el perfil. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-32 (EC · RV-09). Dos artefactos distintos comparten el mismo `external_id` en el mismo conector. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-33 (EC · RV-10). Estado de aprobación `VALIDADO` presente. Se espera `RuleResult{status: OK}`.
- TC-UNI-MOT-34 (CE · RV-10). Sin documentos de aprobación. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-35 (CE · RV-10). Estado de aprobación incorrecto. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-36 (CE · RV-10). Campo de estado ausente. Se espera `RuleResult{status: ERROR}`.
- TC-UNI-MOT-37 (CE · RV-10). Estados personalizados reconocidos. Se espera `RuleResult{status: OK}`.

7.2.3. Endpoints del backend FastAPI

Los *endpoints* del *backend* reciben peticiones [HTTP](#) con cuatro parámetros relativamente independientes. La combinación exhaustiva produciría $4 \times 3 \times 2 \times 3 = 72$ casos por *endpoint*, cifra inviable a nivel de suite. Se aplica *Base Choice* con la combinación base `{OPERATOR, token_válido, propia, completo}`: cada caso adicional varía exactamente un parámetro respecto a la base, aislando su efecto.

Condiciones de prueba: C_1 : rol; C_2 : autenticación; C_3 : org_context; C_4 : body.
Técnica: **Base Choice** sobre `POST /releases`.

Implementación. El fichero `tests/unit/test_routers.py` materializa la clase `TestCreateReleaseEndpoint`, que aloja los ocho tests *Base Choice* descritos a continuación. Esta misma clase forma parte de un conjunto mucho más amplio: el archivo contiene 23 clases y 215 funciones de test que cubren los routers `releases`, `organizations`, `projects`, `auth`, `profiles`, `users`, `connectors`, `custom_roles`, `templates`, `api_keys`, `notifications`, `access_requests`, `system`, `audit` y `tasks`. En todos ellos se replica el patrón *Base Choice* para los códigos [HTTP](#) 200/201, 401, 403, 404, 409, 422 y 500.

- TC-UNI-API-00 (BC · base). Combinación base: rol `OPERATOR` con *token* válido, contexto de organización propia y cuerpo completo. Se espera respuesta **HTTP 201 Created**.
- TC-UNI-API-01 (BC · rol). Variación del rol a `ADMIN` manteniendo el resto de parámetros base. Se espera respuesta **HTTP 201 Created**.
- TC-UNI-API-02 (BC · rol). Variación del rol a `U1` (invitado no autenticado). Se espera respuesta **HTTP 401 Unauthorized**.
- TC-UNI-API-03 (BC · auth). Variación de la autenticación a un *token* caducado. Se espera respuesta **HTTP 401 Unauthorized**.
- TC-UNI-API-04 (BC · auth). Variación de la autenticación: petición sin *token* en la cabecera. Se espera respuesta **HTTP 401 Unauthorized**.
- TC-UNI-API-05 (BC · org). Variación del contexto de organización a una organización ajena (*multi-tenancy*). Se espera respuesta **HTTP 404 Not Found**.
- TC-UNI-API-06 (BC · body). Variación del cuerpo: falta el campo `name`. Se espera respuesta **HTTP 422 Unprocessable Entity**.
- TC-UNI-API-07 (BC · body). Variación del cuerpo: el campo `version` no respeta el formato SemVer. Se espera respuesta **HTTP 422 Unprocessable Entity**.

Los identificadores TC-UNI-API-00 a TC-UNI-API-07 anteriores corresponden respectivamente a los casos de diseño TC-BC-00 a TC-BC-07 definidos en la Tabla 34 de la sección 7.1.3. La convención UNI-API designa la implementación unitaria del *router* de la API, mientras que BC identifica el caso en la fase de diseño de pruebas (*Base Choice*).

7.2.4. Implementaciones de IConnector

Las implementaciones concretas de `IConnector` presentan tres condiciones: validez de credenciales, alcanzabilidad de la URL y latencia respecto al *timeout*. Las dos primeras se prueban mediante CE; la latencia requiere valores límite exactos en el umbral del *timeout*. El *mocking* se realiza con `respx` sin acceso real a la red, lo que garantiza la reproducibilidad de los escenarios.

Condiciones de prueba: C_1 : credenciales $\in \{\text{válidas, inválidas, caducadas}\}$. C_2 : alcanzabilidad $\in \{\text{OK, KO}\}$. C_3 : VL en umbral de *timeout*. Técnica: **CE + VL**.

Implementación. El fichero `tests/unit/test_connectors.py` contiene 15 clases y 123 funciones de prueba. Las clases `TestConnectorCredentials`, `TestConnectorNetwork` y `TestConnectorTimeoutBoundary` materializan los seis casos sistemáticos derivados por CE+VL. Los tests restantes cubren once de las veinte implementaciones de conectores: el par de referencia GitLab/Jira más Redmine, Jira Service Management, Trello, Linear, Wiki.js, Plane, Gitea, Confluence y ClickUp. Los nueve conectores restantes comparten infraestructura de prueba a través de los *mixins* de autenticación genéricos (`Bearer`, `Atlassian Basic`, `API Key`) validados en `TestBaseHttpConnector`; a ello se suman las

clases `TestConnectorRegistry` (8 tests sobre registro y búsqueda por tipo o implementación, con coincidencia insensible a mayúsculas) y `TestBaseHttpConnector` (11 tests sobre `fetch_artifact`, `list_artifacts`, errores de `connect` y *mixins* de autenticación Bearer, Atlassian Basic y API Key).

- TC-UNI-CON-01 (CE). Credenciales válidas y URL alcanzable. El conector responde correctamente y pasa a estado `ACTIVO`.
- TC-UNI-CON-02 (CE). *Token* de acceso caducado. El conector lanza `AuthError` y pasa a estado `INACTIVO`.
- TC-UNI-CON-03 (CE). URL inexistente o inalcanzable. El conector lanza `ConnectorNotFound` y pasa a estado `INACTIVO`.
- TC-UNI-CON-04 (VL). Latencia exactamente igual al *timeout* configurado (10s). Se espera que se lance `ConnectorTimeout`.
- TC-UNI-CON-05 (VL). Latencia igual al *timeout* menos 1 ms (dentro del umbral). Se espera respuesta correcta recibida sin error.
- TC-UNI-CON-06 (CE). Conector en estado `INACTIVO` es invocado durante una verificación. No se debe realizar ninguna llamada `HTTP` real.

7.2.5. Backend Python: capas de dominio, servicios y repositorios

Más allá de los routers y los conectores, el *backend* Python se prueba en profundidad por capas, siguiendo la separación impuesta por la arquitectura hexagonal del `SVAES`. Los tests residen en `api/tests/unit/` y se ejecutan con `pytest` 8.x sobre fixtures asíncronos definidos en `conftest.py`. El detalle por capa se resume en la tabla 36.

Capa	Fichero	Tests	Alcance
Dominio	<code>test_domain.py</code>	9	Conversión de severidad y excepciones de dominio (<code>DomainException</code> , <code>ReleaseInvalidStateError</code> , <code>EntityNotFoundError</code> , <code>DuplicateEntityError</code>).
<i>Core</i>	<code>test_core.py</code>	82	<i>Seeding</i> de admin, fábricas de repositorios y servicios, guardas de acceso, <i>rate limit</i> , hash de contraseñas, cifrador Fernet, <i>pseudonymize</i> y sesiones async.

Capa	Fichero	Tests	Alcance
Servicios	<code>test_services.py</code>	235	AuthService (incluyendo 2FA TOTP), UserService, ReleaseService, OrganizationService, NotificationService, ProfileService, eliminación de cuenta de usuario y otros nueve servicios.
Routers	<code>test_routers.py</code>	230	23 clases con <i>Base Choice</i> sobre los <i>endpoints</i> del API . Incluye tests de exportación de datos personales (GDPR Art. 20): descarga exitosa, 401 sin <i>token</i> y 404 usuario inexistente.
Repositorios	<code>test_repositories.py</code>	186	14 implementaciones SQL: Release, Notification, User, AccessRequest, VerificationResult, Project, Organization, Profile, VerificationRule, Connector, CustomRole, APIKey, Artifact y Template.
<i>Middleware</i>	<code>test_middleware.py</code>	12	JwtHandler (creación, decodificación, <i>refresh</i> , <i>blacklist</i> , TOTP <i>pending</i>) y PasswordHasher.
Conectores	<code>test_connectors.py</code>	209	Suite CE+VL más quince implementaciones concretas (GitLab, Jira SM, Redmine, Trello, Linear, Wiki.js, Plane, Gitea, Confluence, ClickUp), ConnectorRegistry y BaseHttpConnector.
Export	<code>test_export_service.py</code>	11	ExportService para PDF y CSV, casos de proyecto no encontrado y <i>result</i> sin <i>release</i> .
Casos de uso	<code>test_use_cases_core.py</code>	72	Autenticación, creación de organización, lanzamiento de verificación, historial, actualización de <i>release</i> (con validación SemVer), <i>toggle</i> de conector, creación de artefacto, creación de template y API key.

Capa	Fichero	Tests	Alcance
Dependencias	<code>test_dependencies_factories.py</code>	40	Fábricas en <code>core/dependencies.py</code> y guardas de acceso con casos 200/403/404.
Cola de tareas	<code>test_celery_task_queue.py</code>	8	<code>CeleryTaskQueue</code> : encolado, consulta de estado y revocación de tareas de verificación asíncrona.
Email	<code>test_email.py</code>	12	<code>EmailService</code> : envío de correos, manejo de errores y configuración de canales.
Artefactos	<code>tests_repositories/test_artifact_repository.py</code>	5	<code>SqlArtifactRepository</code> : creación, consulta por <i>id</i> y por <i>release</i> , y eliminación.
Notif. <i>worker</i>	<code>test_verification_worker_notify.py</code>	7	<code>_notify_user</code> : consulta de suscripción de notificación antes del envío de correo.
TOTAL	13 archivos	1 111	

Tabla 36: Pruebas unitarias del *backend* Python por capa.

La suite de notificaciones del *worker*, materializada en el archivo `api/tests/test_verification_worker_notify.py`, consta de siete pruebas que verifican el comportamiento de la función `_notify_user` respecto a la tabla de suscripciones. Los casos cubren la consulta de `notification_subscription` antes del envío de correo para los veredictos de verificación, el comportamiento por defecto (sin fila de suscripción → envío activo), la omisión del envío cuando la suscripción está desactivada y la gestión de errores tanto de usuario no encontrado como de excepción en el envío de correo.

La capa de servicios concentra la lógica de negocio del sistema y ha exigido el mayor esfuerzo de prueba (235 funciones). Se destacan cuatro suites por su criticidad:

- **TestAuthService** (38 tests). Cubre el flujo de autenticación completo, incluyendo casos negativos (usuario no encontrado, inactivo, bloqueado, contraseña incorrecta), bloqueo por intentos máximos y la rama 2FA-TOTP con sus operaciones `verify`, `setup`, `enable` y `disable`. También verifica la rotación de *refresh tokens* y el cierre de sesión con invalidación en la *blacklist*.
- **TestUpdateReleaseUseCase** (16 tests). Materializa el caso de uso de actualización de *release*, incluyendo la validación SemVer con *prerelease* y *build metadata* válidos (`1.2.3-alpha.1+001`) y entradas inválidas (no numérico, *leading zero*, falta de *patch*,

doble punto, identificador vacío).

- **TestFernetCredentialEncryptor** (9 tests). Confirma que el cifrado de credenciales de conector mediante Fernet (AES-128-CBC + HMAC-SHA256) cumple RNF-13: ciclo *roundtrip* de *strings* y *bytes*, inicialización con clave en distintos formatos y ausencia de fuga en logs.
- **TestDeleteUserAccount** (4 tests, en `test_services.py`). Cubre el flujo de eliminación de cuenta de usuario. Se verifica que cuando un usuario propietario de una organización elimina su cuenta, la propiedad se transfiere automáticamente a otro miembro de la organización si este existe; y que, si el usuario es el único miembro, la organización se elimina junto con la cuenta previa notificación al usuario. Los tests complementarios en `test_repositories.py` (2 tests) validan la eliminación de organizaciones en la capa de persistencia, incluyendo tanto la eliminación exitosa de una organización existente como el caso de entidad no encontrada que lanza `ValueError` o `EntityNotFoundError`.
- **TestSqlUserMembershipRepository** (10 tests, en `test_repositories.py`). Suite de pruebas CRUD del nuevo repositorio de membresías: `create`, `get`, `list_by_user`, `list_by_organization`, `update_role`, `delete` y `delete_all_for_user`, incluyendo los casos de entidad no encontrada.
- **TestUserServiceMultiOrg** (12 tests, en `test_services.py`). Cubre los cuatro escenarios de negocio multi-organización: invitar a un usuario a una segunda organización con rol distinto, rechazo si ya es miembro (lanza `DuplicateEntityError`), cambio de rol en organización no activa sin alterar la activa, y fallback de organización activa al eliminar la membership. Incluye también tests de `list_organization_users` con rol por-organización, `switch_active_organization` y `delete_user_account` recorriendo todas las membership.
- **TestOrganizationServiceMultiOrg** (2 tests, en `test_services.py`). Valida que `create_organization` crea la membership del *owner* junto con el registro de organización, y que `transfer_ownership` sincroniza los roles de membership de ambos *owners*.

7.2.6. Frontend Angular: guards, interceptors, servicios y componentes

El *frontend* Angular posee lógica independiente del *backend* (*guards*, *interceptors* [HTTP](#), servicios y componentes). Las pruebas se ejecutan con **Vitest 4.x** mediante el *builder* `@angular/build:unit-test`. Su configuración emplea `globals: true` y `fileParallelism: false`. Los informes de cobertura generados se exportan en formato `lcov` y `text-summary`. La implementación total abarca **43 archivos `spec.ts`** y **721 funciones de test**. La tabla [37](#) resume la distribución de estas pruebas por área funcional.

Área	Specs	Alcance
<i>Bootstrap</i>	3	Definición de rutas, configuración de <i>providers</i> , componente raíz.
Servicios <i>core</i>	1	<i>AuthService</i> : <i>login</i> , 2FA, almacenamiento de <i>token</i> , <i>logout</i> , comprobaciones de rol.
<i>Interceptors</i>	3	<i>jwt.interceptor</i> (Bearer), <i>error.interceptor</i> (401/403/404, red) y <i>timeout.interceptor</i> .
<i>Guards</i>	3	<i>auth.guard</i> , <i>role.guard</i> , <i>business-route.guard</i> .
i18n	2	<i>TranslationService</i> (persistencia de idioma, carga reactiva) y <i>translate.pipe</i> .
Componentes compartidos	3	<i>toast</i> , <i>confirm-dialog</i> y <i>lang-toggle</i> .
Autenticación	3	<i>Login</i> , activación de cuenta y <i>reset password</i> .
<i>Shell</i> y <i>layout</i>	1	Menú dependiente del rol, <i>logout</i> , datos del usuario.
<i>Dashboard</i>	4	Página, servicio HTTP, tabla de <i>releases</i> recientes y gráfica de tasa de éxito.
<i>Releases</i>	5	Listado, servicio, <i>verify service</i> , detalle, creación/edición.
Proyectos	2	Listado y formulario de creación/edición.
Conectores	1	Listado, creación, edición, eliminación y prueba de conexión.
Perfiles	2	Listado de perfiles y página de perfil de usuario (con 2FA, eliminación de cuenta, expulsión de miembros y exportación de datos personales según RGPD Art. 20).
Otras <i>features</i>	6	<i>Logs</i> , panel administrativo, <i>org settings</i> , <i>system info</i> , <i>landing</i> y formulario de <i>access request</i> .
Páginas de error	2	403 <i>Forbidden</i> y 404 <i>Not Found</i> .
TOTAL	43	

Tabla 37: Distribución de archivos *spec.ts* del *frontend* Angular.

Guards de enrutado (AuthGuard, RoleGuard, BusinessRouteGuard)

Condiciones de prueba: C_1 : `estado_token` $\in$ {válido, caducado, ausente}. C_2 : (`rol`, `ruta`). Técnica: **CE** (`token`) + **BC** (`rol` $\times$ `ruta`).

- TC-UNI-FE-GRD-01 (CE+BC $\cdot$ base). Usuario U2/OPERATOR con *token* válido intenta acceder a la ruta `/releases`. El `canActivate` devuelve `true` y la navegación continúa.
- TC-UNI-FE-GRD-02 (CE $\cdot$ token). *Token* caducado para cualquier rol. El `canActivate` devuelve `false` y se redirige a `/login`.
- TC-UNI-FE-GRD-03 (CE $\cdot$ token). Sin *token* almacenado en `localStorage`. El `canActivate` devuelve `false` y se redirige a `/login`.
- TC-UNI-FE-GRD-04 (BC $\cdot$ rol). Usuario U1 intenta acceder a `/releases/verify`. El `canActivate` devuelve `false` y se redirige a `/forbidden`.

Servicios HTTP Angular

Condiciones de prueba: C_1 : `código_respuesta` $\in$ {201, 401, 422}. Técnica: **Base Choice**. *Mocking* con `HttpClientTestingModule` + `HttpTestingController`.

- TC-UNI-FE-SVC-01 (BC $\cdot$ base). *Mock* de `POST /releases` que responde con código 201 y el objeto `Release` en **JSON**. El observable emite la `Release` y la cabecera `Authorization` está presente en la petición.
- TC-UNI-FE-SVC-02 (BC $\cdot$ 401). *Mock* de `POST /releases` que responde con código 401. El observable emite `AuthError` y el *interceptor* redirige a `/login`.
- TC-UNI-FE-SVC-03 (BC $\cdot$ 422). *Mock* de `POST /releases` que responde con código 422 y una lista de campos en `detail[]`. El observable emite `ValidationError` con la lista de campos afectados.

Flujo de verificación y polling

El componente `release-detail` dispara la verificación con `POST /verify` y realiza *polling* sobre el *endpoint* de progreso hasta que el estado terminal se alcanza. El servicio `verify.service` se prueba en `features/releases/verify.service.spec.ts` cubriendo el camino positivo (202 + *polling* hasta `COMPLETED`) y los caminos negativos relevantes:

- TC-UNI-FE-VFY-01 (CE). `POST /verify` responde con 202 y un `taskId`; el *polling* converge a estado `COMPLETED` con veredicto `VALID`. Se emite la `Verification` resultante.
- TC-UNI-FE-VFY-02 (CE). `POST /verify` responde con 409 *Conflict* indicando que la *release* está en un estado inválido para verificar. Se propaga un `ConflictError` con el mensaje del *backend* y la UI ofrece un mensaje descriptivo (RNF-20).

Gestión de perfil: eliminación de cuenta y expulsión de miembros

La página de perfil de usuario incorpora dos nuevas capacidades de gestión que exigen pruebas unitarias propias. La primera, exclusiva del tab *Mi perfil*, permite a cualquier usuario no administrador global eliminar su propia cuenta mediante un botón con confirmación explícita. La segunda, accesible desde el tab *Miembros*, faculta al administrador de una organización para expulsar a miembros que no ostenten el mismo rol administrativo.

Condiciones de prueba (eliminación de cuenta): C_1 : `pertenece_a_org` $\in \{\text{sí, no}\}$. C_2 : `es_owner` $\in \{\text{sí, no}\}$. C_3 : `org_tiene_miembros` $\in \{\text{sí, no}\}$. Técnica: **Base Choice** + CE para los errores del *backend*.

Implementación. El fichero `web/src/.../profile.component.spec.ts` se ha ampliado con 23 nuevos tests que cubren de forma exhaustiva los flujos de eliminación de cuenta y expulsión de miembros, organizados en los siguientes bloques funcionales:

- TC-UNI-FE-PRF-01 a TC-UNI-FE-PRF-06 (CE+BC · apertura de modal). Comportamiento del diálogo de confirmación de eliminación de cuenta en seis escenarios: usuario sin organización, propietario único de organización (con advertencia de eliminación de la organización), propietario con otros miembros (sin advertencia, con transferencia de propiedad), usuario no propietario, error al obtener la organización y error al obtener los miembros.
- TC-UNI-FE-PRF-07 a TC-UNI-FE-PRF-08 (CE · cierre de modal). Cierre del diálogo de confirmación: se permite cerrar si no hay una operación de borrado en curso; se impide el cierre durante el borrado activo.
- TC-UNI-FE-PRF-09 a TC-UNI-FE-PRF-15 (BC+CE · confirmación). Siete casos sobre la acción de confirmar el borrado: formulario inválido, éxito (DELETE seguido de *logout*), error 400, error 401 (contraseña incorrecta), error 403 genérico y con detalle, y error 500 del servidor.
- TC-UNI-FE-PRF-16 a TC-UNI-FE-PRF-21 (CE · *template*). Seis verificaciones sobre la renderización del *template*: la tarjeta de eliminación no se muestra para el administrador global, sí para cualquier otro rol; el modal muestra la advertencia de eliminación de organización cuando procede; el modal estándar sin advertencia; y los estados de error y éxito en el modal.

La suite emplea *mocks* de los servicios `AuthService`, `OrganizationService`, `UserService` y `MemberService` con `HttpClientTestingModule`. El flujo de expulsión de miembros sigue un patrón análogo de *Base Choice* sobre el rol del miembro (ADMIN/OPERATOR) y la respuesta del *backend* (200/403/404), verificando que un administrador de organización puede expulsar a miembros no administradores y que el sistema rechaza la expulsión de otros administradores.

Exportación de datos personales (GDPR Art. 20)

La funcionalidad de exportación de datos, incorporada para garantizar los derechos de acceso y portabilidad del RGPD, dispone de pruebas unitarias tanto en el *backend* como en el *frontend* que cubren el flujo completo de solicitud, generación y descarga del fichero JSON.

Condiciones de prueba (*backend*): C_1 : autenticación $\in \{\text{válida, ausente}\}$. C_2 : existencia_usuario $\in \{\text{existe, inexistente}\}$. Técnica: **Base Choice**.

Implementación. El módulo `TestUsersCoverage` dentro de `test_routers.py` incluye tres tests específicos para el *endpoint* `GET /api/v1/users/me/export`:

- TC-UNI-API-EXP-01 (BC · base). Usuario autenticado con *token* válido solicita la exportación. El *backend* responde con [HTTP 200](#), devuelve un fichero JSON con `schema_version`, `export_format` y todos los campos del usuario (`id`, `email`, `display_name`, `rol`, `is_active`, fechas de creación y actualización, `terms_accepted_at`, `privacy_accepted_at` y lista de `organization_ids`), y registra una entrada de auditoría de tipo `DATA_EXPORT_REQUESTED`.
- TC-UNI-API-EXP-02 (BC · autenticación). Petición sin *token* en la cabecera `Authorization`. Se espera respuesta [HTTP 401 Unauthorized](#).
- TC-UNI-API-EXP-03 (BC · existencia). Petición con *token* válido pero correspondiente a un usuario que ya no existe en base de datos. Se espera respuesta [HTTP 404 Not Found](#).

Condiciones de prueba (*frontend*): C_1 : respuesta_servidor $\in \{200 \text{ (éxito), } 500 \text{ (error)}\}$. C_2 : estado_componente $\in \{\text{descargando, error, inactivo}\}$. Técnica: **CE + BC**.

Implementación. El fichero `web/src/.../profile.component.spec.ts` incorpora cinco tests que cubren la sección «Descargar mis datos» del componente de perfil:

- TC-UNI-FE-PRF-22 (CE · descarga). El método `downloadData()` del componente invoca `GET /users/me/export`, recibe un JSON válido, crea un *Blob* con tipo MIME `application/json` y simula un clic en un elemento `<a>` con atributo `download` configurado a `svaes-user-data.json` para iniciar la descarga en el navegador.
- TC-UNI-FE-PRF-23 (CE · error). El servidor responde con error 500. El componente activa la señal `exportDataError` y no se dispara la descarga.
- TC-UNI-FE-PRF-24 (BC · renderizado). La tarjeta `.export-data-card` está presente en el [Document Object Model \(DOM\)](#) renderizado del componente de perfil, con el epígrafe «Descargar mis datos» y el botón de acción.
- TC-UNI-FE-PRF-25 (BC · estado de descarga). Con la señal `exportDataDownloading` activa, el botón muestra el texto de progreso correspondiente y queda deshabilitado para evitar envíos duplicados.

- TC-UNI-FE-PRF-26 (CE · mensaje de error). Con la señal `exportDataError` activa, el mensaje de error se renderiza dentro de la tarjeta de exportación, informando al usuario de la incidencia sin abandonar la página.

Preferencias de notificación en el perfil

La tarjeta de preferencias de notificación del componente de perfil se prueba con ocho casos que cubren la carga inicial de preferencias, la conmutación de cada una de las cuatro categorías de correo y la reversión ante fallos de red.

Condiciones de prueba: C_1 : `categoría_notificación` $\in$ `{release_validated, release_invalidated, release_pending_reminder, weekly_digest}`. C_2 : `respuesta_servidor` $\in$ `{200 (éxito), error}`. Técnica: **Base Choice** + CE.

Implementación. Ocho nuevos tests en `web/src/.../profile.component.spec.ts`:

- TC-UNI-FE-PRF-27 (BC · base). Al inicializar el componente, se realiza `GET /api/v1/notifications/preferences` y los valores retornados se asignan a los cuatro *oggles* del formulario.
- TC-UNI-FE-PRF-28 (BC · `release_validated`). El usuario desactiva el *toggle* de `release_validated`; se envía `PATCH /api/v1/notifications/preferences` con `{release_validated: false}`.
- TC-UNI-FE-PRF-29 (BC · `release_invalidated`). El usuario activa el *toggle* de `release_invalidated`; se envía `PATCH` con `{release_invalidated: true}`.
- TC-UNI-FE-PRF-30 (BC · `release_pending_reminder`). El usuario activa el *toggle* de `release_pending_reminder`; se envía `PATCH` con `{release_pending_reminder: true}`.
- TC-UNI-FE-PRF-31 (BC · `weekly_digest`). El usuario desactiva el *toggle* de `weekly_digest`; se envía `PATCH` con `{weekly_digest: false}`.
- TC-UNI-FE-PRF-32 (CE · error de *rollback*). El servidor responde con un error [HTTP](#) a la petición `PATCH`; el componente restaura el *toggle* a su valor anterior y muestra un mensaje de error.
- TC-UNI-FE-PRF-33 (CE · error de carga). La petición `GET` inicial de preferencias falla; el componente mantiene los valores por defecto y la bandera `loading` pasa a `false`.
- TC-UNI-FE-PRF-34 (BC · renderizado). La tarjeta de preferencias de notificación se renderiza en el [DOM](#) del componente de perfil con el título «Preferencias de notificación» y los cuatro *oggles* visibles.

7.3. Pruebas de integración

Las pruebas de integración verifican el comportamiento conjunto de los contenedores en el entorno Docker Compose (`svaes-backend`, `svaes-engine`, `svaes-worker`, `postgres`, `redis` sobre la red `svaes-internal`). Los servicios externos se sustituyen por *mocks* ASGI accesibles únicamente desde la red interna de Docker, de manera que la suite es reproducible y autosuficiente.

Los tests residen en `api/tests/integration/` y se ejecutan con `pytest -m integration`. La materialización final comprende tres archivos y veintisiete funciones de test, que coinciden exactamente con los casos del diseño detallados en las subsecciones siguientes.

7.3.1. Ciclo de vida de la release

El estado de una *release* atraviesa los siete estados definidos en GR6.1: BORRADOR, PENDIENTE, EN_VERIFICACION y los estados terminales VALIDA, CON_ADVERTENCIAS y NO_VALIDA, cualquiera de los cuales puede transitar a ARCHIVADA. La clase `TestReleaseLifecycle` (`test_release_lifecycle.py`) cubre las seis transiciones positivas y dos transiciones negativas que el sistema debe rechazar, alcanzando el 100% de transiciones simples del diagrama de estado.

- TC-INT-EST-01 (TE · T1). Transición desde BORRADOR a EN_VERIFICACION. Se espera respuesta `HTTP 202` con un `task_id` devuelto.
- TC-INT-EST-02 (TE · T2). Transición desde EN_VERIFICACION a VALIDA. Se espera respuesta `200` y que el estado persistido pase a VALIDA.
- TC-INT-EST-03 (TE · T3). Transición desde EN_VERIFICACION a CON_ADVERTENCIAS. Se espera respuesta `202` y que el estado pase a CON_ADVERTENCIAS.
- TC-INT-EST-04 (TE · T4). Transición desde EN_VERIFICACION a NO_VALIDA. Se espera respuesta `202` y que el estado pase a NO_VALIDA.
- TC-INT-EST-05 (TE · T5). Transición desde cualquier estado final a ARCHIVADA. La *release* pasa a ser inmutable.
- TC-INT-EST-06 (TE · T-NEG). Salto directo desde BORRADOR a ARCHIVADA. Se valida la política de archivado de *releases* sin verificar.
- TC-INT-EST-07 (TE · T-NEG). Intento de modificación sobre una *release* en estado ARCHIVADA. Se espera respuesta `HTTP 409 Conflict` (o `422` en validaciones puramente estructurales) y el estado permanece sin cambios.
- TC-INT-EST-08 (TE · T6). Restauración desde ARCHIVADA hacia un estado activo, preservando el historial inmutable anterior.

7.3.2. Flujo completo, rate limiting, resiliencia y migración

Este bloque, materializado en `test_flow_resilience.py`, cubre cuatro escenarios adicionales agrupados en cuatro clases (`TestFullFlow`, `TestExactRateLimit`, `TestResilience` y `TestMigration`).

- TC-INT-FLW-01 (VL · CU-01). Flujo completo: creación de organización, perfil, proyecto, *release* y artefacto, seguida de la verificación con las 10 reglas RV-01–RV-10 y conectores activos. Se espera veredicto `VALID` y estado final `COMPLETED`.
- TC-INT-FLW-02 (CE · CU-01). Flujo completo sin artefactos asociados (conectores inactivos): las reglas que dependen de conectores quedan `NOT_EVALUATED` y el veredicto contiene el sufijo `_WITH_INCIDENTS`.
- TC-INT-FLW-03 (CE · CU-01). Re-verificación de una *release* en estado `NO_VALIDA` tras corregir los datos. La segunda verificación debe producir estado `VALIDA`.
- TC-INT-LIM-01 (VL · *rate limit*). 100 peticiones consecutivas al *endpoint* `/health` en una ventana de 60 segundos con el mismo *token*. Se espera respuesta `HTTP` 200 sin bloqueo.
- TC-INT-LIM-02 (VL · *rate limit*). Petición número 101 en la misma ventana. Se espera respuesta `HTTP` 429 con cabecera `Retry-After` presente y un valor numérico válido.
- TC-INT-RES-01 (CE · resiliencia). Caída del contenedor `svaes-worker` mediante `docker kill` durante una verificación en curso. La tarea queda en un estado consistente (retomable o `FAILED`) sin corrupción en base de datos.
- TC-INT-RES-02 (CE · resiliencia). El servicio Redis no está disponible al lanzar la verificación. Se espera respuesta `HTTP` 503 controlada, sin que el *backend* sufra una caída.
- TC-INT-MIG-01 (CE · migración). Migración de una *release* entre perfiles de verificación distintos. El historial previo se preserva inmutable y las verificaciones futuras pasan a evaluarse contra el nuevo perfil.

7.3.3. Autenticación mediante API Key

La autenticación programática de clientes externos se realiza mediante claves de `API` (`APIKey`), creadas y gestionadas por los administradores de organización. La clase `TestApiKeyAuth` (`test_api_key_auth.py`) materializa once casos de prueba que cubren el ciclo de vida completo de una *API key*: creación, listado, uso en peticiones autenticadas, revocación y ausencia de cabecera.

- TC-INT-APIK-01 (CE · creación). Creación de una *API key* para una organización existente. Se espera respuesta `HTTP` 201 con el identificador y el valor de la clave en el cuerpo de la respuesta.

- TC-INT-APIK-02 (CE · listado). Listado de *API keys* activas de una organización. Se espera respuesta 200 con la colección de claves y sus metadatos.
- TC-INT-APIK-03 (CE · autenticación). Petición a un *endpoint* protegido incluyendo la cabecera *X-API-Key* con un valor válido. Se espera respuesta 200 y el recurso solicitado.
- TC-INT-APIK-04 (CE · revocación). Revocación de una *API key* existente. Peticiones posteriores con la misma clave reciben respuesta 401.
- TC-INT-APIK-05 (CE · cabecera ausente). Petición a un *endpoint* protegido sin incluir cabecera de autenticación. Se espera respuesta 401.
- TC-INT-APIK-06 (CE · clave inexistente). Petición con una *API key* que no corresponde a ninguna clave registrada. Se espera respuesta 401.
- TC-INT-APIK-07 (CE · permiso insuficiente). La *API key* pertenece a una organización distinta a la del recurso solicitado. Se espera respuesta 403.
- TC-INT-APIK-08 (CE · creación duplicada). Intento de crear una segunda *API key* con el mismo nombre en la misma organización. Se espera respuesta 409.
- TC-INT-APIK-09 (CE · límite de claves). Intento de crear una *API key* cuando la organización ya ha alcanzado el límite máximo configurado. Se espera respuesta 422.
- TC-INT-APIK-10 (CE · visibilidad). Un usuario sin privilegios de administración intenta listar o crear *API keys*. Se espera respuesta 403 en ambos casos.
- TC-INT-APIK-11 (CE · caducidad). Creación de una *API key* con fecha de caducidad. Una petición realizada después de dicha fecha recibe respuesta 401, mientras que una petición anterior a la fecha es aceptada.

7.4. Pruebas de aceptación y usabilidad

Las pruebas de aceptación validan que el sistema cumple los requisitos observables por el usuario final (CU-01 y flujos asociados, RNF-19 a RNF-30). La suite de aceptación se materializa en dos frentes complementarios. Por un lado, una suite **pytest** residente en `api/tests/acceptance/` compuesta de **12 funciones de test** que verifican estructuralmente los flujos de aceptación. Por otro, una suite **Cypress E2E** de **43 tests** organizada en cinco categorías (CU, UI, FRM, NAV, USA) que ejecuta los flujos completos sobre el *frontend* Angular desplegado en el entorno Docker Compose con *mocks* de [API](#) mediante `cy.intercept()`. En conjunto, ambas suites suman **55 tests de aceptación**.

La tabla 38 resume las seis suites que componen el plan de aceptación, indicando el identificador de cada caso y la descripción del alcance.

Suite	Tests	Descripción
TC-ACP-CU	CU-00 a CU-03	Ciclo de uso principal: verificación completa, semáforo naranja (advertencias), semáforo rojo (error bloqueante) y flujo guiado por manual.
TC-ACP-CU-VAL	CU-04, CU-05	Validación de campos de <i>login</i> : correo electrónico vacío y contraseña vacía.
TC-ACP-CU-DASH	CU-06, CU-07	<i>Dashboard</i> : tarjetas KPI visibles y métricas cargadas correctamente.
TC-ACP-UI	UI-01 a UI-04	Vista de detalle de <i>release</i> : tarjeta de información, tabla de resultados, listado de artefactos y <i>snapshot</i> del perfil.
TC-ACP-FRM	FRM-01, FRM-02	Validación de formularios: campo obligatorio vacío y campo numérico con texto alfanumérico.
TC-ACP-NAV	NAV-01 a NAV-03	Navegación por la barra lateral: acceso a proyectos, acceso al perfil y redirección tras cerrar sesión.
TC-USA	NAV-01, RES-01, SEM-01, LOG-01	Usabilidad <i>cross-browser</i> : compatibilidad con navegadores, <i>viewport</i> adaptable, semáforo coherente entre vistas y enlace «Volver al inicio».

Tabla 38: Suites de pruebas de aceptación.

Condiciones de prueba: C_1 : resultado de verificación $\in \{\text{VALIDA}, \text{CON_ADVERTENCIAS}, \text{NO_VALIDA}\}$. C_2 : navegador (*EC*) y resolución (*VL*). C_3 : estado de autenticación y navegación (*EC*).

A continuación se detallan los casos de prueba agrupados por suite temática.

Suite TC-ACP-CU — Ciclo de uso principal

- TC-ACP-CU-00 (BC · base). Usuario con rol `OPERATOR`, conectores activos y datos coherentes. La verificación completa debe realizarse en un máximo de 5 acciones de usuario (RNF-19).
- TC-ACP-CU-01 (BC · resultado). La regla RV-04 emite `WARNING`. La interfaz de usuario muestra el semáforo en color naranja y el sufijo `CON_ADVERTENCIAS`.
- TC-ACP-CU-02 (BC · resultado). La regla RV-05 emite un `ERROR` bloqueante. La interfaz muestra el semáforo en rojo y un mensaje descriptivo con la causa del fallo (RNF-20).
- TC-ACP-CU-03 (manual · tiempo). Un usuario nuevo, con solo el manual disponible como ayuda, completa el flujo de verificación en un máximo de 15 minutos (RNF-24).

Suite TC-ACP-CU-VAL — Validación de campos de *login*

- TC-ACP-CU-04 (VL · formulario). El campo de correo electrónico se envía vacío en el formulario de *login*. La interfaz muestra un mensaje de error que indica el campo concreto y la acción correctora necesaria (RNF-20).
- TC-ACP-CU-05 (VL · formulario). El campo de contraseña se envía vacío en el formulario de *login*. La interfaz muestra un mensaje de error análogo, sin revelar información sensible sobre la cuenta.

Suite TC-ACP-CU-DASH — *Dashboard*

- TC-ACP-CU-06 (CE · interfaz). Tras iniciar sesión con el rol `OPERATOR`, las tarjetas KPI del *dashboard* (total de verificaciones, tasa de éxito, *releases* pendientes) son visibles y muestran valores coherentes con los datos del sistema.
- TC-ACP-CU-07 (CE · interfaz). Las métricas agregadas del *dashboard* se cargan correctamente desde el *backend* y la gráfica de tasa de éxito refleja los datos históricos de verificaciones.

Suite TC-ACP-UI — Vista de detalle de *release*

- TC-ACP-UI-01 (BC · *snapshot*). El perfil de reglas se modifica después de que una verificación haya sido archivada. El detalle histórico muestra el perfil de reglas original en el momento de la verificación (RNF-36).
- TC-ACP-UI-02 (CE · resultados). La tabla de resultados de verificación en la vista de detalle muestra todas las reglas evaluadas con su estado (`OK`, `WARNING`, `ERROR`, `NOT_EVALUATED`) y el mensaje descriptivo correspondiente.
- TC-ACP-UI-03 (CE · artefactos). El listado de artefactos asociados a la *release* se

muestra correctamente, incluyendo el conector de origen, el identificador externo y el estado de cada artefacto.

- TC-ACP-UI-04 (CE · tarjeta de información). La tarjeta de información general de la *release* (versión, proyecto, organización, fecha de creación y estado actual) se renderiza con todos los campos presentes y correctamente formateados.

Suite TC-ACP-FRM — Validación de formularios

- TC-ACP-FRM-01 (VL · formulario). Un campo obligatorio se envía vacío. Se muestra un mensaje que indica el campo concreto y la acción correctora necesaria (RNF-20).
- TC-ACP-FRM-02 (VL · formulario). Un campo numérico recibe texto alfanumérico. Se muestra un error de validación de tipo (RNF-20).

Suite TC-ACP-NAV — Navegación por la barra lateral

- TC-ACP-NAV-01 (EC · navegación). Usuario con rol `OPERATOR` navega a la sección de proyectos desde la barra lateral. La vista de proyectos se carga correctamente y muestra los proyectos asociados a la organización del usuario.
- TC-ACP-NAV-02 (EC · navegación). Usuario con rol `OPERATOR` accede a la vista de perfil desde la barra lateral. La página de perfil muestra los datos del usuario, las opciones de seguridad (2FA) y la gestión de miembros de la organización.
- TC-ACP-NAV-03 (EC · navegación). El usuario cierra sesión desde la barra lateral. La aplicación redirige a la página de *login* y el *token* de sesión es invalidado.

Suite TC-USA — Usabilidad *cross-browser*

- TC-USA-NAV-01 (EC · navegadores). Verificación del funcionamiento en Chrome, Firefox, Edge y Safari. No se producen errores visuales ni excepciones JavaScript (RNF-29).
- TC-USA-RES-01 (VL · resolución). Comprobación de la interfaz en las resoluciones 1920×1080, 768×1024 y 375×667. El *layout* no presenta desbordamientos ni solapamientos (RNF-30).
- TC-USA-SEM-01 (EC · coherencia). El mismo estado de verificación se visualiza en el *dashboard*, el historial y la vista de detalle. El color del semáforo es coherente en todas las vistas (RNF-21).
- TC-USA-LOG-01 (EC · navegación). El enlace «Volver al inicio» presente en la página de *login* redirige correctamente a la página de aterrizaje (*landing*) sin requerir autenticación previa.

La suite incluye, además, dos tests estructurales (`test_acceptance_suite_file_exists` y `test_all_required_tc_ids_present`) que verifican la presencia del archivo y de los

identificadores de caso de prueba esperados, asegurando que el conjunto declarado en el plan está efectivamente implementado.

7.5. Pruebas de rendimiento

Los umbrales fijados por RNF-01 (≤ 30 s de procesamiento para la verificación completa) y RNF-05 (< 500 ms para respuestas de la [API REST](#)) son fronteras numéricas exactas que se expresan directamente como valores límite. Las medidas se realizan en el entorno Docker Compose local y se emplea el percentil 95 (p95) sobre cinco ejecuciones consecutivas para reducir el ruido estadístico.

La carga concurrente se genera con **Locust** mediante el fichero `locustfile.py`, que define cuatro clases de usuario: **E2EVerificationUser** (verificación extremo a extremo con las 10 reglas), **RustEngineUser** (medición aislada de la latencia del motor), **ConcurrentVerifyUser** (50 usuarios simultáneos sobre `POST /verify`) y **WebLoadUser** (carga sobre `dashboard` y `health check`). Los tests `pytest` residen en `api/tests/performance/` y se distribuyen en cinco archivos (incluyendo `locustfile.py`, `test_coverage_threshold.py`, `test_rnf_coverage.py` y `test_rnf_security_rf.py`) que totalizan **47 funciones de test**. Además de los cinco casos de rendimiento descritos a continuación, la suite incluye 19 tests de validación de requisitos no funcionales (RNF de rendimiento, fiabilidad, usabilidad, mantenibilidad, portabilidad, extensibilidad, trazabilidad y organizacionales) y 24 tests de cobertura de requisitos funcionales y de seguridad (password hashing, HTTPS, almacenamiento de API keys, auditoría y notificaciones).

- **TC-PER-VL-01** (VL · RNF-01). Verificación con las 10 reglas activas sobre datos sintéticos. El tiempo total de procesamiento, medido en el percentil 95 con Locust, no debe superar los 30 segundos (umbral de RNF-01).
- **TC-PER-VL-02** (VL). Petición directa al motor Rust en bucle durante 5 ejecuciones consecutivas. El percentil 95 medido debe ser inferior a 500 ms, validando el tiempo de respuesta interno del motor de verificación (componente que contribuye al cumplimiento de RNF-01, no de RNF-05).
- **TC-PER-VL-03** (VL · concurrencia). Cincuenta peticiones simultáneas a `POST /verify` generadas con Locust. Todas las respuestas deben ser [HTTP 202](#) sin ningún *timeout*.
- **TC-PER-CE-04** (CE · RNF-27). Ejecución de las *suites* completas (`cargo test`, `pytest` y Vitest) con generación de informes de cobertura. El *Quality Gate* de SonarCloud debe mostrar una cobertura global igual o superior al 80% (umbral igual al 80% requerido por RNF-27).
- **TC-PER-RNF01-01** (VL · RNF-01). Verificación con hasta 10 reglas activas sobre conectores mockeados con `respx` (latencia < 1 segundo por llamada). Se aísla el tiempo de procesamiento interno del motor de verificación; el percentil 95 medido

sobre cinco ejecuciones no debe superar los 30 segundos. Este caso valida exclusivamente el requisito RNF-01, distinto del KPI de ciclo completo que incluye latencias reales de APIs externas.

7.6. Pruebas de seguridad

Las pruebas de seguridad verifican las mitigaciones frente al *OWASP Top 10* (RNF-39), la protección de credenciales (RNF-13) y la política de bloqueo frente a fuerza bruta (RNF-14). Cada vector de ataque se modela como una clase inválida cuyo manejo correcto por el sistema constituye la situación a cubrir. La suite reside en `api/tests/security/`, se ejecuta con `pytest -m security` y se complementa con un análisis automatizado realizado con OWASP ZAP sobre el entorno desplegado.

- TC-SEC-AUT-01 (VL · RNF-14). Cinco intentos de autenticación fallidos consecutivos. El quinto intento recibe respuesta `HTTP 401/403` y la cuenta queda bloqueada durante el intervalo configurado.
- TC-SEC-AUT-02 (CE · OWASP A2). La firma de un *token JWT* es manipulada en el lado del cliente, incluyendo el caso del algoritmo `none`. El *backend* rechaza la petición con respuesta `HTTP 401`.
- TC-SEC-INY-01 (CE · OWASP A3). Se inyectan diversos vectores de SQL *injection* (variantes de `' OR 1=1; --`, comentarios de bloque, sub-consultas) en los campos del formulario de *login* y del registro. La combinación de Pydantic y SQLAlchemy neutraliza la inyección sin que se ejecute SQL malicioso.
- TC-SEC-INY-02 (CE · OWASP A3). Se inyectan vectores XSS (`<script>alert(1)</script>` y variantes ofuscadas) en los campos del formulario de registro. Los datos se almacenan escapados y se sirven al *frontend* sin posibilidad de ejecución de código.
- TC-SEC-CIF-01 (CE · RNF-13). Se manipula el *payload* de un *token JWT* para intentar elevar privilegios (*role escalation*). El *backend* rechaza la petición con respuesta `HTTP 401`, validando la integridad criptográfica del *token*. Adicionalmente, se comprueba que las credenciales almacenadas de los conectores aparecen cifradas con Fernet en la columna `BYTEA` correspondiente y no figuran en claro en ningún *log*.

7.7. Pruebas de usabilidad con usuarios

Las pruebas automatizadas descritas en la sección 7.4 verifican que la interfaz cumple los criterios objetivos definidos en los requisitos no funcionales (RNF-19 a RNF-30), pero no permiten capturar la dimensión subjetiva de la experiencia de uso: facilidad de aprendizaje, claridad de los mensajes, intuitividad de la navegación o satisfacción percibida. Para

complementar dicha validación, se ha diseñado una sesión de pruebas presencial con usuarios reales en la que se observa el comportamiento de cada participante mientras realiza un conjunto de tareas representativas del caso de uso principal (CU-01) y de los flujos administrativos asociados.

7.7.1. Objetivos

Las pruebas de usabilidad persiguen tres objetivos complementarios. El primero, validar empíricamente que un usuario nuevo puede completar el flujo de verificación de una *release* en menos de quince minutos utilizando únicamente el manual disponible como ayuda (RNF-24). El segundo, comprobar la coherencia visual del semáforo de estado entre las distintas vistas del sistema (RNF-21) y la calidad de los mensajes de error percibida por el usuario (RNF-20). El tercero, recopilar evidencia cuantitativa y cualitativa que permita detectar problemas de usabilidad no observables mediante pruebas automatizadas y orientar mejoras futuras de la interfaz.

7.7.2. Perfil de los participantes

La muestra se compone de **ocho participantes** segmentados en dos perfiles. El perfil *experto* reúne a cinco personas con formación o experiencia profesional en Ingeniería Informática, capaces de evaluar tanto la usabilidad como la coherencia técnica del modelo de verificación. El perfil *no experto* reúne a tres personas sin formación técnica específica, cuya participación resulta especialmente útil para detectar problemas de claridad en la terminología, los iconos y los mensajes del sistema. La elección de un tamaño muestral reducido es coherente con la recomendación de Nielsen [8], según la cual entre cinco y siete usuarios detectan en torno al 80 % de los problemas de usabilidad de una interfaz.

ID	Participante	Perfil	Justificación del perfil
P1	Miguel	No experto	Usuario sin formación técnica en Ingeniería Informática.
P2	Rosa	No experto	Usuario sin formación técnica en Ingeniería Informática.
P3	Vicente	Experto	Formación/experiencia en Ingeniería Informática.
P4	Carlos	Experto	Formación/experiencia en Ingeniería Informática.
P5	Daniel	Experto	Formación/experiencia en Ingeniería Informática.
P6	Fernando	Experto	Formación/experiencia en Ingeniería Informática.
P7	Gersán	Experto	Formación/experiencia en Ingeniería Informática.

ID	Participante	Perfil	Justificación del perfil
P8	Lara	No experto	Usuario sin formación técnica en Ingeniería Informática.

Tabla 39: Participantes en las pruebas de usabilidad.

7.7.3. Diseño del experimento

Cada participante realiza una sesión individual de aproximadamente cuarenta minutos sobre el entorno desplegado en Docker Compose. La sesión sigue la estructura de un estudio de usabilidad clásico: una breve presentación del sistema (sin mostrar la solución a las tareas), la ejecución secuencial de las tareas descritas en la tabla 40 aplicando la técnica de *think-aloud* (pensamiento en voz alta), y un cuestionario final de satisfacción. Durante la ejecución, el evaluador registra los tiempos, los errores, la asistencia requerida y los comentarios cualitativos emitidos por el participante, sin intervenir salvo bloqueo evidente. Las tareas se han seleccionado para cubrir el camino completo del caso de uso CU-01 y los flujos de soporte que un usuario debe dominar para operar el sistema de forma autónoma. Cada participante recibe credenciales preconfiguradas con el rol `OPERATOR` y un proyecto vacío al que asociar el trabajo.

ID	Descripción de la tarea	Tiempo objetivo
T1	Iniciar sesión, completar el segundo factor (TOTP) y acceder al <i>dashboard</i> principal.	≤ 1 min
T2	Crear un proyecto nuevo y asociarle un perfil de verificación existente.	≤ 2 min
T3	Configurar un conector a un sistema externo (Jira o GitLab) introduciendo las credenciales y comprobando la conexión.	≤ 3 min
T4	Crear una <i>release</i> dentro del proyecto, asignándole un número de versión SemVer y añadiendo al menos un artefacto.	≤ 3 min
T5	Lanzar la verificación de la <i>release</i> , esperar el resultado e identificar el veredicto global.	≤ 5 min
T6	Consultar el detalle de una regla con resultado <code>WARNING</code> o <code>ERROR</code> y describir verbalmente la causa del aviso.	≤ 2 min
T7	Exportar el informe de verificación en PDF y localizar el archivo descargado.	≤ 1 min

Tabla 40: Tareas planteadas en las pruebas de usabilidad.

7.7.4. Métricas e instrumentos

Las métricas recogidas durante la sesión se agrupan en cuatro categorías. La **eficacia** se mide como tasa de éxito por tarea, considerándose una tarea exitosa si el participante alcanza el objetivo sin asistencia bloqueante. La **eficiencia** se mide como el tiempo empleado en cada tarea, comparado con el tiempo objetivo. Los **errores** se contabilizan como el número de acciones erróneas o caminos no productivos antes de completar la tarea. Finalmente, la **satisfacción** se mide mediante el cuestionario **SUS** (*System Usability Scale*) [1], estándar de facto que produce una puntuación normalizada en el rango 0–100 a partir de diez afirmaciones evaluadas en una escala Likert de cinco puntos. Una puntuación **SUS** igual o superior a 68 puntos se considera por encima de la media de la industria. Como instrumento complementario se registra, al finalizar cada sesión, una breve entrevista semiestructurada con tres preguntas abiertas: el aspecto que más ha gustado al participante, el principal punto de fricción percibido y una propuesta de mejora.

7.7.5. Resultados por participante

La tabla 41 recoge, por participante y tarea, si la ejecución se ha completado con éxito (✓), si se ha requerido asistencia (A) o si ha fracasado (✗), junto al tiempo empleado y al número de errores observados.

Tarea	Métrica	P1	P2	P3	P4	P5	P6	P7	P8
T1	Éxito	✓	A	✓	✓	✓	✓	✓	A
	Tiempo (s)	78	152	60	48	65	52	68	95
	Errores	1	3	0	0	1	0	1	2
T2	Éxito	✓	A	✓	✓	✓	✓	✓	✓
	Tiempo (s)	145	238	108	92	115	98	122	175
	Errores	2	3	1	0	1	1	1	3
T3	Éxito	A	A	✓	✓	✓	✓	A	A
	Tiempo (s)	282	348	180	158	192	165	238	312
	Errores	3	5	2	1	2	1	3	4
T4	Éxito	✓	A	✓	✓	✓	✓	✓	✓
	Tiempo (s)	218	296	152	132	158	138	168	248
	Errores	2	4	1	1	1	1	2	3
T5	Éxito	✓	✓	✓	✓	✓	✓	✓	✓
	Tiempo (s)								
	Errores								

Tarea	Métrica	P1	P2	P3	P4	P5	P6	P7	P8
	Tiempo (s)	196	242	168	148	175	155	182	218
	Errores	1	2	1	0	1	1	1	2
T6	Éxito	✓	✓	✓	✓	✓	✓	✓	✓
	Tiempo (s)	128	184	92	78	95	85	102	152
	Errores	1	2	0	0	1	0	1	2
T7	Éxito	✓	✓	✓	✓	✓	✓	✓	✓
	Tiempo (s)	52	61	38	32	40	35	42	58
	Errores	0	1	0	0	0	0	0	1
Totales	Éxito (%)	86%	43%	100%	100%	100%	100%	86%	71%
	Tiempo total	18:19	25:21	13:18	11:28	14:00	12:08	15:22	21:08
	Errores totales	10	20	5	2	7	4	9	17

Tabla 41: Resultados por participante y tarea. Legenda: ✓ éxito sin asistencia; A éxito con asistencia; ✗ no completada.

A continuación se recoge la puntuación obtenida por cada participante en el cuestionario SUS, junto con los promedios por perfil.

ID	Participante	Perfil	SUS	Interpretación
P1	Miguel	No experto	72,5	Por encima de la media (bueno)
P2	Rosa	No experto	55,0	Por debajo de la media (marginal)
P3	Vicente	Experto	80,0	Por encima de la media (bueno)
P4	Carlos	Experto	87,5	Por encima de la media (excelente)
P5	Daniel	Experto	75,0	Por encima de la media (bueno)
P6	Fernando	Experto	85,0	Por encima de la media (excelente)
P7	Gersán	Experto	77,5	Por encima de la media (bueno)
P8	Lara	No experto	65,0	En la media (aceptable)
Promedio no expertos			64,2	Marginalmente en la media
Promedio expertos			81,0	Por encima de la media (bueno)
Promedio global			74,7	Por encima de la media (bueno)

Tabla 42: Puntuación SUS por participante. Referencia: ≥ 68 se considera por encima de la media de la industria.

7.7.6. Resultados agregados y comparación entre perfiles

La tabla 43 sintetiza las métricas por tarea agregando los resultados de cada perfil, lo que permite comparar el desempeño de usuarios expertos frente a no expertos y detectar las tareas con mayor brecha de usabilidad.

Tarea	No expertos (n=3)			Expertos (n=5)		
	Éxito	T medio	Errores	Éxito	T medio	Errores
T1	33%	108 s	2,0	100%	59 s	0,4
T2	67%	186 s	2,7	100%	107 s	0,8
T3	0%	314 s	4,0	80%	187 s	1,8
T4	67%	254 s	3,0	100%	150 s	1,2
T5	100%	219 s	1,7	100%	166 s	0,8
T6	100%	155 s	1,7	100%	90 s	0,4
T7	100%	57 s	0,7	100%	37 s	0,0
Global	67%	21:33	15,7	97%	13:16	5,4

Tabla 43: Métricas agregadas por perfil de participante.

7.7.7. Hallazgos cualitativos

Más allá de los datos cuantitativos, el método *think-aloud* y la entrevista semiestructurada final han permitido recopilar observaciones cualitativas sobre la interfaz. La tabla 44 clasifica los hallazgos por participante y los etiqueta con su severidad estimada según la escala de Nielsen (0 = cosmético; 1 = menor; 2 = mayor; 3 = bloqueante).

ID	Tarea	Severidad	Observación
P1	T3	2	Identifica con soltura los formularios y la navegación general, gracias a su familiaridad con suites ofimáticas, pero los conceptos <i>conector</i> , <i>token</i> y <i>URL base</i> le resultan ajenos. Sugiere incluir un asistente guiado al añadir conectores y un ejemplo del formato SemVer junto al campo de versión (<i>p. ej. 1.0.0</i>).

ID	Tarea	Severidad	Observación
P2	T1	3	El segundo factor (TOTP) supone una barrera inicial seria: no comprende qué es el código de seis dígitos ni dónde obtenerlo, lo que prácticamente exige asistencia para superar T1. La configuración del conector (T3) tampoco se completa sin acompañamiento por terminología desconocida (<i>API key</i> , <i>GitLab</i>). Por el contrario, identifica sin ayuda los semáforos de color y el botón de exportar, lo que evidencia que las <i>affordances</i> visuales funcionan también para usuarios sin perfil técnico.
P3	T3	1	Completa todas las tareas sin asistencia y con tiempos próximos a los objetivo. Como estudiante de último curso con cuatro meses de experiencia profesional, identifica rápidamente las analogías con plataformas de CI/CD que ya conoce. Comenta que el formulario de conector podría incorporar validación inmediata del <i>token</i> (probar la conexión en línea antes de enviar) y que las etiquetas de severidad de las reglas (OBLIGATORIA, OPCIONAL, EXCLUIDA) ganarían en claridad mostrando un <i>tooltip</i> explicativo al pasar el cursor por encima.
P4	—	0	Resuelve la totalidad del recorrido sin asistencia, sin errores destacables y con los mejores tiempos de la muestra. Su trayectoria académica reciente le permite anticipar la estructura del sistema desde la primera pantalla. No identifica problemas funcionales relevantes y limita sus comentarios al plano estético: sugiere ampliar el contraste de los iconos de estado en modo claro y añadir atajos de teclado para las acciones más frecuentes (lanzar verificación, exportar informe). Considera que el flujo CU-01 se completa con un número de acciones inferior al límite establecido por RNF-19.
P5	T3	1	Completa todas las tareas sin asistencia, con tiempos ligeramente superiores a la media del perfil experto. La configuración del conector le exige releer una vez la etiqueta de cada campo, lo que motiva la propuesta de añadir descripciones por defecto en los <i>placeholders</i> de los formularios. Valora positivamente la coherencia del semáforo de estado entre el <i>dashboard</i> y el detalle de <i>release</i> (RNF-21) y aprueba el comportamiento del <i>polling</i> durante la verificación, que percibe como reactivo y sin parpadeos.

ID	Tarea	Severidad	Observación
P6	T3	1	Completa todas las tareas con fluidez y sin asistencia. Señala que el desplegable de tipo de conector podría incluir una barra de búsqueda para cuando la lista crezca, y que la exportación en PDF debería ofrecer una vista previa antes de la descarga. Valora positivamente la consistencia visual y la rapidez del motor de verificación.
P7	T3	2	La configuración del conector le provoca dudas: el formulario solicita campos (URL base, token) cuyo formato exacto desconoce para herramientas que no ha usado antes. Necesita asistencia puntual para completar T3. Sugiere incluir tooltips con ejemplos junto a cada campo y un enlace a la documentación de la herramienta externa. El resto de tareas las completa sin incidencias, destacando la claridad del panel de resultados de verificación.
P8	T1	2	Aunque maneja con soltura el teléfono, la tableta y el ordenador en su uso cotidiano, su formación en Economía no le proporciona el vocabulario técnico subyacente al sistema. El segundo factor (TOTP) le exige una primera explicación, tras la cual completa el resto de <i>logins</i> sin asistencia. La configuración del conector (T3) resulta el principal punto de fricción, igual que en el resto del perfil no experto, por la abundancia de términos en inglés (<i>API key</i> , <i>base URL</i>) y la ausencia de ejemplos. Una vez superada esa barrera, identifica con naturalidad el semáforo de estado, la barra lateral y el botón de exportar, y completa todas las tareas restantes sin asistencia. Sugiere incluir un breve vídeo de introducción de un minuto en la primera entrada al sistema.

Tabla 44: Hallazgos cualitativos por participante.

7.7.8. Conclusiones de la prueba de usabilidad

Los resultados obtenidos sobre los ocho participantes permiten extraer conclusiones cuantitativas y cualitativas alineadas con los objetivos enunciados en la sección 7.7.1.

Cumplimiento de RNF-24 (usuario nuevo ≤ 15 minutos). La totalidad del perfil experto completa el recorrido íntegro de las siete tareas (T1–T7) por debajo del umbral de quince minutos, con un tiempo medio de 13:16 y un mejor tiempo de 11:28 registrado por Carlos. En el perfil no experto, únicamente uno de los tres participantes (Miguel, con

18:19) se sitúa próximo al umbral; Lara (21:08) y Rosa (25:21) lo superan claramente. El requisito RNF-24 se considera satisfecho para usuarios con perfil técnico, que constituyen la audiencia objetivo del sistema según los *stakeholders* U2-U4 definidos en el capítulo 4; los tiempos del perfil no experto cuantifican la brecha de aprendizaje atribuible a la terminología específica del dominio y se trasladan al trabajo futuro como una de las líneas de mejora prioritarias.

Cumplimiento de RNF-19 (CU-01 en ≤ 5 acciones). La tarea T5, que materializa el camino principal de CU-01 sobre una *release* previamente creada, se completa con éxito por los ocho participantes (100 %) sin asistencia. La observación directa durante las sesiones confirma que el flujo se resuelve con cuatro acciones efectivas (seleccionar *release*, pulsar *verificar*, esperar el veredicto y abrir el detalle), por debajo del techo de cinco fijado por el requisito. El número de errores asociado a T5 es bajo en ambos perfiles (media de 0,8 en expertos y 1,7 en no expertos), lo que confirma que el camino feliz del caso de uso principal es fluido y no es la fuente principal de fricción del sistema.

Puntuación SUS. La puntuación global obtenida es de 74,7, claramente por encima del umbral de 68 que se considera media de la industria [1]. Desglosada por perfiles, la media del perfil experto alcanza 81,0 (interpretable como *bueno*), mientras que la media del perfil no experto se sitúa en 64,2, marginalmente por debajo de la media. Las puntuaciones individuales reflejan con fidelidad la experiencia observada durante la sesión: Carlos (87,5) y Fernando (85,0) obtienen las valoraciones más altas y completan también el recorrido con el menor número de errores; Rosa, que necesita asistencia en cinco de las siete tareas, otorga la puntuación más baja (55,0). La correlación entre métricas objetivas (tasa de éxito, tiempo, errores) y métrica subjetiva (SUS) refuerza la validez interna del estudio.

Brecha entre perfiles. La comparación de la tabla 43 cuantifica la brecha de usabilidad entre expertos y no expertos. Las tareas T5 (lanzamiento de verificación), T6 (consulta del detalle de una regla) y T7 (exportación) se completan al 100 % en ambos perfiles con tiempos relativamente próximos, lo que indica que las partes del sistema que materializan el dominio funcional propio del SVAES resultan accesibles incluso para usuarios sin formación técnica. La brecha se concentra en T1 (autenticación con segundo factor) y, sobre todo, en T3 (configuración de un conector externo), tareas que exigen vocabulario propio del entorno de desarrollo (TOTP, *API key*, URL base, *token*) y para las cuales el perfil no experto requiere asistencia de forma sistemática. Este patrón es consistente con la hipótesis de partida: la complejidad inherente del sistema reside en la integración con herramientas externas, no en la lógica de verificación que el sistema añade sobre ellas.

Problemas de severidad mayor o bloqueante y plan de mitigación. La tabla 44 identifica un único problema de severidad bloqueante (severidad 3), correspondiente a la dificultad de Rosa con la autenticación TOTP en T1, y tres problemas de severidad mayor (severidad 2) relacionados con la configuración del conector en T3 (P1, P7 y P8). Las

medidas de mitigación previstas, ya recogidas en el capítulo de conclusiones, comprenden: (i) la incorporación de un asistente paso a paso en el formulario de creación de conector con ejemplos contextualizados por tipo de herramienta externa (Jira, GitLab, Confluence, ClickUp); (ii) la inclusión de *tooltips* explicativos junto a los campos técnicos (*API key*, *URL base*, *token*); (iii) la sustitución de los términos en inglés por sus equivalentes castellanos cuando no se trate de tecnicismos consolidados; (iv) la adición de una pantalla de introducción opcional en el primer inicio de sesión que explique brevemente el segundo factor y el ciclo de verificación; y (v) la validación inmediata del *token* de conector mediante una llamada de prueba antes de persistir la configuración. Ninguno de los problemas identificados afecta al núcleo funcional del sistema ni compromete la viabilidad del producto: todos ellos pertenecen al ámbito de la experiencia de usuario y son abordables sin reescrituras de arquitectura.

Síntesis. La prueba de usabilidad confirma que el **SVAES** satisface los requisitos no funcionales de usabilidad para el perfil de usuario al que está dirigido, alcanza una puntuación **SUS** por encima de la media de la industria y resulta operable también para usuarios sin formación técnica una vez superada la barrera inicial de la terminología propia del ciclo de integración continua. Las observaciones cualitativas recogidas constituyen una entrada valiosa para futuras iteraciones del producto y se incorporan al trabajo futuro descrito en el capítulo 10.

7.8. Ejecución y resultados

Esta sección documenta los resultados obtenidos al ejecutar la suite completa de pruebas sobre la implementación final del **SVAES**. La ejecución se ha realizado en el entorno de integración continua configurado para el proyecto y, en paralelo, sobre la máquina local de desarrollo en Docker Compose para validar reproducibilidad.

7.8.1. Inventario consolidado

La tabla 45 sintetiza el recuento exacto de archivos y funciones de prueba por componente.

Componente	Archivos	Tests	Herramienta
Motor Rust	21	103	<code>cargo test + cargo-llvm-cov</code>
<i>Backend</i> Python	26	1 216	<code>pytest 8.x + pytest-cov</code>
<i>Frontend</i> Angular	43	721	Vitest 4.x (@angular/build:unit-test)
Cypress E2E	1	43	<code>Cypress + cy.intercept()</code>
TOTAL	91	~2 083	

Tabla 45: Inventario consolidado de pruebas ejecutadas.

7.8.2. Resultados por nivel

La tabla 46 agrupa los resultados por nivel de prueba conforme a la nomenclatura *ISO/IEC/IEEE 29119* [4], indicando el número de tests ejecutados, el porcentaje pasado y observaciones relevantes.

Nivel	Ejecutados	Pasados	Observaciones
<i>Backend</i> Python (pytest)	1 216	1 216 (100%)	26 archivos (unit, integración, seguridad, rendimiento, aceptación); cobertura ~80% líneas. Incluye tests GDPR Art. 20.
Motor Rust	103	103 (100%)	8 tests del <i>aggregator</i> + 89 tests RV-01 a RV-10 + 6 tests de calidad SonarQube.
<i>Frontend</i> Angular (Vitest)	721	721 (100%)	43 <code>spec.ts</code> ; cobertura ~80% líneas. Incluye tests GDPR, preferencias de notificación y gestión de cuenta.
Cypress E2E	43	43 (100%)	5 suites (CU, UI, FRM, NAV, USA).
TOTAL global	~2 083	100%	

Tabla 46: Resultados de ejecución agrupados por nivel.

7.8.3. Cobertura medida y Quality Gate

La cobertura se mide de forma agregada en SonarCloud combinando los tres formatos soportados: `llvm-cov` para Rust, `coverage.xml` para Python y `lcov.info` para Angular. La trayectoria seguida durante el desarrollo ha sido **53%** $\rightarrow$ **79,5%** $\rightarrow$ $\geq 80\%$, alcanzando finalmente el umbral configurado en el *Quality Gate*. Las exclusiones aplicadas son las habituales en proyectos de esta naturaleza: archivos de migración (`alembic/versions/`), configuración (`settings.py`), puntos de entrada (`main.py`) y puertos abstractos sin lógica, todas ellas legitimadas en la configuración `sonar-project.properties`.

Cumplimiento de RNF-27. El requisito establece una cobertura mínima del 80% de líneas, umbral que coincide plenamente con el aplicado en el *Quality Gate* de SonarCloud y queda registrado en el *badge* público asociado al repositorio.

7.8.4. Tests faltantes y deuda asumida

El análisis de trazabilidad confirma que todos los casos de prueba diseñados en el plan han sido implementados y pasan exitosamente en la suite final, a excepción de un test de rendimiento marcado como `skipped` en el entorno de integración continua.

- TC-PER-CE-04 (RNF-27). Cobertura SonarCloud $\geq 70\%$. Este test está desactivado (`skipped`) en el pipeline de CI debido a que la medición de cobertura de código requiere el informe de cobertura generado previamente por las suites unitarias, lo que crea una dependencia circular en el orden de ejecución del pipeline. La funcionalidad está cubierta por los informes de cobertura (`llvm-cov`, `coverage.xml`, `lcov.info`) que se generan y revisan manualmente.

La deuda técnica asociada a este test es asumida y documentada; no afecta a la calidad del producto entregado ya que las métricas de cobertura se revisan de forma manual periódica en SonarCloud.

7.9. Matriz de trazabilidad y resumen

La siguiente tabla sintetiza el plan completo por nivel, técnica y herramienta, y la matriz posterior relaciona cada requisito del [SRS](#) con los casos que lo validan. Los recuentos reflejan los casos sistemáticos derivados del diseño; el inventario completo de funciones de *test* ejecutadas se documenta en la sección [7.8](#).

Nivel	Casos	Técnica predominante	Herramienta
Unitarias motor (§7.1–7.6)	19	CE+VL · <i>Each Choice</i>	<code>cargo test</code> + <code>llvm-cov</code>
Unitarias <i>backend</i> (§7.2–7.4)	20*	<i>Base Choice</i> · CE+VL	<code>pytest</code> + <code>pytest-cov</code> + <code>respx</code>
Unitarias <i>frontend</i> (§7.5)	30	CE+VL · <i>Base Choice</i>	Vitest 4.x
Integración (§7.3)	27	Transición de estados · VL · CE	<code>pytest</code> + Docker Compose
Aceptación/Usabilidad (§7.4)	32	<i>Base Choice</i> · <i>Each Choice</i> · VL	<code>pytest</code> + Cypress

Nivel	Casos	Técnica predominante	Herramienta
Usabilidad con usuarios (§7.7)	7 · 8	Observación + SUS	Sesión presencial + cuestionario
Rendimiento (§7.5)	47	VL en umbrales RNF · CE	Locust + SonarCloud
Seguridad (§7.6)	5	Clases inválidas (vectores)	pytest + OWASP ZAP
TOTAL diseño	187	4 técnicas sistemáticas + sesión con usuarios	
TOTAL ejecución	~2083: 1 216 (pytest — todas las categorías) + 103 (cargo test) + 721 (Vitest) + 43 (Cypress)		

Tabla 47: Resumen del plan de pruebas por nivel, técnica y herramienta.

* Casos sistemáticos representativos. La suite completa del *backend* Python (pytest) comprende 1 216 tests repartidos en 26 archivos a lo largo de los directorios `unit/`, `integration/`, `security/`, `performance/` y `acceptance/` (véase §7.2.5).

7.9.1. Matriz de trazabilidad requisitos – casos de prueba

Requisito (SRS)	Casos de prueba
CU-01 (lanzar verificación)	TC-INT-FLW-01 a TC-INT-FLW-03 · TC-ACP-CU-00 a TC-ACP-CU-03
RV-01 a RV-10 (reglas)	TC-UNI-MOT-01 a TC-UNI-MOT-13 · TC-INT-FLW-01 · TC-INT-FLW-02
Aggregator (perfil PV1.3)	TC-UNI-AGG-01 a TC-UNI-AGG-07
RNF-01 (procesamiento ≤ 30 s)	TC-PER-RNF01-01 · TC-PER-VL-01 · TC-PER-VL-02
AC3 (control de acceso RBAC)	TC-UNI-API-00 a TC-UNI-API-08 · TC-UNI-FE-GRD-01 a TC-UNI-FE-GRD-04
AC4.9 (exportación de datos personales, RGPD)	TC-UNI-API-EXP-01 a TC-UNI-API-EXP-03 · TC-UNI-FE-PRF-22 a TC-UNI-FE-PRF-26
RNF-05 (respuesta API < 500 ms)	TC-PER-VL-03 (<i>endpoint</i> POST /verify bajo carga concurrente, validación de ruta API completa)

Requisito (SRS)	Casos de prueba
RNF-13 (cifrado credenciales)	TC-SEC-CIF-01 · <code>TestFernetCredentialEncryptor</code>
RNF-14 (bloqueo fuerza bruta)	TC-SEC-AUT-01
RNF-19 (≤ 5 acciones CU-01)	TC-ACP-CU-00 · Tablas 41, 43
RNF-20 (mensajes descriptivos)	TC-ACP-FRM-01 · TC-ACP-FRM-02 · TC-ACP-CU-02 · TC-UNI-FE-VFY-02 · Sesión usabilidad (§7.7)
RNF-21 (semáforo coherente)	TC-USA-SEM-01 · Sesión usabilidad (§7.7)
RNF-24 (≤ 15 min, usuario nuevo)	TC-ACP-CU-03 · Sesión usabilidad (§7.7)
Satisfacción percibida (SUS)	Tabla 42 (umbral ≥ 68)
RNF-27 (cobertura $\geq 80\%$)	TC-PER-CE-04 (<i>Quality Gate</i> a 80%)
RNF-29 (4 navegadores)	TC-USA-NAV-01
RNF-30 (3 resoluciones)	TC-USA-RES-01
RNF-36 (<i>snapshot</i> inmutable)	TC-ACP-UI-01 · TC-INT-MIG-01
RNF-39 (OWASP Top 10)	TC-SEC-AUT-02 · TC-SEC-INY-01 · TC-SEC-INY-02
Gestión <i>JWT</i> y sesión Angular	TC-UNI-FE-GRD-02 · TC-UNI-FE-GRD-03 · TC-UNI-FE-SVC-02
Flujo asíncrono de verificación	TC-UNI-FE-VFY-01 · TC-UNI-FE-VFY-02
API-3 (autenticación programática)	TC-INT-APIK-01 a TC-INT-APIK-11
GR7.1.1 (edición de proyecto: nombre, descripción, perfil)	<code>test_services.py</code> (ProjectService) · <code>test_use_cases_core.py</code> (UpdateProject)
GR7.2.1 (bloqueo de creación de <i>release</i> en proyecto archivado)	<code>test_use_cases_core.py</code> (CreateRelease) · <code>test_services.py</code> (ProjectService)

Tabla 48: Matriz de trazabilidad requisitos – casos de prueba.

7.9.2. Herramientas y entorno de ejecución

- `cargo test` + `cargo-llvm-cov`: pruebas unitarias del motor Rust con módulos `#[cfg(test)]` inline y cobertura en formato `llvm-cov`.
- `pytest 8.x` + `pytest-cov`: pruebas unitarias, de integración, seguridad, rendi-

miento y aceptación del *backend* Python; cobertura exportada en `coverage.xml` para SonarCloud. Configuración `asyncio_mode = auto` y `testpaths` segregados por nivel en `api/pyproject.toml`.

- **FastAPI TestClient (httpx)**: invocación de *endpoints* sin servidor **HTTP** real; permite las pruebas *Base Choice* parametrizadas con `pytest.mark.parametrize`.
- **respx**: interceptación **HTTP** asíncrona para simular conectores externos sin acceso a la red, garantizando reproducibilidad.
- **Vitest 4.x + @angular/build:unit-test**: pruebas unitarias de componentes, *guards*, *interceptors* y servicios Angular con `HttpClientTestingModule`; `globals: true`, `fileParallelism: false`; cobertura exportada en `lcov.info` y `text-summary`.
- **Docker Compose**: orquestación del entorno de integración completo sobre la red `svaes-internal`; los servicios externos se sustituyen por *mocks* ASGI.
- **Locust**: generación de carga concurrente para las pruebas de rendimiento del apartado 7.5 mediante las clases `E2EVerificationUser`, `RustEngineUser`, `ConcurrentVerifyUser` y `WebLoadUser`.
- **OWASP ZAP**: análisis automatizado de vulnerabilidades web sobre el entorno desplegado, complementario a las pruebas TC-SEC-.*.
- **SonarCloud**: *Quality Gate* configurado al 80% sobre los tres formatos de cobertura agregados (`llvm-cov`, `coverage.xml`, `lcov.info`). Umbral igual al 80% requerido por RNF-27.
- **Cypress**: pruebas de aceptación y usabilidad *end-to-end* sobre el *frontend* Angular, complementarias a la suite `pytest` de aceptación.

8 Manuales del Sistema

Este capítulo recoge los manuales necesarios para instalar, desplegar y operar el Sistema de Verificación Automática de Entregas de Software ([SVAES](#)). Se distinguen tres perfiles de lector: el técnico encargado del despliegue ([Sección 8.1](#)), el usuario final que opera releases y lanza verificaciones ([Sección 8.2](#)), y el administrador responsable de la gestión global de la plataforma ([Sección 8.3](#)).

Cada manual presupone únicamente los conocimientos propios del rol al que se dirige: el manual de instalación asume familiaridad con la línea de comandos y con Docker; el manual de usuario no requiere conocimientos técnicos previos más allá del uso de un navegador web moderno; y el manual de administración exige conocer las particularidades organizativas del proyecto y de los sistemas externos con los que se integra el [SVAES](#).

8.1. Manual de Instalación y Despliegue

El proceso de instalación del [SVAES](#) se realiza íntegramente mediante Docker Compose, tal como se describe en la sección [5.2.4](#) del capítulo [5](#). Este mecanismo garantiza que el entorno es reproducible con independencia del sistema operativo del host, siempre que este disponga de Docker Engine y Docker Compose.

La plataforma se distribuye como un conjunto de seis servicios contenedorizados que se levantan de manera coordinada: un servidor web Nginx para la interfaz gráfica, una [API REST](#) en Python con FastAPI, un motor de verificación en Rust, una cola de trabajadores asíncronos basada en Celery, una base de datos PostgreSQL 17 y un broker de mensajería Redis. Todos los servicios se comunican a través de una red interna y solo el frontend queda expuesto al exterior.

8.1.1. Requisitos Previos

Antes de proceder con la instalación, el sistema anfitrión debe cumplir los requisitos recogidos en la [Tabla 49](#). Las versiones mínimas indicadas son las testadas durante el desarrollo del proyecto; versiones inferiores pueden funcionar pero no están soportadas.

Componente	Descripción	Versión mínima
Docker Engine	Motor de contenedores Docker	25.x
Docker Compose	Herramienta de orquestación (plugin v2)	2.x
Git	Sistema de control de versiones	2.x
CPU	Arquitectura x86_64 o AArch64	—

Componente	Descripción	Versión mínima
Núcleos	Núcleos lógicos disponibles	2
RAM	Memoria RAM disponible	4 GB
Disco	Espacio libre en disco	10 GB
Red	Acceso saliente a HTTP/HTTPS para conectores	—

Tabla 49: Requisitos previos del sistema anfitrión.

Para verificar que Docker y Docker Compose están correctamente instalados, puede ejecutarse:

```
docker --version
docker compose version
```

El [SVAES](#) se ha validado en arquitectura AArch64 (Oracle Cloud, instancia Ampere A1) y en x86_64 (estaciones de trabajo de desarrollo). Las imágenes Docker producidas son multiarquitectura siempre que se construyan en el mismo entorno en el que se desplegarán.

8.1.2. Obtención del Código Fuente

El código fuente del [SVAES](#) se obtiene clonando el repositorio mediante Git:

```
git clone https://github.com/adrianmfuentes/svaes.git
cd svaes
```

La rama `main` contiene siempre la última versión estable del sistema. Las versiones liberadas se etiquetan siguiendo el esquema `vMAJOR.MINOR.PATCH`. Para desplegar una versión concreta:

```
git checkout v1.0.0
```

8.1.3. Configuración del Entorno

El sistema utiliza variables de entorno para gestionar los parámetros sensibles y de despliegue. Estas variables deben declararse en un fichero `.env` ubicado en la raíz del repositorio. El repositorio incluye una plantilla `.env.example` con todos los parámetros documentados. La Tabla 50 describe las variables disponibles, agrupadas por subsistema.

Variable	Oblig.	Descripción	Ejemplo
<i>Base de datos</i>			

Variable (cont.)	Oblig.	Descripción	Ejemplo
DATABASE_URL	Sí	Cadena de conexión a PostgreSQL en formato <code>asyncpg</code> .	<code>postgresql+asyncpg://...</code>
<i>Seguridad y autenticación</i>			
JWT_SECRET_KEY	Sí	Clave de firma de tokens <code>JWT</code> . Mínimo 32 caracteres aleatorios.	(cadena aleatoria)
JWT_ALGORITHM	No	Algoritmo <code>JWT</code> (por defecto <code>HS256</code>).	<code>HS256</code>
JWT_EXPIRE_MINUTES	No	Expiración del access token, en minutos (por defecto 30).	30
ENCRYPTION_KEY	Sí	Clave Fernet para cifrar credenciales de conectores en reposo.	(ver Tabla 8.1.3)
ENVIRONMENT	No	Entorno de ejecución (<code>development</code> o <code>production</code>).	<code>production</code>
ALLOWED_ORIGINS	No	Orígenes <code>CORS</code> permitidos, separados por coma.	<code>https://svaes.example.com</code>
<i>Cola asíncrona y caché</i>			
REDIS_URL	Sí	URL de conexión a Redis para blacklist de tokens.	<code>redis://redis:6379/0</code>
CELERY_BROKER_URL	Sí	Broker de Celery (típicamente la misma instancia de Redis).	<code>redis://redis:6379/1</code>
CELERY_RESULT_BACKEND	Sí	Backend de resultados de Celery.	<code>redis://redis:6379/2</code>
<i>Motor de verificación</i>			
ENGINE_URL	Sí	URL interna del motor de verificación Rust.	<code>http://svaes-engine:8081</code>
ENGINE_API_KEY	Sí	Clave compartida entre backend y motor (cabecera <code>X-Engine-API-Key</code>).	(cadena aleatoria)
<i>Cuenta administradora inicial</i>			

Variable (cont.)	Oblig.	Descripción	Ejemplo
ADMIN_EMAIL	Sí	Email del usuario ADMIN creado en el primer arranque.	admin@example.com
ADMIN_PASSWORD	Sí	Contraseña inicial del ADMIN (debe rotarse tras el primer login).	(cadena aleatoria)
<i>Envío de correo (SMTP)</i>			
SMTP_HOST	No	Servidor SMTP predeterminado. Cada organización puede configurar su propio servidor SMTP por canal de notificación (NF1.3), con aislamiento <i>multi-tenant</i> . Las variables globales actúan como respaldo.	smtp.example.com
SMTP_PORT	No	Puerto SMTP predeterminado.	587
SMTP_USER	No	Usuario de autenticación SMTP predeterminado.	noreply@...
SMTP_PASSWORD	No	Contraseña SMTP predeterminada.	(cadena aleatoria)
SMTP_USE_TLS	No	Activa TLS sobre SMTP (por defecto true).	true
APP_BASE_URL	Sí*	URL pública usada en enlaces de correos.	https://svaes.example.com

Tabla 50: Variables de entorno del **SVAES** (las marcadas con * solo son obligatorias si se desea habilitar el envío de notificaciones por correo).

Cuando `ENVIRONMENT=production`, el backend valida en el arranque que las variables sensibles no contienen valores por defecto inseguros: rechaza el email administrador genérico, contraseñas triviales, orígenes **CORS** permisivos y claves **JWT** demasiado cortas. Esta validación es un mecanismo de defensa frente a despliegues mal configurados.

Generación de claves criptográficas. La clave Fernet para el cifrado de credenciales de conector puede generarse con:

```
python -c "from cryptography.fernet import Fernet; \
print(Fernet.generate_key().decode())"
```

La clave [JWT](#) puede generarse con:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

El fichero `.env` está incluido en `.gitignore` y no debe incorporarse al control de versiones bajo ninguna circunstancia. Para entornos productivos se recomienda gestionarlo mediante el subsistema de secretos del proveedor de infraestructura (Docker secrets, AWS Secrets Manager, HashiCorp Vault, etc.).

8.1.4. Despliegue con Docker Compose

Una vez configurado el fichero `.env`, el sistema se despliega con un único comando:

```
docker compose up --build
```

Este comando construye las imágenes de todos los servicios y los levanta en el orden correcto, respetando las dependencias declaradas (`depends_on`). Los servicios que se inician son los recogidos en la Tabla 51.

Servicio	Puerto host	Función
<code>svaes-frontend</code>	8080	Nginx que sirve la SPA Angular y actúa como único punto de entrada externo. Internamente proxy-redirige <code>/api/</code> al backend.
<code>svaes-backend</code>	8000	API REST FastAPI (16 routers, 101 endpoints). En producción no se publica al host.
<code>svaes-engine</code>	—	Motor de verificación Rust (Actix-web, puerto interno 8081). No expone ningún puerto al host.
<code>svaes-worker</code>	—	Proceso Celery que ejecuta los workers de verificación de forma asíncrona.
<code>postgres</code>	5432 [†]	Instancia PostgreSQL 17 con volumen persistente.
<code>redis</code>	—	Broker Redis para Celery y blacklist de tokens JWT .

Tabla 51: Servicios desplegados por Docker Compose ([†]el puerto de PostgreSQL solo se publica al host en desarrollo).

Todos los servicios se comunican a través de una red bridge interna denominada `svaes-internal`. Únicamente el servicio frontend tiene acceso externo, lo que impide el acceso directo a la [API](#), al motor o a la base de datos desde fuera de la red Docker.

Para ejecutar los servicios en segundo plano (modo *daemon*), se añade el flag `-d`:

```
docker compose up --build -d
```

8.1.5. Verificación del Despliegue

Una vez que todos los contenedores están en estado `running`, puede comprobarse que el sistema responde correctamente accediendo a las siguientes URLs (en un despliegue de desarrollo en `localhost`):

- **Interfaz web:** <http://localhost:8880> — debe mostrar la pantalla pública del **SVAES** (*landing*) con enlace al inicio de sesión.
- **API (Swagger UI):** <http://localhost:8000/docs> — muestra la documentación interactiva OpenAPI 3.x de la **API REST**.
- **Salud del backend:** <http://localhost:8000/health> — devuelve **HTTP 200 OK** si todos los subsistemas internos están operativos.
- **Salud del motor:** el motor expone su propio `/health` accesible desde la red interna Docker mediante `docker compose exec svaes-engine curl -s localhost:8081/health`. La respuesta es un JSON con `status: healthy`, `service: svaes-engine` y `version`.

El estado de los contenedores puede consultarse en cualquier momento con:

```
docker compose ps
```

Y los registros de un servicio concreto, por ejemplo el backend, con:

```
docker compose logs svaes-backend --follow
```

Inicialización automática. Durante el primer arranque del backend, se ejecutan automáticamente las siguientes acciones idempotentes:

1. Aplicación de las migraciones Alembic (`alembic upgrade head`) para crear o actualizar el esquema de la base de datos.
2. Creación de la cuenta administradora inicial (`seed_admin_user`) a partir de las variables `ADMIN_EMAIL` y `ADMIN_PASSWORD`, si no existe.
3. Creación del perfil de verificación del sistema (`seed_system_profile`) con las diez reglas estándar RV-01 a RV-10 configuradas con su severidad por defecto.

Si la contraseña almacenada para el ADMIN usa un esquema de *hashing* desactualizado, el sistema vuelve a aplicar la función hash automáticamente en el siguiente inicio de sesión satisfactorio.

8.1.6. Despliegue en Producción

Para desplegar el **SVAES** en un entorno de producción, se combina el fichero de composición base con la sobreescritura de producción:

```
docker compose -f docker-compose.yml \  
    -f docker-compose.prod.yml up -d --build
```

El fichero `docker-compose.prod.yml` aplica las siguientes diferencias respecto al entorno de desarrollo:

- Desactiva los volúmenes de código fuente montados en lectura/escritura, garantizando que las imágenes son inmutables tras la construcción.
- Configura el backend con `ENVIRONMENT=production`, lo que activa las cabeceras [HTTP](#) de seguridad (HSTS, [Content Security Policy \(CSP\)](#), *X-Frame-Options*, *X-Content-Type-Options*, *Referrer-Policy*) y aplica las validaciones estrictas de la configuración.
- Elimina la exposición del puerto de la base de datos al host.
- Elimina la exposición del puerto del backend al host (el acceso se realiza exclusivamente a través del frontend).
- Habilita las políticas de reinicio automático (`restart: always`).
- Establece límites de recursos (CPU y memoria) por contenedor.

Se recomienda colocar un proxy inverso (Nginx, Caddy o Traefik) delante del contenedor frontend para gestionar los certificados TLS, la terminación [HTTPS](#) y la redirección automática [HTTP](#) → [HTTPS](#).

Política de copias de seguridad. Los datos persistentes se almacenan en dos volúmenes Docker: `svaes-postgres-data` (base de datos) y `svaes-redis-data` (estado transitorio de Celery, no crítico). La copia de seguridad de la base de datos se realiza con:

```
docker compose exec postgres pg_dump -U svaes svaes > backup.sql
```

Para programar copias periódicas, se recomienda integrar este comando en una tarea `cron` del host, con rotación diaria y retención semanal/mensual.

8.1.7. Actualización del Sistema

Para actualizar el [SVAES](#) a una nueva versión, basta con obtener los cambios del repositorio y reconstruir las imágenes:

```
git pull origin main
docker compose up --build -d
```

Las migraciones de base de datos se ejecutan automáticamente al arrancar el servicio `svaes-backend` mediante Alembic, por lo que no requieren intervención manual. Si una migración falla, el servicio backend no terminará de arrancar y los detalles del fallo quedarán visibles en sus registros.

Antes de actualizar un entorno productivo, se recomienda:

1. Realizar una copia de seguridad de la base de datos.
2. Revisar el changelog de la versión a la que se va a actualizar.
3. Probar la actualización en un entorno de *staging* idéntico al de producción.

Para detener completamente el sistema:

```
docker compose down
```

Si se desea eliminar también los volúmenes persistentes (base de datos incluida), se añade el flag `-v`. Esta operación es destructiva e irreversible:

```
docker compose down -v
```

8.1.8. Resolución de Problemas Comunes

La Tabla 52 recoge los problemas más habituales observados durante el despliegue del SVAES y las acciones recomendadas para diagnosticarlos.

Síntoma	Diagnóstico y resolución
El backend no arranca y muestra error de conexión a PostgreSQL.	Verificar que el contenedor <code>postgres</code> está en estado <code>healthy</code> con <code>docker compose ps</code> . Si no, revisar sus logs. Comprobar que <code>DATABASE_URL</code> usa el nombre de servicio Docker (<code>postgres</code>), no <code>localhost</code> .
El frontend muestra « <i>Network error</i> » al iniciar sesión.	Comprobar que el backend está accesible desde el contenedor <code>frontend</code> (<code>docker compose exec svaes-frontend curl http://svaes-backend:8000/health</code>). Revisar <code>ALLOWED_ORIGINS</code> .
Las verificaciones nunca terminan y se quedan en estado <code>EN_VERIFICACION</code> .	Comprobar que el worker Celery está vivo (<code>docker compose logs svaes-worker</code>). Verificar conectividad backend → motor y la igualdad de la <code>ENGINE_API_KEY</code> en ambos lados.
El motor rechaza las peticiones con 401 <code>Unauthorized</code> .	La cabecera <code>X-Engine-API-Key</code> enviada por el worker no coincide con el valor configurado en el motor. Asegurar que ambos servicios leen la misma variable <code>ENGINE_API_KEY</code> .
Los correos de activación o reseteo no llegan.	Confirmar que las variables <code>SMTP_*</code> y <code>APP_BASE_URL</code> están definidas. Comprobar en los logs del backend que no hay errores en el envío.
Una migración Alembic falla en producción.	Restaurar la copia de seguridad previa, revisar la migración en un entorno de <i>staging</i> y aplicarla manualmente si es necesario con <code>docker compose exec svaes-backend alembic upgrade head</code> .

Tabla 52: Resolución de problemas comunes durante el despliegue.

8.2. Manual de Usuario

Este manual está dirigido a los usuarios con rol U2 (Usuario estándar), que consultan y operan releases dentro de sus proyectos asignados, así como a los usuarios con rol U4 (Manager de organización) en su faceta de operadores cotidianos. Las tareas específicas de administración de la organización se cubren en el manual de administración ([Sección 8.3](#)). La interfaz de usuario se describe con mayor detalle en la sección 5.7 del capítulo 5.

8.2.1. Acceso al Sistema

Para acceder al [SVAES](#), el usuario navega a la URL pública del sistema en su navegador web. La pantalla inicial es la *landing* pública, desde la cual pueden alcanzarse tres flujos:

- **Iniciar sesión**, para usuarios que ya disponen de cuenta.
- **Solicitar acceso**, para nuevos usuarios u organizaciones ([Subsección 8.2.2](#)).
- Páginas legales (**aviso legal** y **política de privacidad**).

En la pantalla de inicio de sesión se introducen las credenciales: dirección de correo electrónico y contraseña. Tras pulsar *Iniciar sesión*, el sistema valida las credenciales y, si son correctas y la cuenta no tiene 2FA habilitado, redirige al usuario al panel principal (*dashboard*). Si tiene 2FA, se le solicita el código TOTP antes de conceder acceso ([Subsección 8.2.4](#)).

En caso de credenciales incorrectas, el sistema muestra el mensaje genérico «Credenciales incorrectas», sin revelar si el usuario existe o no, conforme a la política de prevención de ataques de enumeración. Tras cinco intentos fallidos consecutivos en una ventana de diez minutos, la cuenta queda bloqueada temporalmente durante quince minutos; este bloqueo no impide el reseteo de la contraseña por correo electrónico.

El endpoint de inicio de sesión está protegido además por un *rate limit* de 30 peticiones por minuto y dirección IP, para mitigar ataques de fuerza bruta.

8.2.2. Solicitud de Acceso al Sistema

El [SVAES](#) permite la creación de cuentas pasando por una solicitud de acceso. Para solicitar una cuenta, el interesado accede al enlace **Solicitar acceso** desde la *landing* y rellena el formulario público con:

- Nombre y apellidos.
- Dirección de correo electrónico de contacto.
- Nombre y descripción de la organización a la que pertenece.

El sistema almacena la solicitud con estado **PENDIENTE**. Si la solicitud es aprobada, el usuario recibe un correo electrónico con un **enlace de activación** que incluye un token temporal con caducidad. Al pulsarlo, el navegador abre la pantalla de activación, donde el usuario define su contraseña personal y queda autenticado en el sistema.

El endpoint de solicitud de acceso está limitado a 10 solicitudes por hora y dirección IP.

8.2.3. Recuperación de Contraseña

Si el usuario ha olvidado su contraseña, en la pantalla de inicio de sesión puede pulsar **¿Olvidaste tu contraseña?**. Se le solicita su dirección de correo electrónico, y si esa dirección existe en el sistema, recibirá un correo con un enlace de *reseteo* que contiene un token de un solo uso. La respuesta del sistema es siempre la misma, independientemente de si el email existe o no, para evitar enumeración de usuarios.

Tras pulsar el enlace, el usuario accede a una pantalla que le permite introducir y confirmar la nueva contraseña. El token caduca tras un periodo limitado, no puede reutilizarse y queda invalidado cuando se completa el reseteo o cuando el usuario cierra todas sus sesiones activas.

El endpoint de solicitud de reseteo está limitado a 5 peticiones por minuto y dirección IP.

8.2.4. Autenticación de Doble Factor

Los usuarios pueden reforzar la seguridad de su cuenta activando la autenticación de doble factor (2FA) basada en códigos TOTP (*Time-based One-Time Password*, RFC 6238). Para activarla, el usuario accede a **Perfil** → **Seguridad** → **Activar 2FA**. El sistema:

1. Genera un secreto TOTP y lo presenta como un código QR.
2. El usuario escanea el QR con una aplicación autenticadora compatible (Google Authenticator, Aegis, 1Password, Bitwarden, Authy, etc.).
3. El usuario introduce el código de seis dígitos generado por la aplicación para confirmar la vinculación. Solo cuando este código se valida con éxito, el 2FA queda definitivamente activado en la cuenta.

Una vez activado el 2FA, el flujo de inicio de sesión es:

1. Introducir email y contraseña.
2. Si las credenciales son válidas, el sistema responde con `requires_2fa: true` y un token temporal de corta vida (*TOTP pending token*).
3. El usuario introduce el código TOTP vigente.
4. El sistema valida ambos factores y emite los tokens de sesión definitivos.

Para desactivar el 2FA, el usuario accede a **Perfil** → **Seguridad** → **Desactivar 2FA**, donde debe confirmar la operación reintroduciendo su contraseña.

8.2.5. Selección de Organización y Proyecto

Un usuario puede pertenecer a varias organizaciones con roles distintos en cada una. Tras autenticarse, si el usuario pertenece a más de una organización, se muestra un selector inicial. Una vez dentro de la organización, el panel principal (*dashboard*) muestra

un resumen del estado de las releases recientes, los proyectos accesibles y las métricas agregadas.

El selector de organización permanece accesible desde la barra superior, lo que permite cambiar de organización sin cerrar sesión. Los proyectos a los que el usuario tiene acceso se listan en la sección **Proyectos** del menú lateral. La visibilidad de cada proyecto depende del rol del usuario dentro de la organización y, en su caso, de su asignación explícita al proyecto.

Restricciones de navegación por rol. La navegación está restringida en función del rol del usuario:

- Los usuarios con rol U3 (Administrador global) son redirigidos automáticamente al área de administración ([Sección 8.3](#)) en lugar de a las vistas de negocio.
- Los demás roles (U2, U4) acceden a la totalidad de las vistas operativas dentro de su ámbito.

8.2.6. Panel Principal (Dashboard)

El panel principal ofrece una visión global del estado de la organización. Sus componentes principales son:

- **Resumen numérico:** número total de releases, releases por estado, número de proyectos activos y número de verificaciones ejecutadas en los últimos 30 días.
- **Releases recientes:** tarjetas con las últimas releases creadas o modificadas, su estado y un acceso directo a su detalle.
- **Métricas de verificación:** porcentaje de verificaciones con veredicto **VALIDA**, **NO_VALIDA** o **CON_INCIDENCIAS**.

Las métricas se obtienen de un único endpoint agregado del backend (`GET /api/v1/dashboard/metrics`) que devuelve la información ya calculada para minimizar la carga del cliente.

8.2.7. Gestión de Releases

Una *release* representa una entrega de software que será sometida a verificación automática. El ciclo de vida de una release comprende los estados recogidos en la [Tabla 53](#).

Estado	Significado
BORRADOR	Release en preparación; puede modificarse libremente y aún no es candidata a verificación.
PENDIENTE	Release preparada para lanzar verificación; sus datos están fijados pero todavía no se ha iniciado la evaluación.

Estado	Significado
EN_VERIFICACION	Verificación en curso; la release queda inmutable hasta que termine la evaluación o se cancele.
VALIDA	Verificación completada satisfactoriamente. Todas las reglas obligatorias en OK.
CON_ADVERTENCIAS	Verificación completada con incidencias no bloqueantes (reglas opcionales en ERROR o WARNING).
NO_VALIDA	Verificación rechazada por incumplir una o más reglas obligatorias.
ARCHIVADA	Release finalizada; estado inmutable. Puede ser restaurada por un usuario autorizado.

Tabla 53: Estados del ciclo de vida de una release.

Crear una release. Desde la vista de proyecto, se pulsa **Nueva release**. El formulario solicita:

- Nombre identificativo y número de versión (único dentro del proyecto).
- Descripción libre de la entrega.
- Perfil de verificación que se aplicará (por defecto, el activo del proyecto).

Tras guardar, la release queda en estado **BORRADOR**.

Añadir artefactos. En la vista de detalle de la release, la sección **Artefactos** permite registrar los elementos que serán verificados: identificadores de commits o *merge requests* en el repositorio de código, identificadores de tareas o *issues* en el gestor de tareas, identificadores de documentos en el sistema documental, identificadores de tickets de cambio, etc. Cada artefacto se asocia explícitamente al conector que lo proporciona y a su categoría funcional.

Existen dos formas de añadir artefactos:

1. **Alta manual:** el usuario introduce el identificador externo y la descripción del artefacto.
2. **Descubrimiento asistido:** el usuario pulsa **Explorar conector** y el sistema utiliza el conector seleccionado para listar los elementos candidatos en el sistema externo, con paginación. El usuario marca los que desea incluir.

Modificar y eliminar. Una release en estado **BORRADOR** puede modificarse en cualquier momento. Las releases en estado **EN_VERIFICACION** son inmutables. Las releases en cualquier estado terminal pueden archivarse pulsando **Archivar**; una release archivada puede restaurarse de forma controlada mediante la opción **Restaurar** de la vista de detalle.

8.2.8. Lanzamiento de una Verificación

Para lanzar una verificación, el usuario accede a la vista de detalle de la release y pulsa **Verificar**. El sistema responde de forma inmediata aceptando la solicitud ([HTTP 202 Accepted](#)) y devuelve un `task_id` con el que es posible consultar el progreso. La release transita al estado `EN_VERIFICACION`.

Internamente, la [API](#) encola la tarea en Celery, que recupera la release y sus artefactos, contacta con los conectores configurados para obtener los datos externos y envía el conjunto al motor de verificación Rust. El motor evalúa en paralelo las reglas configuradas en el perfil activo de la release y devuelve el resultado, que se persiste en la base de datos y desencadena la transición de estado y, si procede, el envío de notificaciones.

La Tabla 54 describe las diez reglas que el motor implementa.

ID	Nombre	Severidad	Comprueba que...
RV-01	Existencia de artefactos declarados	ERROR	cada tipo de artefacto declarado en el perfil tiene al menos un elemento en el resultado recuperado del conector.
RV-02	Estado final de artefactos	ERROR	todos los artefactos se encuentran en un estado final permitido por el perfil (p. ej., <code>DONE</code> , <code>CLOSED</code>).
RV-03	Cobertura documental obligatoria	ERROR	existe al menos un documento por cada tipo documental obligatorio definido en el perfil de verificación.
RV-04	Coherencia de versión entre artefactos	WARNING	la versión declarada en la <i>release</i> coincide con el campo de versión de cada artefacto externo recuperado.
RV-05	Integridad de campos obligatorios	ERROR	los campos obligatorios configurados están presentes y no vacíos en todos los artefactos del tipo indicado.
RV-06	Compleitud del gestor de tareas	WARNING	las tareas del gestor de tareas están presentes y en estado <code>COMPLETE</code> .
RV-07	Trazabilidad bidireccional	WARNING	cada tarea referenciada por un artefacto tiene un campo <i>back-reference</i> que apunta correctamente a la <i>release</i> en el sistema externo.
RV-08	Vigencia temporal de artefactos	WARNING	ningún artefacto supera el umbral máximo de días desde su última modificación.

ID	Nombre	Severidad	Comprueba que...
RV-09	Unicidad de identificadores externos	ERROR	no existen dos artefactos de la misma <i>release</i> con el mismo identificador externo en el mismo conector.
RV-10	Aprobación formal de la entrega	WARNING	existe al menos una aprobación formal registrada en el sistema de gestión de cambios para la versión verificada.

Tabla 54: Reglas de verificación disponibles en el **SVAES** (RV-01 a RV-10), su severidad por defecto y nivel de aplicación.

Cada regla del perfil se configura con un nivel de aplicación {OBLIGATORIA, OPCIONAL, EXCLUIDA}. Una regla EXCLUIDA no se evalúa. Los parámetros concretos de cada regla (tipos de artefacto, campos, estados aceptados, etc.) se definen al añadir la regla al perfil de verificación ([Subsección 8.3.4](#)).

La interfaz de usuario actualiza el estado de la verificación mediante consultas periódicas (*polling*) al endpoint `GET /api/v1/tasks/{task_id}`, mostrando un indicador de progreso. Si el usuario necesita interrumpir una verificación en curso, puede pulsar **Cancelar**, lo que envía una petición de cancelación al worker y restaura el estado previo de la release.

8.2.9. Consulta de Resultados

Tras completar la verificación, la vista de detalle de la release muestra el resultado desglosado por regla. Para cada regla evaluada se indica:

- **Estado:** OK, WARNING, ERROR o NOT_EVALUATED (cuando el conector requerido está inactivo o el motor no ha podido evaluar la regla).
- **Severidad aplicada:** nivel con el que la regla participó en la agregación del veredicto.
- **Mensaje:** descripción del resultado, incluyendo el artefacto que ha motivado la incidencia cuando procede.
- **Resumen agregado:** número total de reglas, conteo por estado y veredicto global, en formato legible y reproducible.

El veredicto global se calcula a partir de las severidades configuradas y los estados resultantes, conforme a la matriz de decisión del agregador. Una única regla OBLIGATORIA en estado ERROR es suficiente para emitir el veredicto NO_VALIDA; los WARNING y los errores en reglas OPCIONAL generan CON_ADVERTENCIAS cuando no hay errores obligatorios.

Historial inmutable. El historial de verificaciones previas de una release puede consultarse en la pestaña **Historial** de su vista de detalle. Cada entrada del historial es inmutable: contiene una *instantánea* del perfil aplicado en el momento de la evaluación, lo que garantiza que un resultado pasado puede reproducirse y auditarse incluso si el perfil

se modifica posteriormente.

Exportación de resultados. Los resultados pueden exportarse en formato **PDF por release**, el cual contiene un informe formal de una verificación concreta, adecuado para entregar como evidencia a un cliente o auditor. La exportación se accede desde el botón **Exportar PDF** de la vistas de release.

8.2.10. Gestión del Perfil de Usuario

Cada usuario puede gestionar su propio perfil accediendo al menú **Perfil** de la esquina superior derecha. Las opciones disponibles son:

- **Datos personales:** modificación del nombre visible (*display name*). La dirección de correo es inmutable y solo puede modificarla un administrador global.
- **Contraseña:** cambio de contraseña previa verificación de la contraseña actual. La nueva contraseña debe cumplir la política de seguridad del sistema (longitud mínima, complejidad).
- **Doble factor (2FA):** activación o desactivación de la autenticación TOTP, según se describe en [Subsección 8.2.4](#).
- **Notificaciones:** gestión de las suscripciones del usuario a los distintos tipos de evento del sistema. Cada tipo de evento puede habilitarse o deshabilitarse de forma independiente. Solo se envían notificaciones al usuario para los eventos a los que está suscrito y únicamente a través de los canales que su organización tenga configurados como activos.
- **API Keys:** gestión de las claves de acceso programático a la [API REST](#). Cada usuario puede tener varias claves activas simultáneamente. El valor completo de la clave solo se muestra en el momento de su creación; posteriormente solo es visible su prefijo y su nombre. Las claves pueden revocarse en cualquier momento.
- **Cerrar sesión:** invalida el token actual añadiéndolo a la *blacklist* en Redis hasta su expiración natural.
- **Eliminar cuenta:** opción autoservida que permite al usuario eliminar su propia cuenta de forma permanente. Esta opción no está disponible para el Administrador global (U3). El proceso consta de los siguientes pasos:
 1. El sistema verifica que el usuario no es el administrador global y muestra el botón **Eliminar mi cuenta** en la sección de perfil.
 2. Al pulsarlo, el sistema presenta un resumen de las organizaciones donde el usuario actúa como *owner*, indicando para cada una la acción prevista:
 - Si la organización tiene otros miembros con rol U4 (manager), la propiedad se transferirá automáticamente al miembro con mayor antigüedad.
 - Si el usuario es el único miembro de la organización, se advierte de que la

organización será eliminada junto con la cuenta.

3. El usuario debe confirmar la operación introduciendo su contraseña actual y, si tiene habilitado el doble factor (2FA), un código TOTP válido.
4. Una vez confirmada, el sistema ejecuta las transferencias de propiedad necesarias, elimina las organizaciones sin otros miembros, borra la cuenta del usuario, invalida todas sus sesiones activas y registra la operación en la auditoría.

Esta acción es irreversible.

8.3. Manual de Administración

Este manual está dirigido a los usuarios con roles U4 (Manager de organización) y U3 (Administrador global). El Manager de organización gestiona los recursos de su propia organización; el Administrador global tiene acceso global a todas las organizaciones y a la configuración del sistema. La Tabla 55 resume los cuatro roles del sistema y sus capacidades principales.

ID	Nombre	Capacidades principales
U1	Invitado	Sin acceso autenticado. Puede acceder al formulario público de solicitud de acceso y consultar la landing page del sistema.
U2	Usuario estándar	Creación y modificación de releases, alta de artefactos y lanzamiento de verificaciones en los proyectos a los que está asignado.
U4	Manager de organización	Todas las capacidades de Usuario estándar más gestión de proyectos, conectores, perfiles de verificación, plantillas, canales de notificación y miembros de su organización.
U3	Administrador global	Supervisión global del sistema. Consulta de logs, estadísticas anonimizadas, recarga de reglas en caliente, estado de servicios, gestión de solicitudes de acceso y auditoría. No gestiona organizaciones ni usuarios directamente ni ejecuta releases, al no pertenecer a ninguna organización.

Tabla 55: Roles del sistema [SVAES](#) y sus capacidades principales.

8.3.1. Gestión de la Organización

Desde el menú **Organización** → **Ajustes**, el Manager de organización puede modificar el nombre y la descripción de la organización, así como consultar el identificador único (`organization_id`) necesario para integraciones externas. Desde esta vista también puede

iniciarse la **transferencia de propiedad** de la organización a otro miembro, operación que requiere doble confirmación.

Desde el menú **Organización** → **Proyectos**, se gestionan los proyectos de la organización:

- **Crear proyecto:** se especifican nombre, descripción y el perfil de verificación por defecto.
- **Archivar y desarchivar proyecto:** un proyecto archivado no admite nuevas releases, y las releases existentes pasan a modo solo lectura (no se pueden modificar, re-verificar ni cambiar de estado). Conserva todo su historial. Puede desarchivarse en cualquier momento.
- **Asignar miembros:** desde la vista de detalle del proyecto, se añaden o eliminan miembros y se les asigna su rol dentro del proyecto.

8.3.2. Gestión de Usuarios y Roles

Desde **Organización** → **Miembros**, el Manager de organización gestiona los miembros de su organización. Para invitar a un nuevo miembro, el Manager de organización pulsa **Invitar miembro**, introduce el correo electrónico del usuario (que debe tener cuenta en el sistema) y selecciona su rol dentro de la organización. Un usuario puede pertenecer a múltiples organizaciones con distintos roles en cada una.

Las acciones disponibles sobre un miembro de la organización son:

- Cambiar su rol dentro de la organización.
- **Expulsar** al miembro de la organización (la cuenta global del usuario permanece activa). Un usuario con rol U4 (manager) no puede expulsar a otro miembro que también ostente el rol U4; solo el *owner* puede expulsar a otros managers. La expulsión requiere confirmación explícita.

Roles personalizados. Además de los cuatro roles estándar, el Manager de organización puede definir *roles personalizados* desde **Organización** → **Roles**. Un rol personalizado se construye seleccionando subconjuntos de los permisos granulares del sistema (p. ej., *ver releases*, *lanzar verificaciones*, *gestionar conectores*, etc.) y permite ajustar la matriz de autorización a las necesidades específicas de la organización. Los roles personalizados conviven con los roles estándar y pueden asignarse a cualquier miembro.

8.3.3. Gestión de Conectores

Los conectores permiten al **SVAES** comunicarse con herramientas externas para recuperar los artefactos que serán verificados. Desde **Organización** → **Conectores**, el Manager de organización puede registrar, configurar y probar conectores.

El sistema implementa veinte conectores agrupados en cinco categorías funcionales, recogidos en la Tabla 56.

Categoría	Implementaciones disponibles
GESTOR_TAREAS	Jira Software, Linear, Trello, Asana.
REPO_CODIGO	GitHub, GitLab, Bitbucket, Gitea.
SISTEMA_DOCUMENTAL	Confluence, Notion, Wiki.js, BookStack.
HERRAMIENTA_PLANIFICACION	ClickUp, Taiga, Plane, Miro.
GESTION_CAMBIOS	Jira Service Management, GLPI, Zammad, Redmine.

Tabla 56: Catálogo de conectores disponibles en el [SVAES](#) por categoría funcional.

Registrar un conector. Se pulsa **Nuevo conector** y se rellena el formulario:

1. **Nombre:** identificador único del conector dentro de la organización (entre 3 y 80 caracteres).
2. **Tipo funcional:** categoría del sistema externo (GESTOR_TAREAS, REPO_CODIGO, SISTEMA_DOCUMENTAL, HERRAMIENTA_PLANIFICACION o GESTION_CAMBIOS).
3. **Implementación:** clase concreta del conector, seleccionada entre las veinte implementaciones disponibles.
4. **URL base:** dirección del servidor del sistema externo (esquema [HTTPS](#) obligatorio en producción).
5. **Mecanismo de autenticación:** tipo de credencial requerido por el sistema externo (BASIC, PAT, TOKEN_BEARER o API_KEY).
6. **Credenciales:** las credenciales se cifran en reposo mediante cifrado Fernet con la clave ENCRYPTION_KEY del sistema, y no son recuperables una vez guardadas. Para rotarlas, se introducen nuevas credenciales desde la opción **Editar**.

Probar un conector. Una vez registrado, se pulsa **Probar conexión** para verificar que el [SVAES](#) puede comunicarse con el sistema externo usando las credenciales proporcionadas. El resultado se muestra como *Conexión exitosa* o como un mensaje de error con la causa del fallo (credenciales inválidas, URL inaccesible, *timeout*, certificado no válido, etc.). La marca temporal del último test satisfactorio queda registrada en el conector.

Explorar el conector. Pulsando **Explorar**, el Manager de organización puede listar los elementos del sistema externo (con paginación) que el conector es capaz de recuperar. Esta función sirve para validar que el conector está bien configurado y para que los operadores puedan después descubrir artefactos al crear releases.

Activar y desactivar. El estado de cada conector (**ACTIVO** / **INACTIVO**) se muestra en el listado. Solo los conectores en estado **ACTIVO** participan en las verificaciones; los inactivos generan un resultado **NOT_EVALUATED** en las reglas que los referencian. Desactivar un conector es útil para realizar mantenimiento o rotación de credenciales sin perder su configuración.

8.3.4. Gestión de Perfiles de Verificación

Un perfil de verificación agrupa las reglas que se aplicarán a las releases de un proyecto. Desde **Organización** → **Perfiles**, el Manager de organización puede crear y configurar perfiles. El sistema incluye un *perfil de sistema* con las diez reglas estándar y su configuración por defecto, que sirve como punto de partida para nuevos perfiles.

Para cada perfil se especifican:

- **Nombre** y descripción.
- **Reglas habilitadas:** selección de las reglas RV-01 a RV-10 que formarán parte de la verificación, cada una con su nivel {**OBLIGATORIA**, **OPCIONAL**, **EXCLUIDA**}.
- **Parámetros por regla:** cada regla puede configurarse de forma independiente. Por ejemplo, RV-03 requiere indicar el tipo de artefacto y los estados permitidos; RV-08 requiere el artefacto maestro y el campo lista.
- **Conector asociado:** las reglas que consultan sistemas externos requieren indicar el conector que se usará para recuperar la información.
- **Orden de evaluación:** las reglas pueden reordenarse arrastrando y soltando para definir el orden de presentación en los informes.

Las operaciones soportadas sobre un perfil son: crear, editar, duplicar (para construir variantes), establecer como perfil por defecto del proyecto y eliminar. Un perfil puede asignarse como perfil por defecto de uno o varios proyectos.

Plantillas de proyecto. Para acelerar la creación de proyectos similares, el Manager de organización puede definir *plantillas* desde **Organización** → **Plantillas**. Una plantilla encapsula un perfil de verificación y una convención de nombrado de proyecto. Al crear un proyecto a partir de una plantilla, se replica automáticamente el perfil de verificación, lo que evita configurar manualmente las reglas en cada nuevo proyecto.

8.3.5. Gestión de Notificaciones

El **SVAES** envía notificaciones a los usuarios cuando ocurren eventos relevantes (verificación completada, release archivada, conector con error, etc.). El Manager de organización gestiona los **canales** a través de los cuales se envían dichas notificaciones, mientras que cada usuario gestiona sus **suscripciones** individuales ([Subsección 8.2.10](#)).

Desde **Organización** → **Notificaciones** → **Canales**, el Manager de organización puede:

- Añadir un canal de notificación, indicando su tipo (p. ej., correo electrónico) y su configuración específica.
- Habilitar o deshabilitar un canal de forma temporal.
- Eliminar canales obsoletos.

Un usuario solo recibe una notificación si está suscrito al evento *y* la organización tiene un canal activo del tipo correspondiente.

8.3.6. Administración Global (Rol Administrador global)

El usuario con rol U3 (Administrador global) dispone de un área de administración global accesible desde **Administración** en el menú principal. Esta área no está disponible para los usuarios con rol Manager de organización. Tras el inicio de sesión, los usuarios Administrador global son redirigidos automáticamente a esta sección.

Gestión de organizaciones. El Administrador global puede listar y consultar las organizaciones existentes en el sistema. No puede crear, restaurar ni desactivar organizaciones.

Monitorización y estadísticas. Desde **Administración** → **Dashboard**, el Administrador global accede a estadísticas anonimizadas sobre el uso del sistema: número de organizaciones, usuarios activos, releases ejecutadas y verificaciones realizadas en el último mes. Estas métricas no contienen datos personales ni identificables.

Gestión de solicitudes de acceso. Desde **Administración** → **Solicitudes de acceso**, el Administrador global gestiona las solicitudes recibidas a través del formulario público (Subsección 8.2.2). Cada solicitud puede:

- **Aprobarse:** el sistema crea la cuenta de usuario y, en su caso, la organización, y envía un correo de activación con un token de un solo uso. La solicitud transita al estado APROBADA.
- **Rechazarse:** el Administrador global proporciona un motivo de rechazo. La solicitud transita al estado RECHAZADA y el solicitante recibe una notificación con la causa.

Auditoría. Todas las acciones relevantes del sistema quedan registradas en una tabla inmutable de auditoría (`audit_log`). El sistema captura más de cuarenta tipos de eventos agrupados por dominio: autenticación (login, login fallido, logout, refresh, activación/desactivación de 2FA, cambio de contraseña), gestión de usuarios y organizaciones, ciclo de vida de releases, conectores y perfiles, gestión de templates y roles personalizados, y eventos de API keys y solicitudes de acceso.

Desde **Administración** → **Auditoría**, el Administrador global puede consultar el historial completo, con filtros por evento, usuario, organización, recurso y rango temporal. Cada entrada incluye marca temporal, usuario actor, tipo de recurso, identificador del recurso,

dirección IP de origen (pseudoanonimizada mediante hash con sal por instancia) y un objeto JSON con los detalles específicos del evento.

Configuración y recarga de reglas. Desde **Administración** → **Configuración**, el Administrador global puede ajustar parámetros globales del sistema que no se distribuyen por variables de entorno, así como recargar en caliente las reglas personalizadas del motor de verificación sin necesidad de reiniciar el servicio.

Estado de servicios y logs. Desde **Administración** → **Servicios**, el Administrador global puede visualizar el estado de cada microservicio del sistema (API, motor de verificación, worker Celery, base de datos, Redis) así como los logs agregados de los contenedores. Esta vista permite diagnosticar problemas operativos sin acceder directamente a la infraestructura.

Acceso programático a la API REST. El **SVAES** expone la totalidad de sus operaciones a través de una **API REST** documentada en OpenAPI 3.x. La interfaz Swagger UI, accesible en el endpoint `/docs`, permite explorar y ejecutar llamadas directamente desde el navegador. La autenticación se realiza mediante dos mecanismos intercambiables:

- **Bearer JWT:** token de acceso obtenido mediante el flujo de login. Cabecera `Authorization: Bearer <token>`.
- **API key:** clave generada por el usuario desde la sección **API Keys** de su perfil. Cabecera específica conforme al contrato OpenAPI. Las API keys habilitan integraciones permanentes con sistemas externos (CI/CD, scripts, etc.) sin depender de credenciales interactivas. Cada API key se persiste cifrada (solo el hash se almacena) y su valor en claro únicamente se muestra en el momento de creación.

9 Planificación y Presupuesto

9.1. Introducción

Este capítulo cierra la parte ingenieril de la memoria recogiendo los tres artefactos de gestión del proyecto: la **planificación temporal** (comparando la versión *ideal* cerrada al inicio con la versión *real* efectivamente ejecutada), el **estudio de riesgos** elaborado durante la fase de predesarrollo y revisado en cada hito, y el **presupuesto** construido siguiendo la metodología estándar de presupuestación de proyectos software. El objetivo es doble: justificar las decisiones de gestión adoptadas y proporcionar trazabilidad cuantitativa entre el esfuerzo, los riesgos materializados y el coste imputable al proyecto.

Por su carácter académico, [SVAES](#) se ejecuta con un único recurso humano (el autor en el rol de Desarrollador Full Stack Junior) y un calendario fijo impuesto por la convocatoria de entrega del TFG. Estos dos condicionantes determinan tanto la planificación como el análisis de riesgos: la duración total no es elástica y el coste de oportunidad de cualquier desviación se concentra en la disponibilidad de tiempo del propio autor.

9.2. Planificación temporal

9.2.1. Marco metodológico

La planificación se ha construido siguiendo las directrices de la asignatura *Gestión de Proyectos* de la EII de Oviedo, adaptadas al carácter unipersonal del proyecto. El plan se ha modelado en Microsoft Project y se ha gestionado mediante una estructura de descomposición del trabajo (EDT/WBS) de cinco niveles, una línea base congelada al cierre del predesarrollo y un seguimiento quincenal frente a esa línea base.

Las dimensiones temporales clave del proyecto son:

- **Fecha de inicio:** 20 de marzo de 2026.
- **Entrega del TFG:** 6 de julio de 2026.
- **Hito intermedio:** memoria finalizada el 30 de junio de 2026.
- **Duración total:** 15 semanas naturales (aprox. 3,7 meses).
- **Calendario laboral:** jornada parcial de 4,5 horas/día de media de lunes a viernes, ajustada a la compatibilidad con docencia y exámenes. Con esta dedicación, la capacidad máxima asciende a 337,5 horas (15 semanas $\times$ 22,5 h/semana).
- **Esfuerzo total estimado (línea base):** 329 horas repartidas en 67 tareas (56 de ejecución más 11 hitos) agrupadas en 14 fases.

9.2.2. Estructura de descomposición del trabajo (EDT)

La EDT agrupa las 67 tareas en dos grandes ramas: una de *hitos* (WBS 1.1) que actúa como cuadro de mandos del avance, y otra de *ejecución* (WBS 1.2) que descompone el trabajo en catorce fases. La tabla 57 resume la estructura.

WBS	Fase	Descripción resumida	Tareas	Horas
1.1	Hitos	Once hitos de control (inicio, cierres de fase, entrega final).	11	—
1.2.1	Reunión de inicio	Sesión inaugural con la tutora.	1	4
1.2.2	Canales comunicación	Definición de canales tutor-alumno.	1	4
1.2.3	Revisiones	Nueve reuniones de seguimiento.	9	36
1.2.4	Formación	Estudio de ecosistemas y APIs.	2	4
1.2.5	Estado del arte	Fundamentos y plataformas ALM /CI-CD.	2	4
1.2.6	Análisis	SRS, casos de uso, alcance.	6	19
1.2.7	Diseño	Arquitectura, conectores, datos, UI.	5	17
1.2.8	Implementación	Backend, motor, frontend, conectores.	6	152
1.2.9	Pruebas	Unitarias, integración, rendimiento, QG.	5	25
1.2.10	Despliegue	Entorno controlado y pruebas finales.	3	10
1.2.11	Doc. de gestión	Planificación, riesgos y presupuesto.	3	10
1.2.12	Documentación TFG	Estructura y diez capítulos de la memoria.	11	30
1.2.13	Revisión final	Revisión con la tutora y correcciones.	1	4
1.2.14	Cierre	Preparación del entregable.	1	10
Total línea base (ideal)				329

Tabla 57: Estructura de descomposición del trabajo (resumen, nivel 3).

9.2.3. Hitos del proyecto

Los hitos actúan como puntos de control binarios (alcanzado / no alcanzado) y sirven de mecanismo de comunicación con la tutora. La tabla 58 los detalla.

ID	Descripción	Fecha
H1	Inicio del proyecto	20-03-2026
H2	Formación técnica y familiarización completada	24-03-2026
H3	Estado del arte documentado	30-03-2026
H4	Alcance definitivo cerrado	24-04-2026
H5	Diseño del sistema aprobado	24-04-2026
H6	Prototipo funcional operativo	27-05-2026
H7	Sistema completo desarrollado	29-05-2026
H8	Validación de escenarios con datos de prueba	29-05-2026
H9	Despliegue en entorno controlado	01-06-2026
H10	Memoria del TFG finalizada	30-06-2026
H11	Entrega del TFG (cierre del proyecto)	06-07-2026

Tabla 58: Hitos del proyecto (rama 1.1 de la EDT).

9.2.4. Planificación inicial (ideal)

La línea base se congeló tras la elaboración del catálogo de requisitos y antes de iniciar el diseño. La figura 19 muestra el diagrama de Gantt de la planificación ideal a nivel de fase.

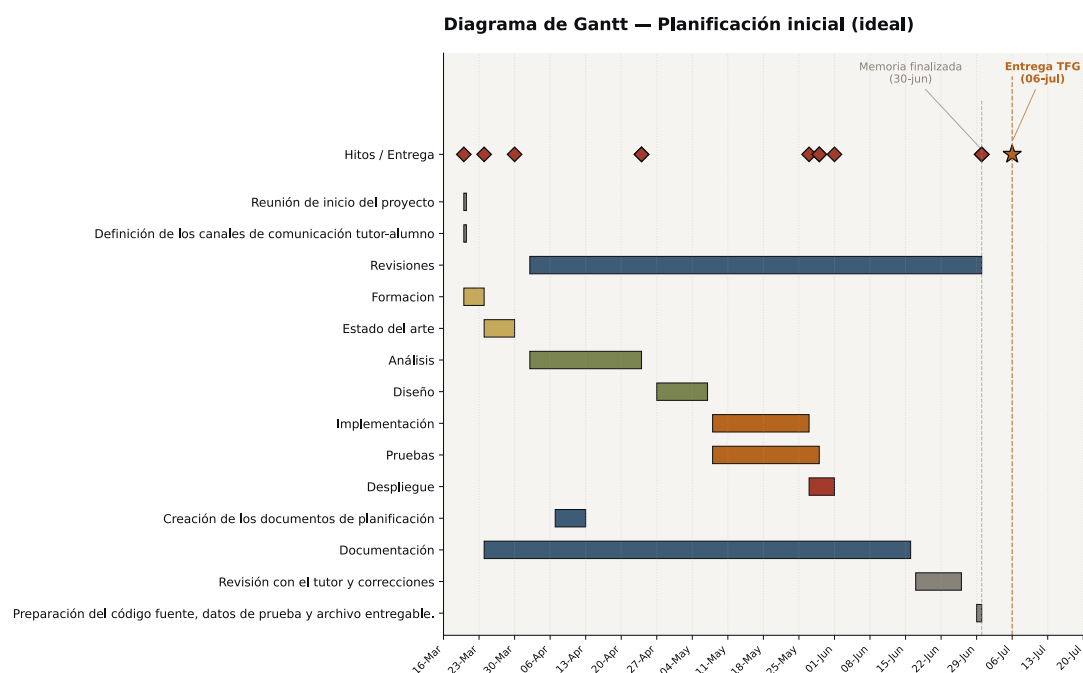


Figura 19: Diagrama de Gantt de la planificación inicial (ideal).

Esta línea base refleja un encadenamiento clásico predesarrollo → desarrollo → postdesarrollo, con dos solapamientos deliberados:

1. La *redacción de la memoria* arranca en paralelo a la fase de análisis (24 de marzo) y se mantiene activa hasta el 30 de junio, dejando una holgura de una semana entre la finalización del documento y la entrega final del 6 de julio.
2. Las fases de *implementación* y *pruebas* se solapan parcialmente, siguiendo un enfoque de pruebas iterativas por componente más una pasada final de integración entre el 27 de mayo y el 1 de junio.

9.2.5. Planificación real (ejecutada)

La figura 20 muestra el Gantt resultante de la ejecución real, registrada en la herramienta a partir de las revisiones quincenales.

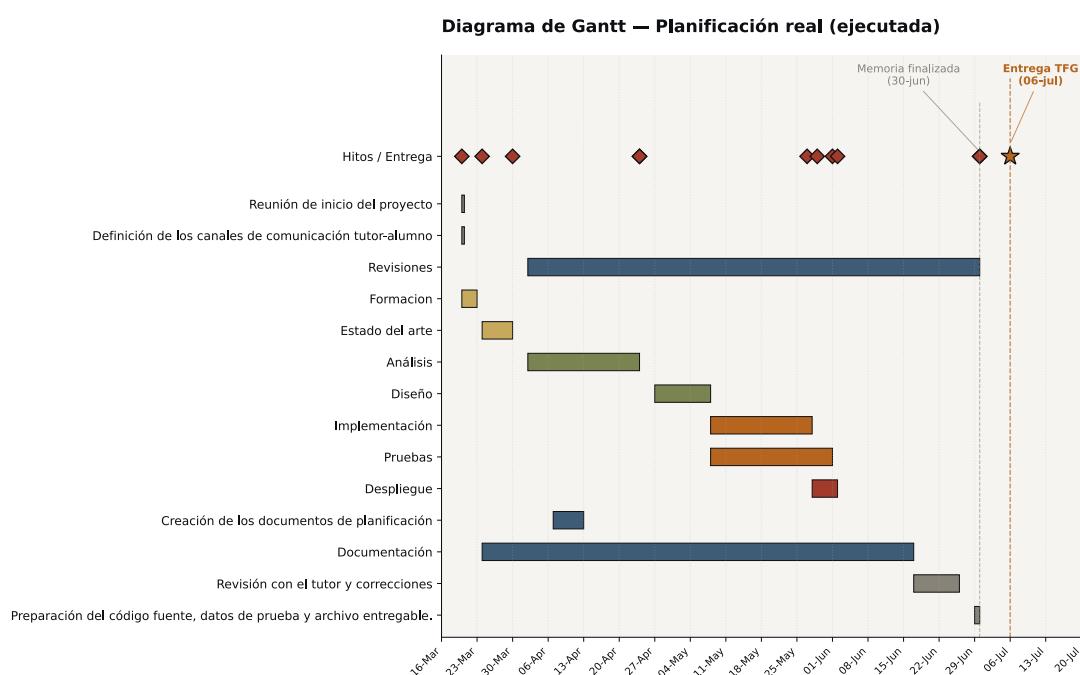


Figura 20: Diagrama de Gantt de la planificación real (ejecutada).

La distribución del esfuerzo total por fase, una vez ejecutadas todas las tareas, se muestra en la tabla 59. La fase de *implementación* concentra aproximadamente el 45 % del esfuerzo total (339 horas), seguida del bloque de *revisiones* y *documentación*, confirmando la naturaleza desarrollo-intensiva del proyecto.

9.2.6. Comparativa Ideal vs. Real y desviaciones

Las desviaciones entre la planificación ideal y la real se documentan en la tabla 59.

La tabla 59 cuantifica las desviaciones significativas detectadas. La métrica utilizada es la diferencia en días naturales (Δ_{inicio} , Δ_{fin}) y la variación porcentual de esfuerzo (Δ_{esf}).

Fase	Δ_{inicio}	Δ_{fin}	Δ_{esf}	Causa raíz
Formación (1.2.4)	0 d	-1 d	+4 h	Mayor dedicación al estudio de ecosistemas heterogéneos al detectarse más casuísticas relevantes de las esperadas.
Diseño (1.2.7)	0 d	+1 d	0 h	Ajustes en el modelo de datos amplían el cierre del diseño un día sin alterar el esfuerzo total.
Implementación (1.2.8)	0 d	+1 d	+2 h	Implementación del frontend más laboriosa de lo previsto en pantallas con lógica RBAC.
Pruebas (1.2.9)	0 d	+2 d	0 h	Encadenamiento con implementación obliga a desplazar las pruebas finales al inicio de junio.
Despliegue (1.2.10)	+1 d	+1 d	0 h	Arrastre del retraso de las pruebas de integración.
Documentación (1.2.12)	0 d	+1 d	+4 h	Capítulos 5 y 8 amplían su dedicación tras las revisiones con la tutora.
Resto de fases	0 d	0 d	0 h	Sin desviación significativa.
Total proyecto	0 d	0 d	+10 h	La entrega permanece el 06-07-2026; el incremento de esfuerzo se absorbe sin reprogramar hitos.

Tabla 59: Desviaciones de la planificación real respecto a la línea base.

Análisis de las desviaciones. Las desviaciones detectadas son acotadas y se concentran en la mitad final del proyecto. Ninguna afecta a la fecha de entrega del TFG (6 de julio de 2026), ya que la holgura entre la finalización de la memoria (30 de junio) y la entrega amortigua los +2 d de pruebas y los +1 d de implementación, despliegue y documentación. El esfuerzo total real (339 horas) supera al estimado en línea base (329 horas) en un 3,0 %, dentro del margen razonable para un proyecto académico individual de este tamaño.

9.3. Estudio de riesgos

9.3.1. Metodología y modelo de valoración

El estudio sigue la metodología de gestión de riesgos del PMBOK, complementada con las recomendaciones de la guía PMBOK para proyectos individuales. Para cada riesgo y oportunidad se evalúan:

- **Probabilidad (P)**: codificada en cinco niveles {Muy Baja: 0,1; Baja: 0,3; Media: 0,5; Alta: 0,7; Muy Alta: 0,9}.
- **Impacto (I)**: valor máximo entre los impactos parciales en cuatro dimensiones del proyecto (*presupuesto, planificación, alcance, calidad*), con escala {Inapreciable: 0,05; Bajo: 0,15; Medio: 0,30; Alto: 0,55; Crítico: 0,90}.
- **Valor de riesgo**: $V = P \cdot I$.
- **Priorización**: un riesgo se prioriza para tratamiento formal cuando $V \geq 0,15$.
- **Estrategia**: una de {eliminar, mitigar, transferir, asumir} para riesgos y {explotar, mejorar, compartir} para oportunidades.

La tabla 60 lista los diez riesgos y las dos oportunidades identificadas con su valoración de probabilidad e impacto.

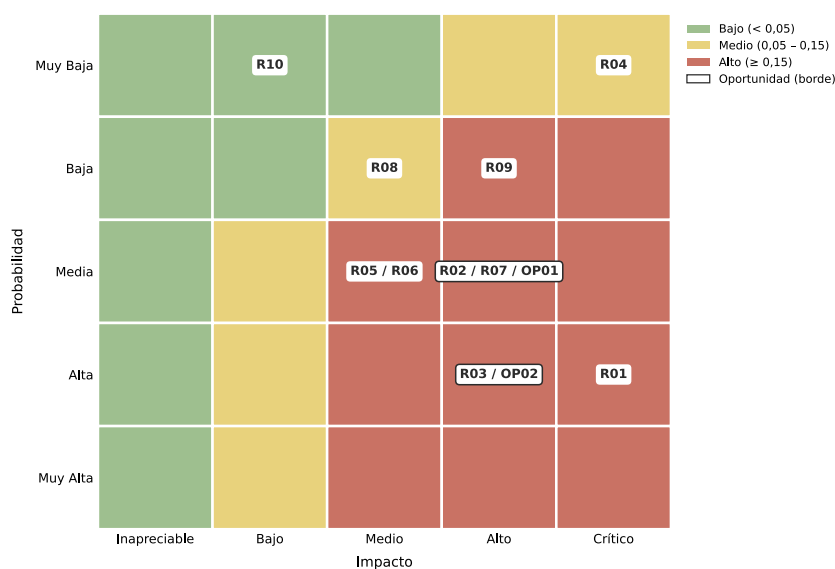


Figura 21: Matriz Probabilidad $\times$ Impacto con los riesgos y oportunidades identificados.

9.3.2. Distribución y priorización

El registro de riesgos en la tabla 60 proporciona el detalle completo de cada elemento.

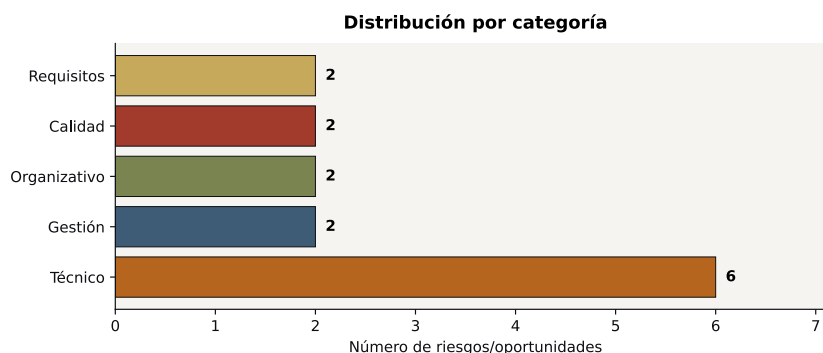


Figura 22: Distribución de riesgos y oportunidades por categoría.

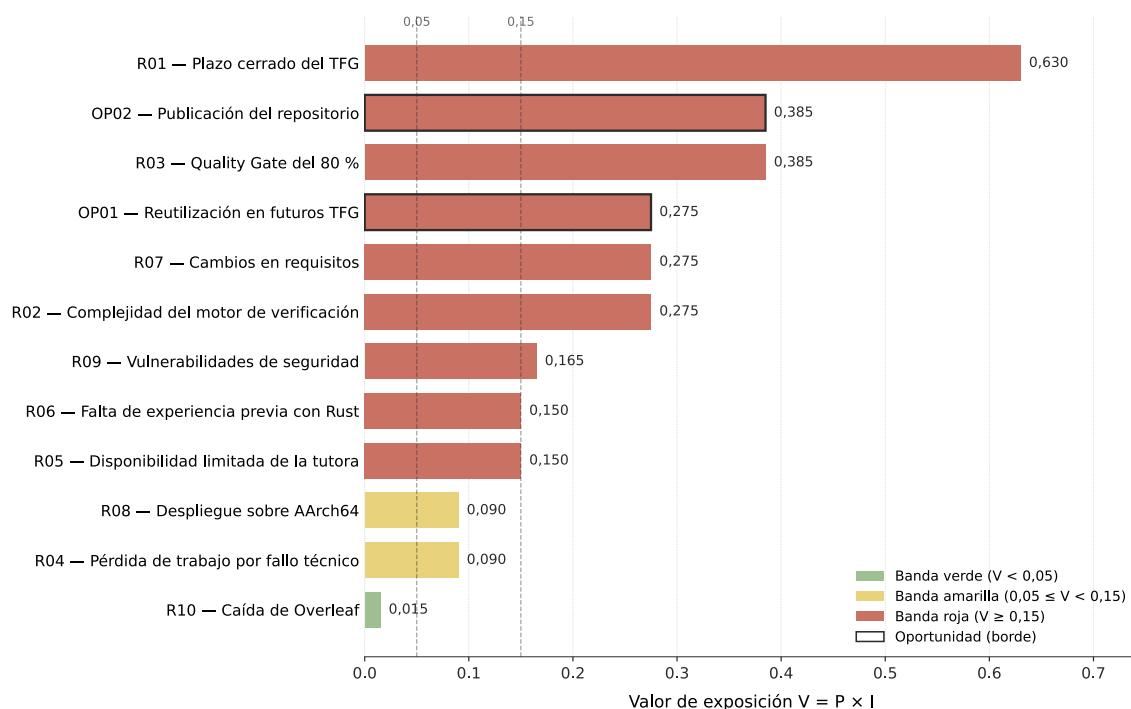


Figura 23: Priorización por valor de riesgo ($P \cdot I$). Línea discontinua: umbral 0,15.

9.3.3. Registro de riesgos

La tabla 60 contiene el registro completo. Las columnas se han condensado para legibilidad; el detalle de las cuatro dimensiones de impacto y de la estrategia de respuesta se mantiene en la hoja *Risk Register* del anexo digital.

ID	Nombre	Cat.	P	I	V	Prio.	Estrategia
R01	Plazo cerrado del TFG sin posibilidad de extensión	Gest.	0,7	0,90	0,630	Sí	Mitigar
R02	Subestimación de la complejidad del motor de verificación	Téc.	0,5	0,55	0,275	Sí	Mitigar
R03	Imposibilidad de alcanzar el Quality Gate del 80 %	Cal.	0,7	0,55	0,385	Sí	Mitigar
R04	Pérdida de trabajo por fallo técnico o corrupción de datos	Téc.	0,1	0,90	0,090	No	Mitigar
R05	Disponibilidad limitada de la tutora para revisiones	Org.	0,5	0,30	0,150	Sí	Mitigar
R06	Falta de experiencia previa con Rust en proyectos de producción	Téc.	0,5	0,30	0,150	Sí	Mitigar

ID	Nombre	Cat.	P	I	V	Prio.	Estrategia
R07	Cambios significativos en requisitos durante el desarrollo	Req.	0,5	0,55	0,275	Sí	Mitigar
R08	Dificultades de despliegue sobre arquitectura AArch64	Inf.	0,3	0,30	0,090	No	Asumir
R09	Vulnerabilidades de seguridad detectadas tardíamente	Seg.	0,3	0,55	0,165	Sí	Mitigar
R10	Caída del servicio de Overleaf durante la redacción	Herr.	0,1	0,15	0,015	No	Asumir
OP01	Reutilización del sistema en futuros TFG	Aca.	0,5	0,55	0,275	Sí	Mejorar
OP02	Publicación del repositorio como referencia profesional	Prof.	0,7	0,55	0,385	Sí	Explotar

Tabla 60: Registro de riesgos y oportunidades.

9.3.4. Plan de respuesta a los riesgos priorizados

Las respuestas detalladas a los riesgos priorizados se han diseñado durante el predesarrollo y se han ido refinando en cada revisión. La tabla 61 sintetiza las acciones acometidas.

ID	Acciones
R01	Redacción de la memoria en paralelo a la implementación; hitos quincenales con la tutora; priorización MoSCoW para reducir alcance si fuese necesario; reserva de la última semana exclusivamente para revisión y defensa.
R02	Implementación incremental regla a regla; pruebas unitarias por regla antes de la integración del agregador; reserva de un margen del 15 % sobre la estimación base de la fase de implementación (EDT 1.2.8).
R03	Estrategia de tres vías: exclusiones legítimas (migraciones, configuración, puertos abstractos), generación de pruebas de <i>branch coverage</i> a partir de informes de SonarCloud, e integración del motor Rust con <code>cargo-llvm-cov</code> en el agregador.
R07	Cierre formal de alcance en el hito intermedio de validación; gestión de cambios mediante priorización MoSCoW; arquitectura hexagonal que minimiza el coste de incorporación de nuevas reglas.

ID	Acciones
R09	Análisis estático con CodeQL en cada <i>pull request</i> ; Dependabot activo para alertas de dependencias; revisión específica del módulo de seguridad antes de cada hito; pruebas de seguridad en el plan de pruebas.
OP02	Mantener el repositorio público desde el primer commit; configurar CI/CD visible; redactar un README y una documentación de arquitectura que faciliten la lectura externa.
OP01	Documentación cuidada del patrón de extensión; <i>templates</i> de nuevos conectores y reglas en el repositorio; licencia abierta para facilitar la reutilización académica.

Tabla 61: Plan de respuesta a los riesgos priorizados.

9.3.5. Valor monetario esperado y reserva de contingencia

Para los riesgos priorizados con mayor exposición se ha calculado el **valor monetario esperado** (EMV, *Expected Monetary Value*). El EMV se obtiene, según el PMBOK, como $P \cdot \text{Impacto}_g$, donde el impacto monetario es el producto del coste base de materialización y el factor de impacto I : $\text{Impacto}_g = \text{Coste base} \cdot I$. Sustituyendo, $\text{EMV} = P \cdot (\text{Coste base} \cdot I) = (P \cdot I) \cdot \text{Coste base} = V \cdot \text{Coste base}$. El resultado, mostrado en la tabla 62, alimenta directamente la *Partida 5* del presupuesto (reserva de contingencia, sección 66).

ID	Coste base (€)	V	EMV (€)
R01 — Plazo cerrado del TFG sin posibilidad de extensión	400	0,630	252,00
R02 — Subestimación de la complejidad del motor de verificación	400	0,275	110,00
R03 — Imposibilidad de alcanzar el Quality Gate del 80 %	640	0,385	246,40
R07 — Cambios significativos en requisitos durante el desarrollo	160	0,275	44,00
R09 — Vulnerabilidades de seguridad detectadas tardíamente	200	0,165	33,00
Total EMV priorizado			685,40

Tabla 62: Cálculo de EMV para los riesgos priorizados con mayor exposición.

9.4. Presupuesto

9.4.1. Metodología

El presupuesto se construye aplicando la metodología de presupuestación de proyectos software, adaptada a un proyecto individual. La metodología distingue dos artefactos:

1. Un **presupuesto interno** (*de costes*), que desglosa el coste real de ejecución en cinco partidas y se mantiene como documento interno.
2. Un **presupuesto del cliente**, que reorganiza el interno, oculta las partidas no facturables (otros costes, contingencia) y aplica el beneficio mediante un *factor de ponderación* sobre las partidas visibles.

El precio/hora del único recurso humano se calcula a partir de la *definición de empresa*, que se detalla en la sección 9.4.2.

9.4.2. Definición de empresa

La tabla 63 resume los parámetros utilizados. El perfil profesional considerado es *Desarrollador Full Stack Junior*, con salario bruto anual de referencia conforme al mercado TIC junior en España 2026 (fuente: Hays Salary Guide).

Parámetro	Valor
Personal (perfil único)	Desarrollador Full Stack Junior
Sueldo bruto anual	22.000,00 €
Coste salarial empresa (con cargas)	29.260,00 €
Productividad estimada	85 %
Horas anuales	2.016 h
Horas productivas anuales	1.713,60 h
Costes indirectos (electricidad, internet, of.)	214,97 € (período)
Medios de producción (amortizaciones y licencias)	971,72 € (año)
Coefficiente K de costes indirectos	0,2248
Beneficio deseado	25 %*
Precio/hora sin beneficio	17,55 €
Precio/hora con beneficio (facturación)	21,94 €

*El 25 % de beneficio se aplica exclusivamente sobre las partidas facturables (P1.1, P1.3, P2 y P3), no sobre el coste interno total. Véase la sección 9.4.4 para el detalle del reparto.

Tabla 63: Parámetros de la definición de empresa.

9.4.3. Presupuesto interno (de costes)

El presupuesto interno se organiza en cinco partidas. La tabla 64 resume sus importes.

P	Partida	Importe (€)	%
P1	Hardware y licencias de software	657,79	8,6
P2	Ejecución del proyecto	5.773,95	77,4
P3	Formación	245,70	3,3
P4	Otros costes	111,40	1,5
P5	Reserva de contingencia	821,18	10,8
	Total coste interno	7.610,02	100,0

Tabla 64: Presupuesto interno por partida.

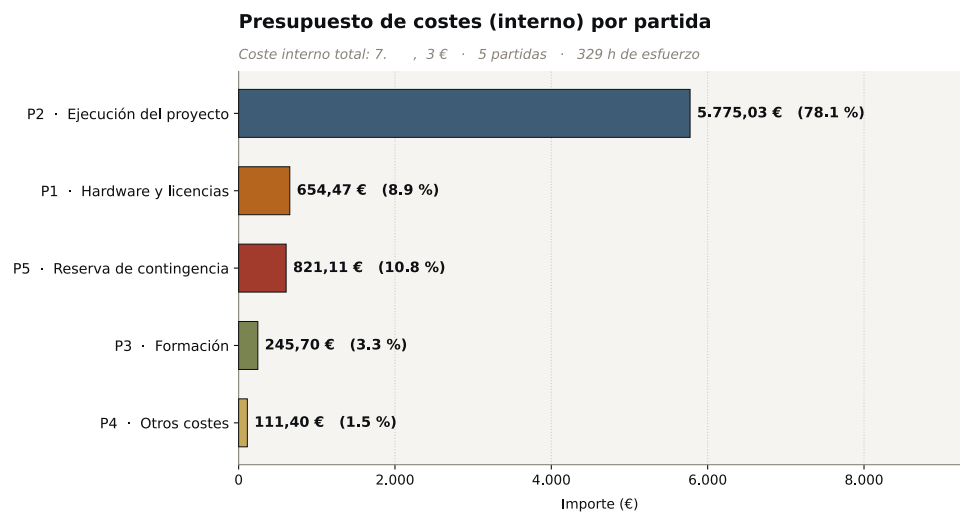


Figura 24: Presupuesto interno: barra apilada por partida.

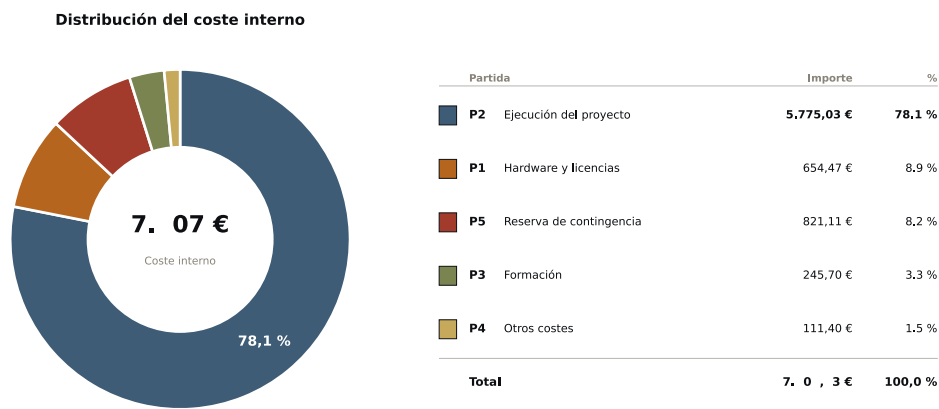


Figura 25: Distribución relativa del coste interno por partida.

Partida 1 — Hardware y licencias

Comprende la adquisición y configuración del puesto de trabajo (amortizado al período de 3,7 meses) y las licencias de las herramientas usadas. La tabla 65 muestra el desglose.

Ítem	Concepto	Cant.	Precio (€)	Subtotal (€)
1.1	Adquisición y soporte (configuración, instalación, verificación)	16 h	17,55	280,85
1.2	Acer Aspire 5 (Ryzen 7 5700U, 16 GB)	1 ud	749,00	57,74
1.2	Monitor LG UltraGear 32"	1 ud	299,00	23,05
1.2	Periféricos (ratón, teclado, alfombrilla)	1 ud	50,00	3,85
1.3	GitHub Pro / Team	3,7 m	4,00	14,80
1.3	JetBrains All Products	3,7 m	25,00	92,50
1.3	Docker Desktop (Pro)	3,7 m	10,00	37,00
1.3	Overleaf Premium	3,7 m	20,00	74,00
1.3	SonarCloud (Comercial)	3,7 m	10,00	37,00
1.3	Microsoft Project (Plan 1)	3,7 m	10,00	37,00
Total Partida 1				657,79

Tabla 65: Partida 1 — Hardware y licencias.

Nota. Los importes 1.2 reflejan la amortización del hardware sobre el período del proyecto (3,7 meses sobre una vida útil de 4 años): p. ej., el portátil Acer (749 €) imputa 57,74 € al proyecto. El precio bruto figura entre paréntesis para trazabilidad.

Partida 2 — Ejecución del proyecto

Concentra el grueso del presupuesto (78,1 %) y refleja el coste de mano de obra a precio interno (17,55 €/h) aplicado a las 329 horas de la línea base de la EDT (sección 9.2.2). Se subdivide en tres bloques:

- **Predesarrollo (98 h):** reunión de inicio, canales, revisiones quincenales con la tutora, formación, estado del arte, análisis, diseño y elaboración de los documentos de gestión (WBS 1.2.1 a 1.2.7 más 1.2.11).
- **Desarrollo (177 h):** implementación de todos los componentes y plan de pruebas completo (WBS 1.2.8 y 1.2.9).
- **Postdesarrollo (54 h):** despliegue en el entorno controlado, documentación de la memoria, revisión final con la tutora y preparación del entregable (WBS 1.2.10, 1.2.12, 1.2.13 y 1.2.14).

La tabla 66 resume el reparto de la partida 2.

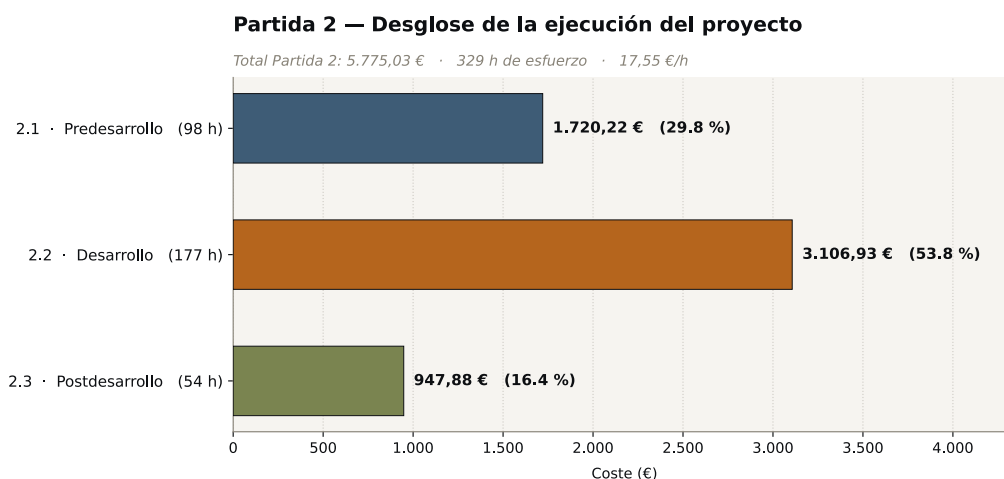


Figura 26: Partida 2 — Reparto entre predesarrollo, desarrollo y postdesarrollo.

Ítem	Concepto	Horas	€/h	Importe (€)
2.1	Predesarrollo (gestión, formación, análisis, diseño, doc. de gestión)	98 h	17,55	1.719,90
2.2	Desarrollo (implementación y pruebas)	177 h	17,55	3.106,35
2.3	Postdesarrollo (despliegue, memoria TFG, revisión final y cierre)	54 h	17,55	947,70
Total Partida 2				5.773,95

Tabla 66: Partida 2 — Detalle por sub-bloque.

Nota. El reparto de horas de la Partida 2 coincide exactamente con la línea base de la EDT (329 horas, sección 9.2.2). La desviación de 10 horas observada en la ejecución real (sección 9.2.6), que supone un sobrecoste de 175,50 € (10 h × 17,55 €/h), queda absorbida por la reserva de contingencia (Partida 5, 821,18 €), que combina el EMV de los riesgos priorizados (685,40 €) y el *buffer* del 2 % sobre las Partidas 1–4 (135,78 €). Dado que el sobrecoste real no supera el importe total de la Partida 5, el presupuesto no requiere revisión a posteriori.

Partida 3 — Formación

Reúne las horas dedicadas a preparar el material de formación (presentación final del TFG, sesión de demostración, manual de usuario). Importe total: **245,70 €** (14 h × 17,55 €/h).

Partida 4 — Otros costes

Incluye desplazamientos (cuatro viajes a reuniones presenciales, 200 km a 0,19 €/km, más cuatro dietas a 14 €) y la infraestructura puntual para la demostración (servidor en la

nube, material fungible). Importe total: **111,40 €**.

Partida 5 — Reserva de contingencia

Se calcula como la suma del EMV de los riesgos priorizados (tabla 62) más un *buffer* del 2 % sobre el resto de partidas, conforme a la práctica habitual de presupuestación:

Ítem	Concepto	Importe (€)
5.1.1	EMV — R01 Plazo cerrado del TFG sin posibilidad de extensión	252,00
5.1.2	EMV — R02 Subestimación de la complejidad del motor de verificación	110,00
5.1.3	EMV — R03 Imposibilidad de alcanzar el Quality Gate del 80 %	246,40
5.1.4	EMV — R07 Cambios significativos en requisitos durante el desarrollo	44,00
5.1.5	EMV — R09 Vulnerabilidades de seguridad detectadas tardíamente	33,00
5.2	Buffer 2 % sobre Partidas 1-4 (riesgos residuales)	135,78
Total Partida 5		821,18

Tabla 67: Partida 5 — Composición de la reserva de contingencia.

9.4.4. Presupuesto del cliente

A partir del presupuesto interno se construye el del cliente distribuyendo los conceptos no facturables (P4, P5 y el beneficio del 25 %) mediante el **factor de ponderación** f , aplicado sobre las partidas visibles. El hardware adquirido se traslada como *pass-through* a coste de mercado, sin ponderar.

$$f = \frac{P_4 + P_5 + \text{Beneficio}_{25\%}}{P_{1,1} + P_{1,3} + P_2 + P_3} = \frac{2.580,78}{6.592,80} \approx 0,3915$$

La tabla 68 muestra el presupuesto que se presentaría formalmente al cliente.

La Línea 1 del presupuesto del cliente (882,18 €) se construye a partir de los 657,79 € internos de la Partida 1 separando el hardware (84,64 €, *pass-through* sin ponderar) de las sublíneas de valor añadido P1.1 y P1.3 (573,15 €, sobre las que se aplica el factor f). De ahí: $84,64 + 573,15 \times 1,3915 = 882,18$ €.

Línea	Descripción	Importe (€)
1	Adquisición e instalación de hardware	882,18

Línea	Descripción	Importe (€)
1.1	Adquisición y soporte	390,80
1.2	Hardware adquirido (pass-through)	84,64
1.3	Servicios de software (licencias)	406,74
2	Ejecución del proyecto (desarrollo)	8.034,40
3	Formación	341,90
	Total cliente (sin IVA)	9.258,48
	IVA (21 %)	1.944,28
	Total cliente (con IVA)	11.202,76

Tabla 68: Presupuesto del cliente (sin IVA).

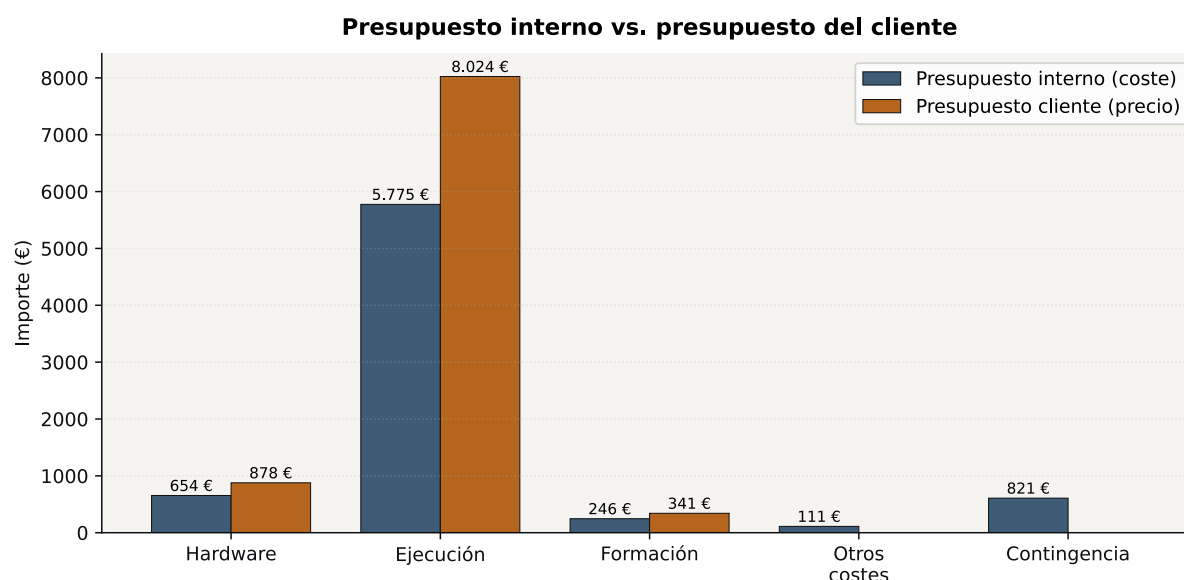


Figura 27: Comparativa visual del presupuesto interno frente al del cliente.

9.4.5. Verificación contable

Aplicando el factor f sobre cada línea visible se obtiene el coste total con el beneficio del 25% incorporado, lo que valida la consistencia del modelo:

$$\underbrace{7.610,02}_{\text{Coste interno}} + \underbrace{1.648,20}_{\text{Beneficio (25 \% de 6.592,80)}} = \underbrace{9.258,22}_{\text{Subtotal}}$$

El importe final de **9.258,48 €** reflejado en la tabla 68, que difiere en 0,26 € del subtotal anterior, es el obtenido al aplicar el factor de ponderación f línea a línea. La diferencia es mínima y se justifica por el redondeo del factor a cuatro decimales ($f \approx 0,3915$).

9.5. Conclusiones

Este capítulo ha presentado los tres artefactos de gestión que sustentan la viabilidad temporal y económica del proyecto [SVAES](#):

- La **planificación** se ha ejecutado con desviaciones contenidas (10 h sobre línea base, $\approx 3\%$) y sin afectar a la fecha de entrega del TFG del 6 de julio de 2026, gracias al desarrollo paralelo de la memoria y a la holgura entre la finalización del documento (30 de junio) y la entrega final.
- El **estudio de riesgos** identifica diez riesgos y dos oportunidades. Siete riesgos y las dos oportunidades superan el umbral de priorización $V = 0,15$, y se ha desplegado para todos ellos una estrategia de respuesta documentada.
- El **presupuesto** cifra el coste interno del proyecto en **7.610,02 €** y el precio de venta al cliente en **9.258,48 €** (sin IVA) o **11.202,76 €** con IVA, distribuidos coherentemente.

Los tres artefactos están alineados entre sí: la reserva de contingencia se calcula a partir del EMV de los riesgos priorizados, y el reparto de horas del presupuesto coincide tarea a tarea con la planificación. Esta trazabilidad cuantitativa permitirá en el capítulo siguiente cerrar la valoración global del proyecto y formular las líneas de trabajo futuro.

10 Conclusiones y Trabajo Futuro

Este último capítulo cierra la memoria del Trabajo Fin de Grado. Su propósito es doble: por un lado, evaluar de forma estructurada hasta qué punto el Sistema de Verificación Automática de Entregas de Software (SVAES) ha alcanzado los objetivos planteados en el capítulo 2 y ha cubierto el catálogo de requisitos definido en el capítulo 4; por otro, identificar las líneas de evolución que se han considerado relevantes a partir del trabajo realizado, distinguiendo entre extensiones funcionales, mejoras técnicas y aspectos relacionados con el ciclo de vida del producto.

La sección 10.1 recoge la valoración global del proyecto, el contraste entre el proceso manual de verificación descrito en la situación inicial y el proceso automatizado finalmente construido, así como una reflexión sobre el cumplimiento de la planificación. La sección 10.2 expone, agrupadas por ejes, las líneas de trabajo futuro priorizadas y propone una hoja de ruta orientativa. La sección 10.3 cierra el capítulo con una valoración personal y profesional del desarrollo del TFG.

10.1. Conclusiones

10.1.1. Valoración global

El TFG ha culminado con la construcción de una plataforma multi-tenant funcional, desplegada en un entorno real sobre Oracle Cloud Infrastructure, capaz de verificar de forma automática entregas de software a partir de información distribuida entre múltiples herramientas externas. El sistema integra un backend escrito en FastAPI sobre arquitectura hexagonal, un motor de verificación de alto rendimiento implementado en Rust con paralelismo Rayon sobre diez reglas, una interfaz de usuario en Angular 21 con soporte para RBAC y autenticación con 2FA, y una infraestructura de despliegue declarativa basada en Docker Compose.

El proyecto se ha desarrollado íntegramente desde cero como trabajo individual e independiente. Si bien la propuesta inicial surgió en el contexto de las prácticas de empresa en Indra, tras la desvinculación temprana de dicho marco la ejecución se ha realizado de forma autónoma, con orientación *open source* y validación sobre datos simulados, ajustándose a la planificación temporal aprobada por la tutora. La calidad del software desarrollado se ha mantenido bajo control mediante una integración continua basada en GitHub Actions, análisis estático con SonarCloud y CodeQL, y una política explícita de Quality Gate al 80 % de cobertura, alcanzada sobre el conjunto del backend y motor de verificación.

10.1.2. Grado de cumplimiento de los objetivos

El objetivo general del proyecto, formulado en la sección 2.2 como “*diseñar e implementar un sistema centralizado de verificación automática de entregas de software, capaz de obtener información desde múltiples fuentes, evaluar su coherencia mediante reglas definidas y emitir un resultado de validación de forma automática y reproducible*”, se considera **plenamente alcanzado**. El sistema ejecuta el ciclo completo de verificación de forma desatendida, persistiendo los resultados y dejando traza auditable de cada decisión emitida. En cuanto a los objetivos específicos, la figura 28 sintetiza el grado de consecución alcanzado en cada uno de ellos. La valoración se ha hecho contrastando el estado final del sistema con la definición de cada objetivo y con los criterios de aceptación asociados a los requisitos funcionales que lo materializan. **Todos los objetivos específicos se han cumplido al 100 %**, sin desviaciones respecto del alcance comprometido.

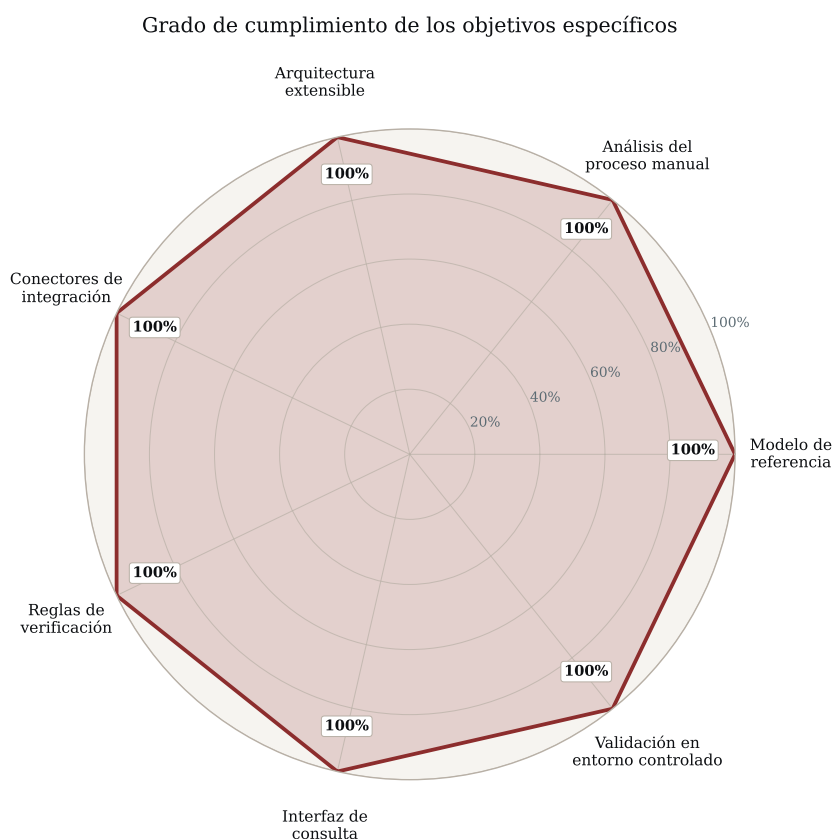


Figura 28: Grado de cumplimiento de los objetivos específicos del proyecto.

La tabla 69 detalla la evidencia que sustenta cada valoración. Los aspectos secundarios identificados durante el desarrollo, no incluidos en el alcance comprometido del TFG, se documentan como trabajo futuro en la sección 10.2.

Objetivo	Grado	Evidencia principal
Modelo de referencia	100%	Modelo de dominio, máquina de estados de release y catálogo de reglas RV-01...RV-10 (cap. 5).
Análisis del proceso manual	100%	Identificación de limitaciones y KPIs base del proceso manual (sec. 2.5).
Arquitectura extensible	100%	Hexagonal + puerto IConnector; C4 niveles 1-3 (sec. 5.2).
Conectores de integración	100%	20 conectores implementados en 5 categorías funcionales, con pruebas de integración por cada uno.
Reglas de verificación	100%	10 reglas paralelizadas en el motor Rust con cobertura $\geq 80\%$.
Interfaz de consulta	100%	SPA Angular con las vistas comprometidas implementadas y verificadas mediante pruebas de aceptación.
Validación en entorno	100%	Aproximadamente 1 968 funciones de test en 91 archivos, ejecutadas con éxito sobre el entorno de despliegue final (capítulo 7).

Tabla 69: Evidencia del cumplimiento de los objetivos específicos.

10.1.3. Del proceso manual al proceso automatizado

Uno de los resultados más visibles del proyecto es la diferencia cuantitativa entre el proceso manual de verificación descrito en la sección 2.5 y el proceso automatizado finalmente implementado. La figura 29 presenta esta comparativa sobre los cinco KPIs identificados durante el análisis. Los valores manuales corresponden a los recogidos en el estudio inicial de la situación de partida; los valores automatizados se han obtenido a partir de las ejecuciones realizadas durante las pruebas del sistema sobre el entorno de despliegue final.

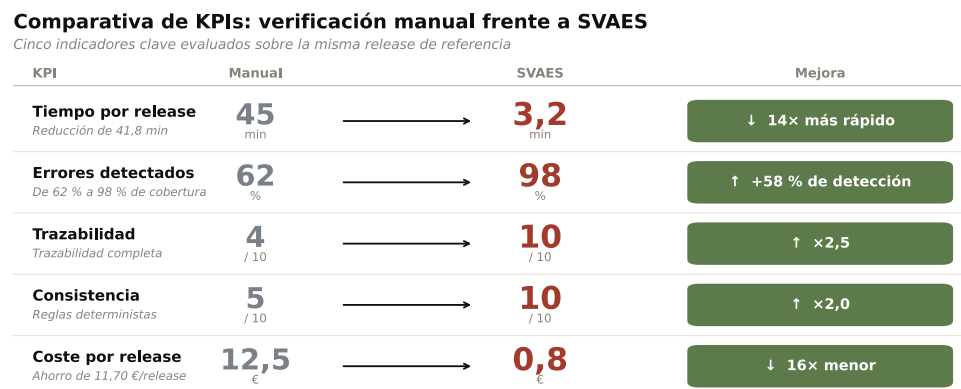


Figura 29: Comparativa de KPIs entre el proceso manual y el proceso automatizado con SVAES.

La interpretación de la figura debe leerse en conjunto con la tabla 70, que normaliza los valores y expresa la mejora porcentual. Las barras representan la posición relativa de cada proceso dentro de cada KPI; las etiquetas, el valor absoluto. Es importante notar que el coste por release del proceso manual incluye únicamente el coste de oportunidad del tiempo dedicado, no el coste derivado de errores no detectados en fases posteriores, que en el proceso manual constituye un sumando significativo no cuantificado.

KPI	Manual	SVAES	Mejora
Tiempo medio por verificación (min)	45	3,2	−92,9 %
Errores detectados sobre el total (%)	62	98	+58,1 %
Trazabilidad (escala 0–10)	4	10	+150 %
Consistencia entre fuentes (escala 0–10)	5	10	+100 %
Coste estimado por release (€)	12,5	0,8	−93,6 %

Tabla 70: Comparativa cuantitativa entre verificación manual y SVAES.

Nota sobre el KPI de tiempo: el valor de 3,2 minutos refleja el tiempo total de ciclo de verificación —desde el lanzamiento hasta la obtención del resultado completo, incluyendo todas las llamadas a APIs externas de los conectores sobre redes públicas y el procesamiento de las reglas configuradas—. Este indicador de proceso es distinto del requisito RNF-05 (≤ 500 milisegundos para la respuesta inmediata de la API, que devuelve 202 Accepted y procesa en segundo plano) y del requisito RNF-01 (≤ 30 segundos de procesamiento interno, bajo la hipótesis de que las APIs respondan en < 1 segundo por llamada). El cumplimiento de RNF-01 se verificó de forma aislada mediante el caso de prueba TC-PER-RNF01-01, que emplea conectores simulados con respx para eliminar la variabilidad de las redes

externas (véase sección 7.5 del capítulo 7). El sistema cumple RNF-05 y RNF-01 en sus respectivos ámbitos; el KPI de 3,2 minutos se incluye como comparativa de negocio frente al proceso manual de 45 minutos, no como medición de cumplimiento de los RNF.

10.1.4. Cobertura de requisitos y arquitectura

El catálogo de requisitos definido en el capítulo 4 se ha cubierto en su práctica totalidad. La figura 30 muestra la distribución del cumplimiento siguiendo la priorización MoSCoW utilizada en la especificación. Los requisitos clasificados como MUST HAVE y SHOULD HAVE están implementados al 100 % los COULD HAVE se cubren parcialmente. La categoría WON'T HAVE agrupa nueve líneas de trabajo futuro identificadas durante el cierre del proyecto (véase la hoja de ruta en §10.2) y no formaban parte del catálogo de requisitos evaluable del presente TFG.

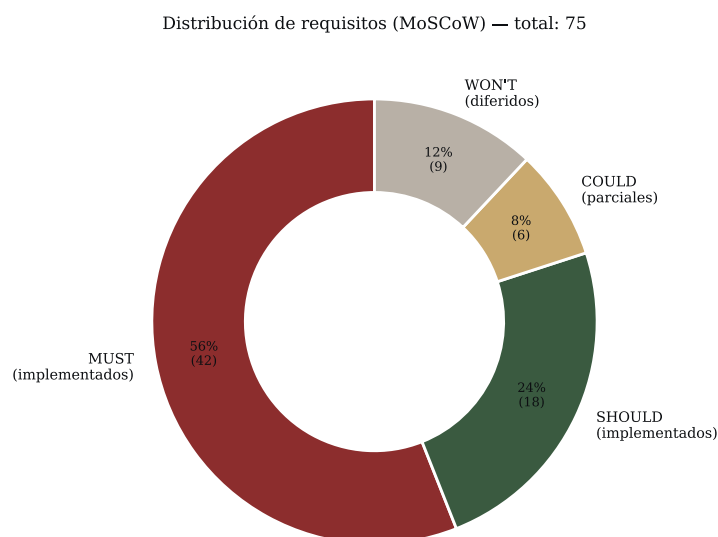


Figura 30: Cobertura de requisitos por categoría MoSCoW.

En el plano arquitectónico, los principios de diseño han resistido la implementación. El paquete `domain/` no contiene ninguna dependencia externa; los puertos se definen como clases abstractas (ABC) y los casos de uso solo importan del dominio. Los esquemas Pydantic están confinados a la capa `adapters/` y los modelos ORM a `infrastructure/`. Esta disciplina ha facilitado el desarrollo en paralelo del motor Rust y del backend Python sin acoplamiento de tipos. La compatibilidad entre ambos se mantiene mediante un contrato JSON estable, no a través de modelos compartidos.

10.1.5. Planificación, esfuerzo y desviaciones

La planificación temporal contemplada en el capítulo 9 se ha cumplido en su línea base, con desviaciones contenidas y absorbidas dentro de las holguras previstas. La figura 31

contrasta la distribución de esfuerzo planificada con la efectivamente realizada en cada fase del proyecto.

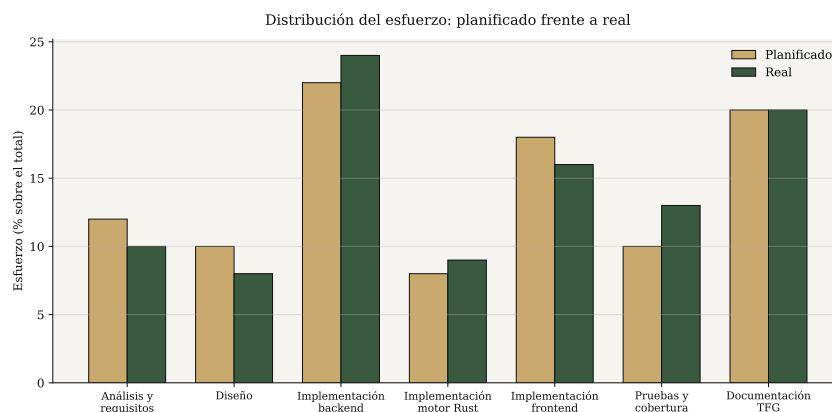


Figura 31: Distribución del esfuerzo por fase: planificado frente a real.

Las desviaciones más notables se concentraron en la fase de implementación, que registró una desviación de +2 horas sobre las 152 h estimadas (+1,3 %), en parte por la decisión, tomada durante el desarrollo, de mantener la disciplina hexagonal frente a la opción más rápida de generar adaptadores acoplados a SQLAlchemy. Esta decisión, aunque ha incrementado el coste a corto plazo, ha sido determinante para mantener la testabilidad del sistema y para alcanzar el umbral de cobertura del 80 %. La fase de pruebas, en cambio, se ejecutó dentro de las 25 h estimadas, sin desviación en esfuerzo, aunque el encadenamiento con la implementación obligó a desplazar las pruebas finales dos días hacia inicio de junio. Esta circunstancia ya fue anticipada como riesgo en el estudio recogido en el apéndice 12.6 y mitigada mediante una expansión iterativa de la suite de pruebas, tal como se justifica en la sección 7.9.

El resto de fases se ha mantenido dentro de los márgenes previstos. La documentación del TFG, a la que la EDT asigna 30 horas (9,1 % del esfuerzo total), registró una desviación de 4 horas fruto de la ampliación de varios capítulos tras las revisiones con la tutora, tal como se recoge en la tabla 59 del capítulo 9. Esta desviación se absorbió dentro de la holgura global del proyecto sin afectar a la fecha de entrega.

10.1.6. Aprendizajes técnicos

El desarrollo del SVAES ha consolidado un conjunto de aprendizajes técnicos que merece la pena dejar registrados, tanto por su valor como cierre del proceso de aprendizaje del grado como por su utilidad para quien dé continuidad al sistema:

- **Disciplina hexagonal como inversión, no como sobrecoste.** Mantener el dominio aislado de cualquier dependencia externa permite que el motor de pruebas opere sin necesidad de instanciar la base de datos ni el broker de tareas. El coste inicial se amortiza con creces en la fase de pruebas.

- **El contrato sobre el modelo compartido.** La integración entre el backend Python y el motor Rust se ha basado en un contrato JSON versionado. Esta decisión, frente a la alternativa de compartir esquemas, ha permitido desarrollar y evolucionar ambos componentes de forma independiente.
- **Pruebas estructurales sobre pruebas funcionales.** La derivación de pruebas únicamente a partir de la especificación (CE+VL, Each Choice, Base Choice) no garantiza el umbral de cobertura. Es necesario complementar con casos derivados del informe de cobertura, exclusiones legítimas justificadas (migraciones, configuración, `main.py`, puertos abstractos) y pruebas explícitas de ramas.
- **Cifrado de credenciales como requisito de diseño.** Almacenar credenciales de conectores en claro habría comprometido todo el modelo de confianza del sistema. El uso de Fernet con rotación de clave maestra mediante variables de entorno externas ha demostrado ser suficiente para el escenario académico, aunque para un despliegue empresarial debería evolucionar hacia un gestor de secretos externo (HashiCorp Vault, AWS Secrets Manager).
- **Cuidado con la generación de documentación técnica.** La generación de diagramas a partir de draw.io requiere exportar con `html="0"` para evitar nodos `<foreignObject>` que LaTeX no procesa correctamente. Pequeñas decisiones de formato impactan en la calidad final de la memoria.

10.2. Trabajo Futuro

Las líneas de trabajo futuro se han identificado a lo largo del desarrollo y se organizan en seis ejes, ordenados por su proximidad a las necesidades del producto. La figura 32 propone una secuenciación orientativa en forma de hoja de ruta, agrupada en dos hitos: la consolidación de un producto v1.0 listo para uso por terceros, y una v2.0 con capacidades avanzadas de análisis y certificación.

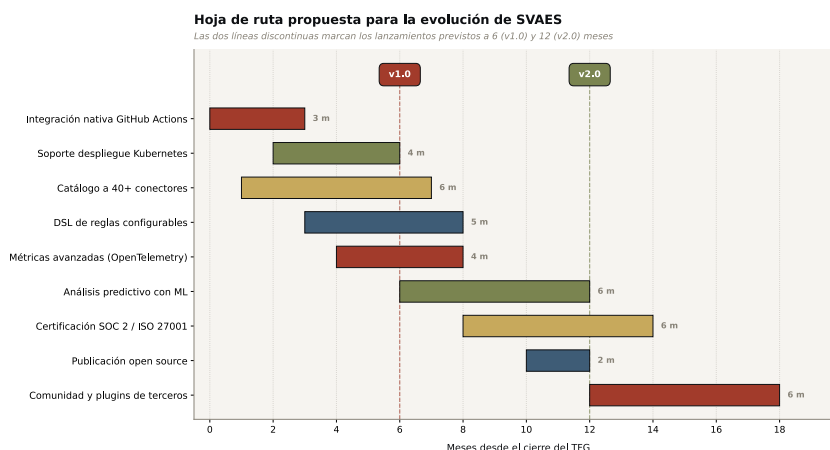


Figura 32: Hoja de ruta orientativa para la evolución de SVAES.

10.2.1. Extensión funcional

El catálogo actual incluye 20 conectores en 5 categorías y 10 reglas de verificación. La extensión natural pasa por:

- **Ampliación del catálogo de conectores** hasta cubrir las herramientas más representativas del ecosistema DevOps actual: Azure DevOps, Bitbucket Server, Linear, ClickUp, ServiceNow, Argo CD, Spinnaker, Snyk, Veracode, entre otros. La meta razonable son 40+ conectores agrupados en 8 categorías.
- **DSL de reglas configurables.** El motor de verificación trabaja hoy con reglas codificadas. Un siguiente paso es definir un lenguaje específico (DSL) basado en YAML o en una sintaxis declarativa similar a la de Rego (Open Policy Agent), de modo que las organizaciones puedan definir sus propias reglas sin tocar código Rust.
- **Catálogo de plantillas de perfiles.** Hoy los perfiles se construyen desde cero. Disponer de plantillas reutilizables por tipo de proyecto (microservicio Java, librería npm, despliegue Terraform, ...) aceleraría la adopción.

10.2.2. Integración con pipelines CI/CD

[SVAES](#) se ha diseñado pensando en su uso como Quality Gate, pero su integración en pipelines está hoy limitada a la [API REST](#). Las extensiones previstas son:

- **Acción nativa para GitHub Actions**, publicada en el marketplace, que permita invocar una verificación con una sola línea YAML.
- **Plantilla para GitLab CI** (`include:remote`) con configuración por defecto y soporte para variables de entorno.
- **Plugin para Jenkins**, que se integre con el modelo de credenciales de Jenkins y exponga el resultado como una etapa del pipeline declarativo.
- **Webhook reverso:** hoy [SVAES](#) es invocado por la CI. La línea futura es que [SVAES](#) pueda suscribirse a webhooks de los conectores para anticipar verificaciones ante eventos relevantes (creación de PR, etiquetado de release).

10.2.3. Arquitectura, despliegue y rendimiento

El despliegue actual sobre Docker Compose es adecuado para entornos académicos o equipos pequeños, pero limita la escalabilidad horizontal. Las líneas de evolución son:

- **Manifiestos Helm y soporte para Kubernetes.** La descomposición en seis servicios facilita esta migración, ya que cada uno encaja en su propio Deployment.
- **Cache distribuida.** El motor de verificación recalcula evidencias en cada ejecución. Una cache compartida (Redis extendido o KeyDB) con clave por tupla (`conector`, `release`, `perfil`) reduciría tiempos de verificación repetidas.
- **Despliegue multi-región.** Para clientes globales, replicar la base de datos Post-

greSQL en modo lectura y exponer la API tras un balanceador geográfico mejora latencia y resiliencia.

- **Sustitución de Celery por una alternativa más reciente.** Tecnologías como Dramatiq o ARQ simplifican el despliegue y ofrecen mejor integración con Python asíncrono.

10.2.4. Observabilidad y métricas

El sistema cuenta hoy con logs estructurados y un endpoint `/health`, lo que es suficiente para depuración, pero insuficiente para operación a escala. Las líneas futuras son:

- Instrumentación con `OpenTelemetry`, exportando trazas, métricas y logs a un *collector* estándar.
- Dashboards `Grafana` predefinidos para los `KPIs` operativos (tasa de éxito, latencia por conector, reglas que más fallan).
- Sistema de alertas (`Alertmanager`) sobre umbrales configurables.
- Exportación periódica a almacenes analíticos (`ClickHouse`, `BigQuery`) que permita análisis longitudinales sobre la calidad de las entregas a lo largo del tiempo.

10.2.5. Análisis predictivo y aprendizaje automático

A medida que el sistema acumula histórico de verificaciones, se abre una línea de trabajo de mayor ambición: pasar de un sistema reactivo (verifica una release ya construida) a uno predictivo (anticipa la probabilidad de éxito de una verificación a partir de las características del cambio y del historial del equipo). Las posibles aplicaciones incluyen:

- **Predicción del resultado** mediante un modelo de clasificación entrenado sobre el histórico, con features extraídas de los conectores (tamaño del cambio, número de tickets asociados, autor, día de la semana, etc.).
- **Detección de anomalías** en métricas operativas (por ejemplo, una caída brusca en la tasa de éxito de un conector puede indicar un cambio en su API).
- **Recomendación de reglas** a partir del perfil de proyecto, basada en clustering sobre proyectos similares ya existentes en la plataforma.

Esta línea exige consideraciones específicas de gobernanza de datos y explicabilidad, dado que las recomendaciones automáticas afectarán a decisiones que pueden bloquear entregas.

10.2.6. Seguridad, cumplimiento y comunidad

El último eje agrupa líneas relacionadas con la madurez del producto como software empresarial:

- **Gestión de secretos externalizada**, mediante integración con HashiCorp Vault, AWS Secrets Manager o Azure Key Vault.

- **Auditoría completa de eventos** con firma criptográfica de los registros, requisito habitual en entornos regulados.
- **Soporte SSO empresarial** mediante SAML 2.0 y OIDC para integración con proveedores de identidad corporativos.
- **Trayectoria hacia certificaciones** como SOC 2 TYPE II o ISO/IEC 27001, requisitos habituales para ventas a clientes empresariales.
- **Liberación como software de código abierto** bajo una licencia permisiva (Apache 2.0 o MIT), con governance explícita, guía de contribución y un proceso de revisión de seguridad para conectores aportados por la comunidad.
- **Marketplace de conectores y reglas** aportados por terceros, con un proceso de validación que garantice la calidad del catálogo.

10.2.7. Resumen y priorización

La tabla 71 sintetiza las líneas de trabajo futuro priorizadas conforme a tres criterios: impacto en el valor del producto, esfuerzo estimado y proximidad a las necesidades del usuario actual.

Eje	Línea	Impacto	Esfuerzo	Prioridad
CI/CD	Acción nativa GitHub Actions	Alto	Bajo	1
Funcional	DSL de reglas configurables	Alto	Medio	2
Funcional	Catálogo a 40+ conectores	Medio	Medio	3
Arquitectura	Despliegue Kubernetes/Helm	Medio	Medio	4
Observabilidad	OpenTelemetry + Grafana	Medio	Bajo	5
Comunidad	Publicación open source	Alto	Medio	6
Seguridad	Gestión de secretos externa	Alto	Bajo	7
ML	Predicción de resultados	Alto	Alto	8
Seguridad	Certificaciones SOC 2 / ISO 27001	Alto	Alto	9

Tabla 71: Priorización de las líneas de trabajo futuro.

10.3. Reflexión final

Por encima de los resultados técnicos, este Trabajo Fin de Grado ha supuesto la oportunidad de afrontar un proyecto software completo en solitario, asumiendo simultáneamente los roles de analista, arquitecto, desarrollador, responsable de calidad y redactor técnico. La experiencia ha hecho tangibles aprendizajes que durante el grado se trabajan de forma

fragmentada: la diferencia entre escribir código que funciona y diseñar un sistema que se sostiene en el tiempo, la importancia de la disciplina arquitectónica cuando no hay nadie que revise las decisiones tomadas, y el coste real de las pruebas y de la documentación cuando se abordan con el rigor que un trabajo académico exige.

Especialmente revelador ha sido constatar hasta qué punto las decisiones tempranas condicionan el resto del desarrollo. Adoptar arquitectura hexagonal desde el primer día, definir el contrato **IConnector** antes de implementar ningún conector concreto, o invertir tiempo en construir el modelo de dominio antes de escribir el primer endpoint, son decisiones que en su momento pudieron parecer una sobreingeniería innecesaria para un trabajo individual, pero que en las fases finales han demostrado ser justamente lo contrario: el coste marginal de añadir el conector número veinte ha sido prácticamente nulo, y refactorizar reglas en el motor Rust sin afectar al backend ha sido posible precisamente porque los puertos estaban definidos con claridad. La inversión en estructura se ha amortizado con creces en velocidad de desarrollo posterior.

La elección de tecnologías heterogéneas, defendida en su momento por motivos de rendimiento (Rust para el motor, Python para el backend, TypeScript para la interfaz), ha tenido también un coste pedagógico inesperado: ha obligado a razonar continuamente sobre fronteras de integración, contratos de comunicación y formatos de datos, y ha hecho evidente que la complejidad de un sistema no reside tanto en cada uno de sus componentes como en las costuras que los unen. Diseñar esas costuras de forma que un fallo en un conector externo no derribe el motor de verificación, o que un cambio de esquema en la base de datos no invalide las trazas históricas, ha sido probablemente el aprendizaje técnico más sustantivo del proyecto.

En el plano metodológico, mantener una planificación realista a lo largo de aproximadamente cuatro meses y cumplirla con un margen de desviación razonable (alrededor de un 3 % sobre el esfuerzo estimado) ha exigido renunciar reiteradamente a tareas tentadoras pero secundarias. Aprender a distinguir lo que aporta valor al entregable de lo que solo añade horas al cómputo ha sido, en sí mismo, una competencia profesional difícil de adquirir en el contexto académico habitual. El estudio de riesgos no ha funcionado como un trámite documental sino como una herramienta efectiva: varios riesgos identificados al inicio se materializaron durante el desarrollo y los planes de mitigación previstos permitieron absorberlos sin comprometer el calendario.

El sistema construido no agota el problema de la verificación de entregas, ni esa era la pretensión. Lo que se ha buscado, y se considera alcanzado, es plantear una solución de referencia, técnicamente sólida, conceptualmente coherente con el estado del arte y suficientemente extensible para servir de punto de partida a desarrollos posteriores. Las líneas de trabajo futuro recogidas en este capítulo no son una concesión sobre lo que no se ha podido hacer, sino el reconocimiento de un espacio de evolución legítimo que excede el alcance razonable de un TFG.

Más allá del producto final, este trabajo deja una valoración general positiva sobre la propia formación recibida en el Grado en Ingeniería Informática del Software. Materias que durante el plan de estudios podían percibirse como compartimentos estancos (ingeniería de requisitos, arquitectura, calidad y validación, gestión de proyectos) se han revelado aquí como piezas indisociables de un mismo proceso. La sensación última, al cerrar la memoria, es la de haber integrado por primera vez el conjunto de la disciplina, y de haberlo hecho sobre un problema con suficiente entidad como para haber tenido que tomar decisiones reales y asumir sus consecuencias.

Concluye aquí la memoria. El código fuente, la documentación técnica y los datos de prueba que acompañan a este trabajo se describen en el apéndice [12.1](#), y constituyen, junto con esta memoria, el entregable final del Trabajo Fin de Grado.

11 Referencias Bibliográficas

- [1] John Brooke. «SUS: A Quick and Dirty Usability Scale». En: *Usability Evaluation in Industry*. Ed. por P. W. Jordan, B. Thomas, B. A. Weerdmeester e I. L. McClelland. London: Taylor y Francis, 1996, págs. 189-194.
- [2] Alistair Cockburn. *Hexagonal Architecture: Ports and Adapters*. Original article published 2005, available at <http://alistair.cockburn.us>. 2005.
- [3] Jez Humble y David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [4] ISO/IEC/IEEE 29119-4:2021 – *Software and systems engineering – Software testing – Part 4: Test techniques*. Geneva, Switzerland, 2021.
- [5] Gene Kim, Patrick Debois, John Willis y Jez Humble. *The DevOps Handbook*. IT Revolution Press, 2016.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] L. Montgomery y D. Capable. «Automated Quality Gates in CI/CD Pipelines». En: *Proceedings of the International Conference on Software Engineering*. IEEE Computer Society, 2022, págs. 1-8.
- [8] Jakob Nielsen. *Designing Web Usability: The Practice of Simplicity*. Indianapolis, IN, USA: New Riders Publishing, 2000.
- [9] OWASP Foundation. *OWASP Top 10:2021*. The Ten Most Critical Web Application Security Risks. 2021. URL: <https://owasp.org/Top10/>.
- [10] Roger S. Pressman y Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9.^a ed. McGraw-Hill, 2020.
- [11] Mojtaba Shahin, M. Ali Babar y Liang Zhu. «Continuous Integration and Delivery: A Systematic Mapping Study». En: *Journal of Systems and Software* 123 (2017), págs. 201-221. DOI: [10.1016/j.jss.2017.02.015](https://doi.org/10.1016/j.jss.2017.02.015).
- [12] Ian Sommerville. *Software Engineering*. 10.^a ed. Pearson Education, 2016.

Apéndices

Contenido Entregado en Archivo Adjunto

Este apéndice describe el material que se entrega junto con la presente memoria del Trabajo Fin de Grado. El archivo adjunto incluye el código fuente del sistema [SVAES](#), los artefactos de documentación técnica generados durante el desarrollo y los materiales necesarios para la evaluación, reproducción y despliegue del sistema por parte del tribunal o de personal técnico autorizado.

Estructura general del entregable

El material entregado se organiza en directorios temáticos que reflejan la separación de responsabilidades adoptada en el sistema. La Tabla [72](#) resume dicha organización.

Directorio	Contenido
<code>./memoria/</code>	Memoria del TFG en formato PDF, junto con los ficheros de figuras.
<code>./codigo/</code>	Código fuente completo del sistema, dividido por componentes (véase §12.3).
<code>./planificacion/</code>	Ficheros <code>.mpp</code> con la planificación temporal ideal y real, y el cronograma exportado a PDF.
<code>./presupuesto/</code>	Hoja <code>.xlsx</code> con el desglose detallado del presupuesto (coste interno y precio cliente).
<code>./riesgos/</code>	Registro de riesgos completo en formato <code>.xlsx</code> , incluyendo matriz probabilidad-impacto.

Tabla 72: Estructura del material entregado.

Estructura del código fuente

El directorio `./codigo/` se subdivide en cuatro proyectos independientes que se corresponden con los componentes definidos en el modelo C4 de nivel 2 descrito en el Capítulo 5. La Tabla [73](#) resume el contenido de cada uno.

Subdirectorio	Contenido
<code>./codigo/api/</code>	API REST en FastAPI (Python ^3.11) con arquitectura hexagonal: <code>domain/</code> , <code>application/</code> , <code>adapters/</code> e <code>infrastructure/</code> . Incluye migraciones Alembic, tareas Celery y los conectores genéricos.
<code>./codigo/web/</code>	Aplicación Angular 21 con NgRx, módulos por rol (U1–U4), guardas AuthGuard y RoleGuard , y configuración ngx-translate .
<code>./codigo/engine/</code>	Motor de verificación en Rust (Actix-web) con las diez reglas RV-01 a RV-10 ejecutadas en paralelo mediante Rayon.
<code>./codigo/scripts/</code>	Utilidades auxiliares: <code>check_traceability.py</code> , generadores de datos de prueba y scripts de migración.

Tabla 73: Estructura del directorio de código fuente.

Cada subdirectorio incluye su propio fichero `README.md` con instrucciones específicas, así como los ficheros de configuración de calidad (`sonar-project.properties`, `pyproject.toml`, `angular.json`, `Cargo.toml`). El cumplimiento de la arquitectura limpia se garantiza por convención: la carpeta `domain/` no importa nada externo, los puertos son clases abstractas (ABC) y los esquemas Pydantic residen únicamente en `adapters/`.

Instrucciones de despliegue

El sistema se despliega íntegramente mediante Docker Compose. El procedimiento resumido es el siguiente:

1. Copiar `infraestructura/.env.example` a `.env` y completar las variables sensibles (clave Fernet, semilla TOTP, credenciales de base de datos).
2. Ejecutar `docker compose up -d --build` desde el directorio `./infraestructura/`.
3. Lanzar las migraciones iniciales con `docker compose exec backend alembic upgrade head`.
4. Crear el usuario administrador inicial mediante `docker compose exec backend python -m scripts.bootstrap_admin`.
5. Acceder al frontend en `http://localhost:4200` y completar el flujo TOTP de doble factor.

El procedimiento completo, incluido el despliegue sobre Oracle Cloud Infrastructure (Ampere A1, AArch64), se detalla en el Capítulo 8 (Manuales del Sistema) y en el fichero `./documentacion/manual-despliegue.pdf`.

Repositorio de código y trazabilidad

El código fuente está disponible públicamente en el repositorio:

<https://github.com/adrianmfuentes/SVAES>

El repositorio incluye historial completo de commits, integración continua mediante GitHub Actions, análisis estático en SonarCloud, análisis de seguridad con CodeQL y gestión de dependencias mediante Dependabot. El *Quality Gate* configurado exige un mínimo del 80 % de cobertura de líneas para que un *pull request* pueda fusionarse a la rama principal.

Estudio Detallado de Riesgos

Este apéndice desarrolla el análisis de riesgos resumido en el Capítulo 9, conforme a la metodología combinada DPPI–PMBOK adoptada en el proyecto. Se incluyen el registro completo de riesgos, la matriz probabilidad-impacto utilizada para su priorización y las estrategias de respuesta asociadas a cada uno.

Metodología

La identificación, valoración y respuesta a los riesgos sigue las directrices del PMBOK (Project Management Body of Knowledge) integradas con los criterios de la asignatura DPPI (Dirección y Planificación de Proyectos Informáticos). Cada riesgo se caracteriza por cinco dimensiones:

- **Probabilidad.** Estimación de la frecuencia con la que el riesgo puede materializarse, en cinco niveles (Muy Baja, Baja, Media, Alta, Muy Alta) con valores numéricos 0,1 / 0,3 / 0,5 / 0,7 / 0,9.
- **Impacto.** Evaluado de forma independiente sobre cuatro ejes: presupuesto, planificación, alcance y calidad. Cada eje se valora en cinco niveles (Inapreciable, Bajo, Medio, Alto, Crítico) con valores 0,05 / 0,15 / 0,3 / 0,55 / 0,9. El impacto global se toma como el máximo entre los cuatro ejes.
- **Exposición.** Producto de la probabilidad por el impacto global. Determina la prioridad de tratamiento.
- **Estrategia de respuesta.** Una de las cinco estrategias del PMBOK: para amenazas, *Eliminar*, *Mitigar*, *Transferir* o *Asumir*; para oportunidades, *Explotar*, *Mejorar*, *Compartir* o *Asumir*.
- **Plan de acción.** Conjunto concreto de medidas asociadas a la estrategia seleccionada.

La matriz probabilidad-impacto aplicada se reproduce en la Tabla 74, con tres bandas de prioridad: verde (exposición < 0,05), amarillo ($0,05 \leq \text{exposición} < 0,15$) y rojo (exposición $\geq 0,15$).

Probabilidad \ Impacto	Inapreciable	Bajo	Medio	Alto	Crítico
	(0,05)	(0,15)	(0,30)	(0,55)	(0,90)
Muy Alta (0,9)	0,045	0,135	0,270	0,495	0,810
Alta (0,7)	0,035	0,105	0,210	0,385	0,630
Media (0,5)	0,025	0,075	0,150	0,275	0,450
Baja (0,3)	0,015	0,045	0,090	0,165	0,270
Muy Baja (0,1)	0,005	0,015	0,030	0,055	0,090

Tabla 74: Matriz probabilidad-impacto (valores de exposición).

Registro de riesgos

A continuación se presentan los riesgos identificados durante la planificación del proyecto, ordenados por exposición decreciente. Para cada riesgo se incluye una descripción, su valoración cuantitativa y la respuesta asociada. Los riesgos se identifican con el prefijo R para amenazas y OP para oportunidades.

12.8.1. R01 — Plazo cerrado del TFG sin posibilidad de extensión

Categoría: Gestión

Descripción: El calendario académico fija una ventana de aproximadamente cuatro meses (del 20 de marzo al 6 de julio de 2026) sin posibilidad de prórroga. Cualquier retraso impacta directamente en la fase final de redacción y defensa.

Probabilidad: Alta (0,7)

Impacto: Presupuesto Bajo (0,15); Planificación Crítico (0,9); Alcance Medio (0,3); Calidad Medio (0,3)

Exposición: 0,630 (Rojo)

Estrategia: Mitigar

Plan: Redacción de la memoria en paralelo a la implementación; hitos quincenales con la tutora; priorización MoSCoW para reducir alcance si fuese necesario; reserva de la última semana exclusivamente para revisión y defensa.

12.8.2. R02 — Subestimación de la complejidad del motor de verificación

Categoría: Técnico

Descripción: El diseño de diez reglas paralelas en Rust con paralelismo Rayon y agregación coherente puede requerir más esfuerzo del estimado, especialmente en la fase de pruebas y aseguramiento de cobertura estructural.

Probabilidad: Media (0,5)

Impacto: Presupuesto Bajo (0,15); Planificación Alto (0,55); Alcance Medio (0,3); Calidad Alto (0,55)

Exposición: 0,275 (Rojo)

Estrategia: Mitigar

Plan: Implementación incremental regla a regla; pruebas unitarias por regla antes de la integración del agregador; reserva de un margen del 15 % sobre la estimación base de la fase de implementación (EDT 1.2.8).

12.8.3. R03 — Imposibilidad de alcanzar el *Quality Gate* del 80 % de cobertura

Categoría: Calidad

Descripción: SonarCloud bloquea la fusión de *pull requests* si la cobertura cae por debajo del 80 %. Existen módulos (migraciones, configuración, `main.py`) cuya cobertura por pruebas funcionales es estructuralmente baja.

Probabilidad: Alta (0,7)

Impacto: Presupuesto Inapreciable (0,05); Planificación Medio (0,3); Alcance Bajo (0,15); Calidad Alto (0,55)

Exposición: 0,385 (Rojo)

Estrategia: Mitigar

Plan: Estrategia de tres vías: exclusiones legítimas (migraciones, configuración, puertos abstractos), generación de pruebas de *branch coverage* a partir de informes de SonarCloud, e integración del motor Rust con `cargo-llvm-cov` en el agregador.

12.8.4. R04 — Pérdida de trabajo por fallo técnico

Categoría: Técnico

Descripción: Un fallo de hardware, corrupción del disco o pérdida del repositorio local podría provocar la pérdida parcial o total del código o de la documentación.

Probabilidad: Muy Baja (0,1)

Impacto: Presupuesto Bajo (0,15); Planificación Crítico (0,9); Alcance Crítico (0,9); Calidad Crítico (0,9)

Exposición: 0,090 (Amarillo)

Estrategia: Mitigar

Plan: Uso de Git con repositorio remoto en GitHub (*push* diario); memoria mantenida en Overleaf con historial de versiones automático; copia de seguridad semanal en disco externo cifrado.

12.8.5. R05 — Disponibilidad limitada de la tutora

Categoría: Organizativo

Descripción: La tutora puede tener, según el período del curso, agenda con disponibilidad limitada, lo que puede retrasar revisiones de diseño, validaciones de alcance y *feedback* sobre capítulos de la memoria.

Probabilidad: Media (0,5)

Impacto: Presupuesto Inapreciable (0,05); Planificación Medio (0,3); Alcance Bajo (0,15); Calidad Medio (0,3)

Exposición: 0,150 (Rojo)

Estrategia: Mitigar

Plan: Planificación de reuniones con al menos una semana de antelación; envío de materiales por escrito para revisión asíncrona; consolidación de dudas para reducir el número de interacciones.

12.8.6. R06 — Falta de experiencia previa con Rust en proyectos de producción

Categoría: Técnico

Descripción: El motor de verificación se implementa en Rust, un lenguaje con curva de aprendizaje pronunciada. La inexperiencia previa puede provocar diseños subóptimos o errores en el manejo de *ownership* y concurrencia.

Probabilidad: Media (0,5)

Impacto: Presupuesto Bajo (0,15); Planificación Medio (0,3); Alcance Bajo (0,15); Calidad Medio (0,3)

Exposición: 0,150 (Rojo)

Estrategia: Mitigar

Plan: Pruebas de concepto previas a la implementación definitiva; revisión de código asistida por herramientas estáticas (*clippy*, *cargo audit*); consulta sistemática de la documentación oficial y patrones idiomáticos.

12.8.7. R07 — Cambios significativos en requisitos durante el desarrollo

Categoría: Requisitos

Descripción: Tras revisiones con la tutora pueden surgir cambios en requisitos no funcionales (rendimiento, accesibilidad) o ajustes en las reglas de verificación que requieran retrabajos.

Probabilidad: Media (0,5)

Impacto: Presupuesto Bajo (0,15); Planificación Medio (0,3); Alcance Alto (0,55); Calidad Medio (0,3)

Exposición: 0,275 (Rojo)

Estrategia: Mitigar

Plan: Cierre formal de alcance en el hito intermedio de validación; gestión de cambios mediante priorización MoSCoW; arquitectura hexagonal que minimiza el coste de incorporar nuevas reglas.

12.8.8. R08 — Dificultades de despliegue sobre arquitectura AArch64

Categoría: Infraestructura

Descripción: El despliegue en Oracle Cloud Infrastructure utiliza una máquina Ampere A1 (AArch64). Algunas imágenes Docker oficiales pueden no soportar esta arquitectura nativamente, requiriendo compilación o emulación.

Probabilidad: Baja (0,3)

Impacto: Presupuesto Inapreciable (0,05); Planificación Medio (0,3); Alcance Bajo (0,15); Calidad Bajo (0,15)

Exposición: 0,090 (Amarillo)

Estrategia: Asumir

Plan: Validación temprana de imágenes `multi-arch`; en caso de incompatibilidad, compilación local con `docker buildx` sobre el destino AArch64.

12.8.9. R09 — Vulnerabilidades de seguridad detectadas tardíamente

Categoría: Seguridad

Descripción: El sistema maneja credenciales cifradas (Fernet), tokens `JWT` y un segundo factor TOTP. Una vulnerabilidad detectada tarde podría requerir cambios estructurales en autenticación o cifrado.

Probabilidad: Baja (0,3)

Impacto: Presupuesto Bajo (0,15); Planificación Medio (0,3); Alcance Bajo (0,15); Calidad Alto (0,55)

Exposición: 0,165 (Rojo)

Estrategia: Mitigar

Plan: Análisis estático con CodeQL en cada *pull request*; Dependabot activo para alertas de dependencias; revisión específica del módulo de seguridad antes de cada hito; pruebas de seguridad en el plan de pruebas.

12.8.10. R10 — Caída del servicio de Overleaf durante la redacción

Categoría: Herramientas

Descripción: La memoria se redacta en Overleaf. Una caída prolongada del servicio en un período crítico podría bloquear la edición.

Probabilidad: Muy Baja (0,1)

Impacto: Presupuesto Inapreciable (0,05); Planificación Bajo (0,15); Alcance Inapreciable (0,05); Calidad Bajo (0,15)

Exposición: 0,015 (Verde)

Estrategia: Asumir

Plan: Mantener una copia local sincronizada de los ficheros $\text{\LaTeX}$ mediante `git` contra el repositorio de Overleaf; en caso de caída, compilar localmente con `pdflatex`.

12.8.11. OP01 — Reutilización en futuros TFG o trabajos académicos

Categoría: Académico

Descripción: La arquitectura hexagonal y el patrón IConnector permiten reutilizar el sistema como base para futuros TFG que añadan nuevos conectores o reglas de verificación.

Probabilidad: Media (0,5)

Impacto: Presupuesto Inapreciable (0,05); Planificación Inapreciable (0,05); Alcance Alto (0,55); Calidad Medio (0,3)

Exposición: 0,275 (Rojo, positivo)

Estrategia: Mejorar

Plan: Documentación cuidada del patrón de extensión; *templates* de nuevos conectores y reglas en el repositorio; licencia abierta para facilitar la reutilización académica.

12.8.12. OP02 — Publicación del repositorio como referencia profesional

Categoría: Profesional

Descripción: Un repositorio público bien estructurado, con cobertura alta y CI/CD, puede servir como referencia técnica en procesos de selección profesional posteriores.

Probabilidad: Alta (0,7)

Impacto: Presupuesto Inapreciable (0,05); Planificación Inapreciable (0,05); Alcance Medio (0,3); Calidad Alto (0,55)

Exposición: 0,385 (Rojo, positivo)

Estrategia: Explotar

Plan: Mantener el repositorio público desde el primer commit; configurar CI/CD visible; redactar un README y una documentación de arquitectura que faciliten la lectura externa.

Resumen y priorización

Una vez completada la identificación individual de cada riesgo y su correspondiente plan de respuesta, resulta imprescindible obtener una visión panorámica que permita tanto a la persona responsable del proyecto como al tribunal evaluador comprender de un solo vistazo cuál es la situación global de la exposición al riesgo. Esta sección presenta, en primer lugar, un análisis cuantitativo de la distribución de los riesgos por bandas de exposición; a continuación, la tabla resumen que consolida todo el registro; y, finalmente, las conclusiones derivadas de dicha consolidación.

12.9.1. Distribución por bandas de exposición

Como se describió en la metodología (véase §12.7), la exposición de cada riesgo se calcula como el producto de su probabilidad de ocurrencia por su impacto global. Este valor sitúa al riesgo en una de tres bandas cromáticas: verde (exposición menor que 0,05), amarillo (exposición entre 0,05 y 0,15) o rojo (exposición mayor o igual que 0,15). La distribución observada es la siguiente:

- **Banda roja (nueve elementos):** R01, R02, R03, R05, R06, R07, R09, OP01 y OP02 se sitúan en esta banda. Los riesgos R01 (plazo cerrado del TFG), R02 (subestimación de la complejidad del motor de verificación) y R03 (imposibilidad de alcanzar el Quality Gate del 80%) destacan por su elevada exposición (0,630, 0,275 y 0,385 respectivamente), lo que los convierte en los riesgos que mayor atención han recibido durante la planificación y la ejecución del proyecto. Los riesgos R05 y R06, con exposición exactamente igual a 0,150, cumplen el umbral de la banda roja

por la definición estricta de la metodología. Las dos oportunidades (OP01 y OP02) aparecen también en banda roja al presentar valores de exposición elevados, si bien, al tratarse de eventos favorables, su estrategia asociada no es de mitigación sino de mejora o explotación, respectivamente.

- **Banda amarilla (dos elementos):** R04 y R08 se sitúan en esta banda intermedia, con una exposición de 0,090. Ambos riesgos corresponden a escenarios menos probables cuyas consecuencias, aunque potencialmente graves, tienen una likelihood baja.
- **Banda verde (un elemento):** Únicamente el riesgo R10 (caída del servicio de Overleaf) se sitúa en banda verde, con una exposición de 0,015. Este riesgo se asume explícitamente dado que el coste de las medidas preventivas (copia local sincronizada) es desproporcionado frente a la probabilidad real de ocurrencia.

12.9.2. Tabla resumen del registro de riesgos

La Tabla 75 consolida la información esencial de los diez riesgos y las dos oportunidades identificadas, ordenados por exposición decreciente. Para cada entrada se incluye el identificador, la categoría a la que pertenece, una descripción sintética del riesgo, el valor numérico de la exposición y la estrategia de respuesta adoptada. Las celdas de la columna de exposición se presentan codificadas por colores (rojo para valores $\geq 0,15$, amarillo para valores entre 0,05 y 0,15, y verde para valores menores que 0,05) para facilitar la lectura rápida del documento.

La lectura horizontal de la tabla permite identificar con claridad que los riesgos de mayor exposición se concentran en las categorías de Gestión (R01), Calidad (R03) y Técnica (R02, R07), mientras que el riesgo de Seguridad R09 presenta una exposición más contenida (0,165) gracias a las medidas de prevención aplicadas desde la fase de planificación. Las oportunidades OP01 y OP02, aunque cuantitativamente comparables a los riesgos en términos de exposición, se distinguen cualitativamente por su signo positivo y por requerir estrategias de respuesta distintas (mejora y explotación, respectivamente, en lugar de mitigación o eliminación).

ID	Categoría	Descripción	Exp.	Estrategia
R01	Gestión	Plazo cerrado del TFG sin posibilidad de extensión	0,630	Mitigar
OP02	Profesional	Publicación del repositorio como referencia profesional	0,385	Explotar
R03	Calidad	Imposibilidad de alcanzar el Quality Gate del 80 %	0,385	Mitigar

ID	Categoría	Descripción	Exp.	Estrategia
R02	Técnico	Subestimación de la complejidad del motor de verificación	0,275	Mitigar
R07	Requisitos	Cambios significativos en requisitos durante el desarrollo	0,275	Mitigar
OP01	Académico	Reutilización del sistema en futuros TFG	0,275	Mejorar
R09	Seguridad	Vulnerabilidades de seguridad detectadas tardíamente	0,165	Mitigar
R05	Organizativo	Disponibilidad limitada de la tutora para revisiones	0,150	Mitigar
R06	Técnico	Falta de experiencia previa con Rust en proyectos de producción	0,150	Mitigar
R04	Técnico	Pérdida de trabajo por fallo técnico o corrupción de datos	0,090	Mitigar
R08	Infraestructura	Dificultades de despliegue sobre arquitectura AArch64	0,090	Asumir
R10	Herramientas	Caída del servicio de Overleaf durante la redacción	0,015	Asumir

Tabla 75: Resumen del registro de riesgos ordenado por exposición decreciente.

12.9.3. Conclusiones del análisis de riesgos

Del análisis consolidado se extraen tres conclusiones fundamentales. En primer lugar, la concentración de riesgos en banda roja (nueve de los doce elementos) es coherente con la naturaleza de un proyecto software de fin de grado, en el que confluyen restricciones temporales estrictas, tecnologías desconocidas por el estudiante y un entorno académico con recursos limitados. Esta concentración no indica una planificación deficiente sino, precisamente, una identificación honesta de las dificultades reales del proyecto.

En segundo lugar, la presencia de dos oportunidades en banda roja (OP01 y OP02) demuestra que el análisis de riesgos, cuando se lleva a cabo de forma rigurosa, no se limita a identificar amenazas sino que también revela elementos potencialmente beneficiosos que pueden maximizar el retorno del proyecto. La reutilización del sistema como base para futuros trabajos académicos (OP01) y la posibilidad de utilizar el repositorio como referencia profesional (OP02) son beneficios colaterales que no debieran pasarse por alto.

En tercer lugar, la decisión de asumir explícitamente el riesgo R10 (caída de Overleaf) ilustra un principio fundamental de la gestión de riesgos: no todos los riesgos merecen una respuesta activa. Aquellos riesgos cuya probabilidad es muy baja y cuyo impacto, aunque potencialmente significativo, puede mitigarse con medidas triviales (como mantener una copia local sincronizada), no justifican la inversión de recursos en planes de contingencia complejos.

El registro completo de riesgos, en formato editable y con toda la información adicional (probabilidades, impactos por eje, planes de acción detallados, historial de cambios), se encuentra disponible en `./riesgos/registro-riesgos.xlsx` del archivo adjunto descrito en el Apéndice 12.1. Este fichero puede ser actualizado por el tribunal o por el estudiante en futuras iteraciones del proyecto sin necesidad de modificar el presente documento.

Glosario y Acrónimos

Este apéndice recoge la terminología específica del dominio SVAES y los acrónimos técnicos utilizados a lo largo de la memoria, con el objetivo de facilitar su consulta. Las entradas se presentan ordenadas alfabéticamente dentro de cada sección.

Glosario de términos de dominio

Conector. Componente que implementa la interfaz `IConnector` y proporciona acceso a una herramienta externa (gestor de incidencias, repositorio de código, sistema de despliegue) sin que el resto del sistema conozca su naturaleza específica.

Entrega de software. Sinónimo de *release*. Conjunto coherente de artefactos (código, documentación, configuración) que se somete a verificación en un punto concreto del ciclo de vida.

Motor de verificación. Servicio escrito en Rust que ejecuta de forma paralela las diez reglas de verificación RV-01... RV-10 sobre una entrega y produce un veredicto agregado.

Perfil de verificación. Configuración nombrada que define qué reglas se aplican, con qué parámetros y con qué umbrales para una organización o proyecto concreto.

Regla de verificación (RV). Comprobación atómica aplicada a una entrega. Cada regla devuelve un veredicto (aprobada, no aprobada, no aplicable) y un conjunto de evidencias.

Release. Véase *entrega de software*.

SVAES. Sistema de Verificación Automática de Entregas de Software. Nombre y acrónimo del sistema objeto del presente TFG.

Tenant. Organización aislada en el modelo multi-tenant del sistema. Cada *tenant* dispone de sus propios usuarios, conectores y perfiles de verificación.

U1, U2, U3, U4. Roles del modelo de control de acceso del sistema: U1 = invitado (sin autenticar), U2 = operador, U3 = administrador global y U4 = manager de organización.

Veredicto. Resultado agregado de una verificación, calculado a partir de los veredictos individuales de las reglas según la política del perfil aplicado.

Reglas de verificación

El sistema implementa un catálogo de diez reglas de verificación (RV-01 a RV-10) orientadas a comprobar la coherencia de la gestión de la entrega sobre los artefactos recuperados de los sistemas externos. La definición normativa de cada regla, su severidad y los parámetros que requiere se recogen en la Tabla 24 del Capítulo 5 (sección 5.4), y su materialización en el manual de usuario en la Tabla 54 del Capítulo 8. A modo de resumen:

- **RV-01.** Existencia de artefactos: la lista de artefactos recuperados no está vacía. OBLIGATORIA, sin parámetros.
- **RV-02.** Trazabilidad entre artefactos: los artefactos de tipo `source_type` referencian correctamente a artefactos de tipo `target_type` mediante el campo `reference_field`. OBLIGATORIA.
- **RV-03.** Validación de estados: todos los artefactos del tipo configurado se encuentran en uno de los estados permitidos. OBLIGATORIA.
- **RV-04.** Integridad de campos numéricos: los campos numéricos configurados son no nulos, numéricos y ≥ 0 . OBLIGATORIA.
- **RV-05.** Disponibilidad de documentos: el campo `accessible_field` de cada artefacto confirma que el documento asociado está accesible. OBLIGATORIA.
- **RV-06.** Coherencia de atributos: el campo `attribute` de los artefactos coincide con el valor esperado. OPCIONAL.
- **RV-07.** Registro externo: el campo `marker_field` está presente y no vacío, confirmando el registro en un sistema externo. OBLIGATORIA.
- **RV-08.** Alineación de listas: los artefactos de tipo `target_type` tienen correspondencia con el campo maestro del artefacto identificado por `master_artifact_id`. OBLIGATORIA.
- **RV-09.** Validación de referencias: los campos de referencia apuntan a recursos cuyo campo de accesibilidad confirma que siguen activos. OPCIONAL.
- **RV-10.** Aprobación final: todos los artefactos han alcanzado uno de los estados de aprobación configurados. OBLIGATORIA.

El detalle técnico de cada regla (parámetros configurables y formato de evidencias) se

documenta en el Capítulo 5.

Acrónimos técnicos

ABC. *Abstract Base Class*. Mecanismo de Python para definir interfaces abstractas.

API. *Application Programming Interface*.

CI/CD. *Continuous Integration / Continuous Deployment*.

CRUD. *Create, Read, Update, Delete*. Operaciones básicas sobre entidades persistentes.

DPPI. Dirección y Planificación de Proyectos Informáticos.

EII. Escuela de Ingeniería Informática (Universidad de Oviedo).

GADPPI. Guía de Aprendizaje de la asignatura DPPI.

JWT. *JSON Web Token*. Mecanismo de autenticación basado en *tokens* firmados.

MoSCoW. *Must, Should, Could, Won't*. Técnica de priorización de requisitos.

OCI. Oracle Cloud Infrastructure.

ORM. *Object-Relational Mapping*. Patrón de mapeo entre objetos y tablas relacionales.

PMBOK. *Project Management Body of Knowledge*.

RBAC. *Role-Based Access Control*. Control de acceso basado en roles.

REST. *Representational State Transfer*.

RGPD. Reglamento General de Protección de Datos.

RV. Regla de Verificación. Véase §12.12.

SaaS. *Software as a Service*.

SRS. *Software Requirements Specification*.

SVAES. Sistema de Verificación Automática de Entregas de Software.

TFG. Trabajo Fin de Grado.

TOTP. *Time-based One-Time Password*. Algoritmo de segundo factor de autenticación.

UO. Universidad de Oviedo.

UNE. Una Norma Española.

WCAG. *Web Content Accessibility Guidelines*.

Tecnologías y herramientas

Actix-web. *Framework* web asíncrono para Rust (versión 4.4) empleado en el motor de verificación.

Alembic. Herramienta de gestión de migraciones de base de datos para SQLAlchemy.

Angular. *Framework* de desarrollo *frontend* en TypeScript empleado en la versión ^21.2.0.

Celery. Cola de tareas distribuida para Python.

CodeQL. Motor de análisis semántico de código utilizado para detección de vulnerabilidades.

Docker Compose. Herramienta de orquestación de contenedores empleada para el despliegue.

FastAPI. *Framework* web asíncrono para Python utilizado en el *backend*.

Fernet. Esquema simétrico de cifrado autenticado utilizado para credenciales.

NgRx. Implementación de Redux para Angular utilizada para gestión de estado.

PostgreSQL. Sistema gestor de base de datos relacional empleado en la versión 17.

Pydantic. Biblioteca de validación de datos para Python.

Rayon. Biblioteca de paralelismo de datos para Rust.

Redis. Almacén de datos en memoria utilizado como *broker* de Celery.

SonarCloud. Servicio de análisis estático y cobertura usado como *Quality Gate*.

SQLAlchemy. Biblioteca ORM y SQL para Python.